
Звуковая подсистема Linux

Документация

Версия 4.13.0-rc4+

Сообщество разработчиков ядра

05 сентября 2017 г.

1 Документация по API ядра ALSA	1
1.1 API драйвера ALSA	1
1.2 Написание драйвера ALSA	89
2 Проекты и реализации	145
2.1 Стандартные имена элементов управления ALSA	145
2.2 API отображения каналов ALSA PCM	147
2.3 API Сжатия-разгрузки ALSA	149
2.4 Временная метка ALSA PCM	151
2.5 Элементы управления ALSA Jack	155
2.6 Трассировочные точки в ALSA	156
2.7 Proc-файлы драйверов ALSA	158
2.8 Примечания по режиму энергосбережения	161
2.9 Примечания по эмуляции операционной системы ядра	161
2.10 Эмуляция секвенсора OSS на ALSA	161
3 Уровень ALSA SoC	171
3.1 Обзор уровня SoC ALSA	171
3.2 Драйвер класса кода ALSA SoC	172
3.3 Цифровой аудиointерфейс ALSA SoC (DAI)	173
3.4 Динамическое управление питанием звука для портативных устройств	174
3.5 Драйвер платформы ALSA SoC	175
3.6 Водитель-машины ALSA SoC	180
3.7 Звуковые хлопki и щелчки	181
3.8 Синхронизация звука	183
3.9 Обнаружение гнезда ALSA SoC	184
3.10 Динамический PCM	185
3.11 Создание ссылки codec to codec dai link для ALSA dapm	191
4 Расширенная звуковая архитектура Linux - Руководство по настройке драйвера	193
4.1 Конфигурация ядра	193
4.2 Параметры модуля	193
4.3 Опция Quick AS97	193
4.4 Настройка для отличных от ISAPNP	225
4.5 Поддержка автоматической загрузки модуля	225
4.6 Сопоставление устройств ALSA PCM с устройствами OSS	225
4.7 Интерфейсы Proc (proc/asound)	226
4.8 Раннее выделение буфера	227
4.9 Ссылки и адреса	227
5 HD-Аудио	229
5.1 Дополнительные сведения о драйвере HD-Audio	229
5.2 Модели, специфичные для HD-аудиокодека	240
5.3 Элементы управления микшером, зависящие от HD-аудиокодека	240
5.4 Поддержка HD-аудио DP-MST	249

6 Информация, относящаяся к конкретной карте	251
Поддержка аналогового джойстика в драйверах ALSA	251 6.1
6.2 Краткие сведения о драйвере C-Media 8338/8738/8768/8770 .	252
..... Микшер Sound Blaster Live / код DSP по умолчанию	255
6.4 Микшер Sound Blaster Audigy / код DSP по умолчанию .	256
..... Низкая задержка многоканальный звук с РАЗЪЕМОМ и emu10k1 / emu10k2	267
6.6 Смеситель VIA82xx	268
..... Руководство по исполнению M-Audio Audiophile USB с ALSA и разъемом	269
6.8 Драйвер Alsa для звуковых карт Digigram miXart8 и miXart8AES / EBU .	277
..... Драйвер ALSA BT87x	278
6.10 Примечания по поддержке USB-звука Maya44	279
6.11 Программный интерфейс ALSA-DSP MADI Driver	282
6.12 Последовательный MIDI-драйвер UART 16450/16550	287
..... 6.13 Контроллеры ввода SPDIF Imagination Technologies	289
Указатель	291

ДОКУМЕНТАЦИЯ ПО API ЯДРА ALSA

1.1 API драйвера ALSA

1.1.1 Управление картами и устройствами

Управление картами

аннулировать `snd_device_initialize` (структурировать устройство * *dev*, структурировать `snd_card` * *card*)
Инициализировать структурное устройство для звуковых устройств

Параметры

`структура device` * `dev` устройство для инициализации
`структура snd_card` * `card` карта для назначения, необязательно `int`
`snd_card_new` (устройство структуры * *parent*, `int idx`, `const char * xid`, модуль структуры * *модуль*,
`int extra_size`, структура `snd_card ** card_ret`)
создайте и инициализируйте структуру звуковой карты

Параметры

`структура device` * `parent` объект parent device
`int idx` картотека (адрес) [0... (SNDRV_CARDS-1)]
`постоянный символ` * `xid` идентификация карты (строка ASCII)
`модуль структуры` * `module` модуль верхнего уровня для блокировки
`int extra_size` выделите этот дополнительный размер после структуры
основной звуковой карты `struct snd_card` * * `card_ret` указатель для
хранения созданного экземпляра карты **Описание**

Создает и инициализирует структуру звуковой карты.

Функция выделяет экземпляр `snd_card` через `kzalloc` с заданным
пространством для свободного использования драйвером .
Выделенная структура хранится в заданном указателе `card_ret`. **Возврат**
Ноль в случае успеха или отрицательный код ошибки.

`int snd_card_disconnect` (`struct snd_card * карта`)

отключите все API от файловых

операций (пространство пользователя) **Параметры**

`структура snd_card` * `card` структура
звуковой карты **Описание**

Отключает все API-интерфейсы от файловых операций (пространство пользователя).

Возврат

Ноль, в противном случае отрицательный код ошибки. **Примечание**

Текущая реализация заменяет все активные file-> f_op специальными фиктивными файловыми операциями (они ничего не делают, кроме release).

int snd_card_free_when_closed(структура snd_card * карта памяти)

Отсоедините карту памяти, освободите ее

позже, в конечном итоге **Параметры**

структура snd_card * карта памяти структура

звуковой карты **Описание**

В отличие от `snd_card_free()`, эта функция не пытается немедленно освободить ресурс карты, а сначала пытается отключиться. Когда карта все еще используется, функция возвращается перед освобождением ресурсов. Ресурсы карты будут освобождены, когда количество пересчетов достигнет нуля. int `snd_card_free`(struct snd_card * card) освобождает заданную структуру звуковой карты

Параметры

struct snd_card * card структура

звуковой карты **Описание**

Эта функция автоматически освобождает структуру звуковой карты и все назначенные устройства. То есть вам не нужно освобождать устройства самостоятельно. Эта функция ожидает, пока все ресурсы не будут должным образом освобождены.

Возврат

Ноль. Освобождает все связанные устройства и интерфейс управления, связанный с данной звуковой картой. аннулирует `snd_card_set_id`(структура snd_card * карточка, постоянный символ * nid) задайте идентификационное имя карты

Параметры

структура snd_card * card структура

звуковой карты постоянный символ * nid новая

идентификационная строка **Описание**

Эта функция устанавливает идентификацию карты и проверяет наличие коллизий имен.

int `snd_card_add_dev_attr`(struct snd_card * карта, постоянная структура attribute_group * группа) Добавьте новую группу атрибутов sysfs в card

Параметры

структура snd_card * card экземпляр карты

постоянная структура attribute_group * group группа атрибутов

для добавления int `snd_card_register`(структура snd_card * card) зарегистрируйте звуковую карту

Параметры

структура snd_card * карта памяти структура

звуковой карты **Описание**

Эта функция регистрирует все устройства, назначенные звуковой карте. До вызова этой функции интерфейс управления ALSA заблокирован от внешнего доступа. Таким образом, вы должны вызвать эту функцию в конце инициализации карты.

Возврат

Ноль, в противном случае отрицательный код ошибки, если регистрация завершилась неудачно.

`int snd_component_add(структура snd_card * card, постоянный символ * component)`
добавьте строку компонента

Параметры

`структура snd_card * card` структура звуковой

карты `постоянный символ * component` строка

идентификатора компонента **Описание**

Эта функция добавляет строку идентификатора компонента в список поддерживаемых. На компонент можно ссылаться из

библиотеки alsa. **Возврат**

Ноль, в противном случае отрицательный код ошибки. `int`

`snd_card_file_add(структура snd_card * card, файл структуры * file)`
добавьте файл в список файлов карты

Параметры

`структура snd_card * card` структура

звуковой карты `файл структуры * file` указатель

на файл **Описание**

Эта функция добавляет файл в список связанных файлов карты. Этот связанный список используется для постоянного отслеживания состояния соединения и для предотвращения высвобождения занятых ресурсов при горячем подключении. **Возврат**

нулевой или отрицательный код ошибки.

`int snd_card_file_remove(struct snd_card * card, struct file * file)`
удалите файл из списка файлов

Параметры

`структура snd_card * card` структура

звуковой карты `файл структуры * file` указатель

на файл **Описание**

Эта функция удаляет файл, ранее добавленный на карту с помощью функции `snd_card_file_add()` .
Если удалены все файлы и `функция snd_card_free_when_closed()` была вызвана заранее, она обрабатывает ожидающий освобождения ресурсов.

Возврат

Нулевой или отрицательный код ошибки.

`int snd_power_wait(struct snd_card * card, unsigned int power_state)`
подождите, пока не будет изменено состояние питания.

Параметры

`структура snd_card * card` структура звуковой

карты `unsigned int power_state` ожидаемое

состояние питания **Описание**

Ожидает изменения состояния питания.

Возврат

Ноль в случае успеха или отрицательный код ошибки.

Примечание

блокировка питания должна быть активирована перед вызовом.

Компоненты устройства

int snd_device_new(структура snd_card * карта, перечисление snd_device_type Tun, аннулировать * device_data, struct snd_device_ops * операции)
создание компонента устройства ALSA

Параметры

структура snd_card * card экземпляр card
перечисление типа snd_device_type тип устройства,
SNDRV_DEV_XXX void * device_data указатель данных этого
устройства struct snd_device_ops * ops операторский стол

Описание

Создает новый компонент устройства для данного указателя данных.
Устройство будет назначено карте и будет совместно управляться картой.
Указатель данных также играет роль идентификатора, поэтому
адрес указателя должен быть уникальным и неизменным. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.
void snd_device_disconnect(struct snd_card * card, void * device_data)
отключите устройство

Параметры

структура snd_card * card экземпляр карты
void * device_data указатель данных для

отключения **Описание**

Переводит устройство в состояние отключения, вызывая обратный вызов dev_disconnect, если
устройство уже было зарегистрировано.
Обычно вызывается из snd_card_disconnect().

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое или если устройство не
найдено. недействительно snd_device_free(структура snd_card * карта, аннулирование * device_data)
освободите устройство от карты

Параметры

структура snd_card * карта экземпляр карты
аннулирует * device_data указатель данных

для освобождения **Описание**

Удаляет устройство из списка на карте памяти и вызывает обратные вызовы
dev_disconnect и dev_free, соответствующие состоянию. Затем отпустите устройство. int
snd_device_register(структура snd_card * карточка, аннулирование * device_data)
зарегистрировать устройство

Параметры

структура snd_card * card экземпляр карты

void * device_data указатель данных для

регистрации

Описание

Обычно это вызывается из регистров устройства, которое уже

snd_card_register(), но его можно вызвать позже, если какие-либо новые устройства будут созданы после вызова

snd_card_register().

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое или если устройство не найдено.

Запросы модуля и записи файла устройства

недействительны snd_request_card(int card)

попробуйте загрузить

модуль карты

Параметры

int card номер карты

Описание

Пытается загрузить модуль "snd-card-X" для данного номера карты через

request_module. Немедленно возвращает, если уже загружен. void

* snd_lookup_minor_data (беззнаковый int второстепенный, инт Tun)

получить пользовательские данные зарегистрированного устройства

Параметры

беззнаковый int minor младший

номер тип int тип устройства

(SNDRV_DEVICE_TYPE_XXX)

Описание

Проверяет, зарегистрировано ли второстепенное устройство с указанным типом, и возвращает указатель его пользовательских данных.

Эта функция увеличивает счетчик ссылок экземпляра карты, если найден связанный экземпляр с

заданным второстепенным номером и типом. Вызывающий должен вызвать snd_card_unref() соответствующим

образом позже.

Возврат

Указатель пользовательских данных, если указанное устройство найдено. NULL в противном случае.

int snd_register_device(int Tun, struct snd_card * card, int dev, const struct file_operations

* f_ops, void * private_data, struct device * устройство)

Зарегистрируйте файл устройства ALSA для карты

Параметры

тип int тип устройства, SNDRV_DEVICE_TYPE_XXX

структура snd_card * card экземпляр карты

int dev индекс устройства

const struct file_operations * f_ops файловые операции

void * private_data пользовательский указатель для f_ops->c.func:открыть()

структурировать устройство * device устройство для регистрации

Описание

Регистрирует файл устройства ALSA для данной карты. Операторы должны быть установлены в параметре reg.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_unregister_device(структурировать устройство * dev)

отменить регистрацию устройства на данной карте

Параметры

структура device * dev ЭКЗЕМПЛЯР

устройства

Описание

Отменяет регистрацию файла устройства, уже зарегистрированного через snd_register_device().

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

Помощники по управлению памятью

int copy_to_user_fromio(void __user * летнее время, постоянная изменчивая пустота __iomem * Src, size_t количество)

копирование данных из mmio-пространства в пространство пользователя

Параметры

void __ user * летнее время конечный указатель на

пользовательское пространство const volatile void __iomem * src Исходный указатель

на mmio количество size_t размер данных для копирования в байтах

Описание

Копирует данные из mmio-пространства в user-space.

Возврат

Ноль в случае успеха или ненулевое значение при сбое.

int copy_from_user_toio(volatile void __iomem * летнее время, const void __user * Src, size_t count)

копирование данных из пользовательского пространства в mmio-пространство

Параметры

volatile void __iomem * dst указатель назначения на

mmio-пространство const void __user * src указатель источника на пользовательское

пространство количество size_t размер данных для копирования в байтах

Описание

Копирует данные из пользовательского пространства в mmio-пространство.

Возврат

Ноль в случае успеха или ненулевое значение при сбое.

недействительно * snd_malloc_pages(размер_t размер, gfp_t gfp_flags)

выделять страницы заданного размера

Параметры

размер_t size размер для выделения в байтах

gfp_t gfp_flags условия распределения, GFP_XXX

Описание

Выделяет физически смежные страницы

заданного размера. Возврат

Указатель на буфер или `NULL` если недостаточно памяти.

`void snd_free_pages(void * ptr, размер_t размер)`
освободить страницы

Параметры

`void * ptr` указатель буфера на
освобождение `размер_t size` размер
выделенного буфера

Описание

Освобождает буфер, выделенный с помощью `snd_malloc_pages()`.

`void snd_malloc_dev_iram(struct snd_dma_buffer * dmab, size_t размер)`
выделение памяти из встроенной в микросхему оперативной памяти

Параметры

`структура snd_dma_buffer * dmab` запись о выделении буфера для хранения
выделенных данных `size_t размер` количество байтов, выделяемых из `iram`

Описание

Для этой функции требуется `iram phandle`, предоставленный через `of_node`

`void snd_free_dev_iram(struct snd_dma_buffer * dmab)`
освободить выделенную специальную память из встроенной в микросхему оперативной памяти

Параметры

`struct snd_dma_buffer * dmab` запись о выделении буфера для хранения выделенных данных `int`
`snd_dma_alloc_pages(int Tun, структура устройства * device, размер_t размер, struct snd_dma_buffer`
`* dmab)`
выделите область буфера в соответствии с заданным типом

Параметры

`тип int` тип буфера DMA
`структура device * устройство` указатель устройства
`размер_t size` размер выделяемого буфера
`структура snd_dma_buffer * dmab` запись о выделении буфера для хранения
выделенных данных

Описание

Вызывает функцию распределения памяти для соответствующего типа буфера.

Возврат

Ноль, если буфер заданного размера выделен успешно, в противном случае отрицательное значение при ошибке.

`int snd_dma_alloc_pages_fallback(int Tun, структурное устройство * устройство, размер_t размер, struct`
`snd_dma_buffer * dmab)`
выделите область буфера в соответствии с заданным типом с помощью резервных параметров

Параметры

`тип int` тип буфера DMA
`структура device * устройство` указатель устройства
`размер_t size` размер буфера для выделения
`структура snd_dma_buffer * dmab` запись о выделении буфера для хранения
выделенных данных

Описание

Вызывает функцию распределения памяти для соответствующего типа буфера. Когда свободного места не остается, эта функция уменьшает размер и пытается выделить снова. Фактически выделенный размер сохраняется в аргументе `res_size`.

Возврат

Ноль, если буфер с заданным размером выделен успешно, в противном случае отрицательное значение при ошибке. void `snd_dma_free_pages`(struct `snd_dma_buffer` * *dmab*)
освободить выделенный буфер

Параметры

struct `snd_dma_buffer` * *dmab* запись о выделении буфера для освобождения
Описание
Освобождает выделенный буфер с помощью функции `snd_dma_alloc_pages()`.

1.1.2 PCM API

Ядро PCM

постоянный символ * `snd_pcm_format_name`(`snd_pcm_format_t` *format*)
Возвращает строку имени для данного формата PCM

Параметры

format `snd_pcm_format_t` Формат PCM
int
`snd_pcm_new_stream`(структура `snd_pcm` * *pcm*, int *stream*, int *substream_count*)
создайте новый поток PCM

Параметры

struct `snd_pcm` * *pcm* экземпляр pcm
int *stream* направление потока, SNDRV_PCM_STREAM_XXX
int *substream_count* количество подпотоков

Описание

Создает новый поток для pcm. Соответствующий поток в pcm должен был быть пустым перед вызовом этого, т.е. аргументу должно быть присвоено ноль `snd_pcm_new()`.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое. int
`snd_pcm_new`(структура `snd_card` * *card*, постоянный символ * *id*, int *device*, int *playback_count*, int *capture_count*, структура `snd_pcm` ** *rpcm*)
создайте новый экземпляр PCM

Параметры

структура `snd_card` * *card* экземпляр card
const char * *id* строка id
int *device* индекс устройства (на основе нуля)
int *playback_count* количество подпотоков для воспроизведения
int *capture_count* количество подпотоков для захвата
структура `snd_pcm` * * *rpcm* указатель для хранения нового экземпляра pcm
Описание
Создает новый экземпляр PCM.

Операторы pcm должны быть впоследствии установлены для нового экземпляра с помощью `snd_pcm_set_ops()`.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int `snd_pcm_new_internal` (struct `snd_card` * *карточка*, постоянный символ * *id*, int *device*, int *playback_count*,
int *capture_count*, struct `snd_pcm` ** *rpcm*)
создайте новый внутренний экземпляр PCM

Параметры

структура `snd_card` * `card` экземпляр `card`
постоянный символ * `id` строка `id`
int `device` индекс устройства (на основе нуля - используется совместно
с обычными PCMS) int `playback_count` количество подпотоков для воспроизведения
int `capture_count` количество подпотоков для захвата
структура `snd_pcm` * * `rpcm` указатель для хранения нового экземпляра

pcm Описание

Создает новый внутренний экземпляр PCM без записей пользовательского пространства `device` или `procsfs`. Это используется ASoC серверными PCMS для создания PCM, который будет использоваться только внутри драйверов ядра. т. е. Он не может быть открыт в пользовательском пространстве. Он предоставляет существующим драйверам ASoC components подпоток и доступ к любым личным данным. Впоследствии операторы `pcm` должны быть настроены на новый экземпляр с помощью `snd_pcm_set_ops()`.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.
int `snd_pcm_notify` (структура `snd_pcm_notify` * *уведомлять*, int *nfree*)
Добавить / удалить список уведомлений

Параметры

struct `snd_pcm_notify` * *уведомлять* Список
уведомлений PCM int `nfree` 0 = зарегистрироваться,
1 = отменить регистрацию

Описание

Это добавляет данный уведомитель в глобальный список, так что обратный вызов вызывается для каждого зарегистрированного устройства PCM. Пока это существует только для эмуляции PCM OSS. void `snd_pcm_set_ops` (struct `snd_pcm` * *pcm*, int *направление*, const struct `snd_pcm_ops` * *ops*)
задайте операторы PCM

Параметры

struct `snd_pcm` * `pcm` экземпляр `pcm`
направление ввода направление потока,
SNDRV_PCM_STREAM_XXX постоянная структура `snd_pcm_ops` * `ops` таблица
операторов

Описание

Устанавливает указанные операторы PCM для экземпляра `pcm`.
void `snd_pcm_set_sync` (struct `snd_pcm_substream` * *подпоток*)
задайте идентификатор синхронизации PCM

Параметры

структура `snd_pcm_substream` * `подпоток` подпоток `pcm`

Описание

Устанавливает идентификатор синхронизации PCM для карты.

`int snd_interval_refine` (структура `snd_interval *i`, `const struct snd_interval`, постоянная структура `snd_interval *v`)
уточните значение интервала в конфигураторе

Параметры

`struct snd_interval * i` значение интервала для уточнения

`const struct snd_interval * v` значение интервала, на

которое нужно ссылаться **Описание**

Уточняет значение интервала с помощью опорного значения. Интервал изменяется на диапазон, удовлетворяющий обоим интервалам. Оценивается состояние интервала (минимальный, максимальный, целое число и т.д.).

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный код ошибки. `void`

`snd_interval_div` (`const struct snd_interval *a`, `const struct snd_interval`, постоянная структура `snd_interval *b`, структура `snd_interval *c`)

уточните значение интервала с помощью деления

Параметры

`постоянная структура snd_interval * a` дивиденд

`постоянная структура snd_interval * b` делитель

`struct snd_interval * c` частное

Описание

$c = a / b$

Возвращает ненулевое значение, если значение изменено, и ноль, если не изменено.

`void snd_interval_muldivk` (`постоянная структура snd_interval *a`, `const struct snd_interval`, `постоянная структура snd_interval *b`, без знака `int k`, `struct snd_interval`, структура `snd_interval *c`)
уточните значение интервала

Параметры

`постоянная структура snd_interval * a` делимое 1

`постоянная структура snd_interval * b` делимое 2

`число без знака k` делитель (как целое число)

`структура snd_interval * c` Результат

Описание

$c = a * b / k$

Возвращает ненулевое значение, если значение изменено, и ноль, если не изменено.

`void snd_interval_mulkdiv` (`постоянная структура snd_interval *a`, беззнаковый `int k`, `const struct snd_interval`, `постоянная структура snd_interval *b`, `struct snd_interval`, структура `snd_interval *c`)
уточните значение интервала

Параметры

`const struct snd_interval * a` dividend 1

`unsigned int k` делимое 2 (как целое число)

`постоянная структура snd_interval * b` делитель

`структура snd_interval * c` Результат

Описание

$c = a * k / b$

Возвращает ненулевое значение, если значение изменено, и ноль, если не изменено.

int `snd_interval_ratnum`(struct `snd_interval` **я*, unsigned int *rats_count*, const struct `snd_ratnum` **rats*, unsigned int **nump*, unsigned int **denp*)
уточните значение интервала

Параметры

struct `snd_interval` * *i* интервал для уточнения
беззнаковый int *rats_count* число `ratnum_t`
постоянная структура `snd_ratnum` * *rats* массив `ratnum_t`
unsigned int * *nump* указатель для хранения результирующего
числителя unsigned int * *denp* указатель для сохранения
результирующего знаменателя

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный код ошибки.

int `snd_interval_ratden`(структура `snd_interval` **я*, unsigned int *rats_count*, const struct `snd_ratden` **rats*, unsigned int **nump*, unsigned int **denp*)
уточните значение интервала

Параметры

структура `snd_interval` * *i* интервал для уточнения
unsigned int *rats_count* номер структуры `ratden`
постоянная структура `snd_ratden` * *rats* массив struct `ratden`
беззнаковый int * *nump* указатель для сохранения результирующего
числителя беззнаковый int * *denp* указатель для
сохранения результирующего знаменателя

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный код ошибки.

int `snd_interval_list`(struct `snd_interval` **я*, беззнаковый int *count*, const без знака int * *Список*, без знака int *Маска*)
уточните значение интервала из списка

Параметры

struct `snd_interval` * *i* значение интервала для
уточнения значение unsigned int *count* количество элементов в
списке const unsigned int * *list* список значений
unsigned int *mask* битовая маска для вычисления

Описание

Уточняет значение интервала из списка. Когда маска отлична от нуля, вычисляются только элементы, соответствующие биту 1.

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный код ошибки. int `snd_interval_ranges`(struct `snd_interval` **я*, unsigned int *count*, const struct `snd_interval` **диапазоны*, unsigned int *маска*)
уточните значение интервала из списка диапазонов

Параметры

struct `snd_interval` * *i* значение интервала для уточнения
количество целых чисел без знака количество элементов в списке диапазонов
const struct `snd_interval` * *диапазоны* список диапазонов

`маска unsigned int` битовая

маска для оценки **Описание**

Уточняет значение интервала из списка диапазонов. Когда маска отлична от нуля, вычисляются только элементы, соответствующие биту 1.

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный код

ошибки. `int snd_pcm_hw_rule_add(struct snd_pcm_runtime * среда выполнения, unsigned int cond, int var,`

`snd_pcm_hw_rule_func_t функция, недействительный * Частное, int dep, ...)`

добавьте правило hw-constraint

Параметры

`структура snd_pcm_runtime * среда выполнения` экземпляр

среды выполнения PCM `unsigned int cond` биты условия

`int var` переменная для вычисления

`snd_pcm_hw_rule_func_t func` функция вычисления

`void * private` указатель на личные данные, переданный

функции `int dep` зависимые переменные

■ ■ ■ аргументы переменных

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_pcm_hw_constraint_mask(struct snd_pcm_runtime * время выполнения, snd_pcm_hw_param_t var,
u_int32_t маска)`

примените данное ограничение растровой маски

Параметры

`struct snd_pcm_runtime * среда выполнения` Экземпляр среды

выполнения PCM `snd_pcm_hw_param_t var` переменная

hw_params для применения маски `маска u_int32_t` растровая

маска **Описание**

Примените ограничение данной растровой маски к

32-битному параметру маски. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_pcm_hw_constraint_mask64(struct snd_pcm_runtime * среда выполнения, snd_pcm_hw_param_t var,
u_int64_t маска)`

примените данное ограничение растровой маски

Параметры

`структура snd_pcm_runtime * среда выполнения` Экземпляр среды выполнения

PCM параметр `snd_pcm_hw_param_t` переменная hw_params

для применения маски `маска u_int64_t` 64-битная растровая маска

Описание

Примените ограничение данной растровой маски к параметру

64-битной маски. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_pcm_hw_constraint_integer(структура

snd_pcm_runtime

* время выполнения,

snd_pcm_hw_param_t var)

применение целочисленного ограничения к интервалу

Параметры

struct snd_pcm_runtime * среда выполнения Экземпляр среды выполнения PCM

snd_pcm_hw_param_t var переменная hw_params для применения целочисленного ограничения

Описание

Примените ограничение integer к параметру interval.

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный код ошибки.

int snd_pcm_hw_constraint_minmax(struct snd_pcm_runtime * среда выполнения, snd_pcm_hw_param_t var,

unsigned int min, unsigned int max)

примените ограничение диапазона min / max к интервалу

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр

среды выполнения PCM snd_pcm_hw_param_t var переменная

hw_params для применения диапазона unsigned int min минимальное

значение unsigned int max максимальное значение

Описание

Примените ограничение минимального / максимального

диапазона к параметру интервала. **Возврат**

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный

код ошибки. int snd_pcm_hw_constraint_list(структура snd_pcm_runtime * среда выполнения,

snd_pcm_hw_param_t var, int cond, const struct

snd_pcm_hw_constraint_list * l)

применить список ограничений к параметру

Параметры

struct snd_pcm_runtime * среда выполнения Экземпляр среды выполнения PCM

unsigned int cond биты условий

параметр snd_pcm_hw_param_t переменная hw_params для применения

ограничения списка постоянная структура snd_pcm_hw_constraint_list * l Список

Описание

Примените список ограничений к параметру интервала.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_pcm_hw_constraint_ranges(структура snd_pcm_runtime

int cond, snd_pcm_hw_param_t var, const struct

snd_pcm_hw_constraint_ranges * r)

применить список ограничений диапазона к параметру

* время выполнения, без знака

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр

среды выполнения PCM unsigned int cond биты условий

`snd_pcm_hw_param_t` переменная переменная `hw_params` для применения списка
ограничений диапазона постоянная структура `snd_pcm_hw_constraint_ranges` * `r` диапазоны

Описание

Примените список ограничений диапазона к параметру интервала.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_pcm_hw_constraint_ratnums` (структура `snd_pcm_runtime` * *время выполнения*, без знака
`int cond`, `snd_pcm_hw_param_t var`, `const` структура
*`snd_pcm_hw_constraint_ratnums` * `r`*)
применить ограничение `ratnums` к параметру

Параметры

структура `snd_pcm_runtime` * среда выполнения Экземпляр среды выполнения PCM
`unsigned int cond` биты условий
`snd_pcm_hw_param_t var` переменная `hw_params` для применения ограничения
`ratnums` `const struct snd_pcm_hw_constraint_ratnums` * `r` константы `struct`
`snd_ratnums` **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_pcm_hw_constraint_rattdens` (структура `snd_pcm_runtime` * *время выполнения*, без знака
`int cond`, `snd_pcm_hw_param_t var`, `const` структура
*`snd_pcm_hw_constraint_rattdens` * `r`*)
примените ограничение `ratdens` к параметру

Параметры

структура `snd_pcm_runtime` * среда выполнения Экземпляр среды выполнения PCM
`unsigned int cond` биты условий
параметр `snd_pcm_hw_param_t` переменная `hw_params` для применения ограничения
`ratdens` постоянная структура `snd_pcm_hw_constraint_rattdens` * `r` константы
`struct snd_rattdens` **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_pcm_hw_constraint_msbits` (`struct snd_pcm_runtime` * *время выполнения*, `unsigned int конд`, un-
значение со знаком `int ширина`, без знака `int msbits`)
добавьте правило `msbits` с ограничением `hw`

Параметры

`struct snd_pcm_runtime` * среда выполнения Экземпляр
среды выполнения PCM `unsigned int cond` биты условия
`unsigned int width` ширина выборки битов
`unsigned int msbits` ширина в `msbits`

Описание

Это ограничение установит количество наиболее значимых битов
(`msbits`), если был выбран формат выборки с указанной шириной. Если
значение `width` равно 0, то `msbits` будут установлены для любого
формата выборки с шириной, большей, чем указанные `msbits`. **Возврат**
Ноль в случае успеха или отрицательный код ошибки при сбое.

int `snd_pcm_hw_constraint_step` (структура `snd_pcm_runtime` * *среда выполнения*, `snd_pcm_hw_param_t var`, без знака long *шаг*)
добавьте правило шага ограничения `hw`

Параметры

struct `snd_pcm_runtime` * *среда выполнения* Экземпляр среды выполнения
PCM unsigned int `cond` биты условий
параметр `snd_pcm_hw_param_t` переменная `hw_params` для применения
ограничения шага длинный шаг без знака размер шага

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое. int
`snd_pcm_hw_constraint_pow` 2 (структура `snd_pcm_runtime` * *среда выполнения*, `snd_pcm_hw_param_t var`)
без знака int *cond*,
добавление правила степени 2 для ограничения `hw`

Параметры

struct `snd_pcm_runtime` * *среда выполнения* Экземпляр среды выполнения PCM
unsigned int `cond` биты условий
параметр `snd_pcm_hw_param_t` переменная `hw_params` для применения
ограничения степени 2

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.
int `snd_pcm_hw_rule_noresample` (struct `snd_pcm_runtime` * *среда выполнения*, unsigned int *base_rate*)
добавьте правило, позволяющее отключить повторную выборку `hw`

Параметры

структура `snd_pcm_runtime` * *время выполнения* Экземпляр среды выполнения
PCM unsigned int `base_rate` скорость, с которой аппаратное обеспечение не
выполняет повторную выборку

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.
int `snd_pcm_hw_param_value` (const struct `snd_pcm_hw_params` * *параметры*,
`snd_pcm_hw_param_t var`, int * *реж*)
Возврат **параметры** поле **var** значение

Параметры

const struct `snd_pcm_hw_params` * *параметры* экземпляр `hw_params`
`snd_pcm_hw_param_t` переменная параметр для извлечения
int * `dir` указатель на направление (-1,0,1) или NULL

Возврат

Значение для поля **var** если это исправлено в пространстве конфигурации, определяемом **params**. -EINVAL в
противном случае. int `snd_pcm_hw_param_first` (struct `snd_pcm_substream` * *pcm*, struct `snd_pcm_hw_params` * *параметры*,
`snd_pcm_hw_param_t var`, int * *каталог*)
уточните пространство конфигурации и верните минимальное значение

Параметры

структура `snd_pcm_substream` * *pcm* Экземпляр PCM
struct `snd_pcm_hw_params` * *параметры* экземпляр `hw_params`
`snd_pcm_hw_param_t` var параметр для извлечения

int * dir указатель на направление (-1,0,1) или NULL

Описание

Внутри конфигурационного пространства, определяемого параметрами удалить из var все значения > минимальные. Соответствующим образом сократите пространство для настройки., Возврат

Минимальный или отрицательный код ошибки при сбое.

int snd_pcm_hw_param_last(struct snd_pcm_substream * pcm, struct snd_pcm_hw_params * параметры, snd_pcm_hw_param_t var, int * каталог)
уточните пространство конфигурации и верните максимальное значение

Параметры

структура snd_pcm_substream * pcm Экземпляр PCM
struct snd_pcm_hw_params * параметры экземпляр hw_params
snd_pcm_hw_param_t переменная параметр для извлечения
int * dir указатель на направление (-1,0,1) или NULL

Описание

Внутри конфигурационного пространства, определяемого параметрами удалить из var все значения < максимальные. Соответствующим образом сократите пространство для настройки. Возврат

Максимальный или отрицательный код ошибки при сбое. int

snd_pcm_lib_ioctl(struct snd_pcm_substream * подпоток, unsigned int cmd, void * аргумент)
общий обратный вызов PCM ioctl

Параметры

структура snd_pcm_substream * подпоток экземпляр подпотока pcm
unsigned int cmd команда ioctl
void * arg аргумент ioctl

Описание

Обрабатывает общие команды ioctl для PCM. Может передаваться как обратный вызов ioctl для операций PCM. Возврат
Ноль в случае успеха или отрицательный код ошибки при сбое. void

snd_pcm_period_elapsed(struct snd_pcm_substream * подпоток)
обновите статус pcm для следующего периода

Параметры

структура snd_pcm_substream * подпоток экземпляр подпотока pcm

Описание

Эта функция вызывается из обработчика прерываний, когда PCM обработает размер периода. Это обновит текущий указатель, разбудит спящих и т.д. Даже если с момента последнего вызова прошло более одного периода, вы должны вызвать это только один раз. int

snd_pcm_add_chmap_ctls(struct snd_pcm * pcm, int stream, const struct snd_pcm_chmap_elem * max_channels, unsigned long private_value, * chmap, структура snd_pcm_chmap ** info_ret)
создание элементов управления отображением каналов

Параметры

структура snd_pcm * pcm назначенный экземпляр PCM

поток ввода направление потока

постоянная структура snd_pcm_chmap_elem * chmap элементы карты каналов (для

запроса) int max_channels максимальное количество каналов для потока

длинное значение private_value без знака значение, передаваемое в поле

private_value каждого kcontrol struct snd_pcm_chmap * * info_ret сохранить экземпляр

struct snd_pcm_chmap, если значение не равно НУЛЮ

Описание

Создайте элементы управления отображением каналов, назначенные данному

поток(ам) PCM. **Возврат**

Ноль в случае успеха или отрицательное значение ошибки.

void snd_pcm_stream_lock(struct snd_pcm_substream * *подпоток*)

Блокировка потока PCM

Параметры

структура snd_pcm_substream * подпоток Подпоток

PCM **Описание**

Это блокирует спин-блокировку или мьютекс потока PCM в зависимости от неатомного флага

данного подпотока. Это также требует глобальной блокировки rw канала (или rw sem), чтобы избежать

гонки со связанными потоками. void snd_pcm_stream_unlock(struct snd_pcm_substream * *подпоток*)

Разблокировка потока PCM

Параметры

структура snd_pcm_substream * подпоток Подпоток

PCM **Описание**

Это разблокирует поток PCM, который был заблокирован с помощью *snd_pcm_stream_lock()*.

void snd_pcm_stream_lock_irq(структура snd_pcm_substream * *подпоток*)

Заблокируйте поток PCM

Параметры

структура snd_pcm_substream * подпоток Подпоток

PCM **Описание**

Это блокирует поток PCM следующим образом *snd_pcm_stream_lock()* и отключает локальный IRQ

(только когда неатомный имеет значение false). В неатомном случае это идентично *snd_pcm_stream_lock()*.

void snd_pcm_stream_unlock_irq(структура snd_pcm_substream * *подпоток*)

Разблокируйте поток PCM

Параметры

структура snd_pcm_substream * подпоток Подпоток PCM

Описание

Это противоположная часть функции *snd_pcm_stream_lock_irq()*.

void snd_pcm_stream_unlock_irqrestore(struct snd_pcm_substream * *подпоток*, unsigned long flags)

Разблокировка потока PCM

Параметры

struct snd_pcm_substream * подпоток Подпоток

PCM длинные флаги без знака флаги irq

Описание

Это противоположная часть функции `snd_pcm_stream_lock_irqsave()`.

```
int snd_pcm_hw_params_choose(struct snd_pcm_substream *pcm, struct snd_pcm_hw_params
                             *параметры)
    выберите конфигурацию, определенную с помощью параметров
```

Parameters

```
struct snd_pcm_substream * pcm    Экземпляр PCM
struct snd_pcm_hw_params * параметры    экземпляр hw_params
```

Описание

Выберите одну конфигурацию из конфигурационного пространства, определенного с помощью **params**.
Выбранная конфигурация такова, что получается исправление в следующем порядке: первый доступ,
первый формат, первый подформат, минимальное количество каналов, минимальная скорость,
минимальный период времени, максимальный размер буфера, минимальное время тактирования
Возврат
Ноль в случае успеха или отрицательный код ошибки при сбое.

```
int snd_pcm_start(struct snd_pcm_substream * подпоток)
    запустить все связанные потоки
```

Параметры

```
структура snd_pcm_substream * подпоток    экземпляр
подпотока PCM
```

Возврат

Ноль в случае успеха или отрицательный код ошибки. Блокировка потока должна быть установлена перед вызовом этой функции.

```
int snd_pcm_stop(struct snd_pcm_substream * подпоток, snd_pcm_state_t состояние)
    попытка остановить все запущенные потоки в группе подпотоков
```

Параметры

```
структура snd_pcm_substream * подпоток    экземпляр подпотока PCM
состояние snd_pcm_state_t    Состояние PCM после остановки потока
```

Описание

Затем состояние каждого потока безоговорочно изменяется на заданное.

Возврат

Ноль в случае успеха или отрицательный код ошибки.

```
int snd_pcm_drain_done(struct snd_pcm_substream * подпоток)
    останавливает DMA только при воспроизведении данного
потока
```

Параметры

```
struct snd_pcm_substream * подпоток    подпоток PCM
```

Описание

После остановки состояние изменяется на SETUP. В отличие от `snd_pcm_stop()`,
это влияет только на данный поток.

Возврат

Ноль в случае успеха или отрицательный код ошибки.

```
int snd_pcm_stop_xrun(struct snd_pcm_substream * подпоток)
    останавливает текущие потоки как XRUN
```

Параметры

```
struct snd_pcm_substream * подпоток    экземпляр подпотока PCM
```

Описание

Это останавливает данный запущенный подпоток (и все связанные подпоток) как XRUN . В отличие от `snd_pcm_stop()`, эта функция сама принимает блокировку подпотока.

Возврат

Ноль в случае успеха или отрицательный код ошибки.

`int snd_pcm_suspend(struct snd_pcm_substream * подпоток)`
запускает ПРИОСТАНОВКУ для всех связанных потоков

Параметры

`структура snd_pcm_substream * подпоток` подпоток PCM

Описание

После этого вызова все потоки переводятся в

ПРИОСТАНОВЛЕННОЕ состояние. **Возврат**

Ноль в случае успеха (или **подпоток** равен нулю), или отрицательный

код ошибки. `int snd_pcm_suspend_all(struct snd_pcm * pcm)`
запускает ПРИОСТАНОВКУ для всех подпотоков в данном pcm

Параметры

`структура snd_pcm * pcm` экземпляр PCM

Описание

После этого вызова все потоки переводятся в ПРИОСТАНОВЛЕННОЕ

состояние. **Возврат**

Ноль в случае успеха (или **pcm** равно NULL), или отрицательный код ошибки.

`int snd_pcm_prepare(struct snd_pcm_substream * подпоток, файл структуры * файл)`
подготовьте подпоток PCM к запуску

Параметры

`struct snd_pcm_substream * подпоток` экземпляр подпотока PCM

`файл структуры * file` файл для ссылки на f_flags

Возврат

Ноль в случае успеха или отрицательный код ошибки.

`int snd_pcm_kernel_ioctl(struct snd_pcm_substream * подпоток, unsigned int cmd, void * аргумент)`
Выполнить PCM ioctl в пространстве ядра

Параметры

`структура snd_pcm_substream * подпоток` Подпоток PCM

`unsigned int cmd` IOCTL cmd

`void * arg` Аргумент IOCTL

Описание

Функция предоставляется в основном для драйверов уровня OSS и USB-гаджетов, и она допускает только ограниченный набор ioctls (hw_params, sw_params, prepare, start, drain, drop, forward). `int snd_pcm_lib_default_mmap(структура snd_pcm_substream * подпоток, структура vm_area_struct * область)`
Функция mmap данных PCM по умолчанию

Параметры

`структура snd_pcm_substream * подпоток` Подпоток PCM

структура vm_area_struct * область VMA

Описание

Это обработчик mmap по умолчанию для данных PCM. Когда значение mmap pcm_ops равно нулю, эта функция вызывается неявно. int

snd_pcm_lib_mmap_iomem(структура snd_pcm_substream * *подпоток*, структура vm_area_struct * *область*)

Функция mmap данных PCM по умолчанию для тем ввода-вывода

Параметры

структура snd_pcm_substream * подпоток Подпоток PCM

структура vm_area_struct * область VMA

Описание

Когда ваше оборудование использует iomapped pages в качестве аппаратного буфера и хочет mmap его, передайте эту функцию как mmap pcm_ops. Обратите внимание, что это должно работать только на ограниченных архитектурах. int snd_pcm_stream_linked(struct snd_pcm_substream * *подпоток*)

Проверьте, связан ли подпоток с другими

Параметры

struct snd_pcm_substream * подпоток подпоток для проверки

Описание

Возвращает true, если данный подпоток связан с другими.

snd_pcm_stream_lock_irqsave(*подпоток*, *флаги*)

Блокировка потока

PCM Параметры

подпоток подпоток PCM

флаги флаги irq

Описание

Это блокирует поток PCM, как *snd_pcm_stream_lock()* но с локальным IRQ (только когда nonatomic имеет значение false). В неатомном случае это идентично значению *snd_pcm_stream_lock()*.

snd_pcm_group_for_each_entry(*s*, *подпоток*)

перебор связанных подпотоков

Параметры

s итератор

подпоток подпоток

Описание

Выполните итерацию по всем связанным подпотокам с заданным **подпотоком**. Когда **подпоток** не связан ни с одним другим, это возвращает результат **подпоток** сам по себе один раз.

int snd_pcm_running(struct snd_pcm_substream * *подпоток*)

Проверяет, находится ли подпоток в запущенном состоянии

Параметры

struct snd_pcm_substream * подпоток подпоток

для проверки **Описание**

Возвращает true, если данный подпоток находится в состоянии RUNNING или в состоянии DRAINING для воспроизведения.

ssize_t bytes_to_samples(структура snd_pcm_runtime * *время выполнения*, ssize_t *размер*)

Преобразование единицы измерения размера из байтов в выборки

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды выполнения

PCM размер ssize_t размер в байтах snd_pcm_sframes_t

bytes_to_frames(структура snd_pcm_runtime * *время выполнения*, ssize_t *размер*)

Преобразование единицы измерения размера из байтов в фреймы

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды выполнения

PCM размер ssize_t размер в байтах ssize_t

samples_to_bytes(структура snd_pcm_runtime * *время выполнения*, ssize_t *размер*)

Преобразование единицы измерения размера из выборок в байты

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды выполнения PCM

размер ssize_t размер в выборках ssize_t

frames_to_bytes(структура snd_pcm_runtime * *время выполнения*, snd_pcm_sframes_t *размер*)

Преобразование единицы измерения размера из кадров в байты

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды

выполнения PCM snd_pcm_sframes_t размер размер во фреймах int

frame_aligned(структура snd_pcm_runtime * *время выполнения*, ssize_t *байты*)

Проверьте, соответствует ли размер байта фреймам

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды

выполнения PCM ssize_t байты размер в байтах

size_t snd_pcm_lib_buffer_bytes(структура snd_pcm_substream * *подпоток*)

Получаем размер буфера текущего PCM в байтах

Параметры

структура snd_pcm_substream * подпоток Подпоток PCM size_t

snd_pcm_lib_period_bytes(struct snd_pcm_substream * *подпоток*)

Получить размер периода текущего PCM в байтах

Параметры

структура snd_pcm_substream * подпоток Подпоток PCM

snd_pcm_uframes_t snd_pcm_playback_avail(struct snd_pcm_runtime * *среда выполнения*)

Получение доступного (для записи) пространства для воспроизведения

Параметры

struct snd_pcm_runtime * среда выполнения Экземпляр

среды выполнения PCM **Описание**

Результат находится в диапазоне 0 ... (граница - 1)

snd_pcm_uframes_t snd_pcm_capture_avail(struct snd_pcm_runtime * *среда выполнения*)

Получает доступное (читаемое) пространство для записи

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды выполнения PCM

Описание

Результат находится в диапазоне 0 ... (граница - 1) snd_pcm_sframes_t
snd_pcm_playback_hw_avail (struct snd_pcm_runtime * *среда выполнения*)
Получение места в очереди для воспроизведения

Параметры

struct snd_pcm_runtime * *среда выполнения* Экземпляр среды выполнения PCM
snd_pcm_sframes_t snd_pcm_capture_hw_avail (структура snd_pcm_runtime * *среда выполнения*)
Освободите место для записи

Параметры

структура snd_pcm_runtime * *среда выполнения* Экземпляр среды
выполнения PCM int snd_pcm_playback_ready (struct snd_pcm_substream * *подпоток*)
проверьте, доступен ли буфер воспроизведения

Параметры

struct snd_pcm_substream * *подпоток* экземпляр подпотока pcm

Описание

Проверяет, достаточно ли свободного места в буфере воспроизведения.

Возврат

Ненулевой, если доступен, или нулевой, если нет.

int snd_pcm_capture_ready (struct snd_pcm_substream * *подпоток*)
проверьте, доступен ли буфер захвата

Параметры

структура snd_pcm_substream * *подпоток* экземпляр подпотока pcm

Описание

Проверяет, достаточно ли данных захвата доступно в буфере захвата.

Возврат

Ненулевой, если доступен, или нулевой, если нет.

int snd_pcm_playback_data (struct snd_pcm_substream * *подпоток*)
проверяет, существуют ли какие-либо данные в буфере воспроизведения

Параметры

структура snd_pcm_substream * *подпоток* экземпляр подпотока pcm

Описание

Проверяет, существуют ли какие-либо данные в буфере

воспроизведения. **Возврат**

Ненулевой, если какие-либо данные существуют, или нулевой, если нет. Если значение
stop_threshold больше или равно boundary, то эта функция всегда возвращает ненулевое
значение. int snd_pcm_playback_empty (struct snd_pcm_substream * *подпоток*)
проверяет, пуст ли буфер воспроизведения

Параметры

структура snd_pcm_substream * *подпоток* экземпляр подпотока pcm

Описание

Проверяет, пуст ли буфер воспроизведения.

Возврат

Ненулевой, если пустой, или нулевой, если нет.

int snd_pcm_capture_empty(struct snd_pcm_substream * *подпоток*)
 проверьте, пуст ли буфер захвата.

Параметры

структура snd_pcm_substream * *подпоток* экземпляр подпотока pcm

Описание

Проверяет, пуст ли буфер захвата.

Возврат

Ненулевой, если пустой, или нулевой, если нет.

void snd_pcm_trigger_done(struct snd_pcm_substream * *подпоток*, struct snd_pcm_substream * *master*)
 Отметьте основной подпоток

Параметры

структура snd_pcm_substream * *подпоток* экземпляр подпотока pcm

структура snd_pcm_substream * *master* связанный основной подпоток

Описание

Когда подключено несколько подпотоков одной и той же карты и аппаратное обеспечение поддерживает работу с одним выстрелом, драйвер вызывает это в цикле в *snd_pcm_group_for_each_entry()* для пометки подпотока как "готово". Тогда большинство операций запуска выполняются только для данного основного подпотока. Отметка *trigger_master* стирается при обновлении временной метки в конце операций запуска. unsigned int *params_channels* (постоянная структура snd_pcm_hw_params * *p*)
 Получите количество каналов из параметров hw

Параметры

постоянная структура snd_pcm_hw_params * *p* параметры hw
unsigned int *params_rate* (постоянная структура snd_pcm_hw_params * *p*)
 Получаем частоту дискретизации из параметров hw

Parameters

const struct snd_pcm_hw_params * *p* параметры hw unsigned int
params_period_size (постоянная структура snd_pcm_hw_params * *p*)
 Получите размер периода (в кадрах) из параметров hw

Параметры

постоянная структура snd_pcm_hw_params * *p* параметры hw
unsigned int *params_periods* (постоянная структура snd_pcm_hw_params * *p*)
 Получите количество периодов из параметров hw

Параметры

const struct snd_pcm_hw_params * *p* параметры hw unsigned int
params_buffer_size (постоянная структура snd_pcm_hw_params * *p*)
 Получите размер буфера (в кадрах) из параметров hw

Параметры

постоянная структура snd_pcm_hw_params * p параметры hw
unsigned int params_buffer_bytes(постоянная структура snd_pcm_hw_params * p)
Получаем размер буфера (в байтах) из параметров hw

Parameters

const struct snd_pcm_hw_params * p hw params int
snd_pcm_hw_constraint_single(структура snd_pcm_runtime * время выполнения, snd_pcm_hw_param_t var,
unsigned int значение val)
Ограничить параметр одним значением

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды выполнения
PCM snd_pcm_hw_param_t var Переменная hw_params для
ограничения unsigned int val Значение, которое нужно ограничить

Возврат

Положительный, если значение изменено, нулевой, если оно не изменено, или отрицательный
код ошибки. int snd_pcm_format_cpu_endian(snd_pcm_format_t форматировать)
Убедитесь, что формат PCM соответствует CPU-endian

Параметры

формат snd_pcm_format_t формат для проверки

Возврат

1, если заданный формат PCM является порядковым номером процессора, 0, если наоборот, или отрицательный код ошибки, если
порядковый номер не указан. недействительно snd_pcm_set_runtime_buffer(struct snd_pcm_substream * подпоток, struct
snd_dma_buffer * bufp)
Установите буфер времени выполнения PCM

Параметры

struct snd_pcm_substream * подпоток Требуется установить
подпоток PCM struct snd_dma_buffer * bufp информация о буфере,
значение NULL для очистки

Описание

Скопируйте информацию о буфере в runtime-> dma_buffer, когда значение **bufp** не
равно нулю. В противном случае он очищает информацию о текущем буфере. void
snd_pcm_gettime(struct snd_pcm_runtime * время выполнения, struct timespec * tv)
Заполните временную шкалу в зависимости от режима временной метки

Параметры

структура snd_pcm_runtime * среда выполнения Экземпляр среды выполнения PCM
структура timespec * tv временная спецификация для заполнения int
snd_pcm_lib_alloc_vmalloc_buffer(структура snd_pcm_substream * подпоток, размер_t размер)
выделение виртуального буфера DMA

Параметры

структура snd_pcm_substream * подпоток подпоток, которому нужно выделить
буфер size_t размер запрошенный размер буфера в байтах

Описание

Выделяет буфер подпотока PCM с помощью vmalloc(), т.е. память является
непрерывной в виртуальном пространстве ядра, но не в физической
памяти. Используйте это, если доступ к буферу осуществляется кодом ядра, но не

Возврат

1, если буфер был изменен, 0, если не изменен, или отрицательный код ошибки.

int snd_pcm_lib_alloc_vmalloc_3 2 _buffer(структура snd_pcm_substream * *подпоток*, размер_t *размер*)
выделение 32-разрядного адресуемого буфера

Параметры

структура snd_pcm_substream * *подпоток* подпоток, для которого будет выделен

буфер размер_t *size* запрошенный размер буфера в байтах

Описание

Эта функция работает как *snd_pcm_lib_alloc_vmalloc_buffer()*, но использует *vmalloc_3 2 ()*,
т.е. Страницы выделяются из памяти с 32-разрядным адресом.

Возврат

1, если буфер был изменен, 0, если не изменен, или отрицательный код ошибки.

dma_addr_t snd_pcm_sgbuf_get_addr(структура snd_pcm_substream * *подпоток*, unsigned int *ofs*)
Получает адрес DMA с соответствующим смещением

Параметры

структура snd_pcm_substream * *подпоток* Подпоток PCM

unsigned int *ofs* смещение в байтах void *

snd_pcm_sgbuf_get_ptr(структура snd_pcm_substream * *подпоток*, unsigned int *ofs*)
Получает виртуальный адрес с соответствующим смещением

Параметры

структура snd_pcm_substream * *подпоток* Подпоток PCM

unsigned int *ofs* смещение в байтах unsigned int

snd_pcm_sgbuf_get_chunk_size(структура snd_pcm_substream * *подпоток*, без знака
int ofs, без знака *int размер*)
Вычислите максимальный размер, который умещается в пределах contig.

страница заданного размера **Параметры**

структура snd_pcm_substream * *подпоток* Подпоток PCM

unsigned int *ofs* смещение в байтах

размер unsigned int размер в байтах для проверки

void snd_pcm_mmap_data_open(структура vm_area_struct * *область*)
увеличить счетчик mmap

Параметры

структура vm_area_struct * *область* VMA

Описание

Обратный вызов PCM mmap должен правильно обрабатывать этот счетчик

void snd_pcm_mmap_data_close(структура vm_area_struct * *область*)
уменьшить счетчик mmap

Параметры

структура vm_area_struct * *область* VMA

Описание

Обратный вызов PCM mmap должен правильно обрабатывать этот счетчик

недействительно `snd_pcm_limit_isa_dma_size`(int *dma*, size_t * *макс*)
Получите примерку максимального размера с помощью ISA DMA transfer

Параметры

int *dma* Номер DMA

size_t * *max* указатель для сохранения максимального размера

const char * *snd_pcm_stream_str*(структура `snd_pcm_substream` * *подпоток*)
Получаем строку, указывающую направление потока

Параметры

структура `snd_pcm_substream` * *подпоток* экземпляр подпотока `pcm`

Возврат

Строка, указывающая направление потока.

struct `snd_pcm_substream` * *snd_pcm_chmap_substream*(struct `snd_pcm_chmap` * *Информация*, без знака
int *idx*)

получить подпоток PCM, назначенный данной `chmap` info

Параметры

структура `snd_pcm_chmap` * *info* информация `chmap`

unsigned int *idx* индекс номера подпотока

u64 *pcm_format_to_bits*(`snd_pcm_format_t` *pcm_format*)

Преобразование `pcm_format` co

строгой типизацией в побитовый **Параметры**

`snd_pcm_format_t` *pcm_format* Формат PCM

Помощники формата PCM

int *snd_pcm_format_signed*(`snd_pcm_format_t` *форматировать*)

Проверьте, что формат PCM является
линейным со знаком **Параметры**

формат `snd_pcm_format_t` формат для проверки

Возврат

1, если данный формат PCM является линейным со знаком, 0, если
линейный без знака, и отрицательный код ошибки для нелинейных
форматов. int *snd_pcm_format_unsigned*(`snd_pcm_format_t` *форматировать*)

Убедитесь, что формат PCM является

линейным без знака **Параметры**

формат `snd_pcm_format_t` формат для проверки

Возврат

1, если данный формат PCM линейный без знака, 0, если
линейный со знаком, и отрицательный код ошибки для нелинейных
форматов. int *snd_pcm_format_linear*(`snd_pcm_format_t` *формат*)

Проверьте, является ли формат PCM линейным

Параметры

формат `snd_pcm_format_t` формат для проверки

Возврат

1, если данный формат PCM линейный, 0, если нет.

int `snd_pcm_format_little_endian`(snd_pcm_format_t *форматировать*)
Проверьте, является ли формат PCM строчным в конце строки

Параметры

формат `snd_pcm_format_t` формат для проверки

Возврат

1, если данный формат PCM имеет младший порядковый номер, 0, если большой порядковый номер, или

отрицательный код ошибки, если порядковый номер не указан. int `snd_pcm_format_big_endian`(snd_pcm_format_t *формат*)
Убедитесь, что формат PCM имеет большой конец

Параметры

формат `snd_pcm_format_t` формат для проверки

Возврат

1, если заданный формат PCM имеет порядковый номер, 0, если порядковый номер маленький, или

отрицательный код ошибки, если порядковый номер не указан. int `snd_pcm_format_width`(snd_pcm_format_t *форматировать*)
возвращает битовую ширину формата

Параметры

формат `snd_pcm_format_t` формат

для проверки **Возврат**

Разрядность формата или отрицательный код ошибки, если формат неизвестен.

int `snd_pcm_format_physical_width`(snd_pcm_format_t *форматировать*)
возвращает физическую

разрядность формата **Параметры**

формат `snd_pcm_format_t` формат

для проверки **Возврат**

Физическая разрядность формата или отрицательный код ошибки, если формат неизвестен.

ssize_t `snd_pcm_format_size`(snd_pcm_format_t *формат*, size_t *выборки*)
возвращает размер выборок в байтах в заданном

формате **Параметры**

формат `snd_pcm_format_t` формат для проверки

размер_t выборки частота дискретизации

Возврат

Размер в байтах данных выборок для формата или отрицательный код ошибки, если формат неизвестен.

const unsigned char * `snd_pcm_format_silence_64`(snd_pcm_format_t *форматировать*)
возвращает данные в режиме ожидания в массиве из 8 байт

Параметры

форматировать `snd_pcm_format_t` формат для

проверки **Возврат**

Шаблон формата для заполнения или `NULL` ошибка if.

int `snd_pcm_format_set_silence`(snd_pcm_format_t *форматировать*, void * *data*, unsigned int *образцы*)
установите данные для отключения в буфере

Параметры

формат `snd_pcm_format_t` формат PCM

данные `void *` указатель буфера

образцы `int` без знака количество сэмплов для установки режима тишины

Описание

Устанавливает данные о тишине в буфере для заданных выборок.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое. `int`

`snd_pcm_limit_hw_rates` (`struct snd_pcm_runtime * время выполнения`)

определите поля `rate_min/ rate_max`

Параметры

структура `snd_pcm_runtime *` среда выполнения экземпляр среды выполнения

Описание

Определяет поля `rate_min` и `rate_max` из битов `rates` данной среды выполнения->hw.

Возврат

Ноль в случае успеха.

`unsigned int` `snd_pcm_rate_to_rate_bit` (беззнаковый `int` *Оценить*)

преобразует частоту дискретизации в бит

SNDRV_PCM_RATE_xxx Параметры

`unsigned int rate` частота дискретизации для преобразования

Возврат

Флаг `SNDRV_PCM_RATE_xxx`, соответствующий заданной скорости, или `SNDRV_PCM_RATE_KNOT` для неизвестной скорости.

`unsigned int` `snd_pcm_rate_bit_to_rate` (`unsigned int rate_bit`)

преобразует бит `SNDRV_PCM_RATE_xxx` в частоту

дискретизации Параметры

`unsigned int rate_bit` бит скорости для преобразования

Возврат

Частота дискретизации, соответствующая заданному флагу `SNDRV_PCM_RATE_xxx`, или 0 для бита неизвестной частоты. `unsigned int` `snd_pcm_rate_mask_intersect` (значение без знака *rates_a*, `unsigned int rates_b`)

вычисляет пересечение двух масок ставок

Параметры

`unsigned int rates_a` Маска первой ставки

`unsigned int rates_b` Маска второго уровня

Описание

Эта функция вычисляет скорости, которые поддерживаются обеими масками скорости, переданными функции. Это позаботится о специальной обработке `SNDRV_PCM_RATE_CONTINUOUS` и `SNDRV_PCM_RATE_KNOT`.

Возврат

Маска скорости, содержащая скорости, поддерживаемые как `rates_a`, так и `rates_b`. `unsigned int` `snd_pcm_rate_range_to_bits` (`unsigned int rate_min`, `unsigned int rate_max`)

преобразует диапазон скоростей в бит `SNDRV_PCM_RATE_xxx`

Параметры

бесзнаковый int rate_min минимальная частота дискретизации
бесзнаковый int rate_max максимальная частота дискретизации

Описание

Эта функция имеет неявное допущение: тарифы в данном диапазоне имеют только заранее определенные тарифы, такие как 44100 или 16000.

Возврат

Флаг SNDRV_PCM_RATE_xxx, соответствующий заданному диапазону скоростей, или SNDRV_PCM_RATE_KNOT для неизвестного диапазона.

Управление памятью PCM

int snd_pcm_lib_preallocate_free (struct snd_pcm_substream * *подпоток*)
освободите предварительно выделенный буфер указанного подпотока.

Параметры

структура snd_pcm_substream * подпоток экземпляр подпотока pcm

Описание

Освобождает предварительно выделенный буфер данного подпотока.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_pcm_lib_preallocate_free_for_all (struct snd_pcm * *pcm*)
освободить все предварительно выделенные буферы на pcm

Параметры

структура snd_pcm * pcm экземпляр pcm

Описание

Освобождает все предварительно выделенные буферы на данном pcm.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_pcm_lib_preallocate_pages (struct snd_pcm_substream * *подпоток*, int *Тип*, структура-наоборот * *данные*, размер_t *размер*, size_t *max*)
предварительное выделение для данного типа DMA

Параметры

структура snd_pcm_substream * подпоток экземпляр подпотока pcm

тип int Тип DMA (SNDRV_DMA_TYPE_*)

устройство структуры * данные Данные, зависящие от типа DMA

размер size_t запрошенный размер предварительного выделения в байтах

размер size_t max макс. допустимый размер предварительного распределения

Описание

Выполните предварительное выделение для данного типа буфера DMA.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int `snd_pcm_lib_preallocate_pages_for_all`(структура `snd_pcm *pcm`, инт `Typ`, аннулирование данных * `данные`, размер_t `размер`, размер_t `макс`)

предварительное выделение для непрерывного типа памяти (все подпотoki) **Параметры**

структура `snd_pcm * pcm` экземпляр pcm
тип `int` Тип DMA (`SNDRV_DMA_TYPE_*`)
void * `данные` Данные, зависящие от типа DMA
size_t `размер` запрошенный размер предварительного выделения в байтах
size_t `макс` максимальный, разрешенный размер предварительного выделения **Описание**

Выполните предварительное распределение для всех подпотоков данного pcm для указанного типа DMA. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

структуры * `snd_pcm_sgbuf_ops_page`(структура `snd_pcm_substream` * `подпоток`, без знака длинный `смещение`)
получаем структуру страницы с заданным смещением

Параметры

структура `snd_pcm_substream * подпоток` экземпляр подпотока pcm
длинное `смещение без знака` смещение буфера

Описание

Используется в качестве обратного вызова страницы PCM ops.

Возврат

Структура страницы с заданным смещением буфера. `NULL` при сбое.

int `snd_pcm_lib_malloc_pages`(структура `snd_pcm_substream * подпоток`, размер_t `размер`)
выделение буфера DMA

Параметры

структура `snd_pcm_substream * подпоток` подпоток для выделения буфера DMA
size_t `размер` запрашиваемый размер буфера в байтах

Описание

Выделяет буфер DMA на ШИНЕ указанного ранее типа для `snd_pcm_lib_preallocate_xxx_pages()`.

Возврат

1, если буфер изменен, 0, если не изменен, или отрицательный код при сбое.

int `snd_pcm_lib_free_pages`(struct `snd_pcm_substream * подпоток`)
освободить выделенный буфер DMA.

Параметры

структура `snd_pcm_substream * подпоток` подпоток для освобождения буфера DMA

Описание

Освобождает буфер DMA, выделенный с помощью функции `snd_pcm_lib_malloc_pages()`.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_pcm_lib_free_vmalloc_buffer(структура snd_pcm_substream * *подпоток*)
свободный буфер vmalloc

Параметры

struct snd_pcm_substream * подпоток the подпоток C a буфер выделено Автор:
snd_pcm_lib_alloc_vmalloc_buffer()

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

структуры * snd_pcm_lib_get_vmalloc_page(структура snd_pcm_substream * *подпоток*, без знака
длинный смещение)

сопоставьте смещение буфера vmalloc со

структурой страницы **Параметры**

структура snd_pcm_substream * подпоток the подпоток C a буфер выделено Автор:
snd_pcm_lib_alloc_vmalloc_buffer()

длинное смещение без знака смещение в

буфере **Описание**

Эта функция должна использоваться в качестве

обратного вызова страницы в PCM ops. **Возврат**

Структура страницы, или NULL при сбое.

PCM DMA Engine API

int snd_hwparams_to_dma_slave_config(const struct snd_pcm_substream * *подпоток*,
const структура параметры snd_pcm_hw_params * *параметры*, struct
dma_slave_config * *slave_config*)

Преобразует hw_params в dma_slave_config

Параметры

постоянная структура snd_pcm_substream * подпоток Подпоток
PCM постоянная структура snd_pcm_hw_params * параметры hw_params
структура dma_slave_config * slave_config Конфигурация
подчиненного устройства DMA **Описание**

Эта функция может использоваться для инициализации dma_slave_config из подпотока и

hw_params в реализации драйвера PCM на основе dmaengine. void

snd_dmaengine_pcm_set_config_from_dai_data(постоянная структура snd_pcm_substream
* *подпоток*, struct const
snd_dmaengine_dai_dma_data
* *dma_data*, struct dma_slave_config
* *slave_config* (*подчиненный*))

Инициализирует конфигурацию подчиненного устройства dma, используя данные DAI DMA.

Параметры

const struct snd_pcm_substream * подпоток Подпоток PCM
постоянная структура snd_dmaengine_dai_dma_data * dma_data Данные
DAI DMA структура dma_slave_config * slave_config Конфигурация
ведомого устройства DMA **Описание**

Инициализирует поля {dst,src}_addr, {dst,src}_maxburst, {dst,src}_addr_width и slave_id
конфигурации DMA slave из тех же полей структуры данных DAI DMA. Поля src и dst будут инициализированы

в зависимости от направления подпотока. Если подпоток является потоком воспроизведения, то будут инициализированы поля dst, если это поток захвата, то будут инициализированы поля src. Поле {dst,src}_addr_width будет инициализировано только в том случае, если установлен флаг SND_DMAENGINE_PCM_DAI_FLAG_PACK или если поле addr_width структуры данных DAI DMA не равно DMA_SLAVE_BUSWIDTH_UNDEFINED. Если выполняются оба условия, то последнее имеет приоритет.

```
int snd_dmaengine_pcm_trigger(struct snd_pcm_substream * подпоток, int cmd)
```

реализация PCM-триггера на основе dmaengine

Параметры

структура snd_pcm_substream * подпоток Подпоток PCM

int cmd Команда запуска

Описание

Возвращает 0 при успешном выполнении, в противном случае - отрицательный код ошибки.

Эта функция может использоваться в качестве обратного вызова триггера PCM для реализаций драйверов PCM на основе dmaengine.

```
snd_pcm_uframes_t snd_dmaengine_pcm_pointer_no_residue(структура snd_pcm_substream * SUB-поток)
```

реализация PCM-указателя на основе dmaengine

Параметры

структура snd_pcm_substream * подпоток Подпоток

PCM Описание

Эта функция устарела и не должна использоваться новыми драйверами, так как ее результаты могут быть ненадежными.

```
snd_pcm_uframes_t snd_dmaengine_pcm_pointer(структура snd_pcm_substream * подпоток)
```

реализация указателя PCM на основе dmaengine

Параметры

структура snd_pcm_substream * подпоток Подпоток

PCM Описание

Эта функция может использоваться в качестве обратного вызова указателя PCM для реализаций драйверов PCM на основе dmaengine.

```
struct dma_chan * snd_dmaengine_pcm_request_channel(dma_filter_fn filter_fn, тер_данные, пустота * filter_data)
```

Канал запроса для dmaengine PCM

Параметры

dma_filter_fn filter_fn Функция фильтрации, используемая для запроса

канала DMA аннулирует * filter_data Данные, переданные функции

фильтрации DMA Описание

Возвращает NULL или запрошенный канал DMA.

Эта функция запрашивает канал DMA для использования с dmaengine PCM.

```
int snd_dmaengine_pcm_open(структура snd_pcm_substream * подпоток, структура dma_chan * chan)
```

Открыть подпоток PCM на основе dmaengine

Параметры

структура snd_pcm_substream * подпоток Подпоток PCM

структура dma_chan * chan Канал DMA, используемый для передачи данных

Описание

Возвращает 0 при успешном выполнении, в противном случае - отрицательный код ошибки.

Обычно функцию следует вызывать из обратного вызова `pcm_open`. Обратите внимание, что эта функция будет использовать `private_data` среды выполнения подпотока. Таким образом, он недоступен для вашей реализации драйвера `pcm`.

```
int snd_dmaengine_pcm_open_request_chan(struct snd_pcm_substream * подпоток,
                                     dma_filter_fn filter_fn, void * filter_data)
```

Откройте подпоток PCM на основе `dmaengine` и запросите канал

Параметры

`структура snd_pcm_substream *` `подпоток` Подпоток PCM

`dma_filter_fn filter_fn` Функция фильтрации, используемая для запроса канала DMA

`void *` `filter_data` Данные, переданные функции фильтрации DMA

Описание

Возвращает 0 при успешном выполнении, в противном случае - отрицательный код ошибки.

Эта функция запросит канал DMA, используя переданную функцию фильтрации и данные. Функция должна обычно вызываться из обратного вызова `pcm_open`. Обратите внимание, что эта функция будет использовать поле `private_data` среды выполнения подпотока. Таким образом, оно недоступно для вашей реализации драйвера `pcm`.

```
int snd_dmaengine_pcm_close(struct snd_pcm_substream * подпоток)
```

Закройте подпоток PCM на основе `dmaengine`

Параметры

`структура snd_pcm_substream *` `подпоток` Подпоток PCM

```
int snd_dmaengine_pcm_close_release_chan(struct snd_pcm_substream * подпоток)
```

Закройте подпоток PCM на основе `dmaengine` и отключите

Параметры

`структура snd_pcm_substream *` `подпоток` Подпоток PCM

Описание

Освобождает канал DMA, связанный с подпотоком PCM.

перечисление `dma_transfer_direction` `snd_pcm_substream_to_dma_direction(const структура snd_pcm_substream * подпоток)`

Получаем `dma_transfer_direction` для подпотока PCM

Параметры

`const структура snd_pcm_substream *` `подпоток` Подпоток PCM

PCM структура `snd_dmaengine_dai_dma_data`

Данные конфигурации DAI DMA

Определение

```
структура snd_dmaengine_dai_dma_data {
    dma_addr_t addr;
    перечисление dma_slave_buswidth
    addr_width; u 3 2 maxburst;
    unsigned int slave_id;
    void * filter_data;
    const char * chan_name;
    unsigned int fifo_size;
    флаги unsigned int;
};
```

Участники

`addr` Адрес источника данных DAI или регистра назначения.

addr_width Ширина регистра источника данных DAI или назначения.

maxburst Максимальное количество слов (примечание: слова, выраженные в единицах элемента `src_addr_width`, а не байты) которые могут быть отправлены в DAI или получены от него за один пакет.

slave_id Идентификатор запрашивающего подчиненного устройства для канала DMA.

filter_data Данные пользовательского фильтра канала DMA, обычно они используются при запросе канала DMA.

chan_name Пользовательское имя канала для использования при запросе канала DMA.

fifo_size Размер FIFO контроллера DAI в байтах

флаги Флаги PCM_DAI, пока только SND_DMAENGINE_PCM_DAI_FLAG_PACK

структура `snd_dmaengine_pcm_config`

Конфигурационные данные для PCM на

основе dmaengine **Определение**

```
структура snd_dmaengine_pcm_config {  
  
    int  (* prepare_slave_config) (структура snd_pcm_substream *подпоток, структура snd_pcm_hw_params  
-   * параметры, структура dma_slave_config *slave_config);  
  
    struct dma_chan * (* compat_request_channel) (struct snd_soc_pcm_runtime *rtd, struct snd_pcm_  
-   подпоток *подпоток);  
    dma_filter_fn compat_filter_fn;  
    struct device * dma_dev;  
    const char * chan_names;  
    const struct snd_pcm_hardware * pcm_hardware;  
    unsigned int prealloc_buffer_size;  
};
```

Участники

prepare_slave_config Обратный вызов, используемый для заполнения DMA slave_config для подпотока PCM. Будет вызван из параметров обратного вызова драйверов PCM.

compat_request_channel Обратный вызов для запроса канала DMA для платформ, которые не используют devicetree.

compat_filter_fn Будет использоваться в качестве функции фильтра при запросе канала для платформ, которые не используют devicetree. Параметром фильтра будут данные DMA DAI.

dma_dev Если установлено, запрашивайте канал DMA на этом устройстве, а не на устройстве DAI.

chan_names Если установлено, эти пользовательские названия каналов DMA будут запрошены при регистрации.

pcm_hardware структура snd_pcm_hardware, которая будет использоваться для PCM.

prealloc_buffer_size Размер предварительно выделенного аудиобуфера.

Примечание

Если установлены как `compat_request_channel`, так и `compat_filter_fn`, `compat_request_channel` будет использоваться для запроса канала, а `compat_filter_fn` будет проигнорирован. В противном случае канал будет запрошен с помощью `dma_request_channel` с `compat_filter_fn` в качестве функции фильтрации.

1.1.3 API управления / микшера

Общий интерфейс управления

недействителен `snd_ctl_notify` (структура `snd_card *card`, unsigned int *маска*, структура `snd_ctl_elem_id *id`)

Отправить уведомление в пользовательское пространство

для изменения элемента управления **Параметры**

структура `snd_card * card` карточка для отправки уведомления

маска unsigned int маска события, SNDRV_CTL_EVENT_*

`struct snd_ctl_elem_id * id` идентификатор элемента

ctl для отправки уведомления **Описание**

Эта функция добавляет запись события с заданным идентификатором и маской, добавляется в список и пробуждает пространство пользователя для уведомления. Это может быть вызвано в атомарном

контексте. `int snd_ctl_new(struct snd_kontrol ** kctl, unsigned int count, unsigned int доступ, struct snd_ctl_file * файл)`

создайте новый экземпляр элемента управления с некоторыми элементами

Параметрами

`struct snd_kontrol * * kctl` указатель на сохранение нового экземпляра элемента

управления `unsigned int count` количество элементов в этом элементе управления

`доступ * unsigned int` флаги доступа по умолчанию для элементов в этом

элементе управления `struct snd_ctl_file * file` задаются при блокировке этих

элементов **Описание**

Выделяет объект памяти для нового экземпляра элемента управления. Экземпляр содержит столько элементов, сколько задано число (**count**). Каждому элементу предоставлены права доступа (**access**). Каждый элемент блокируется, когда **файл** дается.

Возврат

0 при успешном выполнении, код ошибки при сбое

структура `snd_control * snd_ctl_new 1 (const struct snd_control_new * ncontrol, void * private_data)`

создайте экземпляр элемента управления из шаблона

Параметры

`const struct snd_control_new * ncontrol` запись инициализации

`void * private_data` личные данные для установки

Описание

Выделяет новый экземпляр `struct snd_control` и инициализирует из данного шаблона. Когда поле доступа `ncontrol` равно 0, предполагается, что это доступ для ЧТЕНИЯ и перезаписи. Когда поле `count` равно 0, оно принимается за единицу. **Возврат**

Указатель на только что созданный экземпляр или `NULL` при сбое.

`void snd_ctl_free_one(struct snd_kontrol * kcontrol)`

освободить экземпляр элемента управления

Параметры

`struct snd_kontrol * kcontrol` управляющий

экземпляр **Описание**

Освобождает экземпляр элемента управления, созданный с помощью функции `snd_ctl_new()` или `snd_ctl_new 1 ()`.

Не вызывайте это после того, как элемент управления был добавлен в `card`.

`int snd_ctl_add(struct snd_card * card, struct snd_control * kcontrol)`

добавьте экземпляр элемента управления в `card`

Параметры

`struct snd_card * card` экземпляр `card`

`struct snd_control * kcontrol` добавляемый экземпляр

элемента управления **Описание**

Добавляет экземпляр элемента управления, созданный с помощью функции `snd_ctl_new()` или `snd_ctl_new 1 ()` для данной карты. Также присваивает уникальный идентификатор, используемый для быстрого поиска.

1.1. API драйвера ALSA 35

Она автоматически освобождает элемент управления, который нельзя добавить. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_ctl_replace(struct snd_card *card, struct snd_control *kcontrol, bool add_on_replace)`
замените управляющий экземпляр карты

Параметры

`структура snd_card * card` экземпляр карты
`структура snd_control * kcontrol` экземпляр элемента управления для замены
`bool add_on_replace` добавьте элемент управления, если он еще не добавлен

Описание

Заменяет данный элемент управления. Если данный элемент управления не существует и установлен флаг `add_on_replace`, то добавляется элемент управления. Если элемент управления существует, он сначала уничтожается. Это автоматически освобождает элемент управления, который не может быть добавлен или заменен. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_ctl_remove(структура snd_card *карточка, struct snd_control *kcontrol)`
удалите элемент управления с карты и освободите его

Параметры

`структура snd_card * card` экземпляр карты
`структура snd_control * kcontrol` экземпляр элемента управления, который нужно удалить

Описание

Удаляет элемент управления с карты, а затем освобождает экземпляр.

Вам не нужно вызывать

`snd_ctl_free_one()`. Вы должны находиться в режиме блокировки записи - `down_write(card-> controls_rwsem)`.

Возврат

0 в случае успеха или отрицательный код ошибки при сбое.

`int snd_ctl_remove_id(структура snd_card *card, struct snd_ctl_elem_id структура snd_ctl_elem_id *id)`
удалите управление данным идентификатором и освободите его

Параметры

`структура snd_card * card` экземпляр карты
`структура snd_ctl_elem_id * id` идентификатор элемента управления для удаления

Описание

Находит экземпляр элемента управления с заданным идентификатором, удаляет его из списка карточек и освобождает. **Возврат**

0 в случае успеха или отрицательный код ошибки при сбое. `int`

`snd_ctl_remove_user_ctl(структура snd_ctl_file *файл, структура snd_ctl_elem_id *id)`
удалите и отпустите разблокированный пользовательский элемент управления

Параметры

`структура snd_ctl_file * файл` активный дескриптор элемента управления
`структура snd_ctl_elem_id * id` идентификатор элемента управления для удаления

Описание

Находит экземпляр элемента управления с заданным идентификатором, удаляет его из списка карточек и освобождает. **Возврат**

0 в случае успеха или отрицательный код ошибки при сбое. int

`snd_ctl_activate_id`(struct snd_card * *card*, struct snd_ctl_elem_id структура snd_ctl_elem_id * *id*, int *активный*)
активировать/деактивировать управление данным идентификатором

Параметры

структура snd_card * card экземпляр карты
struct snd_ctl_elem_id * id идентификатор элемента управления для активации / деактивации
int active ненулевое значение для активации

Описание

Находит экземпляр элемента управления с заданным идентификатором и активирует или деактивирует элемент управления вместе с уведомлением, в случае изменения.
Данные данного идентификатора заполняются полной информацией. **Возврат**
0, если без изменений, 1, если изменено, или отрицательный код ошибки при сбое.

int `snd_ctl_rename_id`(структура snd_card * *card*, struct snd_ctl_elem_id * *src_id*, struct snd_ctl_elem_id * *dst_id*)
замените идентификатор элемента управления на карте

Параметры

структура snd_card * card экземпляр карты
структура snd_ctl_elem_id * src_id старый идентификатор
struct snd_ctl_elem_id * dst_id новый идентификатор
идентификатор

Описание

Находит элемент управления со старым идентификатором с карты и заменяет идентификатор на новый. **Возврат**
Ноль в случае успеха или отрицательный код ошибки при сбое. struct snd_kontrol * `snd_ctl_find_numid`(struct snd_card * *card*, unsigned int *numid*)
найдите экземпляр элемента управления с заданным номером-id

Параметры

структура snd_card * card экземпляр card
беззнаковый int numid номер-идентификатор для поиска

Описание

Находит управляющий экземпляр с заданным номером-идентификатором с карты.

Вызывающий абонент должен опустить карту->controls_rwsem перед вызовом этой функции (если может возникнуть условие гонки). **Возврат**

Указатель экземпляра, если он найден, или NULL если нет. struct snd_control * `snd_ctl_find_id`(struct snd_card * *card*, struct snd_ctl_elem_id * *id*)
найдите экземпляр элемента управления с заданным идентификатором

Параметры

структура snd_card * card экземпляр card
struct snd_ctl_elem_id * id идентификатор для поиска

Описание

Находит экземпляр элемента управления с заданным идентификатором из карты.

Вызывающий должен нажать card-> controls_rwsem перед вызовом этой функции (если может возникнуть условие гонки). **Возврат**

Указатель экземпляра, если он найден, или `NULL` если нет.

`int snd_ctl_register_ioctl(snd_kctl_ioctl_func_t fcn)`
регистрация элементов управления для конкретного устройства-iocls

Параметры

`snd_kctl_ioctl_func_t fcn` функция обратного вызова ioctl

Описание

вызывается из каждого диспетчера устройств, такого как pcm.c, hwdep.c и т.д.

`int snd_ctl_register_ioctl_compat(snd_kctl_ioctl_func_t fcn)`
зарегистрируйте 32-разрядный контроллер

совместимости для конкретного устройства-iocls **Параметры**

`snd_kctl_ioctl_func_t fcn` функция обратного вызова ioctl

`int snd_ctl_unregister_ioctl(snd_kctl_ioctl_func_t fcn)`
отменить регистрацию элементов управления для конкретного устройства-iocls

Параметры

`snd_kctl_ioctl_func_t fcn` функция обратного вызова ioctl для отмены регистрации

`int snd_ctl_unregister_ioctl_compat(snd_kctl_ioctl_func_t fcn)`
отменить регистрацию 32-битного управления совместимостью для конкретного устройства-iocls

Параметры

`snd_kctl_ioctl_func_t fcn` функция обратного вызова ioctl для отмены регистрации `int`

`snd_ctl_boolean_mono_info`(структура `snd_kcontrol * kcontrol`, `struct snd_ctl_elem_info * uinfo`)

Вспомогательная функция для стандартного обратного вызова

логической информации с моноканалом **Параметры**

`структура snd_control * kcontrol` экземпляр kcontrol

`структура snd_ctl_elem_info * uinfo` информация для хранения

Описание

Это функция, которая может использоваться в качестве обратного вызова `info` для стандартного логического элемента управления с одним моноканалом . `int`

`snd_ctl_boolean_stere_info`(структура `snd_control * kcontrol`, структура `snd_ctl_elem_info * uinfo`)

Вспомогательная функция для стандартного обратного вызова логической

информации с двумя стереоканалами **Параметры**

`структура snd_control * kcontrol` экземпляр kcontrol

`структура snd_ctl_elem_info * uinfo` информация для хранения

Описание

Это функция, которая может использоваться в качестве обратного вызова `info` для стандартного логического элемента управления с двумя стереоканалами.

`int snd_ctl_enum_info`(структура `snd_ctl_elem_info * Информация`, беззнаковый `int Каналы`, беззнаковый `int Товары`,

постоянный символ * `const имена`)

заполняет информационную структуру для перечисляемого элемента управления

Параметры

`struct snd_ctl_elem_info * информация` структура, подлежащая заполнению

`unsigned int channels` количество каналов элемента управления; часто один

`элементы int без знака` количество управляющих значений; также размер **имен**

`const char * const names` массив, содержащий имена всех управляющих значений

Описание

Задаёт все обязательные поля в **Информация** к их соответствующим значениям. Если доступность элемента управления не установлена по умолчанию (доступна для чтения и записи), вызывающий объект должен заполнить **Информация**->доступ.

Возврат

Ноль.

API кодека AC97

недействительный `snd_ac97_write` (структура `snd_ac97 * ac97`, короткое без знака `reg`, короткое без знака *значение*)
запишит значение в указанный регистр

Параметры

`структура snd_ac97 * ac97` экземпляр `ac97`

`беззнаковый короткий reg` регистр для

изменения `короткое значение без знака` значение

для установки **Описание**

Записывает значение в указанный регистр. Это вызовет обратный вызов записи непосредственно после проверки регистра. Эта функция не изменяет кэш регистров в отличие от `#c:func:snd_ac97_write_cache()`, поэтому используйте это только когда вы не хотите отражать изменение состояния приостановки / возобновления. `unsigned short snd_ac97_read` (структура `snd_ac97 * ac97`, короткая без знака `reg`)
считывает значение из заданного

регистра **Параметры**

`структура snd_ac97 * ac97` экземпляр `ac97`

`беззнаковый короткий reg` регистр для

чтения **Описание**

Считывает значение из заданного регистра. Это вызовет обратный вызов `read` непосредственно после проверки регистра. **Возврат**

Прочитанное значение. недействительно

`snd_ac97_write_cache` (struct `snd_ac97 * ac97`, unsigned short `reg`, unsigned short *значение*)
запишит значение в указанный регистр и обновите кэш

Параметры

`структура snd_ac97 * ac97` экземпляр `ac97`

`беззнаковый короткий reg` регистр для изменения

`беззнаковое короткое значение` значение, которое нужно

установить **Описание**

Записывает значение в указанный регистр и обновляет кэш регистров.
Кэшированные значения используются для кэшированного чтения и приостановки / возобновления.

int snd_ac97_update(структура snd_ac97 * ac97, unsigned short reg, unsigned short значение)

обновите значение в заданном

регистре **Параметры**

структура snd_ac97 * ac97 экземпляр ac97

unsigned short reg регистр для изменения

unsigned short value значение для

установки **Описание**

Сравнивает значение с кэшем регистров и обновляет значение только при изменении значения.

Возврат

1, если значение изменено, 0, если изменений нет, или отрицательный код при сбое. int

snd_ac97_update_bits(структура snd_ac97 * ac97, беззнаковое сокращение reg, беззнаковое сокращение маска, unsigned short значение)

обновите биты в заданном регистре

Параметры

структура snd_ac97 * ac97 экземпляр ac97

unsigned short reg регистр для изменения

беззнаковая короткая маска битовая маска для

изменения беззнаковое короткое значение значение,

которое нужно установить **Описание**

Обновляет маскированные биты в данном регистре только при

изменении значения. **Возврат**

1, если биты изменены, 0, если изменений нет, или отрицательный код

при сбое. const char * snd_ac97_get_short_name(структура snd_ac97 * ac97)

извлекает имя кодека

Параметры

структура snd_ac97 * ac97 экземпляр кодека

Возврат

Краткое идентифицирующее название кодека.

int snd_ac97_bus(структура snd_card * card, int num, структура snd_ac97_bus_ops * ops, void * private_data, структура snd_ac97_bus ** rbus)

создание компонента шины AC97

Параметры

структура snd_card * card экземпляр карты

int num номер шины

структура snd_ac97_bus_ops * ops таблица обратных вызовов шины

void * private_data указатель личных данных для нового экземпляра

структура snd_ac97_bus ** rbus указатель для хранения нового экземпляра

шины AC97. **Описание**

Создает компонент шины AC97. Заново выделяется и инициализируется экземпляр struct snd_ac97_bus. Таблица ops должна включать допустимые обратные вызовы (по крайней мере, чтение и запись). Другие обратные вызовы, wait и reset , не являются обязательными.

Часы установлены на 48000. Если нужны другие часы, установите (* rbus) -> часы вручную.

Экземпляр шины AC97 зарегистрирован как устройство низкого уровня, поэтому вам не нужно освобождать его вручную. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое. int

snd_ac97_mixer(структура snd_ac97_bus * Автобус, struct snd_ac97_template * шаблон, struct snd_ac97 ** rac97)
создание компонента Codec97

Параметры

структура snd_ac97_bus * bus шина AC97, к которой подключен кодек
структура snd_ac97_template * шаблон шаблон ac97, включая индекс, обратные вызовы и при-
сведения вате.

структура snd_ac97 * * rac97 указатель для
хранения новых ac97 инстанции. **Описание**

Создает компонент Codec97. Экземпляр struct snd_ac97 выделяется заново и инициализируется из шаблона. Затем кодек инициализируется стандартной процедурой. Шаблон должен включать номер кодека (num) и адрес (addr), а также личные данные (private_data). Экземпляр ac97 зарегистрирован как устройство низкого уровня, поэтому вам не нужно освобождать его вручную. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое. int

snd_ac97_update_power(struct snd_ac97 * ac97, int reg, int включение питания)
обновите регистр отключения питания

Параметры

структура snd_ac97 * ac97 экземпляр кодека
int reg регистр скорости, например
AC97_PCM_FRONT_DAC_RATE значение int powerup ненулевое
при включении детали **Описание**

Обновите биты регистра отключения питания AC97 для
данной части. **Возврат**

Ноль. недействительно

snd_ac97_suspend(структура snd_ac97 * ac97)
Общая функция приостановки для кодека AC97

Параметры

структура snd_ac97 * ac97 экземпляр ac97

Описание

Приостанавливает работу кодека, отключает питание чипа.

void snd_ac97_resume(структура snd_ac97 * ac97)
Общая функция возобновления
для кодека AC97

Параметры

структура snd_ac97 * ac97 экземпляр

ac97 Описание

Выполните стандартную процедуру возобновления, включите питание и восстановите старые значения регистра.

int snd_ac97_tune_hardware(структура snd_ac97 *ac97, const struct ac97_quirk *quirk, const char *переопределить)
настроить аппаратное обеспечение

Параметры

struct snd_ac97 *ac97 экземпляр ac97
постоянная структура ac97_quirk *quirk список причуд
постоянный символ *переопределение явное значение quirk
(переопределяет список, если оно не равно NULL) Описание

Выполните некоторое обходное решение для каждого устройства pci, например, переименуйте элемент управления наушниками (true line-out) в "Master".
Список причуд должен завершаться записью, заполненной нулем. Возврат
Ноль в случае успеха или отрицательный код ошибки при сбое.

int snd_ac97_set_rate(структура snd_ac97 *ac97, int reg, unsigned int Оценить)
изменить скорость данного

ввода/вывода. Параметры

struct snd_ac97 *ac97 экземпляр ac97
int reg регистр для изменения
unsigned int rate устанавливаемая частота дискретизации Описание

Изменяет скорость данного ввода/ вывода в кодеке. Если кодек не поддерживает VAR, скорость должна быть 48000 (за исключением SPDIF). Допустимые регистры: AC97_PMC_MIC_ADC_RATE, AC97_PMC_FRONT_DAC_RATE, AC97_PMC_LR_ADC_RATE.
Принимаются значения AC97_PCM_SURR_DAC_RATE и AC97_PCM_LFE_DAC_RATE , если кодек их поддерживает. AC97_SPDIF принимается в качестве псевдо-регистра для изменения битов состояния SPDIF.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.
int snd_ac97_pcm_assign(структура snd_ac97_bus *Автобус, unsigned short pcms_count, const struct ac97_pcm *pcms)
назначить слоты AC97 для заданных потоков PCM

Параметры

структура snd_ac97_bus *bus экземпляр шины ac97
беззнаковое сокращение pcms_count количество PCM,
которые должны быть назначены const struct ac97_pcm *pcm PCM,
которые должны быть назначены Описание

Он назначает доступные слоты AC97 для данных PCM. Если ни один из них или доступны только некоторые слоты, количество участников pcm->xxx.slots и pcm->xxx.rslots[] уменьшается и может равняться нулю.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое. int
snd_ac97_pcm_open(struct ac97_pcm *pcm, беззнаковый int Оценить, перечисление ac97_pcm_cfg cfg, unsigned int flags)
открывает заданный pcm AC97

Параметры

структура ac97_pcm *pcm экземпляр ac97 pcm

беззнаковая скорость ввода частота в Гц, если кодек не поддерживает VRA, это значение должно быть 48000 Гц `ac97_pcm_cfg` характеристики выходного потока `ac97_pcm_cfg` короткие слоты без знака подмножество выделенных слотов

(`snd_ac97_pcm_assign`) для этого `pcm` **Описание**

Он блокирует указанные слоты и устанавливает заданную скорость для регистров AC97.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_ac97_pcm_close(struct ac97_pcm *pcm)`
закрывает данный AC97 `pcm`

Параметры

структура `ac97_pcm` * `pcm` экземпляр `ac97_pcm`

Описание

Это освобождает заблокированные слоты AC97.

Возврат

Ноль.

`int snd_ac97_pcm_double_rate_rules(struct snd_pcm_runtime *runtime)`
установите ограничения двойной скорости

Параметры

структура `snd_pcm_runtime` * `runtime` время выполнения время выполнения

PCM фронтального воспроизведения `ac97` **Описание**

Устанавливает правила аппаратных ограничений для предотвращения использования двойной скорости и более двух каналов одновременно. одновременно.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

API управления виртуальным мастером

структура `snd_ctl` * `snd_ctl_make_virtual_master(символ *Имя, const unsigned int *tlv)`
Создайте виртуальный главный элемент управления

Параметры

символ * имя строка имени элемента управления для создания
`const unsigned int * tlv` необязательный массив TLV int для получения информации из базы данных **Описание**

Создает виртуальный главный элемент управления с заданной строкой имени.

После создания элемента `vmaster` вы можете добавить подчиненные элементы управления с помощью `snd_ctl_add_slave()` или `snd_ctl_add_slave_uncached()`.

Необязательный аргумент `tlv` может использоваться для указания информации о TLV для масштаба dB главного управления

. Это должен быть отдельный элемент с типом `#SNDRV_CTL_TLVT_DB_SCALE`, `#SNDRV_CTL_TLVT_DB_MINMAX` или

`#SNDRV_CTL_TLVT_DB_MINMAX_MUTE` и максимальным значением 0dB.

Возврат

Созданный элемент управления, или `NULL` для ошибок (ENOMEM).

```
int snd_ctl_add_vmaster_hook(struct snd_kcontrol *kcontrol, void (*перехват)(void *private_data, int, void *private_data))
```

Добавление привязки к элементу управления vmaster

Параметры

структура snd_control * kcontrol элемент kctl vmaster

перехват void (*)(void *private_data, int) функция

перехвата void * private_data указатель private_data,

который необходимо сохранить **Описание**

Добавляет данный хук к управляющему элементу vmaster, чтобы он вызывался каждый раз, когда значение изменяется.

Возврат

Ноль. пустота

snd_ctl_sync_vmaster(структура snd_kcontrol *kcontrol, bool hook_only)

Синхронизируйте подчиненные устройства vmaster и hook

Параметры

struct snd_control * kcontrol элемент vmaster kctl

bool hook_only синхронизировать только хук

Описание

Принудительно вызывайте обратный вызов put для каждого подчиненного устройства и вызывайте функцию hook для синхронизации с текущим значением данного элемента vmaster. NOP, когда в kcontrol передается значение NULL. **kcontrol**.

int snd_ctl_add_slave(структура snd_kcontrol *master, структура snd_control *Подчинение)

Добавление виртуального подчиненного элемента управления

Параметры

структура snd_kcontrol * master элемент vmaster

структура snd_control * подчиненный элемент добавляемый

подчиненный элемент **Описание**

Добавьте виртуальный подчиненный элемент управления к данному главному элементу, созданному с помощью snd_ctl_create_virtual_master() предварительно.

Все подчиненные устройства должны быть одного типа (возвращать одну и ту же информацию через обратный вызов info). Функция не проверяет это, так что это ваша ответственность.

Кроме того, некоторые дополнительные ограничения: не более двух каналов, логарифмическая регулировка громкости (уровень дБ), таким образом, отсутствует линейная громкость, мастер может только уменьшать громкость без усиления

Возврат

Ноль в случае успеха или отрицательный код ошибки.

int snd_ctl_add_slave_uncached(структура snd_kcontrol *master, структура snd_control *Подчинение)

Добавление виртуального подчиненного элемента управления

Параметры

структура snd_kcontrol * master элемент vmaster

структура snd_control * подчиненный элемент добавляемый

подчиненный элемент **Описание**

Добавьте виртуальный подчиненный элемент управления к данному ведущему. В отличие от snd_ctl_add_slave(), элемент, добавленный с помощью этой функции, должен иметь изменяемые значения, и обратный вызов get вызывается при каждом запросе от master.

Когда элемент управления просматривает аппаратные значения напрямую и значение может быть изменено другими способами, кроме обратного вызова элемента put, следует использовать эту функцию, чтобы поддерживать значение всегда в актуальном состоянии. **Возврат** Ноль в случае успеха или отрицательный код ошибки.

1.1.4 MIDI API

Необработанный MIDI API

int snd_rawmidi_receive(структура snd_rawmidi_substream * *подпоток*, const без знака символ * *буфер*, int *количество*)
получать входные данные от устройства

Параметры

структура snd_rawmidi_substream * подпоток подпоток rawmidi
буфер const unsigned char * указатель буфера
количество значений int размер данных для чтения

Описание

Считывает данные из внутреннего буфера.

Возврат

Размер считываемых данных или отрицательный код ошибки при сбое.

int snd_rawmidi_transmit_empty(struct snd_rawmidi_substream * *подпоток*)
проверьте, пуст ли выходной буфер

Параметры

структура snd_rawmidi_substream * подпоток подпоток rawmidi
Возврат

1, если внутренний выходной буфер пуст, 0, если нет.

int __snd_rawmidi_transmit_peek(struct snd_rawmidi_substream * *подпоток*, unsigned char * *буфер*, int *count*)
копирование данных из внутреннего буфера

Параметры

структура snd_rawmidi_substream * подпоток подпоток rawmidi
буфер без знака char * указатель буфера
количество входных данных размер данных для передачи

Описание

Это вариант `snd_rawmidi_transmit_peek()` без блокировки вращения.

int snd_rawmidi_transmit_peek(структура snd_rawmidi_substream * *подпоток*, без знака символ * *буфер*, int *count*)
копирование данных из внутреннего буфера

Параметры

структура snd_rawmidi_substream * подпоток подпоток rawmidi
символ без знака * `buffer` указатель буфера
количество входных данных размер данных для передачи

Описание

Копирует данные из внутреннего выходного буфера в указанный буфер.

Вызовите это в обработчике прерываний, когда midi-вывод будет готов, и вызовите `snd_rawmidi_transmit_ack()` после завершения передачи.

Возврат

Размер копируемых данных или отрицательный код ошибки при сбое.

`int __snd_rawmidi_transmit_ack(struct snd_rawmidi_substream * подпоток, значение int count)`
подтвердите передачу

Параметры

`структура snd_rawmidi_substream * подпоток` подпоток rawmidi
`количество int` переданный счет

Описание

Это вариант `__snd_rawmidi_transmit_ack()` без блокировки вращения.

`int snd_rawmidi_transmit_ack(struct snd_rawmidi_substream * подпоток, int count)`
подтвердите передачу

Параметры

`struct snd_rawmidi_substream * подпоток` подпоток rawmidi
`количество входных данных` количество переданных данных

Описание

Увеличивает аппаратный указатель для внутреннего выходного буфера до заданного размера и обновляет условие. Вызовите после завершения передачи.

Возврат

Расширенный размер в случае успеха или отрицательный код ошибки при сбое. `int`
`snd_rawmidi_transmit(struct snd_rawmidi_substream * подпоток, unsigned char * буфер,
int количество)`
копирование из буфера на устройство

Параметры

`struct snd_rawmidi_substream * подпоток` подпоток rawmidi
`unsigned char * buffer` указатель буфера
`количество значений int` размер данных для передачи

Описание

Копирует данные из буфера на устройство и перемещает указатель вперед.

Возврат

Скопированный размер в случае успеха или отрицательный код ошибки при сбое.

`int snd_rawmidi_new(struct snd_card * card, char * id, int устройство, int output_count, int input_count,
структура snd_rawmidi ** rrawmidi)`
создание экземпляра rawmidi

Параметры

`struct snd_card * card` экземпляр карты
`char * id` строка идентификатора
`int device` индекс устройства
`int output_count` количество выходных потоков

`int input_count` количество входных потоков

`struct snd_rawmidi * * rrawmidi` указатель для хранения нового экземпляра rawmidi

Описание

Создает новый экземпляр rawmidi. Использовать `snd_rawmidi_set_ops()` чтобы установить операторы для нового экземпляра.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое. void

`snd_rawmidi_set_ops`(`struct snd_rawmidi * rmidi`, `int ноток`, постоянная структура `snd_rawmidi_ops * ops`)

задайте операторы rawmidi

Параметры

`структура snd_rawmidi * rmidi` экземпляр rawmidi

`int stream` направление потока, SNDRV_RAWMIDI_STREAM_XXX

`постоянная структура snd_rawmidi_ops * ops` таблица операторов

Описание

Задает операторы rawmidi для заданного направления потока.

MPU401-UART API

`irqreturn_t snd_mpu401_uart_interrupt`(`int irq`, `void * dev_id`)

универсальный обработчик

прерываний MPU401-UART

Параметры

`int irq` номер irq

`void * dev_id` экземпляр mpu401

Описание

Обрабатывает прерывание для ввода-вывода MPU401-UART.

Возврат

`IRQ_HANDLED` если прерывание было обработано. `IRQ_NONE` в противном случае.

`irqreturn_t snd_mpu401_uart_interrupt_tx`(`int irq`, `void * dev_id`)

универсальный обработчик irq передачи

MPU401-UART

Параметры

`int irq` номер irq

`void * dev_id` экземпляр mpu401

Описание

Обрабатывает прерывание для вывода MPU401-UART.

Возврат

`IRQ_HANDLED` если прерывание было обработано. `IRQ_NONE` в противном случае.

`int snd_mpu401_uart_new`(`структура snd_card * card`, `int device`, беззнаковое сокращение *аппаратное обеспечение*, без знака длинный *порт*, без знака `int info_flags`, `int irq`, структура `snd_rawmidi ** rrawmidi`)

создание экземпляра MPU401-UART

Параметры

`структура snd_card * card` экземпляр card

`int device` индекс устройства на основе нуля

краткое описание оборудования без подписи тип оборудования,
MPU401_HW_XXXX беззнаковый длинный порт базовый адрес порта MPU401
беззнаковый int info_flags битовые флаги MPU401_INFO_XXX
int irq номер irq ISA, -1, если он не должен быть выделен
struct snd_rawmidi * * rrawmidi указатель для хранения нового экземпляра
rawmidi

Описание

Создает новый экземпляр MPU-401.

Обратите внимание, что в аргументе rrawmidi возвращается экземпляр rawmidi, а не сам экземпляр mpu401. Чтобы получить доступ к экземпляру mpu401, выполните приведение из rawmidi->private_data (co структурой snd_mpu401 magic-cast). **Возврат**
Ноль в случае успеха или отрицательный код ошибки.

1.1.5 Proc Info API

Proc Info интерфейс

int snd_info_get_line(struct snd_info_buffer * buffer, char * line, int len)
считывает одну строку из буфера procfs

Параметры

struct snd_info_buffer * buffer буфер procfs
строка символа * буфер для хранения
int len максимальный размер буфера

Описание

Считывает одну строку из буфера и сохраняет строку.

Возврат

Ноль в случае успеха или 1 в случае ошибки или EOF.

const char * snd_info_get_str(char * dest, const char * Src, int len)
анализ строкового токена

Параметры

char * dest буфер для хранения
строкового токена const char * src исходная строка
int len максимальная. длина токена - 1

Описание

Анализирует исходную строку и копирует токен
в заданный строковый буфер. **Возврат**

Обновленный указатель исходной строки, чтобы его можно было использовать для

следующего вызова. struct snd_info_entry * snd_info_create_module_entry(модуль struct * модуль, const символ
* Имя, struct snd_info_entry * parent)
создайте информационную запись для

данного модуля Параметры

struct module * модуль указатель на модуль
постоянный символ * name имя файла

структура snd_info_entry * parent родительский каталог

Описание

Создает новую информационную запись и присваивает ее
данному модулю. **Возврат**

Указатель на новый экземпляр или `NULL` при сбое.

struct snd_info_entry * snd_info_create_card_entry (структура snd_card * *card*, постоянный символ * *Имя*,
структура snd_info_entry * *родительская*)
создайте информационную запись для данной карты

Параметры

структура snd_card * card экземпляр карты

постоянный символ * name имя файла

struct snd_info_entry * parent родительский каталог

Описание

Создает новую информационную запись и присваивает ее
данной карточке. **Возврат**

Указатель нового экземпляра или `NULL` при сбое. void

snd_info_free_entry (структура snd_info_entry * *запись*)
отпустите ввод информации

Параметры

struct snd_info_entry * запись ввод информации

Описание

Освобождает информационную запись. int

snd_info_register (структура snd_info_entry * *запись*)
зарегистрируйте информационную запись

Параметры

структура snd_info_entry * запись информационная

запись **Описание**

Регистрирует запись информации о процедуре.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

1.1.6 Сжатие разгрузки

Сжатие разгрузки API

int snd_compress_register (структура *snd_compr* * *устройство*)
зарегистрировать сжатое устройство

Параметры

структура snd_compr * устройство сжатое устройство

для регистрации структуры `snd_compressed_buffer`

сжатый буфер

Определение

```
struct snd_compressed_buffer {
    __u32  размер
фрагмента;  __u32 2
фрагменты;  };
```

Участники

fragment_size размер фрагмента буфера в
байтах **фрагменты** количество таких
фрагментов **struct snd_compr_params**
параметры сжатого потока

Определение

```
struct snd_compr_params {
    struct snd_compressed_buffer
буфер;  struct snd_codec
кодек;  __u8  no_wake_mode;
};
```

Участники

буфер описание буфера
кодек параметры кодека
no_wake_mode не пробуждать по истечении
фрагмента структура **snd_compr_timestamp**
дескриптор временной

метки **Определение**

```
структура snd_compr_timestamp {
    __u32 2  byte_offset;
    __u32 2  copied_total;
    __u32 2  pcm_frames;
    __u32 2
pcm_io_frames;  __u32 2
sampling_rate;  };
```

Участники

byte_offset Смещение байта в кольцевом буфере до DSP
copied_total Общее количество байтов, скопированных из / в кольцевой буфер в /с помощью DSP
pcm_frames Кадры, декодированные или закодированные с помощью DSP. Это поле будет развиваться большими шагами и должно использоваться только
для отслеживания прогресса кодирования / декодирования. Оно не должно использоваться для оценки времени.
pcm_io_frames Кадры, отрисованные или принятые DSP в микшер или на аудиовыход / вход. Это поле
следует использовать для A / V синхронизации
или оценки времени. **sampling_rate** частота
дискретизации звука структура **snd_compr_avail**
доступный дескриптор

Определение

```
структура snd_compr_avail {
    __u64 4  доступно;
    struct
snd_compr_timestamp  timestamp;  };
```

Участники

доступно Количество байтов, доступных в
кольцевом буфере для записи / чтения `timestamp` информация о
временной метке структура `snd_compr_caps`

 дескриптор `caps`

Определение

```
структура  snd_compr_caps  {  
  
    __u 3 2    num_codecs;  
    __направление  u 3 2 ;  
    __u 3 2  
    min_fragment_size; __u 3 2  
    max_fragment_size; __u 3 2  
    min_fragments; __u 3 2  
    max_fragments; __u 3 2  
    codecs; __u 3 2    reserved;  
};
```

Участники

num_codecs количество поддерживаемых кодеков

направление поддерживается направление. Типа `snd_compr_direction`

min_fragment_size минимальный фрагмент, поддерживаемый DSP

max_fragment_size максимальный фрагмент, поддерживаемый DSP

min_fragments минимальное количество фрагментов,
поддерживаемых DSP **max_fragments** максимальное количество фрагментов,
поддерживаемых DSP **кодеки** указатель на массив кодеков

зарезервировано зарезервированное поле

структура `snd_compr_codec_caps`

 возможность запроса к

кодеку **Определение**

```
структура  snd_compr_codec_caps  {  
  
    __u 3 2    codec; __u 3 2  
    num_descriptors;  
    дескриптор  структуры  
    snd_codec_desc;  };
```

Участники

codec кодек, для которого запрашивается функциональность

num_descriptors количество дескрипторов кодека

дескриптор массив дескрипторов возможностей кодека

перечисление `sndrv_compress_encoder`

Константы

ДОПОЛНЕНИЕ `SNDRV_COMPRESS_ENCODER_PADDING` количество сэмплов, добавленных кодировщиком в
конец дорожки `SNDRV_COMPRESS_ENCODER_DELAY` количество сэмплов, вставленных кодировщиком в
начало дорожки структура `snd_compr_metadata`

 метаданные
сжатого потока **Определение**

```
структура snd_compr_metadata {
    __ключ__  __у 3 2 ;
    __значение__  __у 3 2 ;
};
```

Участники

- Ключиша идентификатор ключа
- значение значение ключа

```
SNDRV_COMPRESS_IOCTL_VERSION()
```

Описание параметров

SNDRV_COMPRESS_GET_CAPS: возможность запроса DSP SNDRV_COMPRESS_GET_CODEC_CAPS:
Возможность запроса совместимости кодека SNDRV_COMPRESS_SET_PARAMS: установка параметров
кодека и потока

Примечание

только параметры кодека могут быть изменены во время выполнения, а параметры потока не могут
быть SNDRV_COMPRESS_GET_PARAMS: Параметры запроса SNDRV_COMPRESS_TSTAMP: получить текущее
значение временной метки SNDRV_COMPRESS_AVAIL: получить текущее значение доступности буфера.

Это также запрашивает правильную

tstamp- связывает SNDRV_COMPRESS_PAUSE: приостановите текущий поток SNDRV_COMPRESS_RESUME:
возобновите приостановленный поток SNDRV_COMPRESS_START: запустите поток
SNDRV_COMPRESS_STOP: остановите текущий поток, отбрасывая содержимое кольцевого буфера и буферы, в которых в данный
момент находится DSP SNDRV_COMPRESS_DRAIN: Воспроизводите до конца буферов и останавливайтесь
после этого SNDRV_COMPRESS_IOCTL_VERSION: запрашивайте версию API struct snd_enc_vorbis

Определение

```
структура snd_enc_vorbis {
    __с 3 2 качество;
    __управляемый __у 3 2 ; __максимальный
    бит_рейт __у 3 2 ;
    __минимальный бит_рейт
    __у 3 2 ; __понижающее
    микширование __у 3 2 ; };
```

Участники

- Качество Устанавливает качество кодирования равным n, в диапазоне от -1 (низкое) до 10 (высокое). В режиме работы по умолчанию уровень качества равен 3. Нормальный диапазон качества - 0-10.
- управляемый Логический. Установите режим управления битрейтом. Это отключает обычную кодировку VBR, но позволяет кодировщику применять жесткие или мягкие ограничения битрейта. Этот режим может быть медленнее и также может иметь более низкое качество. В первую очередь он полезен для потоковой передачи.
- max_bit_rate Включен, только если значение maxaged равно TRUE
- min_bit_rate Включено, только если значение "Управляемый" равно TRUE
- понижающее микширование Логическое значение. Понижающее микширование ввода из стерео в моно (не влияет на нестереофонные потоки). Полезно для кодирование с более низким битрейтом.

Описание

Эти параметры были извлечены из спецификации OpenMAX IL и свойств Gstreamer vorbisenc
Для обеспечения наилучшего качества пользователям следует указать режим VBR и
установить уровни качества. структура snd_enc_real

Определение


```
структура snd_enc_real {
__u 3 2 quant_bits;
__u 3 2
start_region; __u 3 2
num_regions; };
```

Участники

- quant_bits** количество битов квантования связи в потоке
- start_region** начальная область связи в потоке
- num_regions** значение количества регионов

Описание

Эти параметры были извлечены из спецификации OpenMAX IL

struct **snd_enc_flac**

Определение

```
struct snd_enc_flac {
__число __u 3 2 ;
__коэффициент
усиления __u 3 2 ; };
```

Участники

- число** серийный номер, действительный только для форматов OGG, должен быть установлен приложением **усиление** Добавление тегов усиления воспроизведения
- Описание**

Эти параметры были извлечены из онлайн-документации FLAC по адресу [http://flac.sourceforge.net / documentation_tools_flac.html](http://flac.sourceforge.net/documentation_tools_flac.html)

Чтобы упростить API, предполагается, что пользователь будет выбирать качественные профили. Дополнительные параметры, которые влияют на качество и скорость кодирования, могут быть добавлены позже, если потребуется. По умолчанию кодировщики используют формат подмножества. ТЕГИ, такие как изображения и т.д., не могут обрабатываться выгруженным кодировщиком и не поддерживаются в этом API. структура **snd_compr_runtime**

Определение

```
структура snd_compr_runtime {
snd_pcm_state_t состояние;
структура snd_compr_ops * ops;
void * буфер;
размер буфера __u 6 4 ;
размер фрагмента __u 3 2 ;
__u 3 2 фрагмента;
__u 6 4 total_bytes_available;
__u 6 4 total_bytes_transferred;
wait_queue_head_t sleep;
void * private_data;
};
```

Участники

- состояние** состояние потока
- операции** указатель на обратные вызовы DSP
- буфер** указатель на буфер ядра, действительный только тогда, когда он не находится в режиме mmap или DSP не реализует копирование

buffer_size размер указанного выше буфера

fragment_size размер фрагмента буфера в байтах

фрагменты количество таких фрагментов

total_bytes_available совокупное количество байт, ставших доступными в кольцевом буфере

total_bytes_transferred совокупные байты, переданные с помощью `disload DSP`

режим ожидания режим ожидания опроса

private_data указатель личных данных драйвера

структура `snd_compr_stream`

Определение

```
структура  snd_compr_stream {

const char  *  имя;

структура  snd_compr_ops  *  ops;

структура  snd_compr_runtime  *  среда
выполнения;  структура  snd_compr  *
устройство;  структура  delayed_work
error_work;  перечисление

snd_compr_direction  direction;  bool

metadata_set;  bool  next_track;

void  *  private_data;

};
```

Участники

Имя имя устройства

ops указатель на обратные вызовы DSP

среда выполнения указатель на структуру среды выполнения

устройство указатель устройства

error_work задержка выполнения, используемая при закрытии потока

из-за ошибки **направление** направление потока, воспроизведение /

запись **metadata_set** флаг для установки метаданных, при установке `true`

next_track сигнализирует пользовательскому пространству о переходе на следующий трек, при установке `true`

private_data указатель на личные данные DSP

struct `snd_compr_ops`

Определение

```
struct  snd_compr_ops {

int  ( *  открытый) (структура  snd_compr_stream  *поток);

int  ( *  свободный) (структура  snd_compr_stream  *поток);

int  ( *  set_params) (структура  snd_compr_stream  *поток,  структура  snd_compr_params  *  параметры);

int  ( *  get_params) (структура  snd_compr_stream  *поток,  структура  snd_codec  *  параметры);

int  ( *  set_metadata) (структура  snd_compr_stream  *поток,  структура  snd_compr_metadata
*метаданные);  int  ( *  get_metadata) (структура  snd_compr_stream  *поток,  структура
snd_compr_metadata  *метаданные);  int  ( *  trigger) (структура  snd_compr_stream  *поток,  int  cmd);

int  ( *  указатель) (структура  snd_compr_stream  *поток,  структура  snd_compr_tstamp  *tstamp);

int  ( *  копировать) (  struct  snd_compr_stream  *stream,  char  __user  *buf,  size_t  count);

int  ( *  mmap) (структура  snd_compr_stream  *stream,  структура  vm_area_struct  *vma);

int  ( *  ack) (структура  snd_compr_stream  *stream,  размер_t  байт);

int  ( *  get_caps) (структура  snd_compr_stream  *stream,  структура  snd_compr_caps  *caps);

int  ( *  get_codec_caps) (структура  snd_compr_stream  *поток,  структура  snd_compr_codec_caps  *кодек);

};
```

Участники

- Открыть** Откройте сжатый поток Этот обратный вызов обязателен и должен поддерживать dsp в готовности к приему параметр потока
- Бесплатно** Закройте сжатый поток, обязательно
- set_params** Устанавливает параметры сжатого потока, в обязательном порядке Это может быть вызвано во время потока
- создание только для установки параметров кодека и свойств потока
- get_params** получение параметров кодека, обязательно
- set_metadata** Установка значений метаданных для потока
- get_metadata** извлекает запрошенные значения метаданных из потока
- запускает** Запускает такие операции, как start, pause, resume, drain, stop. Этот обратный вызов является обязательным
- указатель** Извлекает текущую информацию о ч / б указателе. Обязательный
- Копировать** Копирование сжатых данных в / из пользовательского пространства, опция не может быть реализована, если DSP поддерживает mmap **mmap** Метод DSP mmap для mmap
- памяти DSP **подтверждение** Подтверждение для DSP при записи данных в аудиобuffer, необязательно, недопустимо, если реализовано копирование
- get_caps** Получение
- возможностей DSP, обязательно **get_codec_caps** Получение возможностей для определенного кодека, обязательно
- struct snd_compr**

Определение

```
struct snd_compr {  
  
    const char * имя;  
    struct device dev;  
    struct snd_compr_ops * ops;  
    void * private_data;  
    struct snd_card * card;  
    unsigned int direction;  
    struct mutex lock;  
    int device;  
    #ifdef CONFIG_SND_VERBOSE_PROCS  
};
```

Участники

- Имя** Имя устройства DSP
- dev** связанный экземпляр устройства
- ops** указатель на обратные вызовы DSP
- private_data** указатель на данные DSP pvt
- карта** указатель звуковой карты
- направление** Направление воспроизведения
- или захвата **блокировка** блокировка устройства
- устройство** идентификатор устройства

1.1.7 ASoC

Базовый API ASoC

- структура **snd_soc_jack_pin**
- Описывает pin-код для обновления на основе обнаружения разъема

Определение

```
структура snd_soc_jack_pin {
    struct list_head список;
    const char * pin;
    маска int;
    bool инвертировать;
};
```

Участники

список запись внутреннего списка

pin-код имя PIN-кода для обновления

маска биты для проверки в сообщенном состоянии разъема

инвертировать если значение ненулевое, то вывод включен, когда

не сообщается о состоянии структура `snd_soc_jack_zone`

Описывает зоны напряжения для

обнаружения гнезда **Определение**

```
структура snd_soc_jack_zone {
    unsigned int min_mv;
    unsigned int max_mv;
    unsigned int jack_type;
    unsigned int debounce_time;
    struct list_head список;
};
```

Участники

min_mv начальное напряжение в мВ

max_mv конечное напряжение в мВ

jack_type тип разъема, ожидаемый для данного напряжения

время отключения время отключения для разъема драйвер кодека

должен подождать это время, прежде чем считывать показания АЦП

для

напряжений **список** запись внутреннего списка

`struct snd_soc_jack_gpio`

Описывает вывод gpio для

определения разъема **Определение**

```
структура snd_soc_jack_gpio {
    unsigned int gpio;
    unsigned int idx;
    struct device * gpiod_dev;
    const char * name;
    int report;
    int invert;
    int debounce_time;
    bool wake;
    int ( * jack_status_check ) (void *data);
};
```

Участники

gpio устаревший номер gpio

idx индекс дескриптора gpio в функции потребительского устройства

GPIO **gpiod_dev** Потребительское устройство GPIO

Имя имя gpio. Также в качестве идентификатора подключения для функции поиска имени устройства-потребителя GPIO

Сообщить значение для сообщения при обнаружении гнезда

инвертировать сообщить о наличии в низком состоянии

debounce_time время отмены в мс

пробуждение включить в качестве источника пробуждения

jack_status_check функция обратного вызова, которая переопределяет обнаружение для обеспечения более сложных проверок (например,, считывание АЦП).

struct snd_soc_codec * snd_soc_component_to_codec(struct snd_soc_component * component)
Преобразует компонент в КОДЕК, в который он встроен

Параметры

структура **snd_soc_component** * **component** Компонент
для преобразования в КОДЕК **Описание**

Эта функция должна использоваться только с компонентами, которые, как известно, являются кодеками. В противном случае поведение не определено. struct snd_soc_platform

*** snd_soc_component_to_platform(struct snd_soc_component * component)**
Приводит компонент к платформе, в которую он встроен

Параметры

struct **snd_soc_component** * **компонент** Компонент
для приведения к платформе **Описание**

Эта функция должна использоваться только в компонентах, которые, как известно, являются платформами. В противном случае поведение не определено. struct snd_soc_component

*** snd_soc_dapm_to_component(struct snd_soc_dapm_context * dapm)**
Приводит контекст DAPM к компоненту, в который он встроен

Параметры

struct **snd_soc_dapm_context** * **dapm** Контекст
DAPM для приведения к компоненту **Описание**

Эта функция должна использоваться только в контекстах DAPM, которые, как известно, являются частью компонента (например, в драйвере компонента). В противном случае поведение не определено. struct snd_soc_codec

*** snd_soc_dapm_to_codec(struct snd_soc_dapm_context * dapm)**
Преобразует контекст DAPM в кодек, в который он встроен

Параметры

структура **snd_soc_dapm_context** * **dapm** Контекст
DAPM для приведения к КОДЕКУ **Описание**

Эта функция должна использоваться только в контекстах DAPM, которые, как известно, являются частью КОДЕКА (например, в драйвере КОДЕКА). В противном случае поведение не определено. struct snd_soc_platform

*** snd_soc_dapm_to_platform(struct snd_soc_dapm_context * dapm)**
Применяет контекст DAPM к платформе, в которую он встроен

Параметры

структура **snd_soc_dapm_context** * **dapm** Контекст
DAPM для приведения к платформе. **Описание**

Эта функция должна использоваться только в контекстах DAPM, которые, как известно, являются частью платформы (например, в драйвере платформы). В противном случае поведение не определено.

структура `snd_soc_dapm_context` * `snd_soc_component_get_dapm`(struct `snd_soc_component` * *компонент-ponent*)

Возвращает контекст DAPM, связанный с компонентом

Параметры

`struct snd_soc_component` * `компонент` Компонент, для которого нужно получить контекст DAPM
`struct snd_soc_dapm_context` * `snd_soc_codec_get_dapm`(struct `snd_soc_codec` * *codec*)
Возвращает контекст DAPM для КОДЕКА

Параметры

`struct snd_soc_codec` * `кодек` КОДЕК, для которого требуется получить контекст DAPM

Примечание

Используйте эту функцию вместо прямого доступа к полю `dapm` КОДЕКА

`void snd_soc_codec_init_bias_level`(структура * *кодек*, `snd_soc_codec` `snd_soc_bias_level` *Уровень*) *перечисление*
Инициализировать уровень смещения кодека DAPM

Параметры

`struct snd_soc_codec` * `codec` КОДЕК, для которого инициализируется уровень смещения DAPM
перечислите `уровень` `snd_soc_bias_level` Уровень DAPM для инициализации

Описание

Инициализирует уровень смещения DAPM кодека. См. `snd_soc_dapm_init_bias_level()`. *перечислить*
`snd_soc_bias_level` `snd_soc_codec_get_bias_level`(struct `snd_soc_codec` * *codec*)
Получить текущий уровень смещения DAPM КОДЕКА

Параметры

`struct snd_soc_codec` * `codec` КОДЕК, для которого нужно получить уровень смещения DAPM

Возврат

Текущий уровень смещения DAPM КОДЕКА.

`int snd_soc_codec_force_bias_level`(struct `snd_soc_codec` * *codec*, `snd_soc_bias_level` *Уровень*) *перечисление*
Установите уровень смещения DAPM КОДЕКА

Параметры

`структура snd_soc_codec` * `codec` КОДЕК, для которого нужно установить уровень *перечислите* `snd_soc_bias_level` `level` Уровень, который нужно установить на

Описание

Переводит уровень смещения кодека в определенное состояние. Смотрите `snd_soc_dapm_force_bias_level()`.
`struct snd_soc_codec` * `snd_soc_dapm_kcontrol_codec`(структура `snd_kcontrol` * *kcontrol*)

Возвращает кодек, связанный с `kcontrol`

Параметры

`структура snd_kcontrol` * `kcontrol` `kcontrol`

Описание

Эта функция должна использоваться только в контекстах DAPM, которые, как известно, являются частью КОДЕКА (например, в драйвере кодека). В противном случае поведение не определено.
`int snd_soc_cache_sync`(struct `snd_soc_codec` * *кодек*)

Синхронизирует кэш регистров с аппаратным обеспечением

Параметры

`struct snd_soc_codec * codec` КОДЕК

для синхронизации **Примечание**

Эта функция вызовет `regcache_sync()`

`void snd_soc_codec_init_regmap(struct snd_soc_codec * кодек, struct regmap * regmap)`

Инициализировать экземпляр `regmap` для КОДЕКА

Параметры

`структура snd_soc_codec * codec` КОДЕК, для которого нужно инициализировать

экземпляр `regmap` `struct regmap * regmap` Экземпляр `regmap`, который должен

использоваться КОДЕКОМ **Описание**

Эта функция позволяет отложить назначение экземпляра `regmap`, который связан с кодеком.

Используйте это только в том случае, если экземпляр `regmap` еще не готов к

регистрации КОДЕКА. Функция также должна быть вызвана перед первой попыткой

ввода-вывода кодека. `void snd_soc_codec_exit_regmap(struct snd_soc_codec * кодек)`

Деинициализируйте экземпляр `regmap` для КОДЕКА

Параметры

`struct snd_soc_codec * codec` КОДЕК, для которого

нужно деинициализировать экземпляр `regmap` **Описание**

Вызовы `regmap_exit()` на экземпляре `regmap`, связанном с кодеком, и удаляет

экземпляр `regmap` из КОДЕКА. Эту функцию следует использовать

только в том случае, если *функция* `snd_soc_codec_init_regmap()` использовалась

для инициализации экземпляра `regmap`. `struct`

`snd_soc_component * snd_soc_kontrol_component(struct snd_kontrol * kcontrol)`

Возвращает компонент, который зарегистрировал элемент управления

Параметры

`Структура snd_kontrol * kcontrol` Элемент управления, для

которого требуется получить компонент **Примечание**

Эта функция будет работать корректно, если элемент управления зарегистрирован для компонента.

Либо с

помощью `snd_soc_add_codec_controls()` или `snd_soc_add_platform_controls()` или с помощью табличной настройки

для либо кодека, платформы, либо драйвера компонента. В противном случае поведение не определено.

`struct snd_soc_codec * snd_soc_kontrol_codec(структура snd_kontrol * kcontrol)`

Возвращает КОДЕК, который зарегистрировал элемент управления

Параметры

`struct snd_kontrol * kcontrol` Элемент управления, для которого

требуется получить КОДЕК **Примечание**

Эта функция будет работать корректно, только если элемент управления был

был

с зарегистрированными

`snd_soc_add_codec_controls()` или с помощью табличной настройки

случае используется функция

`snd_soc_codec_driver`. поведение не определено. `struct`

`snd_soc_platform * snd_soc_kontrol_platform(структура snd_kontrol * kcontrol)`

Возвращает платформу, которая зарегистрировала элемент управления

Параметры

`структура snd_kontrol * kcontrol` Элемент управления, для

которого требуется получить платформу **Примечание**

Эта функция позволит только работать правильно, если этот элемент управления было зарегистрирован в другом-
`snd_soc_add_platform_controls()` или с помощью табличной настройки
`snd_soc_platform_driver`. поскольку поведение не определено. `struct`
`snd_soc_dai * snd_soc_find_dai (const struct snd_soc_dai_link_component * dlc)`
Найдите зарегистрированный DAI

Параметры

`const struct snd_soc_dai_link_component * dlc` имя DAI и дополнительная информация о компоненте для соответствия

Описание

Эта функция выполнит поиск по всем зарегистрированным компонентам и их DAI, чтобы найти DAI с таким же именем. `Of_node` и `name` компонента также должны совпадать, если они указаны. **Возврат**
указатель DAI или `NULL`, если не найден. `struct snd_soc_dai_link *`
`snd_soc_find_dai_link (struct snd_soc_card * карточка, int id, постоянный символ`
`* Имя, постоянный символ * имя_потока)`
Найти ссылку на DAI

Параметры

`структура snd_soc_card * card` `soc card`
`идентификатор int` Идентификатор ссылки DAI для соответствия
`постоянный символ * имя` Имя канала DAI для соответствия,
необязательно `постоянный символ * имя_потока` Имя потока DAI для соответствия, необязательно **Описание**

Эта функция выполнит поиск по всем существующим ссылкам DAI карты `soc`, чтобы найти ссылку с таким же идентификатором. Поскольку ссылки DAI могут не иметь своего уникального идентификатора, поэтому `name` и название потока также должны совпадать, если они указаны. **Возврат**
указатель ссылки DAI или `NULL`, если не найден.
`int snd_soc_add_dai_link (структура snd_soc_card * card, struct snd_soc_dai_link * dai_link)`
Добавление ссылки DAI динамически

Параметры

`struct snd_soc_card * карточка` Карточка ASoC, к которой добавляется ссылка DAI `struct snd_soc_dai_link * dai_link` Новая ссылка DAI для добавления **Описание**

Эта функция добавляет ссылку DAI в список ссылок карты ASoC.

Примечание

Топология может использовать этот API для добавления ссылок DAI при проверке компонента топологии. И драйверы машин все еще могут определять статические ссылки DAI в массиве `dai_link`.
`void snd_soc_remove_dai_link (struct snd_soc_card * card, struct snd_soc_dai_link * dai_link)`
Удалить ссылку DAI из списка

Параметры

`struct snd_soc_card * карточка` Карточка ASoC, которой принадлежит ссылка `struct snd_soc_dai_link * dai_link` Ссылка DAI для удаления

Описание

Эта функция удаляет ссылку DAI из списка ссылок карты ASoC.

Для ссылок DAI, ранее добавленных topology, topology должна удалить их с помощью dobj, встроенного в ссылку. int

snd_soc_runtime_set_dai_fmt(struct snd_soc_pcm_runtime * rtd, unsigned int dai_fmt)
Измените формат ссылки DAI для среды выполнения ASoC

Параметры

struct snd_soc_pcm_runtime * rtd Среда выполнения, для которой формат ссылки DAI должен быть
изменен unsigned int dai_fmt Новый формат DAI link

Описание

Эта функция обновляет формат DAI link для всех DAI, подключенных к DAI link, для указанной среды
выполнения. **Примечание**

Для настроек со статическим форматом установите поле dai_fmt в соответствующей структуре
snd_dai_link вместо использования этой функции.

Возвращает 0 при успешном завершении, в противном случае отрицательный код ошибки.

int snd_soc_set_dmi_name(struct snd_soc_card * card, const char * вкус)
Регистрируйте имена DMI в card

Параметры

структура snd_soc_card * card Карта для регистрации имен DMI
постоянный символ * вкус Вкусное "отличие" карты среди аналогов.

Описание

Драйвер компьютера Intel может использоваться многими различными устройствами, но пользовательскому пространству
трудно различать названия, поскольку драйверы компьютеров обычно используют свое собственное имя в качестве
краткого названия карты и оставляют длинное имя карты пустым. Чтобы дифференцировать такие устройства и исправить
ошибки из-за отсутствия конфигураций для конкретного устройства, эта функция позволяет использовать информацию DMI
в качестве длинного названия звуковой карты в формате "поставщик-продукт- версия-плата" (символ '-' используется здесь
для разделения различных полей DMI). Это поможет пользовательскому пространству загрузить конфигурации диспетчера
вариантов использования для конкретного устройства (UCM) для карты. Возможные длинные названия карт:
DellInc.-XPS139343-01-0310JH asustek computerinc.-T100TA-1.0- Схема T100TA-MinnowboardMaxD0PLATFORM-D0-MinnowBoardMAX

Эта функция также поддерживает изменение длинного имени карты
для обеспечения дополнительной дифференциации, например "поставщик- продукт-версия-плата-вкус". Мы сохраняем
только цифры и символы алфавита, а также несколько символов-разделителей в длинном имени карты, поскольку
UCM в пространстве пользователя использует длинные имена карт в качестве имен каталога конфигурации карты, а
AudioConf не может поддерживать специальные символы, такие как ПРОБЕЛ.
Возвращает 0 в случае успеха, в противном случае отрицательный код ошибки.

структура snd_control * snd_soc_cnew(постоянная структура snd_control_new * _template, void * данные, const
символ * long_name, const символ * префикс)
создать новый элемент управления

Параметры

постоянная структура snd_control_new * _template шаблон
элемента управления void * data управление частными данными
постоянный символ * длинное_имя длинное имя элемента управления
префикс const char * префикс имени элемента управления

Описание

Создайте новый элемент управления микшером из элемента управления шаблона. Возвращает 0 для успешного выполнения,

иначе ошибка. `int snd_soc_add_component_controls` (структура `snd_soc_component` * *компонент*, `const` структура `snd_control_new` * *элементы управления*, `unsigned int` *num_controls*)
Добавление массива элементов управления в компонент.

Параметры

структура `snd_soc_component` * *component* Компонент для добавления элементов управления в `const struct snd_control_new` * *элементы управления* Массив элементов управления для добавления `unsigned int` *num_controls* Количество элементов в массиве

Возврат

0 для успеха, иначе ошибка.

`int snd_soc_add_codec_controls` (`struct snd_soc_codec` * *codec*, `const struct snd_control_new` * *элементы управления*, `unsigned int` *num_controls*)

добавьте массив элементов управления в кодек. Удобная функция для добавления списка элементов управления. Многие кодеки дублировали этот код.

Параметры

структура `snd_soc_codec` * *codec* кодек для добавления элементов управления в `const struct snd_control_new` * *элементы управления* массив элементов управления для добавления `unsigned int` *num_controls* количество элементов в массиве

Описание

Возвращает 0 в случае успеха, иначе ошибка. `int`

`snd_soc_add_platform_controls` (`struct snd_soc_platform` * *Платформа*, `const struct snd_control_new` * *элементы управления*, `unsigned int` *num_controls*)

добавьте массив элементов управления на платформу. Удобная функция для добавления списка элементов управления.

Параметры

структура `snd_soc_platform` * *платформа* платформа для добавления элементов управления `const struct snd_control_new` * *элементы управления* массив элементов управления для добавления `unsigned int` *num_controls* количество элементов в массиве

Описание

Возвращает 0 при успешном завершении, иначе ошибка.

`int snd_soc_add_card_controls` (`struct snd_soc_card` * *soc_card*, `const struct snd_control_new` * *элементы управления*, `int` *num_controls*)

добавьте массив элементов управления на карту SoC. Удобная функция для добавления списка элементов управления.

Параметры

структура `snd_soc_card` * *soc_card* SoC-карта для добавления элементов управления `const struct snd_control_new` * *элементы управления* массив добавляемых элементов управления `int` *num_controls* количество элементов в массиве

Описание

Возвращает 0 в случае успеха, в противном случае ошибка.

`int snd_soc_add_dai_controls` (структура `snd_soc_dai` * *dai*, `const struct snd_control_new` * *элементы управления*, `int` *num_controls*)
добавьте массив элементов управления в DAI. Удобная функция для добавления списка элементов управления.

Параметры

структура snd_soc_dai * dai DAI для добавления элементов управления

В const struct snd_control_new * элементы управления МАССИВ

элементов управления для добавления int num_controls КОЛИЧЕСТВО

элементов в массиве **Описание**

Возвращает 0 в случае успеха, иначе ошибка.

int snd_soc_dai_set_sysclk(структура snd_soc_dai * dai, int clk_id, unsigned int freq, int dir)

настройте системные или основные часы DAI.

Параметры

структура snd_soc_dai * dai DAI

int clk_id Идентификатор часов, специфичный для

DAI unsigned int freq новая тактовая частота в Гц

int dir новое направление синхронизации - ввод /

вывод. **Описание**

Настраивает синхронизацию DAI master (MCLK) или system (SYSCLK).

int snd_soc_codec_set_sysclk(struct snd_soc_codec * кодек, int clk_id, int Источник, без знака
int freq, int dir)

настройте системный кодек или основную частоту.

Параметры

struct snd_soc_codec * codec КОДЕК

int clk_id Идентификатор синхронизации,

специфичный для DAI int source Источник для синхронизации

беззнаковая int freq новая тактовая частота в Гц

int dir новое направление синхронизации - ввод /

вывод. **Описание**

Настраивает синхронизацию главного кодека (MCLK) или системы (SYSCLK).

int snd_soc_dai_set_clkdiv(struct snd_soc_dai * dai, int div_id, int div)
настройте тактовые разделители DAI.

Параметры

struct snd_soc_dai * dai DAI

int div_id Идентификатор тактового

делителя для конкретного DAI int div новый

тактовый делитель. **Описание**

Настраивает разделители тактовых импульсов. Это используется для получения наилучших битовых и кадровых тактовых сигналов DAI из системных или основных тактовых сигналов. Лучше всего установить бит DAI и частоту кадров как можно ниже для экономии энергии системы. int snd_soc_dai_set_pll(struct snd_soc_dai * dai, int pll_id, int Источник, unsigned int freq_in, unsigned int freq_out)
настройте DAI PLL.

Параметры

структура snd_soc_dai * dai DAI

int pll_id Идентификатор PLL, специфичный для DAI

исходный код int Конкретный источник DAI для PLL

unsigned int freq_in Входная тактовая частота PLL в Гц

unsigned int freq_out запрошенная выходная тактовая частота PLL в Гц

Описание

Настраивает и позволяет PLL генерировать выходные тактовые частоты на основе входных тактовых частот.

int snd_soc_dai_set_bclk_ratio(struct snd_soc_dai * dai, unsigned int *соотношение*)
настройте соотношение BCLK к частоте дискретизации.

Параметры

struct snd_soc_dai * dai DAI

беззнаковый коэффициент int Отношение BCLK к частоте дискретизации.

Описание

Настраивает DAI на заданное соотношение BCLK к частоте дискретизации.

int snd_soc_dai_set_fmt(структура snd_soc_dai * dai, unsigned int *fmt*)
настройка аппаратного аудиоформата DAI.

Параметры

структура snd_soc_dai * dai DAI

неподписанный int *fmt* **SND_SOC_DAIFMT_** значение формата.

Описание

Настраивает аппаратный формат DAI и синхронизацию.

int snd_soc_xlate_tdm_slot_mask(unsigned int *слоты*, unsigned int * *tx_mask*, unsigned int * *rx_mask*)
сгенерируйте маску tx / rx-слота.

Параметры

неподписанные int-слоты Количество используемых слотов.

неподписанный int * *tx_mask* битовая маска, представляющая активные TX-слоты.

unsigned int * *rx_mask* битовая маска, представляющая активные RX-слоты.

Описание

Генерирует маски слотов TDM tx и rx по умолчанию для DAI.

int snd_soc_dai_set_tdm_slot(структура snd_soc_dai * dai, без знака int *tx_mask*, без знака int *rx_mask*, int *слоты*, int *slot_width*)
Настраивает DAI для работы TDM

Параметры

структура snd_soc_dai * dai DAI для настройки

unsigned int *tx_mask* битовая маска, представляющая активные TX-слоты.

unsigned int *rx_mask* битовая маска, представляющая активные слоты RX.

int *slots* Количество используемых слотов.

int *slot_width* Ширина в битах для каждого слота.

Описание

Эта функция настраивает указанный DAI для работы TDM. **слот** содержит общее количество слотов потока TDM и **slot_with** ширину каждого слота в битовых тактах. **tx_mask** и **rx_mask** являются битовыми масками, определяющими активные слоты потока TDM для указанного DAI, т.Е. В какие слоты DAI должен записывать или считывать из них. Если бит установлен, соответствующий слот активен, если бит очищен, соответствующий слот активен.

неактивен. Бит 0 соответствует первому слоту, бит 1 - второму слоту и так далее. Первый активный слот сопоставляется с первым каналом DAI, второй активный слот - со вторым каналом и так далее.

Режим TDM можно отключить, передав значение 0 для **слотов**. В этом случае **tx_mask**, **rx_mask** и **slot_width** будут игнорироваться.

Возвращает 0 при успешном завершении, в противном случае отрицательный код ошибки.

```
int snd_soc_dai_set_channel_map(struct snd_soc_dai *dai, unsigned int tx_num, unsigned int
                                *tx_slot, unsigned int rx_num, unsigned int *rx_slot)
    настройка карты аудиоканалов DAI
```

Параметры

структура snd_soc_dai * dai DAI

unsigned int tx_num сколько каналов передачи

unsigned int * tx_slot указатель на массив, который подразумевает, что канал 0 ~ num-1

использует номер TX-слота unsigned int rx_num сколько RX-каналов

unsigned int * rx_slot указатель на массив, который подразумевает номер RX-слота,

используемый каналом 0 ~ num-1

Описание

настройте взаимосвязь между номером канала и номером слота TDM.

```
int snd_soc_dai_set_tristate(struct snd_soc_dai *dai, int tristate)
    настройте систему DAI или основные часы.
```

Параметры

структура snd_soc_dai * dai DAI

внутреннее трехсоставное состояние включить

трехсоставное состояние

Описание

Преобразует DAI, чтобы другие могли его использовать.

```
int snd_soc_dai_digital_mute(struct snd_soc_dai *dai, int отключить звук, int направление)
    настройте систему DAI или основные
```

часы. Параметры

структура snd_soc_dai * dai DAI

немой звук отключение звука включить

направление ввода поток для отключения

Описание

Отключает DAI DAC.

```
int snd_soc_register_card(структура snd_soc_card * card)
    Зарегистрировать карту в ядре ASoC
```

Параметры

структура snd_soc_card * card Карта для регистрации

int snd_soc_unregister_card(структура snd_soc_card * card)

Отменить регистрацию карты с ядром ASoC

Параметры

структура snd_soc_card * card Карточка для отмены регистрации

недействительна snd_soc_unregister_dais(структура snd_soc_component * компонент)

Отменить регистрацию

DAIs из ядра ASoC

Параметры

структура snd_soc_component * компонент Компонент, для которого DAIs должен быть незарегистрирован

int snd_soc_register_dais (struct snd_soc_component * *компонент*, struct snd_soc_dai_driver

* *dai_drv*, size_t *count*, bool *legacy_dai_naming*)

Зарегистрируйте DAI в ядре ASoC

Параметры

структура snd_soc_component * компонент Компонент, для которого зарегистрированы DAI

структура snd_soc_dai_driver * dai_drv Драйвер DAI, используемый для платформы

size_t count Количество платформ

bool legacy_dai_naming Используйте устаревшую схему именования и позвольте DAI наследовать

родительское имя. int snd_soc_register_dai (struct snd_soc_component * *компонент*, struct snd_soc_dai_driver

* *dai_drv*)

Динамически регистрируйте DAI и создавайте его виджеты

Параметры

struct snd_soc_component * компонент Компонент, для которого зарегистрирован

DAI структура snd_soc_dai_driver * dai_drv Драйвер DAI для использования в DAI

Описание

Топология может использовать этот API для регистрации DAIs при исследовании компонента. Виджеты

этих платформ будут освобождены при очистке карты, а платформа будет освобождена при очистке компонента.

void snd_soc_component_init_regmap (struct snd_soc_component * *компонент*, struct regmap

* *regmap*)

Инициализировать экземпляр regmap для компонента

Параметры

struct snd_soc_component * компонент Компонент, для которого нужно инициализировать

экземпляр regmap struct regmap * regmap Экземпляр regmap, который должен

использоваться компонентом

Описание

Эта функция позволяет отложить назначение экземпляра regmap, связанного с компонентом.

Используйте это, только если экземпляр regmap еще не готов на момент регистрации

компонента. Функция также должна быть вызвана перед первой попыткой ввода-вывода компонента.

void snd_soc_component_exit_regmap (структура snd_soc_component * *компонент*)

Деинициализируйте экземпляр regmap для компонента

Параметры

struct snd_soc_component * компонент Компонент, для которого нужно

деинициализировать экземпляр regmap

Описание

Вызовы regmap_exit() на экземпляре regmap, связанном с компонентом, и удаляет

экземпляр regmap из компонента. Эту функцию следует

использовать только в том случае, если *snd_soc_component_init_regmap()* использовался

для инициализации экземпляра regmap.

void snd_soc_unregister_component (структурировать устройство * *dev*)

Отменить регистрацию компонента в ядре ASoC

Параметры

структурировать устройство * dev Устройство для отмены регистрации int

snd_soc_add_platform (структурировать устройство * *dev*, структурировать snd_soc_platform * *Платформа*, const struct

snd_soc_platform_driver * *platform_drv*)

Добавление платформы в ядро ASoC

Параметры

структура device * dev Родительское устройство для платформы

структура snd_soc_platform * платформа Платформа для добавления

постоянная структура snd_soc_platform_driver * platform_drv Драйвер для платформы int

snd_soc_register_platform(структурное устройство * *разработчик*, const struct snd_soc_platform_driver * *platform_drv*)

Зарегистрировать платформу в ядре ASoC

Параметры

структурировать устройство * dev Устройство для платформы

постоянная структура snd_soc_platform_driver * platform_drv Драйвер для платформы

недействителен snd_soc_remove_platform(структура snd_soc_platform * *Платформа*)

Удаление платформы из ядра ASoC

Параметры

struct snd_soc_platform * платформа платформа для удаления

void snd_soc_unregister_platform(устройство структуры * *разработчик*)

Отменить регистрацию платформы в ядре ASoC

Параметры

структура устройства * dev платформа для отмены регистрации int

snd_soc_register_codec(структура устройства * *dev*, const struct snd_soc_codec_driver * *codec_drv*, struct snd_soc_dai_driver * *dai_drv*, int *num_dai*)

Зарегистрируйте кодек в ядре ASoC

Параметры

структура device * dev Родительское устройство для этого кодека

постоянная структура snd_soc_codec_driver * codec_drv Драйвер

кодека структура snd_soc_dai_driver * dai_drv Связанный

драйвер DAI int num_dai Номер платформы

void snd_soc_unregister_codec(структура устройства * *dev*)

Отменить регистрацию кодека в ядре ASoC

Параметры

структура device * dev кодек для отмены регистрации

int devm_snd_soc_register_component(структура устройство * *dev*, const структура snd_soc_component_driver * *cmpnt_drv*, struct snd_soc_dai_driver * *dai_drv*, int *num_dai*)

регистрация компонента, управляемого ресурсами

Параметры

структура device * dev Устройство, используемое для управления

компонентом постоянная структура snd_soc_component_driver * cmpnt_drv Драйвер

компонента структура snd_soc_dai_driver * dai_drv Драйвер DAI

int num_dai Количество платформ для регистрации

Описание

Зарегистрируйте компонент с автоматической отменой регистрации, когда устройство не зарегистрировано.

int devm_snd_soc_register_platform(структурное устройство * *разработчик*, const struct snd_soc_platform_driver * *platform_drv*)
регистрация платформы, управляемой ресурсами

Параметры

устройство структуры * dev Устройство, используемое для управления платформой
постоянная структура snd_soc_platform_driver * platform_drv платформа для регистрации

Описание

Зарегистрируйте драйвер платформы с автоматической отменой регистрации, когда устройство не зарегистрировано.

int devm_snd_soc_register_card(структурируйте устройство * *dev*, struct snd_soc_card * *карта*)
регистрация карты, управляемой ресурсами

Параметры

устройство структуры * dev Устройство, используемое для управления картой
структура snd_soc_card * карточка Карточка для регистрации

Описание

Зарегистрируйте карту с автоматической отменой регистрации, когда устройство не зарегистрировано.

int devm_snd_dmaengine_pcm_register(структурное устройство * *dev*, const struct snd_dmaengine_pcm_config * *snd_dmaengine_pcm_config* * конфигурация, unsigned int *флаги*)
Регистрация PCM dmaengine с управлением ресурсами

Параметры

структура device * dev Родительское устройство для устройства PCM
const struct snd_dmaengine_pcm_config * config Конфигурация PCM для конкретной платформы
флаги unsigned int Особенности, зависящие от платформы

Описание

Зарегистрируйте PCM-устройство на базе dmaengine с автоматической отменой регистрации, когда устройство не зарегистрировано.
int snd_soc_component_read(struct snd_soc_component * *component*, unsigned int *reg*, без знака int * *val*)
Считанное значение регистра

Параметры

структура snd_soc_component * компонент Компонент для чтения
unsigned int reg Зарегистрируйтесь для чтения
unsigned int * val Указатель на место, где хранится прочитанное значение

Возврат

0 при успешном завершении, в противном случае отрицательный код ошибки.

int snd_soc_component_write(struct snd_soc_component * *component*, unsigned int *reg*, без знака int *val*)
Значение регистра записи

Параметры

структура snd_soc_component * компонент Компонент для записи
unsigned int reg Зарегистрируйтесь для записи
unsigned int val Значение для записи в регистр

Возврат

0 при успешном выполнении, в противном случае - отрицательный код ошибки.

`int snd_soc_component_update_bits` (структура `snd_soc_component` * *компонент*, unsigned int *reg*,
неподписанный int *маска*, unsigned int *значение*)
Выполнить цикл чтения / изменения / записи

Параметры

`структура snd_soc_component` * `компонент` Компонент для обновления

`unsigned int reg` Зарегистрируйтесь для обновления

`unsigned int mask` Маска, указывающая, какие биты обновлять

`unsigned int val` Новое значение для битов, указанных mask

Возврат

1, если операция прошла успешно и значение регистра изменилось, 0, если операция прошла успешно,
но значение не изменилось. В противном случае возвращает отрицательный код ошибки.

`int snd_soc_component_update_bits_async` (структура `snd_soc_component` * *компонент*, без знака
int reg, без знака *int маска*, unsigned int *значение*)
Выполнение асинхронного цикла чтения / изменения / записи

Параметры

`структура snd_soc_component` * `component` Компонент для обновления

`unsigned int reg` Зарегистрируйтесь для обновления

`unsigned int mask` Маска, указывающая, какие биты обновлять

`unsigned int val` Новое значение для битов, указанных в mask

Описание

Эта функция похожа на `snd_soc_component_update_bits()`, но операция обновления запланирована асинхронно. Это означает, что она может не завершиться при возврате функции. Чтобы убедиться, что все запланированные обновления завершены *необходимо* *вызвать* `snd_soc_component_async_complete()` .

Возврат

1, если операция прошла успешно и значение регистра изменилось, 0, если операция прошла успешно,
но значение не изменилось. В противном случае возвращает отрицательный код ошибки.

`void snd_soc_component_async_complete` (структура `snd_soc_component` * *компонент*)
Убедитесь, что асинхронный ввод-вывод завершен

Параметры

`struct snd_soc_component` * `компонент` Компонент,
которого нужно ждать

Описание

Эта функция блокирует до тех пор, пока не будут выполнены все асинхронные операции ввода-вывода,
которые ранее были запланированы с помощью `snd_soc_component_update_bits_async()` завершенно.

`int snd_soc_component_test_bits` (struct `snd_soc_component` * *компонент*, беззнаковый int *reg*, un-
signed int *маска*, unsigned int *значение*)
Проверьте регистр на предмет изменения

Параметры

`структура snd_soc_component` * `компонент` КОМПОНЕНТ

`unsigned int reg` Зарегистрируйтесь для тестирования

`unsigned int mask` Маска, указывающая, какие биты
проверять `значение unsigned int` Значение для
проверки

Описание

Проверяет регистр с новым значением и проверяет, отличается ли новое значение от старого.

Возврат

1 для изменения, в противном случае 0.

int snd_soc_update_bits(struct snd_soc_codec * *codec*, unsigned int *reg*, unsigned int *маска*, unsigned int *значение*)
обновление битов регистра кодека

Параметры

struct snd_soc_codec * *codec* аудиокодек
unsigned int *reg* регистр кодека
unsigned int *mask* регистровая маска
значение без знака int новое значение

Описание

Записывает новое значение регистра.

Возвращает 1 при изменении, 0 при отсутствии изменений или отрицательный код ошибки.

int snd_soc_test_bits(структура snd_soc_codec * *кодек*, неподписанный int *reg*, неподписанный int *маска*, неподписанный int *значение*)
проверьте регистр на предмет изменения

Параметры

структура snd_soc_codec * *codec* аудиокодек
unsigned int *reg* регистр кодека
unsigned int *mask* регистровая маска
значение unsigned int новое значение

Описание

Проверяет регистр с новым значением и проверяет, отличается ли новое значение от старого. Возвращает 1 для изменения, иначе 0. void

snd_soc_runtime_activate(struct snd_soc_pcm_runtime * *rtd*, int *stream*)
Увеличивает количество активных компонентов среды выполнения PCM

Параметры

struct snd_soc_pcm_runtime * *rtd* Активированная среда выполнения
ASoC PCM int *stream* Направление потока PCM

Описание

Увеличивает количество активных для всех DAIS и компонентов, подключенных к среде выполнения PCM. Обычно должно вызываться при открытии потока. Должен быть вызван с удерживаемым rtd->pcm_mutex void
snd_soc_runtime_deactivate(struct snd_soc_pcm_runtime * *rtd*, int *поток*)
Уменьшает количество активных элементов для компонентов среды выполнения PCM

Параметры

структура snd_soc_pcm_runtime * *rtd* Деактивированная среда выполнения ASoC PCM *внутренний поток* Направление потока PCM

Описание

Уменьшает количество активных операций для всех DAIS и компонентов, подключенных к среде выполнения PCM. Обычно должно вызываться при закрытии потока. Должен быть вызван с удерживаемым rtd-> pcm_mutex

`bool snd_soc_runtime_ignore_pmdown_time(struct snd_soc_pcm_runtime * rtd)`
Проверьте, игнорировать ли задержку отключения питания

Параметры

`struct snd_soc_pcm_runtime * rtd` Среда
выполнения ASoC PCM, которую следует проверить. **Описание**

Эта функция проверяет, следует ли игнорировать задержку отключения питания для конкретной среды выполнения PCM. Возвращает true, если задержка равна 0, если канал DAI был настроен на игнорирование задержки или если ни один из компонентов не выигрывает от наличия задержки. `int snd_soc_set_runtime_hwparams(struct snd_pcm_substream * подпоток, const struct snd_pcm_hw_params * hw)`
установите аппаратные параметры среды выполнения

Parameters

`struct snd_pcm_substream * подпоток` подпоток pcm
постоянная структура `snd_pcm_hw_params * hw` параметры

оборудования **Описание**

Задаёт аппаратные параметры среды выполнения подпотока.

`int snd_soc_info_enum_double(struct snd_kcontrol * kcontrol, struct snd_ctl_elem_info * uinfo)`
обратный вызов информации о двойном микшере с перечислением

Параметры

`структура snd_kcontrol * kcontrol` управление микшером
`структура snd_ctl_elem_info * uinfo` информация об элементе

управления **Описание**

Обратный вызов для предоставления информации об элементе управления микшером с двойным перечислением. Возвращает 0 в случае успеха.

`int snd_soc_get_enum_double(struct snd_kcontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)`
получает обратный вызов перечисляемого двойного микшера

Параметры

`struct snd_kcontrol * kcontrol` элемент управления микшером
`структура snd_ctl_elem_value * ucontrol` информация об элементе

управления **Описание**

Обратный вызов для получения значения микшера с двойным перечислением. Возвращает 0 в случае успеха.

`int snd_soc_put_enum_double(struct snd_kcontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)`
перечислил обратный вызов двойного микшера

Параметры

`struct snd_kcontrol * kcontrol` элемент управления микшером
`структура snd_ctl_elem_value * ucontrol` информация об элементе

управления **Описание**

Обратный вызов для установки значения микшера с двойным перечислением. Возвращает 0 в случае успеха.

int snd_soc_read_signed(структура snd_soc_component *компонент, беззнаковый int рег, беззнаковый int маска, беззнаковый int сдвиг, unsigned int sign_bit, int *signed_val)

Считывает регистр кодека и интерпретирует как значение со знаком

Параметры

структура snd_soc_component *компонент КОМПОНЕНТ

unsigned int рег Регистр для чтения

unsigned int mask Маска, используемая после сдвига значения регистра

unsigned int shift Сдвиг значения регистра вправо

unsigned int sign_bit Бит, который описывает, является ли число отрицательным или нет. int * signed_val Указатель на место, где должно быть сохранено прочитанное значение

Описание

Эта функция считывает регистр кодека. Значение регистра сдвигается вправо на биты "shift" и маскируется с помощью заданной "маски". После этого он преобразует данное значение реестра в целое число со знаком, если sign_bit не равен нулю. Возвращает 0 при успешном выполнении, в противном случае значение ошибки int snd_soc_info_volsw(структура snd_kcontrol *kcontrol, структура snd_ctl_elem_info *uinfo)

обратный вызов информации об одном микшере

Параметры

структура snd_kcontrol *kcontrol управление микшером

структура snd_ctl_elem_info *uinfo информация об элементе управления

Описание

Обратный вызов для предоставления информации об одном элементе управления микшером или элементе управления двойным микшером, который охватывает 2 регистра-isters. Возвращает 0 в случае успеха. int snd_soc_info_volsw_sx(структура snd_kcontrol *kcontrol, структура snd_ctl_elem_info *uinfo)

Обратный вызов информации о микшере для элементов управления SX TLV

Параметры

структура snd_kcontrol *kcontrol управление микшером

структура snd_ctl_elem_info *uinfo информация об элементе управления

Описание

Обратный вызов для предоставления информации об одинарном элементе управления микшером или элементе управления двойным микшером, который охватывает 2 регистратора типа SX TLV. Элементы управления SX TLV имеют диапазон, который представляет как положительные, так и отрицательные значения по обе стороны от нуля, но без знака. Возвращает 0 для успешного выполнения. int snd_soc_get_volsw(структура snd_kcontrol *kcontrol, struct snd_ctl_elem_value *ucontrol)

обратный вызов для одного микшера

Параметры

struct snd_kcontrol *kcontrol управление микшером

struct snd_ctl_elem_value *ucontrol информация об элементе управления

Описание

Обратный вызов для получения значения элемента управления с одним микшером или элемента управления с двумя микшерами, который охватывает 2 регистра. Возвращает 0 в случае успеха.

int `snd_soc_put_volsw`(структура `snd_kontrol` * *kcontrol*, struct `snd_ctl_elem_value` * *ucontrol*)
обратный вызов с одним микшером

Параметры

struct `snd_kontrol` * `kcontrol` управление микшером

struct `snd_ctl_elem_value` * `ucontrol` информация об элементе

управления **Описание**

Обратный вызов для установки значения элемента управления с одним микшером или элемента управления с двумя микшерами, который охватывает 2 регистра. Возвращает 0 для успешного выполнения.

int `snd_soc_get_volsw_sx`(структура `snd_kontrol` * *kcontrol*, struct `snd_ctl_elem_value` * *ucontrol*)
обратный вызов для одного микшера

Параметры

структура `snd_control` * `kcontrol` управление микшером

структура `snd_ctl_elem_value` * `ucontrol` информация об элементе

управления **Описание**

Обратный вызов для получения значения элемента управления с одним микшером или элемента управления с двумя микшерами, который охватывает 2 регистра. Возвращает 0 для успешного выполнения.

int `snd_soc_put_volsw_sx`(структура `snd_kontrol` * *kcontrol*, структура `snd_ctl_elem_value` * *ucontrol*)
обратный вызов с двойным микшированием

Параметры

структура `snd_control` * `kcontrol` управление микшером

структура `snd_ctl_elem_value` * `ucontrol` информация об элементе

управления **Описание**

Обратный вызов для установки значения элемента управления с двойным микшированием, который охватывает 2 регистра. Возвращает 0 в случае успеха.

int `snd_soc_info_volsw_range`(структура `snd_kontrol` * *kcontrol*, структура `snd_ctl_elem_info` * *uinfo*)
обратный вызов информации об одном микшере с диапазоном.

Параметры

структура `snd_kontrol` * `kcontrol` управление микшером

структура `snd_ctl_elem_info` * `uinfo` информация об элементе

управления **Описание**

Обратный вызов для предоставления информации в пределах диапазона об одном элементе управления микшером. возвращает 0 в случае успеха.

int `snd_soc_put_volsw_range`(структура `snd_kontrol` * *kcontrol*, структура `snd_ctl_elem_value` * *ucontrol*)
одиночный микшер вводит обратный вызов значения с диапазоном.

Параметры

структура `snd_control` * `kcontrol` управление микшером

struct `snd_ctl_elem_value` * `ucontrol` информация об элементе

управления **Описание**

Обратный вызов для установки значения в пределах диапазона для одного элемента управления микшером. Возвращает 0 в случае успеха.

`int snd_soc_get_volsw_range` (структура `snd_kcontrol` * *kcontrol*, struct `snd_ctl_elem_value` * *ucontrol*)
одиночный микшер получает обратный вызов с диапазоном

Параметров

`struct snd_kcontrol` * `kcontrol` элемент управления микшером

структура `snd_ctl_elem_value` * `ucontrol` информация об элементе
управления **Описание**

Обратный вызов для получения значения в пределах диапазона одного
элемента управления микшером. Возвращает 0 в случае успеха.

`int snd_soc_limit_volume` (структура `snd_soc_card` * *card*, постоянный символ * *Имя*, int *max*)
Установите новое ограничение для существующего регулятора громкости.

Параметры

структура `snd_soc_card` * `card` где искать элемент
управления `const char` * `name` Название элемента
управления `int max` новый максимальный лимит

Описание

Возвращает 0 в случае успеха, иначе ошибка.

`int snd_soc_info_xr_sx` (struct `snd_kcontrol` * *kcontrol*, struct `snd_ctl_elem_info` * *uinfo*)
обратный вызов информации о нескольких регистрах со знаком

Параметры

`struct snd_control` * `kcontrol` элемент управления mreg

структура `snd_ctl_elem_info` * `uinfo` информация об элементе
управления **Описание**

Обратный вызов для предоставления информации об элементе управления, который
может охватывать несколько регистров кодека, которые вместе образуют
одно значение со знаком в формате MSB / LSB. Возвращает 0 в случае успеха. int

`snd_soc_get_xr_sx` (struct `snd_kcontrol` * *kcontrol*, struct `snd_ctl_elem_value` * *ucontrol*)
обратный вызов с несколькими регистрами со знаком get

Параметры

структура `snd_control` * `kcontrol` управление mreg

`struct snd_ctl_elem_value` * `ucontrol` информация об элементе
управления **Описание**

Обратный вызов для получения значения элемента управления, который может охватывать несколько
регистров кодека, которые вместе образуют одно подписанное значение способом MSB / LSB.
Элемент управления поддерживает указание общего количества битов, используемых для разрешения
битовых полей во множестве регистров кодека. Возвращает 0 для успешного выполнения.

`int snd_soc_put_xr_sx` (структура `snd_kcontrol` * *kcontrol*, структура `snd_ctl_elem_value` * *ucontrol*)
обратный вызов со знаком из нескольких регистров

Параметры

структура `snd_control` * `kcontrol` элемент управления mreg

структура `snd_ctl_elem_value` * `ucontrol` информация об элементе управления

Описание

Обратный вызов для установки значения элемента управления, который может охватывать несколько регистров кодека, которые вместе образуют одно подписанное значение способом MSB / LSB. Элемент управления поддерживает указание общего количества битов, используемых для разрешения битовых полей во множестве регистров кодека. Возвращает 0 для успешного выполнения.

int snd_soc_get_strobe (структура snd_kcontrol * *kcontrol*, структура snd_ctl_elem_value * *ucontrol*)
strobe получает обратный вызов

Параметры

структура snd_kcontrol * *kcontrol* управление микшером
структура snd_ctl_elem_value * *ucontrol* информация об элементе

управления Описание

Обратный вызов возвращает значение элемента управления микшером стробоскопа. Возвращает 0 в случае успеха.

int snd_soc_put_strobe (struct snd_kcontrol * *kcontrol*, struct snd_ctl_elem_value * *ucontrol*)
обратный вызов strobe put

Параметры

структура snd_kcontrol * *kcontrol* управление микшером
структура snd_ctl_elem_value * *ucontrol* информация об элементе

управления Описание

Обратный вызов преобразует бит регистра в высокий, затем в низкий (или наоборот) за один проход одного элемента управления перечислением микшера. Возвращает 1 в случае успеха. int snd_soc_new_compress (struct snd_soc_pcm_runtime * *rtd*, int *num*)
создайте новое сжатие.

Параметры

struct snd_soc_pcm_runtime * *rtd* Среда выполнения, для которой мы создадим compress int *num* индексный номер устройства (на основе нуля - используется совместно с обычными PCMS) **Возврат**

0 означает успех, в противном случае ошибка.

ASoC DAPM API

структура snd_soc_dapm_widget * snd_soc_dapm_kcontrol_widget (структура snd_kcontrol * *kcontrol*)
Возвращает виджет, связанный с kcontrol

Параметры

Структура snd_kcontrol * *kcontrol* В kcontrol структура snd_soc_dapm_context
* snd_soc_dapm_kcontrol_dapm (структура snd_kcontrol * *kcontrol*)
Возвращает контекст dapm, связанный с kcontrol

Параметры

структура snd_kcontrol * *kcontrol* kcontrol

Примечание

Эта функция должна использоваться только с kcontrol, которые, как известно, были зарегистрированы для кодека. В остальном поведение не определено.

int snd_soc_dapm_force_bias_level(структура snd_soc_dapm_context *dapm, enum snd_soc_bias_level *Уровень*)

Устанавливает уровень смещения DAPM

Параметры

структура snd_soc_dapm_context * dapm Контекст DAPM, для которого нужно

установить уровень

перечислите snd_soc_bias_level level Уровень для установки

Описание

Переводит уровень смещения DAPM в определенное состояние. Это вызовет обратный вызов уровня смещения контекста DAPM с указанным уровнем. Это произойдет даже в том случае, если контекст уже находится на том же уровне. Кроме того, он не будет проходить обычную последовательность уровней смещения, что означает, что никакие промежуточные состояния между текущим и целевым состоянием вводиться не будут. Обратите внимание, что изменение уровня смещения носит временный характер и произойдет в следующий раз *snd_soc_dapm_sync()* вызывается состояние будет установлено на уровень, определенный ядром DAPM. Функция в основном предназначена для использования во время проверки или возобновления работы с приостановки для включения питания устройства, чтобы можно было выполнить инициализацию, прежде чем ядро DAPM вступит во владение.

int snd_soc_dapm_set_bias_level(структура snd_soc_dapm_context *dapm, перечисление snd_soc_bias_level *Уровень*)

установите уровень смещения для системы

Параметры

структура snd_soc_dapm_context * dapm Контекст DAPM

перечисление snd_soc_bias_level уровень уровень для

настройки Описание

Настройте уровни смещения (мощности) для аудиоустройства

SoC. Возвращает 0 при ошибке success else.

int snd_soc_dapm_dai_get_connected_widgets(структура snd_soc_dai *dai, struct snd_soc_dapm_widget_list (*пользовательское_stop_condition) (struct snd_soc_dapm_widget *, перечисление snd_soc_dapm_direction) bool)

запрос звукового пути и его

виджетов.

Параметры

struct snd_soc_dai * dai soc DAI.

внутренний поток направление потока.

struct snd_soc_dapm_widget_list * * список список активных виджетов для этого потока.

bool (*)(структура snd_soc_dapm_widget *,перечисление snd_soc_dapm_direction) пользовательское_stop_condition

(необязательно) функция, предназначенная для остановки обхода

графика виджета на основе пользовательской логики. Описание

Запрашивает график DAPM относительно того, существует ли допустимый путь аудиопотока для исходного потока, указанного в *name* . При этом учитываются текущие настройки микшера и мультиплексора *kcontrol*. Создает список допустимых виджетов. Дополнительно может быть предоставлена функция, действующая как условие остановки. Эта функция принимает проверяемый в данный момент виджет *dapm* и направление прохождения в качестве аргументов, она должна возвращать *true*, если прохождение должно быть остановлено, и *false* в противном случае. Возвращает количество допустимых путей или отрицательную ошибку. *int snd_soc_dapm_sync_unlocked(struct snd_soc_dapm_context *dapm)*

пути сканирования и

питания *dapm*

Параметры

структура snd_soc_dapm_context * dapm Контекст

DAPM Описание

Обходит все аудиотрассы dapm и активирует виджеты в соответствии

с использованием их потока или дорожки. Требуется

внешняя блокировка. Возвращает 0 для успешного выполнения.

int snd_soc_dapm_sync(структура snd_soc_dapm_context * dapm)
пути сканирования и питания dapm

Параметры

структура snd_soc_dapm_context * dapm Контекст

DAPM Описание

Обходит все аудиотрассы dapm и активирует виджеты в соответствии

с их использованием потока или пути. Возвращает 0 в случае успеха.

int snd_soc_dapm_add_routes(struct snd_soc_dapm_context * dapm, const struct
snd_soc_dapm_route * маршрут, int число)
Добавление маршрутов между виджетами DAPM

Параметры

структура snd_soc_dapm_context * dapm Контекст DAPM

постоянная структура snd_soc_dapm_route * маршрут аудиомаршруты

общее число количество маршрутов

Описание

Соединяет 2 виджета dapm вместе с помощью именованного звукового тракта. Приемник - это виджет, принимающий аудиосигнал, в то время как источником является отправитель аудиосигнала.

Возвращает 0 при ошибке success else. При ошибке все ресурсы могут быть освобождены с помощью вызова snd_soc_card_free().

int snd_soc_dapm_del_routes(struct snd_soc_dapm_context * dapm, const struct
snd_soc_dapm_route * маршрут, int число)
Удаление маршрутов между виджетами DAPM

Параметры

struct snd_soc_dapm_context * dapm Контекст DAPM

постоянная структура snd_soc_dapm_route * маршрут аудиомаршруты

общее число количество маршрутов

Описание

Удаляет маршруты из контекста DAPM.

int snd_soc_dapm_weak_routes(структура snd_soc_dapm_context * dapm, const struct
snd_soc_dapm_route * маршрут, int num)
Помечает маршруты между виджетами DAPM как слабые

Параметры

struct snd_soc_dapm_context * dapm Контекст DAPM

постоянная структура snd_soc_dapm_route * маршрут аудиомаршруты

общее число количество маршрутов

Описание

Пометьте существующие маршруты, соответствующие указанным в переданном массиве, как слабые, что означает, что они игнорируются с целью принятия решений о полномочиях. Основной предполагаемый вариант использования - для путей с боковым уклоном, которые соединяются

аудио между другими независимыми каналами, если они оба активны, для улучшения работы комбинации на уровне пользователя, но которые не предназначены для "использования". Обратите внимание, что драйверам кодеков не следует использовать это, поскольку пути типа sidetone часто также могут использоваться в качестве обходных путей. `int snd_soc_dapm_new_widgets(struct snd_soc_card * card)`
добавить новые виджеты dapm

Параметры

`структура snd_soc_card * card` необходимо проверить карточку на наличие новых виджетов dapm

Описание

Проверяет кодек на наличие любых новых виджетов dapm и создает их, если они найдены. Возвращает 0 в случае успеха.

`int snd_soc_dapm_get_volsw(struct snd_kcontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)`
получение обратного вызова микшера dapm

Параметры

`структура snd_control * kcontrol` управление микшером
`struct snd_ctl_elem_value * ucontrol` информация об элементе

управления Описание

Обратный вызов для получения значения элемента управления микшером dapm. Возвращает 0 в случае успеха.

`int snd_soc_dapm_put_volsw(struct snd_kcontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)`
набор обратного вызова микшера dapm

Параметры

`struct snd_kcontrol * kcontrol` элемент управления микшером
`структура snd_ctl_elem_value * ucontrol` информация об элементе

управления Описание

Обратный вызов для установки значения элемента управления микшером dapm. Возвращает 0 в случае успеха.

`int snd_soc_dapm_get_enum_double(struct snd_control * kcontrol, struct snd_ctl_elem_value * ucontrol)`
двойной микшер с перечислением dapm получает обратный вызов

Параметры

`структура snd_control * kcontrol` управление микшером
`структура snd_ctl_elem_value * ucontrol` информация об элементе

управления Описание

Обратный вызов для получения значения элемента управления двойным микшером с перечислением dapm. Возвращает 0 в случае успеха.

`int snd_soc_dapm_put_enum_double(struct snd_kcontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)`
обратный вызов двойного микшера с перечислением dapm

Параметры

`структура snd_control * kcontrol` управление микшером
`struct snd_ctl_elem_value * ucontrol` информация об элементе управления

Описание

Обратный вызов для установки значения элемента управления двойным микшером с перечислением dapm. Возвращает 0 в случае успеха.

int snd_soc_dapm_info_pin_switch(структура snd_control * kcontrol, структура snd_ctl_elem_info * uinfo)
Информация для pin-переключателя

Параметры

структура snd_control * kcontrol управление микшером
структура snd_ctl_elem_info * uinfo информация об элементе управления

Описание

Обратный вызов для предоставления информации об элементе управления pin-переключателем.

int snd_soc_dapm_get_pin_switch(struct snd_kontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)
Получение информации для pin-переключателя

Параметры

структура snd_control * kcontrol управление микшером
структура snd_ctl_elem_value * ucontrol Значение
int snd_soc_dapm_put_pin_switch(struct snd_kontrol * kcontrol, struct snd_ctl_elem_value * ucontrol)
Задаёт информацию для pin-переключателя

Параметры

структура snd_kontrol * kcontrol элемент управления микшером
структура snd_ctl_elem_value * ucontrol Значение
int snd_soc_dapm_new_controls(struct snd_soc_dapm_context * dapm, const структура snd_soc_dapm_widget * виджет, int num)
создание новых элементов управления dapm

Параметры

структура snd_soc_dapm_context * dapm Контекст DAPM
постоянная структура snd_soc_dapm_widget * виджет МАССИВ виджетов
int num количество виджетов

Описание

Создаёт новые элементы управления DAPM на основе шаблонов. Возвращает 0 для ошибки success else.

void snd_soc_dapm_stream_event(структура snd_soc_pcm_runtime * rtd, int поток, int событие)
отправить событие stream в ядро dapm

Параметры

структура snd_soc_pcm_runtime * rtd Данные времени выполнения PCM
int stream название потока
входящее событие потоковое событие

Описание

Отправляет потоковое событие в ядро dapm. Затем ядро выполняет все необходимые изменения мощности виджета. Возвращает 0 при ошибке success else.

int snd_soc_dapm_enable_pin_unlocked(структура snd_soc_dapm_context * *dapm*, постоянный символ * *pin-код*)
включить pin-код.

Параметры

структура snd_soc_dapm_context * dapm Контекст

DAPM постоянный символ * pin имя pin

Описание

Включает PIN-код ввода / вывода и его родительские или дочерние виджеты при наличии
действительного аудиомаршрута и активного аудиопотока .

Требуется внешняя блокировка.

ПРИМЕЧАНИЕ

snd_soc_dapm_sync() после этого необходимо вызвать DAPM для переключения питания любого
виджета. int snd_soc_dapm_enable_pin(struct snd_soc_dapm_context * *dapm*, const char * *pin*)
включить pin.

Параметры

struct snd_soc_dapm_context * dapm Контекст DAPM

постоянный символ * pin имя pin

Описание

Возможность ввода/вывода и его родители или дети виджеты мкФ с допустимым аудио маршрута и активного
звука поток.

Примечание

snd_soc_dapm_sync() после этого необходимо вызвать функцию DAPM для переключения питания любого
виджета. int snd_soc_dapm_force_enable_pin_unlocked(структура snd_soc_dapm_context * *dapm*, постоянный символ
* *pin*)
принудительно включить pin-код

Параметры

структура snd_soc_dapm_context * dapm Контекст

DAPM постоянный символ * pin имя пин-кода

Описание

Включает вывод ввода / вывода независимо от любого другого состояния. Предназначен для
использования со смещением микрофона расходные материалы, используемые для
определения гнезда микрофона. Требуется внешняя блокировка.

ПРИМЕЧАНИЕ

snd_soc_dapm_sync() после этого необходимо вызвать DAPM для переключения питания любого
виджета. int snd_soc_dapm_force_enable_pin(struct snd_soc_dapm_context * *dapm*, const char * *pin*)
принудительно включить pin-код

Параметры

struct snd_soc_dapm_context * dapm Контекст DAPM

постоянный символ * pin имя pin

Описание

Включает вывод ввода / вывода независимо от любого другого состояния. Это
предназначено для использования со смещением микрофона расходные
материалы, используемые для определения гнезда микрофона. **ПРИМЕЧАНИЕ**

`snd_soc_dapm_sync()` после этого необходимо вызвать функцию DAPM для переключения питания любого виджета. `int snd_soc_dapm_disable_pin_unlocked(struct snd_soc_dapm_context *dapm, const char *pin)` отключить pin.

Параметры

структура `snd_soc_dapm_context` * `dapm` Контекст DAPM
постоянный символ * `pin` имя pin

Описание

Отключает PIN-код ввода / вывода и его родительские или дочерние виджеты. Требуется внешняя блокировка.

ПРИМЕЧАНИЕ

`snd_soc_dapm_sync()` после этого необходимо вызвать DAPM для переключения питания любого виджета. `int snd_soc_dapm_disable_pin(struct snd_soc_dapm_context *dapm, const char *pin)` отключить pin.

Параметры

структура `snd_soc_dapm_context` * `dapm` Контекст DAPM
постоянный символ * `pin-код` имя pin-кода

Описание

Отключает PIN-код ввода / вывода и его родительские или дочерние виджеты.

ПРИМЕЧАНИЕ

`snd_soc_dapm_sync()` после этого необходимо вызвать функцию DAPM для переключения питания любого виджета. `int snd_soc_dapm_nc_pin_unlocked(struct snd_soc_dapm_context *dapm, const char *pin)` навсегда отключить pin.

Параметры

структура `snd_soc_dapm_context` * `dapm` Контекст
DAPM постоянный символ * `pin` имя pin

Описание

Помечает указанный pin-код как не подключенный, отключая его во всех родительских или дочерних виджетах. В настоящее время это идентично `snd_soc_dapm_disable_pin()` но в будущем он будет расширен для выполнения дополнительных действий, таких как отключение элементов управления, которые влияют только на пути через pin-код. Требуется внешняя блокировка. **ПРИМЕЧАНИЕ**

`snd_soc_dapm_sync()` после этого необходимо вызвать DAPM для переключения питания любого виджета. `int snd_soc_dapm_nc_pin(struct snd_soc_dapm_context *dapm, const char *pin)` постоянно отключить pin.

Параметры

struct `snd_soc_dapm_context` * `dapm` Контекст DAPM
постоянный символ * `pin` имя pin

Описание

Помечает указанный pin-код как не подключенный, отключая его во всех родительских или дочерних виджетах. В настоящее время это идентично `snd_soc_dapm_disable_pin()` но в будущем он будет расширен для выполнения дополнительных действий, таких как отключение элементов управления, которые влияют только на пути через pin-код. **ПРИМЕЧАНИЕ**

`snd_soc_dapm_sync()` после этого необходимо вызвать функцию DAPM для переключения питания любого виджета. `int snd_soc_dapm_get_pin_status(struct snd_soc_dapm_context *dapm, const char *pin-код)` получить статус звукового pin-кода

Параметры

структура `snd_soc_dapm_context` * `dapm` Контекст DAPM
постоянный символ * `pin` конечная точка вывода аудиосигнала
(или начальная точка) **Описание**

Получает состояние аудиовыключения - подключено или отключено. Возвращает 1 для подключенного, в противном случае 0.

`int snd_soc_dapm_ignore_suspend(структура snd_soc_dapm_context *dapm, постоянный символ *pin-код)` игнорировать состояние приостановки для конечной точки DAPM

Параметры

структура `snd_soc_dapm_context` * `dapm` Контекст DAPM
постоянный символ * `pin-код` конечная точка вывода аудиосигнала (или начальная точка) **Описание**

Пометьте данную конечную точку или pin-код как игнорирующий suspend. Когда система отключена, путь между двумя конечными точками, помеченными как игнорирующие приостановку, не будет отключен. Путь уже должен быть включен обычным способом во время приостановки, он не будет включен, если он еще не был включен. `void snd_soc_dapm_free(структура snd_soc_dapm_context *dapm)` свободные ресурсы dapm

Параметры

структура `snd_soc_dapm_context` * `dapm` Контекст DAPM **Описание**
Освободите все виджеты и ресурсы dapm.

ASoC DMA Engine API

`int snd_dmaengine_pcm_prepare_slave_config(структура snd_pcm_substream * podпоток, структура snd_pcm_hw_params * параметры, struct dma_slave_config * slave_config)`
Общий обратный вызов prepare_slave_config

Параметры

структура `snd_pcm_substream` * `подпоток` Подпоток PCM
структура `snd_pcm_hw_params` * `параметры` hw_params
структура `dma_slave_config` * `slave_config` Конфигурация ведомого устройства DMA для подготовки **Описание**

Эта функция может использоваться как общий обратный вызов prepare_slave_config для платформ, которые используют структуру snd_dmaengine_dai_dma_data для своих данных DAI DMA. Внутренне функция сначала вызовет snd_hwparams_to_dma_slave_config для заполнения конфигурации подчиненного устройства на основе параметров hw_params, за которой следует snd_dmaengine_set_config_from_dai_data для заполнения оставшихся полей на основе данных DAI DMA. `int snd_dmaengine_pcm_register(структура device *dev, const struct snd_dmaengine_pcm_config *config, unsigned int флагу)`

Зарегистрировать PCM-устройство на базе dmaengine **Параметры**

структурное устройство * dev Родительское устройство для устройства PCM

const struct snd_dmaengine_pcm_config * config Конфигурация PCM для конкретной платформы флаги unsigned int Специфичные для платформы особенности

аннулирование snd_dmaengine_pcm_unregister(struct device * dev)

Удаляет устройство PCM на основе dmaengine

Параметры

структурное устройство * разработчик Родительское устройство, на котором был зарегистрирован PCM

Описание

Удаляет PCM-устройство на базе dmaengine, ранее зарегистрированное в snd_dmaengine_pcm_register.

1.1.8 Прочие функции

API аппаратно-зависимых устройств

int snd_hwdep_new(структура snd_card * card, char * id, int устройство, struct snd_hwdep ** rhwdep)

создайте новый экземпляр hwdep

Параметры

struct snd_card * card экземпляр card

char * id строка идентификатора

int device индекс устройства (на основе нуля)

структура snd_hwdep * * rhwdep указатель для хранения нового экземпляра hwdep

Описание

Создает новый экземпляр hwdep с заданным индексом на карте.

Обратные вызовы (hwdep-> ops) должны быть установлены на возвращаемом экземпляре после этого вызова вызывающей стороной вручную.

Возврат

Ноль в случае успеха, или отрицательный код ошибки при сбое.

API уровня абстракции Jack

перечисление snd_jack_types

Типы Jack, о которых можно сообщать

Константы

SND_JACK_HEADPHONE Наушники

SND_JACK_MICROPHONE Микрофон

SND_JACK_HEADSET Гарнитура

SND_JACK_LINEOUT Линейный вывод

SND_JACK_MECHANICAL Механический переключатель

SND_JACK_VIDEOOUT Видеовыход

SND_JACK_AVOUT Выход AV (аудио-видео)

SND_JACK_LINEARIN Линейный ввод

SND_JACK_BTN_0 Кнопка 0

SND_JACK_BTN_1 Кнопка 1

<code>SND_JACK_BTN_2</code>	Кнопка 2
<code>SND_JACK_BTN_3</code>	Кнопка 3
<code>SND_JACK_BTN_4</code>	Кнопка 4
<code>SND_JACK_BTN_5</code>	Кнопка 5

Описание

Эти значения используются в качестве битовой маски.

Обратите внимание, что это должно быть синхронизировано с таблицей поиска в `sound/core/jack.c`.

`int snd_jack_add_new_kctl(struct snd_jack *jack, постоянный символ *Имя, int маска)`
Создайте новый `snd_jack_kctl` и добавьте его в `jack`

Параметры

`структура snd_jack * jack` экземпляр `jack`, к которому будет привязан `kctl`

`const char * name` имя объекта `snd_control`

`маска int` битовая маска значений `enum snd_jack_type`, которые могут быть обнаружены этим объектом `snd_jack_kctl`. **Описание**

Создает новый объект `snd_control` и добавляет его в список `jack kctl_list`.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое.

`int snd_jack_new(struct snd_card *card, const char *id, int Tun, struct snd_jack **jjack, bool initial_kctl, bool phantom_jack)`
Создание нового гнезда

Параметры

`struct snd_card * card` экземпляр карты

`const char * id` строка, идентифицирующая этот разъем

`тип int` битовая маска значений `enum snd_jack_type`, которые могут быть обнаружены с помощью этого разъема

`структура snd_jack * * jjack` Используется для предоставления выделенного объекта `jack` вызывающей стороне.

`bool initial_kctl` если `true`, создайте `kcontrol` и добавьте его в список `jack`.

`bool phantom_jack` Не создавайте устройство ввода для фантомных разъемов.

Описание

Создает новый объект `jack`.

Возврат

Ноль в случае успеха или отрицательный код ошибки при сбое. В случае успеха **будет инициализирован `jjack`**.

`void snd_jack_set_parent(struct snd_jack *jack, struct device *parent)`
Установите родительское устройство для разъема

Параметры

`структура snd_jack * jack` Разъем для настройки

`структура device * parent` Устройство, которое нужно установить в качестве родительского для разъема. **Описание**

Установите родительский параметр для устройств с разъемом в дереве устройств. Эта функция действительна только до регистрации разъема. Если родительский элемент не настроен, родительским устройством будет звуковая карта.

`int snd_jack_set_key(struct snd_jack *jack, перечислить snd_jack_types Tunы snd_jack_types Tun, int mun ключа)`
Установите соответствие клавиш на разъеме

Параметры

структура snd_jack * jack Гнездо для настройки

Тип enum snd_jack_types Тип отчета Jack для этого

ключа int keytype Тип ключа входного слоя, о котором

нужно сообщить **Описание**

Карта **a SND_JACK_BTN** введите кнопку для ключа входного уровня, позволяя сообщать о ключах на аксессуарах через абстракцию разъема. Если сопоставление не предусмотрено, но для типа разъема включены клавиши, то будет сообщено о цифрах BTN_n кнопок. Если разъемы не сообщают через API ввода, этот вызов не будет иметь эффекта. Обратите внимание, что это предназначено для использования простыми устройствами с небольшим количеством ключей, о которых можно сообщать. Также возможен прямой доступ к устройству ввода - устройствам со сложными возможностями ввода на аксессуарах следует подумать об этом, а не использовать эту абстракцию. Эта функция может быть вызвана только перед регистрацией гнезда. **Возврат**

Ноль в случае успеха или отрицательный код ошибки при сбое.

недействительный snd_jack_report (структура snd_jack * jack, int *Статус*)

Сообщить текущее состояние разъема

Параметры

структура snd_jack * разъем Разъем, для которого нужно сообщить о состоянии

int status Текущее состояние разъема int

snd_soc_codec_set_jack (struct snd_soc_codec * *кодек*, struct snd_soc_jack * *разъем*, void * *данные*)

настройте разъем для кодека.

Параметры

struct snd_soc_codec * кодек КОДЕК

struct snd_soc_jack * разъем структура, используемая для разъема

void * данные может использоваться, если драйверу кодека требуются

дополнительные данные для настройки разъема **Описание**

Настраиивает и включает функцию обнаружения гнезда. int

snd_soc_card_jack_new (struct snd_soc_card * *карточка*, постоянный символ * *id*, *инт* *Тип*, структура *snd_soc_jack* * *jack*, struct *snd_soc_jack_pin* * *контакты*, без знака *инт* *num_pins*)

Создайте новый jack

Параметры

struct snd_soc_card * card карточка ASoC

const char * id строка, идентифицирующая этот разъем

тип int битовая маска значений enum snd_jack_type, которые могут быть обнаружены с

помощью этого разъема структура snd_soc_jack * jack структура, используемая для разъема

структура snd_soc_jack_pin * контакты Массив Джек штыри, которые будут добавлены к

гнезду или null беззнаковый инт num_pins Количество элементов **булавки** массив

Описание

Создает новый объект jack.

Возвращает ноль в случае успеха или отрицательный код ошибки при сбое. В случае успеха jack будет инициализирован.

недействительно `snd_soc_jack_report` (структура `snd_soc_jack *jack`, int *Статус*, int *маска*)
Сообщает текущее состояние для гнезда

Параметры

структура `snd_soc_jack` * гнездо Гнездо
состояние int битовая маска значений enum `snd_jack_type`, которые
обнаружены в данный момент. `маска` int битовая маска значений enum
`snd_jack_type`, о которых сообщается. **Описание**

Если настроено с использованием `snd_soc_jack_add_pins()` затем соответствующие контакты DAPM будут включены
или отключены в зависимости от обстоятельств и DAPM будет синхронизирован.

Примечание

Эта функция использует мьютексы и должна вызываться из контекста, который может находиться в режиме ожидания (например,
рабочей очереди). int `snd_soc_jack_add_zones` (struct `snd_soc_jack *разъем`, int *count*, struct `snd_soc_jack_zone`
* *зоны*)
Связывают зоны напряжения с гнездом

Параметры

struct `snd_soc_jack` * `jack` ASoC jack
общее количество Количество зон
struct `snd_soc_jack_zone` * *зоны* Множество зон

Описание

После вызова этой функции зоны, указанные в массиве, будут связаны с гнездом. int
`snd_soc_jack_get_type` (struct `snd_soc_jack *разъем`, входное значение *micbias_voltage*)
На основе значения смещения микрофона эта функция возвращает тип разъема из зон,
объявленных в тип разъема

Параметры

структура `snd_soc_jack` * `jack` Разъем ASoC
int `micbias_voltage` напряжение смещения микрофона на канале АЦП при подключении
разъема **Описание**

На основе переданного значения смещения микрофона эта функция помогает определить тип
разъема из уже объявленных зон подключения int
`snd_soc_jack_add_pins` (struct `snd_soc_jack *разъем`, int *count*, struct `snd_soc_jack_pin` * *контакты*)

Связать контакты DAPM с разъемом ASoC

Параметры

struct `snd_soc_jack` * *разъем* ASoC jack
количество входов Количество выводов
struct `snd_soc_jack_pin` * *выводы* Набор
контактов **Описание**

После вызова этой функции состояние контактов DAPM, указанных в массиве `pins`, будет
обновляться чтобы отражать текущее состояние разъема всякий раз, когда обновляется состояние разъема. void
`snd_soc_jack_notifier_register` (структура `snd_soc_jack *jack`, структура `notifier_block` * *примечание*)

Зарегистрируйте уведомитель для

статуса jack **Параметры**

структура `snd_soc_jack` * `jack` ASoC jack

struct notifier_block * *примечание* Блок

уведомителя для регистрации **Описание**

Зарегистрируйтесь для получения уведомлений о текущем состоянии домкрата. Обратите внимание, что невозможно сообщать о дополнительных событиях подключения при обратном вызове от уведомителя, это предназначено для поддержки таких приложений, как включение электрического обнаружения только при возникновении события механического обнаружения. void snd_soc_jack_notifier_unregister (struct snd_soc_jack * *jack*, struct notifier_block * *примечание*)

Отмените регистрацию уведомителя для статуса jack

Параметры

struct snd_soc_jack * *jack* ASoC jack

struct notifier_block * *nb* Блок уведомлений для отмены

регистрации **Описание**

Прекратить отправку уведомлений об изменениях

состояния. int snd_soc_jack_add_gpios (struct snd_soc_jack * *jack*, int *count*, struct *snd_soc_jack_gpio* * *gpios*)

Связать контакты GPIO с разъемом ASoC

Параметры

структура snd_soc_jack * *jack* Разъем ASoC

количество *входов* количество выводов

структура snd_soc_jack_gpio * *gpios* массив выводов

gpio **Описание**

Эта функция запросит gpio, установит направление данных и запросит irq для каждого gpio в массиве.

int snd_soc_jack_add_gpioids (структура устройства * *gpiod_dev*, структура snd_soc_jack * *гнездо*, int *количество*, структура *snd_soc_jack_gpio* * *gpios*)

Связать контакты дескриптора GPIO с разъемом

ASoC Параметры

структурное устройство * *gpiod_dev* Потребительское

устройство GPIO структура snd_soc_jack * *jack* Разъем

ASoC количество *входов* количество контактов

структура snd_soc_jack_gpio * *gpios* массив выводов

gpio **Описание**

Эта функция запросит gpio, установит направление данных и запросит irq для каждого gpio в массиве.

void snd_soc_jack_free_gpios (struct snd_soc_jack * *jack*, int *count*, struct *snd_soc_jack_gpio* * *gpios*)

Высвобождает ресурсы выводов GPIO разъема ASoC

Параметры

struct snd_soc_jack * *разъем* ASoC jack

количество *входов* количество выводов

struct snd_soc_jack_gpio * *gpios* массив выводов

gpio **Описание**

Освободите ресурсы gpio и irq для выводов gpio, связанных с разъемом ASoC.

Помощники ISA DMA

недействительны `snd_dma_program` (unsigned long *dma*, unsigned long *addr*, беззнаковый int *размер*, без знака короткий *режим*)
программа передачи Иса ДМА

Параметры

неподписанные долго дма количество ДМА
unsigned long addr физический адрес буфера
размер unsigned int размер передачи ДМА
короткий режим без знака режим передачи ДМА,

DMA_MODE_XXX Описание

Программирует передачу ISA DMA для данного буфера.

void `snd_dma_disable` (без знака long *dma*)
остановка передачи ISA DMA

Параметры

без знака long dma номер dma

Описание

Останавливает передачу ISA DMA.

unsigned int `snd_dma_pointer` (unsigned long *dma*, unsigned int *размер*)
возвращает текущий указатель на буфер передачи ДМА

в байтах Параметры

длина dma без знака номер dma
размер int без знака размер передачи данных dma

Возврат

Текущий указатель в буфере передачи ДМА в байтах.

Другие вспомогательные макросы

`snd_printk` (*fmt*,...)
оболочка printk

Параметры

`fmt` форматная строка
... аргументы переменной

Описание

Работает как `printk` () но печатает файл и строку вызывающего объекта при настройке с помощью CONFIG_SND_VERBOSE_PRINTK. `snd_printd` (*fmt*,...)
отладочный printk

Параметры

`fmt` форматная строка
... аргументы переменной

Описание

Работает как `snd_printk` () для целей отладки. Игнорируется, если параметр CONFIG_SND_DEBUG не установлен.

`snd_BUG()`

выдает сообщение с предупреждением
об ошибке и трассировкой стека

Параметры

Описание

Вызывает функцию `WARN()` если установлен параметр `CONFIG_SND_DEBUG`.

Игнорируется, если параметр `CONFIG_SND_DEBUG` не установлен. `snd_printd_ratelimit()`

Параметры

`snd_BUG_ON(cond)`

макрос проверки

отладки

Параметры

`cond` условие для оценки

Описание

Имеет то же поведение, что и `WARN_ON`, когда установлен
параметр `CONFIG_SND_DEBUG`, в противном случае просто вычисляет
условие и возвращает значение. `snd_printdd(форматировать,...)`
отладочный `printf`

Параметры

`форматировать` строка

формата `...` аргументы

переменной

Описание

Работает как `snd_printk()` для целей отладки. Игнорируется,
если параметр `CONFIG_SND_DEBUG_VERBOSE` не установлен.

1.2 Написание драйвера ALSA

Автор Такаши Иваи <tiwai@suse.de>

Дата 15 октября 2007 г.

Издание 0.3.7

1.2.1 Предисловие

В этом документе описывается, как написать драйвер ALSA (Advanced Linux Sound Architecture). В документе основное внимание уделяется звуковым картам PCI. В случае с другими типами устройств API также может отличаться. Однако, по крайней мере, API ядра ALSA является согласованным, и поэтому это все равно немного помогло бы при их написании. Этот документ предназначен для людей, которые уже достаточно владеют языком C и обладают базовыми знаниями программирования на ядре Linux. Этот документ не объясняет общую тему кодирования ядра Linux и не охватывает детали низкоуровневой реализации драйвера. В нем описывается только стандартный способ написания звукового драйвера PCI на ALSA. Если вы уже знакомы со старым API ALSA версии 0.5.x, вы можете проверить драйверы, такие как `sound/pci-es1938.c` или `sound/pci/maestro3.c` которые также имеют почти одинаковую кодовую базу в дереве ALSA 0.5.x, так что вы можете сравнить различия. Этот документ все еще является черновой версией. Любые отзывы и исправления, пожалуйста!!

1.2.2 Древовидная структура файлов

Общая информация

Драйверы ALSA предоставляются двумя способами.

Один из них - это деревья, предоставляемые в виде архива или через cvs с ftp-сайта ALSA, а другой - 2.6 (или более поздней версии) дерево ядра Linux. Чтобы синхронизировать оба, дерево драйверов ALSA разделяется на два разных дерева: alsa-ядро и alsa-драйвер. Первое содержит исключительно исходный код для дерева Linux 2.6 (или более поздней версии). Это дерево предназначено только для компиляции в среде 2.6 или более поздней версии. Последний, alsa-driver, содержит множество неочевидных файлов для компиляции драйверов ALSA за пределами дерева ядра Linux, функции-оболочки для старых ядер 2.2 и 2.4, адаптирующие новейший kernel API, и дополнительные драйверы, которые все еще находятся в разработке или в тестировании. Драйверы из дерева драйверов alsa будут перемещены в ядро alsa (и, в конечном итоге, в дерево ядра 2.6), когда они будут завершены и будет подтверждено, что они работают нормально. Древовидная структура файла ALSA driver показана ниже. И alsa-kernel, и alsa-driver имеют почти одинаковую файловую структуру, за

исключением каталога "core". Он назван "acore" в дереве драйверов alsa.

```

/ядро
      /операционная
система /seq
      /операционная система
      /instr

/iocctl 3 2

/включить /драйверы
      /mpu 4 0 1
      /opl 3

/i 2 c
      /l 3

/синтезатор
      /emux

/pci
      / (карты)

/isa
      / (карты)

/arm
/ppc /sparc
/usb /pcmcia
/ (карты)
/операционные
системы
```

базовый каталог

Этот каталог содержит средний уровень, который является сердцем драйверов

ALSA. В этом каталоге хранятся собственные модули ALSA. Подкаталоги

содержат разные модули и зависят от конфигурации ядра.

core/oss

В этом каталоге хранятся коды для модулей эмуляции PCM и mixer OSS.

Эмуляция rawmidi OSS включена в код ALSA rawmidi, поскольку он довольно маленький. Код

секвенсора хранится в каталоге core/seq/oss (см. *ниже*).

core/ioctl32

Этот каталог содержит 32-битные оболочки ioctl для 64-битных архитектур, таких как x86-64, ppc64 и sparc64. Для 32-битных и альфа-архитектур они не компилируются.

core / seq

Этот каталог и его подкаталоги предназначены для секвенсора ALSA. Этот каталог содержит секвенсор

core и основные модули секвенсора, такие как snd-seq-midi, snd-seq-virmidi и т.д. Они компилируются

только тогда, когда `CONFIG_SND_SEQUENCER` устанавливается в конфигурации ядра.

core/seq/oss

Содержит коды эмуляции OSS sequencer.

core/seq/instr

Этот каталог содержит модули для инструментального уровня секвенсора.

включить каталог

Это место для общедоступных заголовочных файлов драйверов ALSA, которые должны быть

экспортированы в пространство пользователя или включены несколькими файлами в разные каталоги.

В принципе, файлы личных заголовков не следует размещать в этом каталоге, но вы все равно можете

найти там файлы по историческим причинам :) **каталог драйверов**

Этот каталог содержит код, совместно используемый разными драйверами на разных архитектурах.

Следовательно, предполагается, что они не зависят от конкретной архитектуры. Например, фиктивный

драйвер rsm и драйвер serial MIDI находятся в этом каталоге. В подкаталогах есть код для компонентов,

которые независимы от архитектуры шины и центрального процессора.

драйверы/mpu401

Модули MPU401 и MPU401-UART хранятся здесь.

драйверы/opl3 и opl4

FM-синтезаторы OPL3 и OPL4 находятся здесь.

каталог i2c

Он содержит компоненты ALSA i2c.

Хотя в Linux существует стандартный уровень i2c, у ALSA есть свой собственный код i2c для некоторых

карт, потому что звуковой карте требуется только простое управление, а стандартный API i2c слишком

сложен для такой цели. **i2c / I3**

Это подкаталог для ARM L3 i2c.

каталог синтезатора

Он содержит модули среднего уровня `synth`.

Пока что в подкаталоге `synth` / `emux` находится только драйвер синтезатора `Emu8000` / `Emu10k1`.

каталог pci

Этот каталог и его подкаталоги содержат модули видеокарт верхнего уровня для звуковых карт PCI и код, специфичный для шины PCI. Драйверы, скомпилированные из одного файла, хранятся непосредственно в каталоге `pci`, в то время как драйверы с несколькими исходными файлами хранятся в их собственном подкаталоге (например, `emu10k1`, `ice1712`).

каталог isa

В этом каталоге и его подкаталогах хранятся модули видеокарт верхнего уровня для звуковых карт ISA.

каталоги arm, ppc и sparc

Они используются для модулей карты верхнего уровня, специфичных для одной из этих архитектур.

каталог usb

Этот каталог содержит драйвер USB-audio. В последней версии драйвер USB MIDI интегрирован в драйвер `usb-audio`.

каталог pcmcia

Драйверы PCMCIA, особенно PCCard, будут размещены здесь. Драйверы CardBus будут находиться в каталоге `pci`, потому что их API идентичен API стандартных карт PCI.

каталог oss

Исходные файлы OSS / Lite хранятся здесь, в дереве Linux 2.6 (или

более поздней версии). В архиве драйверов ALSA этот каталог, конечно

же, пуст :) **1.2.3 Базовый поток для драйверов PCI**

Схема

Минимальный поток для звуковых карт PCI следующий:

- определите таблицу идентификаторов PCI (см. Раздел *Записи PCI*).
- создайте `probe` обратный вызов.
- создать `remove` удалить обратный вызов.
- создайте `struct pci_driver` структуру, содержащую три указателя выше.
- создайте функцию `init`, просто вызывающую `pci_register_driver()` чтобы зарегистрировать таблицу `pci_driver` де-юре, указанную выше.
- создайте функцию `exit` для вызова функции `pci_unregister_driver()`.

Полный пример кода

Пример кода показан ниже. Некоторые части на данный момент остаются невыполненными, но будут заполнены в следующих разделах. Цифры в строках комментариев к `snd_mychip_probe()` функция приведена в деталях, описанных в следующем разделе.

```
# включить <linux/init.h>

#   включить <linux/pci.h>

#   включить <linux/slab.h>

#   включить <sound/core.h>

#   включить <sound/initval.h>

/* параметры модуля (см. "Параметры модуля") */
/* SNDRV_CARDS: максимальное количество карт, поддерживаемых этим модулем */

статический индекс int[SNDRV_CARDS] = SNDRV_DEFAULT_IDX;
статический символ * id[SNDRV_CARDS] = SNDRV_DEFAULT_STR;
static bool enable[SNDRV_CARDS] = SNDRV_DEFAULT_ENABLE_PNP;

/* определение записи, зависящей от чипа */
struct mychip {

    структура snd_card *card; /*
    оставшая часть реализации будет описана в разделе

    * "Управление
    ресурсами PCI" */
};

/* деструктор для конкретного чипа
 * (см. "Управление
ресурсами PCI") */ static
int
snd_mychip_free(struct mychip *чип) {

    .... /* будет реализован позже... */
}

/* компонент-деструктор
 * (см. "Управление картами и
компонентами") */ статический
int snd_mychip_dev_free(структура
snd_device *устройство) {

    возвращает snd_mychip_free(device-> device_data);
}

/* конструктор для конкретного чипа
 * (см. "Управление картами и компонентами")
 */ статический int
snd_mychip_create(структура snd_card *card,

                                структура pci_dev *pci,
                                структура mychip **rchip)

{

    struct mychip *чип;
    ошибка ввода; статическая
    структура snd_device_ops ops = {

        .dev_free = snd_mychip_dev_free,

    };

    * rchip = NULL;

    /* проверьте доступность PCI
здесь
    * (см. "Управление ресурсами PCI") */
    ....

    /* выделите данные, относящиеся к чипу, с заполнением нуля */
```

```

chip = kzalloc(sizeof(*чип), GFP_KERNEL);
если (чип == NULL)
    возврат -ENOMEM;

чип-> карта = card;

/* оставшая часть инициализации здесь;
будет реализована
* позже смотрите "Управление ресурсами PCI" */
....

err = snd_device_new(карта, SNDRV_DEV_LOWLEVEL,
чип и операционные системы); если (ошибка < 0) {
    snd_mychip_free (чип);
    возвращает ошибку;
}

* rchip = чип;
возвращает 0;
}

/* конструктор -- смотрите подраздел "Конструктор
драйвера" */ static int snd_mychip_probe(struct pci_dev
постоянная структура pci_device_id *pci_id)
*pci, {

    static int dev;

    struct snd_card
    * карта; struct mychip
    * чип; ошибка ввода;

    /* (1) */ if
    (dev >= SNDRV_CARDS)

        return -ENODEV;
    если (!включить[dev]) {
        dev++;
        return -ENOENT;
    }

    /* (2) */ ошибка = snd_card_new(&pci->dev, индекс[dev],
    идентификатор[dev], THIS_MODULE,
                                0, &card);

    if (ошибка < 0)
        возвращает ошибку;

    /* (3) */ ошибка =
    snd_mychip_create(карта, pci и чип);
    если (ошибка < 0) {
        snd_card_free(карточка);
        возвращает ошибку;
    }

    /* (4) */ strcpy (карта-> драйвер, "Мой чип");
    strcpy (карта-> краткое имя, "Мой
    собственный чип 1 2 3"); sprintf (карта->
    длинное имя, "%s при 0x%lx irq %i",
                                карта-> краткое имя, чип-> ioport, чип-> irq);

    /* (5) */ .... /* реализовано
    позже */ /* (6) */ ошибка
    = snd_card_register(карточка);

    if (ошибка < 0) {

```

```
        snd_card_free (карта);
        возвращает ошибку;
    }

    /* ( 7 ) */
    pci_set_drvdata(pci, карта);
    dev++;
    возвращает 0 ;
}

/* деструктор - смотрите подраздел
"Деструктор" */ static void snd_mychip_remove(struct
pci_dev *pci) {
    snd_card_free(pci_get_drvdata(pci));
    pci_set_drvdata(pci, NULL);
}
```

Конструктор драйверов

Реальным конструктором драйверов PCI является `зонд` обратный вызов. The `зонд` обратный вызов и другие компоненты- конструкторы, которые вызываются из `зонд` обратный вызов не может использоваться с `__init` префикс, потому что любое устройство PCI может быть устройством горячего подключения.

В обратном вызове `probe` часто используется следующая схема.

1) Проверьте и увеличьте индекс устройства.

```
static int dev;
....
if (dev >= SNDRV_CARDS)
    return -ENODEV;
if (!включить[dev]) {
    dev++;
    return -ENOENT;
}
```

где `включить[dev]` - опция модуля.

Каждый раз, когда вызывается обратный вызов `probe` , проверяйте доступность устройства. Если оно недоступно, просто увеличьте индекс устройства и верните результаты. `dev` также будет увеличен позже (*шаг 7*).

2) Создайте экземпляр карты

```
struct snd_card *card;
ошибка ввода;
.... ошибка = snd_card_new(&pci->dev,
индекс[dev], идентификатор[dev], THIS_MODULE,
0, &card);
```

Подробности будут объяснены в разделе *Управление картами и компонентами*.

3) Создайте основной компонент

В этой части распределяются ресурсы PCI.

```

struct mychip *чип;
.... ошибка =
snd_mychip_create(карта, pci и чип);
если (ошибка < 0) {

    snd_card_free (карта);

    возвращает ошибку.;
}

```

Подробности будут объяснены в разделе [Управление ресурсами PCI](#).

4) Установите идентификатор драйвера и строки имени.

```

strcpy(карта-> драйвер, "Мой чип"); strcpy
(карта-> краткое имя, "Мой собственный чип 1 2 3"); sprintf
(карта-> длинное имя, "%s при 0x%x irq %i",

    card-> краткое название, chip-> ioport, chip-> irq);

```

Поле драйвера содержит строку минимального идентификатора чипа. Это используется конфигуратором `alsa-lib`, поэтому сделайте его простым, но уникальным. Даже один и тот же драйвер может иметь разные идентификаторы драйверов, чтобы различать функциональность каждого типа микросхемы.

Поле `shortname` представляет собой строку, отображаемую как более подробное название. Поле длинного имени содержит информацию, показанную в `/proc/asound/cards`.

5) Создайте другие компоненты, такие как микшер, MIDI и т.д.

Здесь вы определяете основные компоненты, такие как *PCM*, микшер (например, *AC97*), MIDI (например, *MPU-401*) и другие интерфейсы. Кроме того, если вам нужен файл *proc*, определите его и здесь.

6) Зарегистрируйте экземпляр карты.

```

err = snd_card_register(карточка);
if (ошибка < 0) {

    snd_card_free(карточка);

    возвращает ошибку;
}

```

Также будет объяснено в разделе [Управление картами и компонентами](#).

7) Установите данные драйвера PCI и верните ноль.

```

pci_set_drvdata (pci,
карта); dev ++;
возвращает 0 ;

```

В приведенном выше примере запись о карте сохраняется. Этот указатель также используется в обратных вызовах `remove callback` и `power-management`.

Деструктор

Деструктор `remove callback` просто освобождает экземпляр `card`. Затем средний слой ALSA автоматически освободит все прикрепленные компоненты. Обычно это выглядит следующим образом:

```
статическая пустота snd_mychip_remove(struct
pci_dev *pci) {

    snd_card_free(pci_get_drvdata(pci));

    pci_set_drvdata(pci, NULL);

}
```

Приведенный выше код предполагает, что указатель карты установлен на данные драйвера PCI.

Заголовочные файлы

Для приведенного выше примера необходимы по крайней мере следующие включаемые файлы.

```
# включить <linux/init.h>

# включить <linux/pci.h>

# включить <linux/slab.h>

# включить <sound/core.h>

# включить <sound/initval.h>
```

где последнее необходимо только тогда, когда параметры модуля определены в исходном файле. Если код разбит на несколько файлов, файлам без параметров модуля они не нужны.

В дополнение к этим заголовкам вам понадобятся `<linux/interrupt.h>` для обработки прерываний и `<asm/io.h>` для доступа к вводу-выводу. Если вы используете `mdelay()` или `udelay()` функции, которые вам нужно будет включить `<linux/delay.h>` тоже.

Интерфейсы ALSA, такие как PCM и control API, определены в других `<sound/xxx.h>` заголовочных файлах. Они должны быть включены после `<sound/core.h>`.

1.2.4 Управление картами и компонентами

Экземпляр карты

Для каждой звуковой карты должна быть выделена запись "card".

Запись на карточке является центром звуковой карты. Она управляет всем списком устройств (компонентов) на звуковой карте, таких как PCM, микшеры, MIDI, синтезатор и так далее. Кроме того, запись карты содержит идентификатор и строки имени карты, управляет корневым каталогом файлов ргос и контролирует состояния управления питанием и отключения от горячих подключений. Список компонентов в записи карты используется для управления правильным высвобождением ресурсов при уничтожении.

Как упоминалось выше, чтобы создать экземпляр карты, вызовите `snd_card_new()`.

```
struct snd_card *card;

int err;

err = snd_card_new(&pci-> dev, индекс, идентификатор, модуль, extra_size, &card);
```

Функция принимает шесть аргументов: указатель родительского устройства, картотечный номер, строку идентификатора, указатель модуля (обычно `THIS_MODULE`), размер пространства дополнительных данных и указатель для возврата экземпляра карты. Аргумент `extra_size` используется для выделения `card->private_data` для данных, специфичных для чипа. Обратите внимание, что эти данные распределяются с помощью `snd_card_new()`. Первый аргумент, указатель `struct device`, указывает родительское устройство. Для устройств PCI, обычно здесь передается `&pci->`.

Компоненты

После создания карты можно присоединить компоненты (устройства) к экземпляру карты. В драйвере ALSA, компонент представлен как `struct snd_device` объект. Компонентом может быть экземпляр PCM, интерфейс управления, необработанный MIDI-интерфейс и т.д. Каждый такой экземпляр имеет одну запись компонента.

Компонент может быть создан с помощью `snd_device_new()` функции.

```
snd_device_new (карта, SNDRV_DEV_XXX, чип и операционные системы);
```

Для этого используется указатель карты, уровень устройства (`SNDRV_DEV_XXX`), указатель данных и указатели обратного вызова (`&ops`). Уровень устройства определяет тип компонентов и порядок регистрации и снятия с регистрации. Для большинства компонентов уровень устройства уже определен. Для компонента, определенного пользователем, вы можете использовать `SNDRV_DEV_LOWLEVEL`. Эта функция сама по себе не выделяет пространство для данных. Данные должны быть распределены заранее вручную, и в качестве аргумента передается указатель на них. Этот указатель (`chip` в приведенном выше примере) используется в качестве идентификатора для экземпляра. Каждый предварительно определенный компонент ALSA, такой как `ac97` и `rct`, вызывает `snd_device_new()` внутри его конструктора. Деструктор для каждого компонента определен в указателях обратного вызова. Следовательно, вам не нужно заботиться о вызове деструктора для такого компонента. Если вы хотите создать свой собственный компонент, вам нужно установить для функции деструктора значение обратного вызова `dev_free` в `ops`, чтобы его можно было автоматически освободить с помощью `snd_card_free()`. В следующем примере будет показана реализация данных, зависящих от чипа.

Данные, зависящие от чипа

Информация, относящаяся к чипу, например адрес порта ввода-вывода, указатель его ресурса или номер `irq`, хранится в записи, относящейся к чипу.

```
struct mychip {
    . . . . .
};
```

В общем, существует два способа выделения записи чипа.

1. Выделение через `snd_card_new()`.

Как упоминалось выше, вы можете передать дополнительную длину данных в 5-й аргумент `snd_card_new()`, т.е.

```
err = snd_card_new(&pci->dev, индекс[dev], идентификатор[dev], THIS_MODULE,
                  sizeof(структура mychip), &card);
```

`struct mychip` - это тип записи чипа. В свою очередь, к выделенной записи можно получить доступ

```
как struct mychip *chip = card-> private_data;
```

При использовании этого метода вам не нужно выделять дважды. Запись освобождается вместе с экземпляром карты.

2. Выделение дополнительного устройства.

После выделения экземпляра карты с помощью `snd_card_new()` (с 0 на 4-м аргументе) вызовите `kzalloc()`.

```
struct snd_card *карта;

struct mychip *чип;

err = snd_card_new(&pci->dev, индекс[dev], идентификатор[dev], THIS_MODULE,
                  0, &card);

.... чип =
kzalloc(sizeof(*chip), GFP_KERNEL);
```

В записи о чипе должно быть поле, в котором должен содержаться указатель карты, по крайней мере,

```
struct mychip {
    struct
        snd_card * card; ....
};
```

Затем установите указатель карты в возвращаемом

```
экземпляре чипа. чип-> карта = card;
```

Затем инициализируйте поля и зарегистрируйте эту запись чипа как низкоуровневое устройство с указанным значением ops,

```
статическая структура snd_device_ops ops = {
    .dev_free = snd_mychip_dev_free,
}; ....
snd_device_new(карта, SNDRV_DEV_LOWLEVEL, чип
и операционные системы);
```

snd_mychip_dev_free() это функция устройства-деструктора, которая вызовет

```
реальный деструктор. статический int snd_mychip_dev_free(структура
snd_device * устройство) {
    возвращает snd_mychip_free(устройство-> device_data);
}
```

где snd_mychip_free() является реальным деструктором.

Регистрация и освобождение

После назначения всех компонентов зарегистрируйте экземпляр карты, вызвав `snd_card_register()`. На этом этапе разрешен доступ к файлам устройства. То есть до вызова функции `snd_card_register()` компоненты надежно недоступны с внешней стороны. Если этот вызов завершается неудачей, выйдите из функции проверки после освобождения карты с помощью `snd_card_free()`. Для освобождения экземпляра карты вы можете вызвать просто `snd_card_free()`. Как упоминалось ранее, все компоненты автоматически освобождаются при этом вызове. Для устройства, допускающего горячее подключение, вы можете использовать `snd_card_free_when_closed()`. Эта функция будет переносить уничтожение до тех пор, пока все устройства не будут закрыты.

1.2.5 Управление ресурсами PCI

Пример полного кода

В этом разделе мы завершим ввод конструктора, деструктора и PCI для конкретного чипа.

Пример кода показан первым ниже.

```
struct mychip {
    struct snd_card
        * карта; struct pci_dev
        * pci; длинный порт без
        знака; int irq;
};

статический int snd_mychip_free(структура
mychip * чип) {
    /* отключите оборудование здесь, если оно
    есть */ .... /* (не реализовано в этом
    документе) */ / * запустите irq */
```

```

    if (chip-> irq >= 0)

        free_irq(чип-> irq,
микроплата); /* освободить порты
ввода-вывода и память */
pci_release_regions(чип-> pci); /*
отключить запись PCI */
pci_disable_device(чип-> pci); /*
освободить данные */ kfree (чип);
вернуть 0;
}

/* конструктор для конкретного чипа */
статический int snd_mychip_create(struct snd_card *card,
                                структура pci_dev *pci,
                                структура mychip **rchip)
{
    struct mychip *чип;
    ошибка ввода; статическая
    структура snd_device_ops ops = {
        .dev_free = snd_mychip_dev_free,
    };

    * rchip = NULL;

    /* инициализируем запись PCI
    */ err =
pci_enable_device(pci); если (ошибка < 0)

        возвращать ошибку;

    /* проверить доступность PCI (28-битный DMA) */
    если (pci_set_dma_mask(pci, DMA_BIT_MASK(28)) < 0 ||
        pci_set_consistent_dma_mask(pci, DMA_BIT_MASK(28)) < 0) {
        printk(KERN_ERR "ошибка установки 28-битной маски
DMA\n"); pci_disable_device(pci);
        возврат -ENXIO;
    }

    чип = kzalloc(sizeof(*чип), GFP_KERNEL);
    if (чип == NULL) {
        pci_disable_device(pci);
        возврат -ENOMEM;
    }

    /* инициализируем содержимое
    */ чип-> карта
    = card; чип-> pci =
pci; чип-> irq = -1;

    /* (1) Распределение ресурсов PCI
    */ ошибка =
pci_request_regions(pci, "Мой чип"); если (ошибка < 0) {
        kfree (микроплата);
        pci_disable_device (pci);
        возвращает ошибку;
    } чип-> порт = pci_resource_start(pci,
0); if (request_irq(pci->irq,
snd_mychip_interrupt,
                    IRQF_SHARED, KBUILD_MODNAME, чип)) {
        printk (KERN_ERR "не удается захватить irq %d\n",
pci-> irq); snd_mychip_free(чип);
        возврат -EBUSY;
    } чип-> irq
    = pci-> irq;

```



```

/* ( 2 ) инициализация аппаратного обеспечения
микросхемы */ (в этом документе не реализовано) */

err = snd_device_new(карта, SNDRV_DEV_LOWLEVEL,
чип и операционные системы); если (ошибка < 0) {

    snd_mychip_free (чип);
    возвращает ошибку;

}

* rchip = чип;
возвращает 0;

}

/* Идентификаторы PCI */ статическая структура
pci_device_id snd_mychip_ids[] = {
    { PCI_VENDOR_ID_FOO, PCI_DEVICE_ID_BAR,
      PCI_ANY_ID,
      PCI_ANY_ID,
      0, 0, 0, }, .... { 0, }
}; MODULE_DEVICE_TABLE(pci,
snd_mychip_ids);

/* определение pci_driver */ статическая
структура драйвера pci_driver = {

    .name = KBUILD_MODNAME,
    .id_table = snd_mychip_ids,
    .probe = snd_mychip_probe,
    .remove = snd_mychip_remove,
};

/* инициализация модуля */
статический int __init alsa_card_mychip_init(void)
{

    возвращает pci_register_driver(и драйвер);

}

/* очистка модуля */
статическая пустота __завершите alsa_card_mychip_exit(пустота)
{

    pci_unregister_driver(и драйвер);

}

модуль_init(alsa_card_mychip_init)
модуль_exit(alsa_card_mychip_exit)

EXPORT_NO_SYMBOLS; /* только для старых ядер */

```

Некоторые Hafta

Распределение ресурсов PCI выполняется в функции `probe` и обычно дополнительной `xxx_create()` функция написана для этой цели. В случае устройств

PCI сначала необходимо вызвать функцию `pci_enable_device()` функции перед выделением ресурсов.

Кроме того, вам необходимо установить соответствующую маску PCI DMA, чтобы ограничить доступ

к диапазону ввода-вывода. В некоторых случаях, вам может потребоваться вызвать `pci_set_master()` тоже

функция. Предположим, что 28-битная маска и добавляемый код будут такими:

```

err = pci_enable_device(pci);

если (ошибка < 0)

    возвращает ошибку;

```

```
if (pci_set_dma_mask(pci, DMA_BIT_MASK( 2 8 )) < 0 ||
    pci_set_consistent_dma_mask(pci, DMA_BIT_MASK( 2 8 )) < 0) {
    printk(KERN_ERR "ошибка установки 2 8-битной маски
DMA\n"); pci_disable_device(pci);
    возврат -ENXIO;
}
```

Распределение ресурсов

Распределение портов ввода-вывода и irq осуществляется с помощью стандартных функций ядра. В отличие от ALSA версии 0.5.x., для этого нет вспомогательных средств. И эти ресурсы должны быть освобождены с помощью функции деструктора (см. Ниже). Кроме того, в ALSA 0.9.x вам не нужно выделять (псевдо-) DMA для PCI, как в ALSA 0.5.x. Теперь предположим, что устройство PCI имеет порт ввода-вывода с 8 байтами и прерыванием. Тогда struct mychip будет иметь следующие поля:

```
struct mychip {
    struct snd_card *карточка;

    длинный порт без
    знака; int irq;
};
```

Для порта ввода-вывода (а также области памяти) вам необходимо иметь указатель ресурса для стандартного управления ресурсами. Для irq вы должны сохранить только номер irq (целое число). Но вам нужно инициализировать это число как -1 перед фактическим распределением, поскольку допустим irq 0. Адрес порта и указатель на его ресурс могут быть инициализированы как null с помощью kzalloc() автоматически, поэтому вам не нужно заботиться о их сбросе. Выделение порта ввода-вывода выполняется следующим образом:

```
err = pci_request_regions(pci, "Мой чип");
if (ошибка < 0) {
    kfree(чип);
    pci_disable_device(pci);
    возвращает ошибку;
} chip-> port = pci_resource_start(pci,
0 );
```

Это зарезервирует область порта ввода-вывода в 8 байт для данного устройства PCI. Возвращаемое значение, chip-> res_port, выделяется через kcalloc() Автор:request_region(). Указатель должен быть освобожден с помощью kfree(), но с этим есть проблема. Эта проблема будет объяснена позже. Выделение источника прерывания выполняется следующим образом:

```
if (request_irq(pci->irq, snd_mychip_interrupt,
                IRQF_SHARED, KBUILD_MODNAME, чип)) {
    printk(KERN_ERR "не удастся перехватить irq %d\n", pci->
irq); snd_mychip_free(чип);
    return -EBUSY;
}
чип-> irq = pci-> irq;
```

где snd_mychip_interrupt() определен обработчик прерывания позже. Обратите внимание, что chip-> irq следует определять только тогда, когда request_irq() выполнено успешно. На шине PCI прерывания могут быть разделены. Таким образом,, IRQF_SHARED используется в качестве флага прерывания повторно request_irq(). Последний аргумент функции request_irq() - это указатель данных, передаваемый обработчику прерывания. Обычно для этого используется специфичная для чипа запись, но вы также можете использовать то, что вам нравится.

На данном этапе я не буду вдаваться в подробности об обработчике прерываний, но, по крайней мере, теперь можно объяснить его внешний вид. Обработчик прерывания обычно выглядит

следующим образом:

```
статический irqreturn_t snd_mychip_interrupt(int irq, void *dev_id)
{
    struct mychip *chip = dev_id;
    ....
    возвращает IRQ_HANDLED;
}
```

Теперь давайте напишем соответствующий деструктор для указанных выше ресурсов.

Роль деструктора проста: отключите оборудование (если оно уже активировано) и освободите ресурсы. Пока у нас нет аппаратной части, поэтому код отключения здесь не написан. Чтобы освободить ресурсы, метод "проверки и освобождения" является

более безопасным способом. Для прерывания сделайте так: `if (chip-> irq >= 0 )`

```
free_irq(chip-> irq, чип);
```

Поскольку номер `irq` может начинаться с 0, вам следует выполнить инициализацию `chip-> irq` с отрицательным значением (например, -1), чтобы вы могли проверить правильность номера `irq`, как указано выше.

Когда вы запросили порты ввода-вывода или области памяти через `pci_request_region()` или `pci_request_regions()` как и в этом примере, освободите ресурс (ы) с помощью соответствующей функции, `pci_release_region()` или

```
pci_release_regions().
pci_release_regions(chip-> pci);
```

При запросе вручную через `request_region()` или `request_mem_region()`, вы можете освободить его через

`release_resource()`. Предположим, что вы сохраняете указатель ресурса, возвращаемый из `request_region()` в

`chip->res_port` процедура освобождения выглядит следующим образом:

```
release_and_free_resource(chip->res_port);
```

Не забудьте вызвать `pci_disable_device()` перед завершением.

И, наконец, опубликуйте запись, относящуюся к конкретному чипу.

```
kfree (чип);
```

Мы не реализовали часть отключения оборудования, описанную выше. Если вам нужно это сделать, пожалуйста,

обратите внимание, что деструктор может быть вызван еще до завершения инициализации чипа. Было бы лучше

иметь флаг, позволяющий пропустить отключение оборудования, если оборудование еще не было инициализировано.

Когда данные чипа назначаются карте с помощью `snd_device_new()` с помощью `SNDRV_DEV_LOWLEVEL`

его деструктор вызывается последним. То есть гарантируется, что все остальные компоненты, такие как PCMs и

элементы управления, уже выпущены. Вам не нужно явно останавливать PCMS и т.д., а просто вызвать

низкоуровневую аппаратную остановку.

Управление областью, отображенной в память, почти такое же, как управление портом ввода-вывода. Вам

понадобятся три поля, подобные следующим:

```
struct mychip {
    .... длинный
    iobase_phys без знака; void
    __iomem * iobase_virt;
};
```

и распределение будет таким, как показано ниже:

```
if ((err = pci_request_regions(pci, "Мой чип")) < 0) {
    kfree(чип);
    возвращает ошибку;
}
```

```

} чип-> iobase_phys = pci_resource_start(pci,
0); чип-> iobase_virt
= ioremap_nocache(чип-> iobase_phys,

                                pci_resource_len(pci, 0));

```

и соответствующий деструктор будет иметь вид:

```

static int snd_mychip_free(struct
mychip *чип) {

    .... if (чип->
iobase_virt)

        iounmap(чип->
iobase_virt); ....
pci_release_regions(чип->pci);
    ....
}

```

Записи PCI

Пока все хорошо. Давайте закончим с недостающими материалами по PCI. Сначала нам понадобится struct pci_device_id таблица

для этого набора микросхем. Это таблица с идентификатором поставщика PCI / устройства и некоторыми масками.

Например,

```

статическая структура pci_device_id snd_mychip_ids[] = {

    { PCI_VENDOR_ID_FOO, PCI_DEVICE_ID_BAR,
      PCI_ANY_ID,
      PCI_ANY_ID,
      0, 0, 0, }, .... { 0, }

};
MODULE_DEVICE_TABLE(pci, snd_mychip_ids);

```

Первое и второе поля структуры pci_device_id structure - это идентификаторы поставщика и устройства.

Если у вас нет причин фильтровать соответствующие устройства, вы можете оставить остальные поля, как указано

выше. Последнее поле в struct pci_device_id struct содержит личные данные для этой записи. Вы можете

указать любое значение здесь, например, чтобы определить конкретные операции для поддерживаемых

идентификаторов устройств. Такой пример находится в драйвере intel8x0.

Последней записью в этом списке является символ завершения. Вы должны указать эту запись, содержащую

все нули. Затем подготовьте struct pci_driver запись:

```

статическая структура pci_driver driver = {

    .name = KBUILD_MODNAME,
    .id_table = snd_mychip_ids,
    .probe = snd_mychip_probe,
    .remove = snd_mychip_remove,

};

```

Функции probe и remove уже были определены в предыдущих разделах. Функции имя поле

- это строка имени этого устройства. Обратите внимание, что вы не должны использовать

косую черту "/" в этой строке. И, наконец, записи модуля:

```

статический int __init alsacard_mychip_init(void)
{

    возвращает pci_register_driver(и драйвер);
}

статическая пустота __выход из
alsacard_mychip_exit(пустота) {

    pci_unregister_driver(и драйвер);
}

```

```
}

модуль_init(alsa_card_mychip_init)
модуль_exit(alsa_card_mychip_exit)
```

Обратите внимание, что эти записи модуля помечены `__init` и `__exit` префиксы.

О, была забыта одна вещь. Если у вас нет экспортированных символов, вам нужно объявить их в ядрах

2.2 или 2.4 (в ядрах 2.6 это необязательно).

```
экспортируй_но_символы;
```

Вот и все!

Интерфейс 1.2.6 PCM

Общая информация

Средний уровень PCM в ALSA достаточно мощный, и каждому драйверу необходимо только реализовать низкоуровневые функции для доступа к своему оборудованию.

Для доступа к слою PCM необходимо включить `<звук/pcm.h>` Первый. Кроме того,

`<sound/pcm_params.h>` может потребоваться, если вы получаете доступ к некоторым функциям, связанным с `hw_param`.

Каждое устройство с картой может иметь до четырех экземпляров `rcm`. Экземпляр `rcm` соответствует файлу устройства

`rcm`. Ограничение количества экземпляров зависит только от доступного размера в битах номеров устройств Linux.

Как только будет использован 64-битный номер устройства, у нас будет доступно больше экземпляров `rcm`. Экземпляр

`rcm` состоит из потоков воспроизведения и захвата `rcm`, и каждый поток `rcm` состоит из одного или нескольких

подпотоков `rcm`. Некоторые звуковые карты поддерживают несколько функций воспроизведения. Например, `eti10k1` имеет

PCM-воспроизведение 32 стереопотоков. В этом случае при каждом открытии (обычно) автоматически выбирается

и открывается бесплатный подпоток. Между тем, когда существует только один подпоток, и он уже был открыт,

успешное открытие приведет либо к блокировке, либо к ошибке с помощью `EAGAIN` в зависимости от режима открытия

файла. Но вам не нужно заботиться о таких деталях в вашем драйвере. Средний уровень PCM позаботится о такой работе.

Пример полного кода

Приведенный ниже пример кода не включает в себя никаких процедур аппаратного доступа, а показывает только скелет

того, как создавать интерфейсы PCM.

```
# включить <звук/pcm.h>

....

/* определение аппаратного обеспечения */

статическая структура snd_pcm hardware snd_mychip_playback_hw = {

    .info = (SNDRV_PCM_INFO_MMAP |
SNDRV_PCM_INFO_INTERLEAVED |
SNDRV_PCM_INFO_BLOCK_TRANSFER |
SNDRV_PCM_INFO_MMAP_VALID),

    .formats = SNDRV_PCM_FMTBIT_S 1 6_LE,
    .тарифы = SNDRV_PCM_RATE_8 0 0 0_4 8 0 0 0,
    .rate_min = 8 0 0 0,
    .rate_max = 4 8 0 0 0,
    .channels_min = 2,
    .channels_max = 2,
    .buffer_bytes_max = 3 2 7 6 8,
    .period_bytes_min = 4 0 9 6,
    .period_bytes_max = 3 2 7 6 8,
    .periods_min = 1,
    .periods_max = 1 0 2 4,
```

```

};

/* определение аппаратного обеспечения */ статическая структура
snd_pcm_hw snd_mychip_capture_hw = {
    .info = (SNDRV_PCM_INFO_MMAP |
             SNDRV_PCM_INFO_INTERLEAVED |
             SNDRV_PCM_INFO_BLOCK_TRANSFER |
             SNDRV_PCM_INFO_MMAP_VALID),
    .formats = SNDRV_PCM_FMTBIT_S16_LE,
    .тарифы = SNDRV_PCM_RATE_8000_48000,
    .rate_min = 8000,
    .rate_max = 48000,
    .channels_min = 2,
    .channels_max = 2,
    .buffer_bytes_max = 32768,
    .period_bytes_min = 4096,
    .period_bytes_max = 32768,
    .periods_min = 1,
    .periods_max = 1024,
};

/* открыть обратный вызов */
статический int snd_mychip_playback_open(структура
snd_pcm_substream *подпоток) {
    struct mychip * chip = snd_pcm_substream_chip(подпоток);
    struct snd_pcm_runtime * время выполнения = подпоток->
    время выполнения; время выполнения-> hw =
    snd_mychip_playback_hw; /* здесь будет выполнена
    дополнительная аппаратная инициализация */ ....
    возвращает 0;
}

/* закрыть обратный вызов */ статический
int
snd_mychip_playback_close(структура snd_pcm_substream *подпоток) {
    struct mychip * chip =
    snd_pcm_substream_chip(подпоток); /* здесь будут указаны
    коды, зависящие от оборудования */ ....
    возвращает 0;
}

/* открыть обратный вызов */
статический int snd_mychip_capture_open(структура
snd_pcm_substream *подпоток) {
    struct mychip * chip = snd_pcm_substream_chip(подпоток);
    struct snd_pcm_runtime * время выполнения = подпоток->
    время выполнения; время выполнения-> hw =
    snd_mychip_capture_hw; /* здесь будет выполнена
    дополнительная аппаратная инициализация */ ....
    возвращает 0;
}

/* закрыть обратный вызов */
статический int snd_mychip_capture_close(структура
snd_pcm_substream *подпоток) {
    struct mychip * chip =
    snd_pcm_substream_chip(подпоток); /* здесь будут указаны
    коды, зависящие от оборудования */ ....
    возвращает 0;
}

```

```

}

/* обратный вызов hw_params */ статический int
snd_mychip_pcm_hw_params(структура snd_pcm_substream *подпоток,
                        struct snd_pcm_hw_params *hw_params)

{
    возвращает snd_pcm_lib_malloc_pages(подпоток,
                                        params_buffer_bytes(hw_params));
}

/* обратный вызов hw_free */
статический int snd_mychip_pcm_hw_free(структура
snd_pcm_substream *подпоток) {
    возвращает snd_pcm_lib_free_pages(подпоток);
}

/* подготовить обратный вызов */
статический int snd_mychip_pcm_prepare(struct
snd_pcm_substream *подпоток) {
    структура mychip *chip = snd_pcm_substream_chip(подпоток);
    структура snd_pcm_runtime * время выполнения = подпоток->
время выполнения; /* настройте оборудование в соответствии
с текущей конфигурацией
* например...
*/
mychip_set_sample_format(чип, среда выполнения->формат);
mychip_set_sample_rate(чип, время выполнения-> скорость);
mychip_set_channels(чип, время выполнения-> каналы);
mychip_set_dma_setup(чип, время выполнения-> dma_addr,
                    чип-> buffer_size,
                    чип-> period_size);

    возвращает 0;
}

/* запуск обратного вызова */ статический int
snd_mychip_pcm_trigger(структура snd_pcm_substream *подпоток,
                      int cmd)

{
    switch (cmd) { регистр
SNDRV_PCM_TRIGGER_START:
        /* сделайте что-нибудь,
чтобы запустить движок
PCM */ .... прервать;

    case SNDRV_PCM_TRIGGER_STOP:
        /* сделайте что-нибудь,
чтобы остановить PCM
engine */ .... перерыв;

    По умолчанию:
        возврат -EINVAL;
    }
}

/* обратный вызов указателя */ статический
snd_pcm_uframes_t snd_mychip_pcm_pointer(структура
snd_pcm_substream *подпоток) {
    struct mychip *chip = snd_pcm_substream_chip(подпоток);
    unsigned int current_ptr;

    /* получаем текущий аппаратный указатель */
    current_ptr = mychip_get_hw_pointer(чип);

    возвращаем current_ptr;
}

```

```
/* операторы */ статическая структура
snd_pcm_ops snd_mychip_playback_ops = {

    .открыть =          snd_mychip_playback_open,
    .закрыть =          snd_mychip_playback_close,
    .ioctl =            snd_pcm_lib_ioctl,
    .hw_params =        snd_mychip_pcm_hw_params,
    .hw_free =          snd_mychip_pcm_hw_free,
    .подготовить =       snd_mychip_pcm_prepare,
    .триггер =          snd_mychip_pcm_trigger,
    .указатель =        snd_mychip_pcm_pointer,

};

/* операторы */ статическая структура
snd_pcm_ops snd_mychip_capture_ops = {

    .открыть =          snd_mychip_capture_open,
    .закрыть =          snd_mychip_capture_close,
    .ioctl =            snd_pcm_lib_ioctl,
    .hw_params =        snd_mychip_pcm_hw_params,
    .hw_free =          snd_mychip_pcm_hw_free,
    .подготовить =       snd_mychip_pcm_prepare,
    .триггер =          snd_mychip_pcm_trigger,
    .указатель =        snd_mychip_pcm_pointer,

};

/*
 * определения захвата здесь опущены...
 */

/* создайте устройство pcm */
статический int snd_mychip_new_pcm(struct
mychip *чип) {

    struct snd_pcm *pcm;
    ошибка ввода;

    err = snd_pcm_new(чип-> карта,
"Мой чип", 0, 1, 1, &pcm); если (ошибка < 0)
        вернуть ошибку;

    pcm-> private_data = чип;
    strcpy(pcm->name, "Мой чип");
    чип-> pcm = pcm;
    /* операторы набора */
    snd_pcm_set_ops(pcm, SNDRV_PCM_STREAM_PLAYBACK,
                    &snd_mychip_playback_ops);

    snd_pcm_set_ops(pcm, SNDRV_PCM_STREAM_CAPTURE,
                    &snd_mychip_capture_ops);

    /* предварительное выделение буферов */
    /* ПРИМЕЧАНИЕ: это может привести к сбою */
    snd_pcm_lib_preallocate_pages_for_all(pcm, SNDRV_DMA_TYPE_DEV,
                                           snd_dma_pci_data(чип->
pci), 64 * 1024, 64 * 1024);

    возвращает 0;

}
```

Конструктор РСМ

Экземпляр рсм выделяется `snd_pcm_new()` функция.

Было бы лучше создать конструктор для рсм, а именно,

```
статический int snd_mychip_new_pcm(структура mychip
* чип) {
```



```
struct snd_pcm *pcm;
ошибка ввода;

err = snd_pcm_new(чип-> карта,
"Мой чип", 0, 1, 1, &pcm); если (ошибка < 0)
    вернуть ошибку;

pcm-> private_data = chip;
strcpy(pcm->имя, "Мой чип");
chip-> pcm = pcm;
....
возвращает 0 ;
}
```

Функция `snd_pcm_new()` принимает четыре аргумента. Первый аргумент - это указатель на карточку, которому назначен этот `pcm`, а второй - строка идентификатора. Третий аргумент (указатель, 0 в приведенном выше примере) - индекс этого нового `pcm`. Он начинается с нуля. Если вы создаете более одного экземпляра `pcm`, укажите в этом аргументе разные числа. Например, индекс = 1 для второго устройства PCM. Четвертый и пятый аргументы - это количество подпотоков для воспроизведения и захвата соответственно. Здесь для обоих аргументов используется 1. Если подпотоки воспроизведения или захвата недоступны, передайте 0 в соответствующий аргумент. Если чип поддерживает несколько обратных воспроизведений или захватов, вы можете указать больше чисел, но они должны быть обработаны должным образом при обратных вызовах открытия / закрытия и т.д. Если вам нужно знать, к какому подпотоку вы обращаетесь, то его можно

получить из `struct snd_pcm_substream` данные передаются в каждый обратный вызов следующим образом:

```
struct snd_pcm_substream *подпоток;

int index = подпоток-> число;
```

После создания `pcm` необходимо задать операторы для каждого потока `pcm`.

```
snd_pcm_set_ops(pcm, SNDRV_PCM_STREAM_PLAYBACK,
                &snd_mychip_playback_ops);

snd_pcm_set_ops(pcm, SNDRV_PCM_STREAM_CAPTURE,
                &snd_mychip_capture_ops);
```

Операторы обычно определяются следующим образом:

```
статическая структура snd_pcm_ops snd_mychip_playback_ops = {
    .открыть = snd_mychip_pcm_open,
    .закрыть = snd_mychip_pcm_close,
    .ioctl = snd_pcm_lib_ioctl,

    .hw_params = snd_mychip_pcm_hw_params,
    .hw_free = snd_mychip_pcm_hw_free,
    .подготовить = snd_mychip_pcm_prepare,
    .триггер = snd_mychip_pcm_trigger,
    .указатель = snd_mychip_pcm_pointer,
};
```

Все обратные вызовы описаны в подразделе [Операторы](#).

После настройки операторов вы, вероятно, захотите предварительно выделить буфер.

Для предварительного выделения просто вызовите следующее:

```
snd_pcm_lib_preallocate_pages_for_all(pcm, SNDRV_DMA_TYPE_DEV,
                                      0, 0,
                                      snd_dma_pci_data(чип->
pci), 64 * 1024, 64 * 1024);
```

По умолчанию он выделит буфер объемом до 64 КБ. Детали управления буфером будут описаны позже в разделе [Управление буфером и памятью](#).

Кроме того, вы можете задать некоторую дополнительную информацию для этого `pcm-> info_flags`.

Доступные значения определяются как `SNDRV_PCM_INFO_XXX` в `<звук/asound.h>`, который используется для определения аппаратного обеспечения (описано позже). Если ваш звуковой чип поддерживает только

полудуплексный режим, укажите его следующим образом:`pcm-> info_flags = SNDRV_PCM_INFO_HALF_DUPLEX;`

... И деструктор?

Деструктор для экземпляра `pcm` не всегда необходим. Поскольку `pcm`-устройство будет освобождено с помощью кода среднего уровня автоматически, вам не нужно вызывать деструктор явно. Деструктор был бы необходим, если бы вы создали специальные записи внутри компании и вам нужно было их освободить. В таком случае установите для функции деструктора значение `pcm-> private_free`:

```
статическая пустота mychip_pcm_free(структура snd_pcm *pcm)
{
    структура mychip *chip =
    snd_pcm_chip(pcm); /* освободите свои
    собственные данные */ kfree(chip->
    my_private_pcm_data); / * делайте то, что
    вам больше нравится */ ....
}

статический int snd_mychip_new_pcm(структура
mychip *чип) {
    создайте snd_pcm *pcm;
    ....
    / * выделите свои собственные данные */
    chip-> my_private_pcm_data = kmalloc(...);
    / * установите деструктор */
    pcm-> private_data = chip;
    pcm-> private_free = mychip_pcm_free;
    ....
}
```

Указатель среды выполнения - хранилище информации PCM

При открытии подпотока PCM выделяется экземпляр среды выполнения PCM, который присваивается подпотоку. Доступ к этому указателю осуществляется через `подпоток-> среда выполнения`. Этот указатель среды выполнения содержит большую часть информации, необходимой вам для управления PCM: копию конфигураций `hw_params` и `sw_params`, указатели буфера, mmap записи, спин-блокировки и т.д. Определение экземпляра среды выполнения находится в `<sound/pcm.h>`. Вот содержимое этого файла:

```
struct _snd_pcm_runtime {
    / * -- Status -- */ struct
    snd_pcm_substream *trigger_master;
    snd_timestamp_t trigger_tstamp; / * временная метка запуска */
    превышение диапазона значений;
    snd_pcm_uframes_t avail_max;
    snd_pcm_uframes_t hw_ptr_base; / * Позиция при перезапуске буфера */
    snd_pcm_uframes_t hw_ptr_interrupt; / * Позиция во время прерывания */

    / * -- Параметры HW -- */
    snd_pcm_access_t доступ; / * режим доступа */
    формат snd_pcm_format_t; / * SNDRV_PCM_FORMAT / * SNDRV_PCM_FORMAT_ * */
    snd_pcm_subformat_t подчиненный / * подформат */
    формат; unsigned int rate; / * частота в Гц */
    unsigned int channels; / * каналы */ / * размер
    snd_pcm_uframes_t period_size; периода */
}
```

```

    беззнаковые периоды ввода;                /* периоды */

    snd_pcm_uframes_t buffer_size;              /* размер буфера */
    unsigned int tick_time;                    /* время такта */
    snd_pcm_uframes_t min_align;               /* Минимальное выравнивание для формата */
    size_t byte_align;

    unsigned int frame_bits;
    unsigned int sample_bits;
    unsigned int info;
    unsigned int rate_num;
    unsigned int rate_den;

    /* -- Параметры SW -- */
    /* временная спецификация структуры */
    /* временная метка mmap обновлена */
    timestamp_t; неподписанный int period_step;
    неподписанный int sleep_min;               /* время перехода в режим ожидания составляет минимум */

    snd_pcm_uframes_t start_threshold;
    snd_pcm_uframes_t stop_threshold; snd_pcm_uframes_t
    silence_threshold; /* Заполнение тишины происходит, когда
                        шум ближе всего к этому */
    /* /
    snd_pcm_uframes_t размер тишины;          * Размер заполнения тишиной */
    /* /
    snd_pcm_uframes_t граница;                / * точка переноса указателей */

    snd_pcm_uframes_t отключенный запуск;
    snd_pcm_uframes_t отключенный размер;

    snd_pcm_sync_id_t синхронизация;          /* идентификатор аппаратной синхронизации */

    /* -- mmap -- */
    /* изменяемая структура snd_pcm_mmap_status */
    /* *status;
    изменяемая структура snd_pcm_mmap_control
    *control; atomic_t mmap_count;
    /* -- блокировка / планирование -- */
    spinlock_t блокировка;
    wait_queue_head_t режим ожидания;
    структура timer_list tick_timer;
    структура fasync_struct * fasync;

    /* -- приватный раздел -- */
    void *private_data;

    void (*private_free)(структура snd_pcm_runtime *
    время выполнения); /* -- описание оборудования -- */
    структура snd_pcm_hardware hw;
    структура snd_pcm_hw_constraints hw_constraints;

    /* -- timer -- */
    unsigned int timer_resolution;             /* разрешение таймера */

    /* -- DMA -- */
    unsigned char *dma_area;                   /* Область DMA */
    dma_addr_t dma_addr;                       /* адрес физической шины (недоступен из основного
    процессора) */
    /* / * размер области DMA */
    size_t dma_bytes;

    struct snd_dma_buffer *dma_buffer_p;       /* выделенный буфер */

#ifdef CONFIG_SND_PCM_OSS || CONFIG_SND_PCM_OSS_MODULE
    /* -- OSS things -- */
    структура snd_pcm_oss_runtime oss;
#endif
};

```

Предполагается, что для операторов (обратных вызовов) каждого звукового драйвера большинство этих записей доступно только для чтения.

Изменяет / обновляет их только средний уровень PCM. Исключениями являются описание оборудования (hw) информация о буфере DMA и личные данные. Кроме того, если вы используете стандартный метод выделения буфера через `snd_pcm_lib_malloc_pages()`, вам не нужно самостоятельно устанавливать информацию о буфере DMA. В разделах ниже приведены важные записи.

Описание оборудования

Дескриптор оборудования (структура `snd_pcm_hw`) содержит определения основных жестких изделий конфигурации. Прежде всего, вам необходимо определить это в *ПКМ открыть обратный звонок*. Обратите внимание, что экземпляры среды выполнения содержит копию дескриптора, а не указатель на существующий дескриптор. То есть в открытом обратном вызове вы можете изменить скопированный дескриптор (время выполнения-> hw) так, как вам нужно. Например, если максимальное количество каналов равно 1 только на некоторых моделях микросхем, вы все равно можете использовать тот же аппаратный дескриптор и изменить `channels_max` позже:

```
структура snd_pcm_runtime * время выполнения = подпоток-> время выполнения;

...

время выполнения-> hw = snd_mychip_playback_hw; /* общее определение */

if (chip->model == VERY_OLD_ONE)

    время выполнения-> hw.channels_max = 1;
```

Обычно у вас будет аппаратный дескриптор, как показано ниже:

```
статическая структура snd_pcm_hw snd_mychip_playback_hw = {

    .info = (SNDRV_PCM_INFO_MMAP |

             SNDRV_PCM_INFO_INTERLEAVED |

             SNDRV_PCM_INFO_BLOCK_TRANSFER |

             SNDRV_PCM_INFO_MMAP_VALID),

    .formats = SNDRV_PCM_FMTBIT_S 1 6_LE,

    .тарифы_min = 8 0 0 0, SNDRV_PCM_RATE_8 0 0 0_4 8 0 0 0,

    .rate_max = 4 8 0 0 0,

    .channels_min = 2,

    .channels_max = 2,

    .buffer_bytes_max = 3 2 7 6 8,

    .period_bytes_min = 4 0 9 6,

    .period_bytes_max = 3 2 7 6 8,

    .periods_min = 1,

    .periods_max = 1 0 2 4,

};
```

Битовые флаги определены в `The Information` поле `struct snd_pcm_hw`. Содержит тип и возможности этого `struct`. `<звук/asound.h>` как `SNDRV_PCM_INFO_XXX`. Здесь, по крайней мере, вы должны указать, поддерживается ли `mmap` и какой формат чередования поддерживается. Если оборудование поддерживает `mmap`, добавьте `SNDRV_PCM_INFO_MMAP` отметьте здесь. Если оборудование поддерживает чередующиеся или попеременно чередующиеся форматы, `SNDRV_PCM_INFO_INTERLEAVED` или `SNDRV_PCM_INFO_NONINTERLEAVED` флаг должен быть установлен, соответственно. Если поддерживаются оба варианта, вы также можете установить оба. В приведенном выше примере, `MMAP_VALID` и `BLOCK_TRANSFER` задаются для режима OSS `mmap`. Обычно задаются оба параметра. Конечно, `MMAP_VALID` устанавливается, только если `mmap` действительно поддерживается. Другими возможными флагами являются `SNDRV_PCM_INFO_PAUSE` и `SNDRV_PCM_INFO_RESUME`. Бит `PAUSE` означает, что `struct` поддерживает операцию "пауза", в то время как бит `RESUME` бит означает, что `struct` поддерживает полную операцию "приостановить/возобновить". Если установлен флаг `PAUSE`, то срабатывает приведенный ниже обратный вызов должен обрабатывать соответствующие команды (приостановить нажатие / отпустить). Команды запуска приостановки / возобновления могут быть определены даже без `возобновить`. Отметить. См. раздел *Управление питанием* раздел для получения подробной информации. Когда подпотоки PCM могут быть синхронизированы (как правило, синхронизированы начало / остановка воспроизведения и потоков захвата), вы можете указать `SNDRV_PCM_INFO_SYNC_START` тоже. В этом случае вам нужно будет проверить связанный список подпотоков PCM в обратном вызове триггера. Это будет описано в следующем разделе.

- `форматы` поле содержит битовые флаги поддерживаемых форматов (`SNDRV_PCM_FMTBIT_XXX`). Если аппаратное обеспечение поддерживает более одного формата, укажите все биты `or'ed`. В приведенном выше примере указан 16-битный формат со знаком в конце.
- `тарифы` поле содержит битовые флаги поддерживаемых тарифов (`SNDRV_PCM_RATE_XXX`). Если чип поддерживает непрерывные тарифы, передайте дополнительно `НЕПРЕРЫВНЫЙ` бит. Предварительно определенные биты скорости предоставляются только для типичных скоростей. Если ваш чип поддерживает нетрадиционные скорости, вам необходимо добавить `УЗЕЛ` разрежьте и настройте аппаратное ограничение вручную (объяснено позже).
- `rate_min` и `rate_max` определите минимальную и максимальную частоту дискретизации. Это должно соответствовать каким-то образом, чтобы `оценивать` биты.
- `channel_min` и `channel_max` определите, как вы уже могли ожидать, минимальное и максимальное количество каналов.
- `buffer_bytes_max` определяет максимальный размер буфера в байтах. Здесь нет поля `buffer_bytes_min` поле, поскольку оно может быть рассчитано исходя из минимального размера периода и минимального количества периодов. Среднее значение- пока, `period_bytes_min` и определите минимальный и максимальный размер периода в байтах. `periods_max` и `periods_min` определите максимальное и минимальное количество периодов в буфере. "Период" - это термин, который соответствует фрагменту в мире OSS. Период определяет размер, с которым генерируется прерывание PCM. Этот размер сильно зависит от оборудования. Как правило, меньший размер периода дает вам больше прерываний, то есть больше элементов управления. В случае захвата этот размер определяет задержку ввода. С другой стороны, размер всего буфера определяет задержку вывода для направления воспроизведения.
- Также есть поле `fifo_size`. Оно определяет размер аппаратного FIFO, но в настоящее время это ни то, ни другое. используется ни в драйвере, ни в `alsa-библиотеке`. Таким образом, вы можете игнорировать это поле.

Конфигурации PCM

Хорошо, давайте снова вернемся к записям времени выполнения PCM. Наиболее часто упоминаемыми записями в экземпляре среды выполнения являются конфигурации PCM. Конфигурации PCM сохраняются в экземпляре среды выполнения после того, как приложение отправляет `hw_params` данные через `alsa-lib`. Существует множество полей, скопированных из структур `hw_params` и `sw_params`. Например, `format` содержит тип формата, выбранный приложением. Это поле содержит значение `enum SNDRV_PCM_FORMAT_XXX`.

Следует отметить, что настроенные размеры буфера и периода сохраняются в "кадрах" во время выполнения.

В мире ALSA, `1 кадр = каналы \ * размер выборки`. Для преобразования между кадрами и байтами,

вы можете использовать вспомогательные функции `frames_to_bytes()` и `bytes_to_frames()` .

```
period_bytes = frames_to_bytes(время выполнения, runtime-> period_size);
```

Кроме того, многие параметры программного обеспечения (`sw_params`) также хранятся во фреймах. Пожалуйста, проверьте тип поля. `snd_pcm_uframes_t` для фреймов используется целое число без знака, в то время как `snd_pcm_sframes_t` является для фреймов целым числом со знаком.

Информация о буфере DMA

Буфер DMA определяется следующими четырьмя полями, `dma_area`, `dma_addr`, `dma_bytes` и `dma_private`.

В поле `dma_area` хранится указатель буфера (логический адрес). Вы можете вызвать `memcpy()` от/к этому указателю.

Между тем, `dma_addr` содержит физический адрес буфера. Это поле указывается только тогда, когда буфер является линейным буфером. `dma_bytes` содержит размер буфера в байтах. `dma_private` используется для распределителя данных ALSA DMA.

Если вы используете стандартную функцию ALSA, `snd_pcm_lib_malloc_pages()` для выделения буфера эти поля устанавливаются средним уровнем ALSA, и вам не следует *не* изменять их самостоятельно. Вы можете прочитать их, но не записать. С другой стороны, если вы хотите выделить буфер самостоятельно, вам нужно будет управлять им в обратном вызове `hw_params` . По крайней мере, `dma_bytes` является обязательным. `dma_area` необходим, когда буфер `mapped`. Если ваш драйвер не поддерживает `mapped`, это поле необязательно. `dma_addr` также необязателен. Вы также можете использовать `dma_private` по своему усмотрению.

1.2. Написание драйвера ALSA 113

Состояние выполнения

Это указатель на структуру `private_data`. Состояние выполнения можно указать через `time->status->hw_ptr`.
`snd_pcm_mmap_status` запись. Например, вы можете получить текущий аппаратный указатель DMA через `runtime->control`, который указывает на `struct snd_pcm_mmap_control` запись. Однако прямой доступ к этому значению не рекомендуется.

Личные данные

Вы можете выделить запись для подпотока и сохранить ее в `private_data`. Обычно это делается в *Обратном вызове PCM open*. Не смешивайте это с `pcm->private_data`. В `pcm->private_data` обычно указывает на экземпляр `chip`, назначенный статически при создании PCM, в то время как `private_data` указывает на динамическую структуру данных, созданную при обратном вызове PCM `open`.

```
статический int snd_xxx_open(структура snd_pcm_substream *подпоток)
{
    struct my_pcm_data *данные;
    .... data = kmalloc(sizeof(*данные),
        GFP_KERNEL); подпоток->
    среда выполнения-> private_data =
        данные; ....
}
```

Выделенный объект должен быть освобожден с помощью *завершающего обратного вызова*.

Операторы

Хорошо, теперь позвольте мне подробно рассказать о каждом обратном вызове `pcm (ops)`. Как правило, каждый обратный вызов должен возвращать 0 в случае успешного выполнения или отрицательный номер ошибки, такой как `-EINVAL`. Чтобы выбрать подходящий номер ошибки, рекомендуется проверить, какое значение возвращают другие части ядра при сбое запроса того же типа. Функция обратного вызова принимает по крайней мере аргумент `struct snd_pcm_substream` указатель. Чтобы извлечь запись чипа из данного экземпляра подпотока, вы можете использовать следующий макрос.

```
int xxx() {
    struct mychip *chip =
        snd_pcm_substream_chip(подпоток); ....
}
```

Макрос читает `подпоток->private_data`, который является копией `pcm->private_data`. Вы можете переопределить первое, если вам нужно назначить разные записи данных для каждого подпотока PCM. Например, `sti8330` драйвер назначает разные значения `private_data` для направлений воспроизведения и захвата, потому что он использует два разных кодека (совместимый с SB и AD) для разных направлений.

PCM открывает обратный вызов.

```
статический int snd_xxx_open(struct snd_pcm_substream *подпоток);
```

Вызывается при открытии подпотока `pcm`.

По крайней мере, здесь вы должны инициализировать запись `runtime->hw`.

```
Обычно это делается следующим образом: статический int
snd_xxx_open(структура snd_pcm_substream *подпоток) {
    struct mychip *chip = snd_pcm_substream_chip(подпоток); struct snd_pcm_runtime
    *время выполнения = подпоток->время выполнения;
```

```
среда выполнения-> hw = snd_mychip_playback_hw;
возвращает 0 ;
}
```

где `snd_mychip_playback_hw` - предопределенное описание оборудования.

Вы можете выделить личные данные в этом обратном вызове, как описано в разделе [Личные данные](#) Раздел.

Если конфигурация оборудования требует дополнительных ограничений, задайте аппаратные ограничения и здесь. Смотрите раздел [Ограничения](#) для получения дополнительной информации.

закрыть обратный вызов

```
статический int snd_xxx_close(struct snd_pcm_substream *подпоток);
```

Очевидно, что это вызывается, когда подпоток `pcm` закрыт.

Любой частный экземпляр для подпотока `pcm`, выделенный в [открыть](#) обратный

вызов будет выпущен здесь.

```
статический int snd_xxx_close(структура
snd_pcm_substream *подпоток) {
    .... kfree(подпоток->
среда выполнения->
private_data); ....
}
```

обратный вызов `ioctl`

Это используется для любого специального вызова `pcm` `ioctls`. `snd_pcm_lib_ioctl()`.

Но обычно вы можете передать общий обратный вызов `ioctl`,

обратный вызов `hw_params`

```
статический int snd_xxx_hw_params(структура snd_pcm_substream *подпоток,
struct snd_pcm_hw_params *hw_params);
```

Это вызывается, когда аппаратный параметр (`hw_params`) настраивается приложением, то есть один раз, когда размер буфера, размер периода, формат и т.д. Определены для подпотока `pcm`.

В этом обратном вызове должно быть выполнено множество аппаратных настроек, включая выделение буферов.

Параметры, подлежащие инициализации, извлекаются с помощью `params_xxx()` макросов. Чтобы выделить

буфер, вы можете вызвать вспомогательную функцию,

```
snd_pcm_lib_malloc_pages(подпоток, params_buffer_bytes(hw_params));
```

функция `snd_pcm_lib_malloc_pages()` доступна только в том случае, если буферы DMA были предварительно

выделены. Смотрите раздел [Типы буферов](#) для получения дополнительной информации. Обратите

внимание, что `this` и `prepare` обратные вызовы могут вызываться несколько раз за инициализацию. Например,

эмуляция OSS может вызывать эти обратные вызовы при каждом изменении через свой `ioctl`. Таким образом,

вам нужно быть осторожным, чтобы не выделять одни и те же буферы много раз, что приведет к утечкам памяти!

Многократный вызов вспомогательной функции, описанной выше, - это нормально. Он автоматически

освободит предыдущий буфер, когда он уже был выделен. Еще одно замечание заключается в том, что этот обратный

вызов по умолчанию неатомный (с возможностью планирования), т. Е. Когда нет `неатомный` установлен флаг.

Это важно, потому что `триггер` обратного вызова является атомарным (не планируемым). То есть мьютексами

или любым другим **1.2. Написание драйвера ALSA**

функции, связанные с расписанием, недоступны в `trigger` обратный вызов. Пожалуйста, смотрите подраздел *Атомарность* для подробностей.

обратный вызов `hw_free`

```
статический int snd_xxx_hw_free(struct snd_pcm_substream *подпоток);
```

Вызывается для освобождения ресурсов, выделенных с помощью `hw_params`. Например,

освобождение буфера с помощью `snd_pcm_lib_malloc_pages()` выполняется путем

вызова следующего: `snd_pcm_lib_free_pages(подпоток);`

Эта функция всегда вызывается перед закрытием обратного вызова. Кроме того, обратный вызов может быть вызван несколько раз. Следите за тем, был ли ресурс уже освобожден.

подготовьте обратный вызов.

```
статический int snd_xxx_prepare(struct snd_pcm_substream *подпоток);
```

Этот обратный вызов вызывается, когда `rcm` "подготовлен". Вы можете установить тип формата, частоту дискретизации и т.д. здесь.

Отличие от `hw_params` заключается в том, что обратный вызов `prepare` будет вызываться каждый раз `snd_pcm_prepare()`

вызывается, т.е. при восстановлении после недозагрузки и т.д. Обратите внимание,

что этот обратный вызов теперь неатомный. В этом обратном вызове можно безопасно использовать функции, связанные с расписанием.

В этом и следующих обратных вызовах вы можете ссылаться на значения через запись времени выполнения, `подпоток->время`

выполнения. Например, чтобы получить текущую скорость, формат или каналы, перейдите к `время выполнения->скорость, время выполнения->`

`> формат` или `среда выполнения->каналы`, соответственно. Физический адрес выделенного буфера устанавливается равным `время`

`выполнения->time->dma_area`. Размеры буфера и периода указаны в `время выполнения->buffer_size` и `время выполнения->period_size`,

соответственно.

Будьте осторожны, этот обратный вызов также будет вызываться много раз при каждой настройке.

запустите обратный вызов

```
статический int snd_xxx_trigger(struct snd_pcm_substream *substream, int cmd);
```

Вызывается при запуске, остановке или приостановке `rcm`.

Какое действие указано во втором аргументе, `SNDRV_PCM_TRIGGER_XXX` в `<звук/pcm.h>`.

По крайней мере, в `START` и `остановка` команды должны быть определены в этом

обратном вызове.

```
switch (cmd) {
    регистр SNDRV_PCM_TRIGGER_START:
        /* сделайте что-нибудь, чтобы
        запустить движок PCM */ break;
    case SNDRV_PCM_TRIGGER_STOP:
        /* сделайте что-нибудь,
        чтобы остановить PCM engine */
        перерыв; по умолчанию:
            возврат -EINVAL;
}
```

Когда `rcm` поддерживает операцию приостановки (указанную в информационном поле таблицы аппаратного обеспечения), значения `PAUSE_PUSH`

и `PAUSE_RELEASE` команды должны обрабатываться и здесь. Первая - это команда для приостановки `rcm`,

а вторая - для повторного запуска `rcm`.

Когда `rcm` поддерживает операцию приостановки / возобновления, независимо от полной или частичной поддержки

порта приостановки / возобновления, также должны выполняться команды `ПРИОСТАНОВИТЬ` и `ВОЗОБНОВИТЬ`. Эти команды выдаются, когда

изменен статус управления питанием. Очевидно, что `приостановить` и `возобновить` команды приостанавливают и повторно вводят подпоток `рст`, и обычно они идентичны командам `остановка` и `команды "пуск"` соответственно. Подробности смотрите в разделе [Управление питанием](#). Как упоминалось, этот обратный вызов по умолчанию является атомарным, если только `неатомный` флаг не установлен, и вы не можете вызывать функции, которые могут находиться в режиме ожидания. Параметр `trigger` обратный вызов должен быть как можно меньше, просто по-настоящему запускать DMA. Остальные параметры должны быть инициализированы `hw_params` и `подготовьте` обратные вызовы должным образом заранее.

обратный вызов указателя

статический `snd_pcm_uframes_t` `snd_xxx_pointer(структура snd_pcm_substream * подпоток)`

Этот обратный вызов вызывается, когда средний уровень PCM запрашивает текущее положение оборудования в буфере. Положение должно быть возвращено в кадрах в диапазоне от 0 до `buffer_size - 1`. Обычно это вызывается из процедуры обновления буфера на среднем уровне `рст`, которая вызывается, когда `snd_pcm_period_elapsed()` вызывается в процедуре прерывания. Затем средний уровень `рст` обновляет позицию и вычисляет доступное пространство, а также пробуждает спящие потоки опроса и т.д. Этот обратный вызов также является атомарным по умолчанию.

Операции `copy_user`, `copy_kernel` и `fill_silence`

Эти обратные вызовы не являются обязательными и в большинстве случаев могут быть опущены. Эти обратные вызовы используются, когда аппаратный буфер не может находиться в обычном пространстве памяти. У некоторых микросхем есть свой собственный буфер на оборудовании, который невозможно отобразить. В таком случае вам придется вручную перенести данные из буфера памяти в аппаратный буфер. Или, если буфер не является непрерывным как в физической, так и в виртуальной памяти, эти обратные вызовы также должны быть определены. Если эти два обратных вызова определены, операции копирования и отключения выполняются ими. Подробности будут описаны в следующем разделе [Управление буфером и памятью](#).

подтверждающий обратный вызов

Этот обратный вызов также не является обязательным. Этот обратный вызов вызывается, когда `appl_ptr` обновляется при операциях чтения или записи. Некоторым драйверам, таким как `eti10k1-fx` и `cs46xx`, необходимо отслеживать текущее значение `appl_ptr` для внутреннего буфера, и этот обратный вызов полезен только для такой цели. Этот обратный вызов по умолчанию является атомарным.

обратный вызов страницы

Этот обратный вызов тоже необязателен. Этот обратный вызов используется в основном для несмежных буферов. `Mmap` вызывает этот обратный вызов для получения адреса страницы. Некоторые примеры будут объяснены в следующем разделе. [Управление буфером и памятью](#) тоже.

Обработчик прерываний PCM

Остальной частью `рст` является обработчик прерываний PCM. Роль обработчика прерываний PCM в звуковом драйвере заключается в обновлении положения буфера и сообщении среднему уровню PCM, когда положение буфера превышает заданный размер периода. Чтобы сообщить об этом, вызовите функцию `snd_pcm_period_elapsed()`. Существует несколько типов звуковых микросхем для генерации прерываний.

Прерывания на границе периода (фрагмента)

Это наиболее часто встречающийся тип: аппаратное обеспечение генерирует прерывание на границе каждого периода. В этом случае вы можете вызвать `snd_pcm_period_elapsed()` при каждом прерывании.

`snd_pcm_period_elapsed()` принимает указатель подпотока в качестве аргумента. Таким образом, вам необходимо сохранить указатель подпотока доступным из экземпляра чипа. Например, определите `подпоток` поле в записи чипа для хранения текущего указателя текущего подпотока и установки значения указателя на `Открыть` обратный вызов (и сброс в `Заккрыть` обратный вызов). Если вы получаете блокировку вращения в обработчике прерываний, и блокировка также используется в других обратных вызовах `pcm`, то вам необходимо снять блокировку перед вызовом `snd_pcm_period_elapsed()`, потому что `snd_pcm_period_elapsed()` вызывает другие обратные вызовы `pcm` внутри.

Типичный код будет выглядеть следующим образом:

```
статический irqreturn_t snd_mychip_interrupt(int irq, void *dev_id)
{
    struct mychip *chip = dev_id;
    блокировка вращения(&chip->блокировка);
    ....
    if (pcm_irq_invoked(chip)) {
        /* вызовите программу обновления,
        разблокируйте перед этим */ spin_unlock(&chip->блокировка);
        snd_pcm_period_elapsed(chip-> подпоток);
        spin_lock(&chip-> блокировка);

        /* подтвердите
        прерывание, если необходимо */
    } .... spin_unlock(&chip->блокировка);

    возвращает IRQ_HANDLED;
}
```

Прерывания высокочастотного таймера

Это происходит, когда аппаратное обеспечение не генерирует прерывания на границе периода, но выдает прерывания по таймеру с фиксированной частотой таймера (например, драйверы `es1968` или `umtfrs1`). В этом случае вам необходимо проверять текущее положение оборудования и накапливать длину обработанной выборки при каждом прерывании. Когда накопленный размер превысит размер периода, вызовите `snd_pcm_period_elapsed()` и сбросьте аккумулятор.

Типичный код будет выглядеть следующим образом.

```
статический irqreturn_t snd_mychip_interrupt(int irq, void *dev_id)
{
    struct mychip *chip = dev_id;
    блокировка вращения(&chip->блокировка);
    ....
    if (pcm_irq_invoked(chip)) {
        unsigned int last_ptr, размер;
        /* получаем текущий аппаратный указатель
        (в кадрах) */ last_ptr =
        get_hw_ptr(чип); /* вычисляем
        обработанные кадры с момента
        * последнее обновление
        */
        if (last_ptr < chip->last_ptr)
            размер = время выполнения->buffer_size + last_ptr
                    - чип->last_ptr;

        ещё

        size = last_ptr - chip-> last_ptr;
        /* запомнить последний обновленный пункт */
        chip->last_ptr = last_ptr;
    }
}
```

```

        /* накопить размер */
        чип-> размер+ = размер;

        /* за границу периода? */ if (chip->
size >= время выполнения-> period_size) {

        /* сброс аккумулятора */ chip->
size %= время выполнения-> period_size;

        /* вызов программы обновления */
        spin_unlock(&chip->блокировка);

        snd_pcm_period_elapsed(подпоток);

        spin_lock(&chip-> блокировка); / *
        подтвердите прерывание, если необходимо */

    } ....

    spin_unlock(&chip->блокировка); возвращает IRQ_HANDLED;
}

```

При вызове `snd_pcm_period_elapsed()`

случаях, даже если больше одного периода `snd_pcm_period_elapsed()` много раз. Вызовите только один раз. завершается, вы этого не делаете, и уровень рст проверит текущий указатель оборудования и обновится до последнего состояния.

вызывается в обоих

Атомарность

Одной из наиболее важных (и, следовательно, трудных для отладки) проблем в программировании ядра являются условия гонки.

В ядре Linux их обычно избегают с помощью спин-блокировок, мьютексов или семафоров. В общем, если

в обработке прерываний может возникнуть состояние гонки, им нужно управлять атомарно, и вы должны использовать

блокировку вращения для защиты критического сеанса. Если критический раздел отсутствует в коде обработчика прерываний и если

выполнение занимает относительно много времени, вам следует вместо этого использовать мьютексы или семафоры.

Как уже было видно, некоторые обратные вызовы рст являются атомарными, а некоторые - нет. Например, обратный вызов `hw_params`

является неатомным, в то время как обратный вызов `trigger` является атомарным. Это означает, что последний вызывается уже в режиме

блокировки, удерживаемом средним слоем PCM. Пожалуйста, учитывайте эту атомарность при выборе схемы блокировки в

обратных вызовах.

В атомарных обратных вызовах вы не можете использовать функции, которые могут вызывать `schedule()` или перейти к `sleep()`. Семафоры

и мьютексы могут находиться в режиме ожидания, и, следовательно, их нельзя использовать внутри атомарных обратных вызовов (например,

`trigger` обратный вызов). Чтобы реализовать некоторую задержку при таком обратном вызове, пожалуйста, используйте `udelay()` или `mdelay()`.

Все три атомарных обратных вызова (триггер, указатель и подтверждение) вызываются с отключенными локальными прерываниями.

Однако недавние изменения в коде ядра PCM позволяют всем операциям PCM быть неатомными. Это предполагает,

что все вызывающие стороны находятся в неатомных контекстах. Например, функция `snd_pcm_period_elapsed()`

обычно вызывается из обработчика прерываний. Но, если вы настроили драйвер на использование обработчика поточных прерываний

, этот вызов также может быть в неатомном контексте. В таком случае вы можете установить `неатомный` файл из `struct`

`snd_pcm` объект после его создания. Когда этот флаг установлен, `mutex` и `rwsem` используются внутри PCM

ядра вместо `spin` и `rwlocks`, так что вы можете безопасно вызывать все функции PCM в неатомном контексте.

Ограничения

Если ваш чип поддерживает нестандартные частоты дискретизации или только ограниченные выборки, вам необходимо установить

ограничение для условия.

Например, чтобы ограничить частоту дискретизации некоторыми поддерживаемыми значениями, используйте

`snd_pcm_hw_constraint_list()`. Вам нужно вызвать эту функцию в открытом обратном вызове.

```
статические беззнаковые входные частоты [] =
    { 4 0 0 0 , 1 0 0 0 0 , 2 2 0 5 0 , 4 4 1 0 0 };

статическая структура snd_pcm_hw_constraint_list constraints_rates = {
    .count = ARRAY_SIZE(ставки),
    .list = ставки,
    .mask = 0 ,
};

статический int snd_mychip_pcm_open(структура
snd_pcm_substream *подпоток) {

    int err; .... err =

    snd_pcm_hw_constraint_list(подпоток-> время выполнения, 0 ,

                                SNDRV_PCM_HW_PARAM_RATE,
                                &constraints_rates);

    if (err < 0 )
        возвращает err;

    . . . . .
}
```

Существует множество различных ограничений. Посмотрите на `звук / pcm.h` для получения полного списка. Вы даже можете определить свои собственные правила ограничения. Например, предположим, что `my_chip` может управлять подпоток из 1 канала тогда и только тогда, когда формат равен `S 1 6 _LE`, в противном случае он поддерживает любой формат, указанный в `struct snd_pcm_hardware` structure (или в любом другом `constraint_list`) . Вы можете создать правило следующим образом:

```
статический int hw_rule_channels_by_format(структура snd_pcm_hw_params *параметры,
                                          структура snd_pcm_hw_rule *правило)

{

    struct snd_interval *c = hw_param_interval(параметры,
                                                SNDRV_PCM_HW_PARAM_CHANNELS);

    struct snd_mask *f = hw_param_mask(параметры,
                                        SNDRV_PCM_HW_PARAM_FORMAT); struct snd_interval ch;

    snd_interval_any(&ch);

    if (f->bits[ 0 ] == SNDRV_PCM_FMTBIT_S 1 6 _LE) {

        ch.min = ch.max = 1 ;
        ch.целое число = 1 ;

        return snd_interval_refine(c, &

ch); }

    возвращает 0 ;
}
```

Затем вам нужно вызвать эту функцию, чтобы добавить ваше правило:

```
snd_pcm_hw_rule_add(подпоток-> время выполнения, 0 , SNDRV_PCM_HW_PARAM_CHANNELS,
                   hw_rule_channels_by_format,
                   ЗНАЧЕНИЕ NULL, SNDRV_PCM_HW_PARAM_FORMAT, - 1 );
```

Функция `rule` вызывается, когда приложение устанавливает формат PCM, и соответственно уточняет количество каналов . Но приложение может установить количество каналов до установки формата. Таким образом, вам также необходимо определить обратное

```
правило: статический int hw_rule_format_by_channels(структура snd_pcm_hw_params *параметры,
                                                    структура snd_pcm_hw_rule *правило)

{

    struct snd_interval *c = hw_param_interval(параметры,
                                                SNDRV_PCM_HW_PARAM_CHANNELS);

    struct snd_mask *f = hw_param_mask(параметры,
                                        SNDRV_PCM_HW_PARAM_FORMAT); struct snd_mask fmt;

    snd_mask_any(&fmt); /* Инициализировать структуру */

    if (c->min < 2 ) {
```

```
    fmt.bits[ 0 ]  &=
    SNDRV_PCM_FMTBIT_S 1 6_LE;  возвращает
    snd_mask_refine(f,  &fmt);
    }  возвращает  0 ;
}
```

... и в открытом обратном вызове:

```
snd_pcm_hw_rule_add(подпоток->  время  выполнения,  0 ,  SNDRV_PCM_HW_PARAM_FORMAT,
                    hw_rule_format_by_channels,  ЗНАЧЕНИЕ  NULL,
                    SNDRV_PCM_HW_PARAM_CHANNELS,  - 1 );
```

Я не буду приводить здесь больше подробностей, скорее я хотел бы сказать: "Люк, используй исходный код".

1.2.7 Интерфейс управления.

Общая информация

Интерфейс управления широко используется для многих переключателей, ползунков и т.д. доступ к которым осуществляется из пользовательского пространства. Его наиболее важным использованием является интерфейс микшера. Другими словами, начиная с ALSA 0.9.x, все компоненты микшера реализованы в API ядра control. У ALSA есть четко определенный модуль управления AC97. Если ваш чип поддерживает только AC97 и ничего больше, вы можете пропустить этот раздел. API управления определен в <звук / управление.h>. Включите этот файл, если вы хотите добавить свои собственные элементы управления.

Определение элементов управления

Чтобы создать новый элемент управления, вам необходимо определить следующие три обратных вызова: `Информация, получаем` и `помещаем`. Затем определяем a `struct snd_control_new` запись, например:

```
статическая структура snd_control_new my_control = {

    .iface = SNDRV_CTL_ELEM_IFACE_MIXER,
    .name = "Переключатель
воспроизведения PCM", .index = 0 ,
    .access = SNDRV_CTL_ELEM_ACCESS_READWRITE,
    .private_value = 0xffff,
    .info = my_control_info,
    .get = my_control_get,
    .put = my_control_put
};
```

В поле `iface` указывается тип элемента управления, `SNDRV_CTL_ELEM_IFACE_XXX`, которым обычно является `МИКШЕР`. Использовать `CARD` для глобальных элементов управления, которые логически не являются частью микшера. Если элемент управления тесно связан с каким-то конкретным устройством на звуковой карте, используйте `HWDEP`, `PCM`, `RAWMIDI`, `ТАЙМЕР`, или `SEQUENCER`, и укажите номер устройства в полях `device` и `subdevice` . В `Имя` является строкой идентификатора имени. Начиная с ALSA 0.9.x, имя элемента управления очень важно, потому что его роль классифицируется по его имени. Существуют предопределенные стандартные имена элементов управления. Подробности описаны в [Имена элементов управления](#) подраздел. В `Указатель` поле содержит порядковый номер этого элемента управления. Если существует несколько разных элементов управления с одинаковым именем, их можно отличить по порядковому номеру. Это тот случай, когда на карте существует несколько кодеков. Если индекс равен нулю, вы можете опустить приведенное выше определение. The `access` поле содержит тип доступа к этому элементу управления. Укажите комбинацию битовых масок, `SNDRV_CTL_ELEM_ACCESS_XXX`, там. Подробности будут объяснены в [Флаги доступа](#) подраздел.

The `private_value` поле содержит произвольное длинное целое значение для этой записи. При использовании

универсального `информация, получить` и `поместить` обратные вызовы, вы можете передать значение через это поле. Если

несколько небольших чисел КНЭ-обходима, вы можете объединить их в побитовых операциях. Или же можно указать

указатель (приведенный к `unsigned long`) некоторой записи и на это поле. В

`tlv` поле может использоваться для предоставления метаданных об элементе управления; смотрите подраздел [Метаданные](#).

Остальные три - это [Обратные вызовы элемента управления](#).

Имена элементов управления

Существуют некоторые стандарты для определения имен элементов управления. Элемент управления обычно определяется

из трех частей как "ФУНКЦИЯ НАПРАВЛЕНИЯ ИСТОЧНИКА".

Первая, `Источник`, указывает источник элемента управления и представляет собой строку, такую как "Master", "PCM", "CD" и

"Line". Существует множество предопределенных источников.

Второе, `DIRECTION` - это одна из следующих строк в зависимости от направления элемента управления: "Воспроизведение",

"Захват", "Воспроизведение в обход" и "Захват в обход". Или его можно опустить, что означает как воспроизведение, так и

направление захвата.

Третья, `функция`, является одной из следующих строк в зависимости от функции элемента управления: "Переключатель",

"Громкость" и "Маршрут". Примером

имен элементов управления являются, таким образом, "Главный переключатель захвата" или "Громкость воспроизведения

PCM". Есть несколько исключений:

Глобальный захват и воспроизведение

"Источник захвата", "Переключатель захвата" и "Громкость захвата" используются для глобального источника захвата (ввода),

переключателя и громкости. Аналогично, "Переключатель воспроизведения" и "Громкость воспроизведения" используются

для глобального вывода переключатель усиления и громкости.

Регуляторы тембра

переключатель управления тоном и громкостью задаются как "Tone Control - XXX", например "Tone Control - переключатель",

"Tone Control - басы", "Tone Control - Центр управления тоном".

3D - элементы управления

Переключатели и объемы 3D-управления задаются как "3D Control - XXX", например "3D Control - Switch", "3D

Control - Center", "3D Control - Space".

Усиление микрофона

Переключатель усиления микрофона установлен в положение "Усиление микрофона" или "Усиление микрофона (6 дБ)".

Более точную информацию можно найти в `Documentation/sound/alsa/ControlNames.txt`.

Флаги доступа

Флаг доступа - это битовая маска, которая определяет тип доступа к данному элементу управления. Тип доступа по

умолчанию `SNDRV_CTL_ELEM_ACCESS_READWRITE`, что означает, что для этого элемента управления разрешено как чтение, так и

запись. Когда флаг доступа опущен (т.е. = 0), это считается как `ЗАПИСЬ` на `ЧТЕНИЕ` доступ по умолчанию.

Если элемент управления доступен только для чтения, передайте вместо этого `SNDRV_CTL_ELEM_ACCESS_READ`. В этом случае вам не нужно определять обратный вызов `put`. Аналогично, когда элемент управления доступен только для записи (хотя это редкий случай), вы можете использовать `WRITE` вместо этого, и вам не понадобится обратный вызов `get`. Если контрольное значение часто меняется (например, VU meter), `VOLATILE` следует указать флажок. Это означает, что элемент управления может быть изменен без *Уведомления об изменении*. Приложения должны опрашивать такой элемент управления постоянно. Когда элемент управления неактивен, установите флажок `NEACTIVE` флаг тоже. Есть `LOCK` и `OWNER` флаги, чтобы изменить разрешения на запись.

Обратные Вызовы Управления

инфо обратный звонок

В информации обратный вызов используется для получения подробной информации об этом элементе управления. В нем должны храниться значения данного объекта `struct snd_ctl_elem_info`. Например, для логического

```
элемента управления с одним элементом: статический int snd_myctl_mono_info(struct snd_kcontrol,
                                struct snd_ctl_elem_info *uinfo)
{
    uinfo->тип =
    SNDRV_CTL_ELEM_TYPE_BOOLEAN; uinfo->количество
    = 1; uinfo->значение.целое число.мин
    = 0; uinfo->значение.целое число.макс =
    1; возвращать 0;
}
```

В тип поле определяет тип элемента управления. Существуют ЛОГИЧЕСКИЕ значения, ЦЕЛОЕ ЧИСЛО, ПЕРЕЧИСЛЯЕМЫЕ, БАЙТЫ, ЕС 9 5 8 и `INTEGER 6 4`. В поле `count` указывается количество элементов в этом элементе управления. Например, для стереосистемы громкость будет иметь значение `count = 2`. Поле `значение` представляет собой объединение, и сохраняемые значения зависят от типа. Логический и целочисленный типы идентичны. Перечисляемый тип немного отличается от других. Вам нужно будет задать строку для текущего заданного индекса элемента.

```
статический int snd_myctl_enum_info(структура snd_kcontrol,
                                struct snd_ctl_elem_info *uinfo)
{
    статический символ * тексты[4] = {
        "Первый", "Второй", "Третий",
        "Четвертый" }; uinfo->
    type = SNDRV_CTL_ELEM_TYPE_ENUMERATED;
    uinfo->count = 1;
    uinfo->value.enumerated.items = 4;
    if (uinfo->value.enumerated.item > 3)
        uinfo-> значение.перечисляемый.элемент
    = 3; strcpy(uinfo->value.enumerated.name,
        тексты[uinfo-> значение.перечисляемый.элемент]);
    возвращает 0;
}
```

Приведенный выше обратный вызов может быть упрощен с помощью вспомогательной функции, `snd_ctl_enum_info()`. Окончательный код выглядит как показано ниже. (Вы можете передать `ARRAY_SIZE(тексты)` вместо 4 в третьем

```
аргументе; это дело вкуса.) статический int snd_myctl_enum_info(struct snd_kcontrol,
                                struct snd_ctl_elem_info *uinfo)
{
    статический символ * texts[4] = {
        "Первый", "Второй", "Третий", "Четвертый"
    };
}
```

```
        возвращает snd_ctl_enum_info(uinfo, 1, 4, тексты);
    }
```

Для вашего удобства доступны некоторые общие обратные вызовы info: `snd_ctl_boolean_mono_info()` и `snd_ctl_boolean_stere_info()`. Очевидно, что первый представляет собой обратный вызов info для моноканального логического элемента, точно так же, как `snd_myctl_mono_info()` выше, а последнее относится к логическому элементу стереоканала.

получить обратный вызов

Этот обратный вызов используется для считывания текущего значения элемента

управления и возврата в пространство пользователя. Например,

```
статический int snd_myctl_get(структура snd_control *kcontrol,
                             структура snd_ctl_elem_value *ucontrol)
{
    struct mychip *chip =
        snd_control_chip(kcontrol); ucontrol->значение.целое число.значение[ 0 ] =
        get_some_value(chip); возвращает 0 ;
}
```

Поле `value` зависит от типа элемента управления, а также от обратного вызова info. Например, драйвер `sb` использует это поле для хранения смещения регистра, битового сдвига и битовой маски. Значение `private_value` поле

задается следующим образом:

```
.private_value = reg | (сдвиг << 16) | (маска << 24)
```

и извлекается в обратных вызовах, таких как

```
статический int snd_sbmixer_get_single(структура snd_control *kcontrol,
                                       структура snd_ctl_elem_value *ucontrol)
{
    int reg = kcontrol-> private_value &
        0xff; int shift = (kcontrol-> private_value
        >> 16) & 0xff; int mask = (kcontrol->
        private_value >> 24) & 0xff; ....
}
```

В обратном вызове `get` вы должны заполнить все элементы, если элемент управления содержит более одного элемента, т.Е. `count`

> 1. В приведенном выше примере мы заполнили только один элемент (`значение.целое число.значение[ 0 ]`) поскольку оно

предполагается как `count = 1`.

поставить обратный вызов

Этот обратный вызов используется для записи значения из пользовательского пространства.

Например,

```
статический int snd_myctl_put(структура snd_kontrol *kcontrol,
                              структура snd_ctl_elem_value *ucontrol)
{
    struct mychip *chip = snd_control_chip(kcontrol);
    значение изменено = 0 ;
    if (chip-> current_value !=
        ucontrol-> значение.целое число.значение[ 0 ]) {
        change_current_value(микросхема,
                              ucontrol->
                              значение.целое число.значение[ 0 ]); изменено = 1 ;
    }
}
```



```
        возврат изменен;
    }
}
```

Как показано выше, вы должны вернуть 1, если значение изменено. Если значение не изменено, верните вместо него 0. Если произойдет какая-либо фатальная ошибка, верните отрицательный код ошибки, как обычно. Как и в обратном вызове `get`, когда элемент управления содержит более одного элемента, все элементы также должны оцениваться в этом обратном вызове.

Обратные вызовы не являются атомарными

Все эти три обратных вызова в основном не являются атомарными.

Конструктор элемента управления

Когда все готово, наконец, мы можем создать новый элемент управления. Для создания элемента управления необходимо вызвать две функции, `snd_ctl_new 1()` и `snd_ctl_add()`. Проще всего это можно сделать следующим образом:

```
err = snd_ctl_add(карта, snd_ctl_new 1(&my_control, чип));

если (ошибка < 0)
    возвращает ошибку;
```

где `my_control` это структура `snd_control_new` объект определен выше, а `chip` - это указатель на объект, который должен быть передан в `kcontrol->private_data`, на который можно ссылаться в обратных вызовах. `snd_ctl_new 1()` выделяет новый экземпляр структуры `snd_kontrol` и `snd_ctl_add()` назначает данный компонент управления плате.

Уведомление об изменении

Если вам нужно изменить и обновить элемент управления в процедуре прерывания, вы можете вызвать `snd_ctl_notify()`. Для примера,

```
snd_ctl_notify(card, SNDRV_CTL_EVENT_MASK_VALUE, id_pointer);
```

Эта функция принимает указатель карты, маску события и указатель идентификатора элемента управления для уведомления. Маска событий определяет типы уведомлений, например, в приведенном выше примере уведомляется об изменении значений элемента управления. Указатель `id` - это указатель `struct snd_ctl_elem_id` для получения уведомления. Вы можете найти несколько примеров в `es 1 9 3 8 .c` или `es 1 9 6 8 .c` для аппаратных прерываний громкости.

Метаданные

Чтобы предоставить информацию о значениях dB элемента управления микшером, используйте `оп` из `DECLARE_TLV_xxx` макросы из `<звук/tlv.h>` чтобы определить переменную, содержащую эту информацию, задайте в поле `tlv.p` значение, указывающее на эту переменную, и включите `SNDRV_CTL_ELEM_ACCESS_TLV_READ` установите флажок в поле `доступ`; вот так:

```
статический DECLARE_TLV_DB_SCALE(db_scale_my_control, -4050, 150, 0);

статическая структура snd_control_new my_control = {

    ... .доступ =
    SNDRV_CTL_ELEM_ACCESS_READWRITE |
                                SNDRV_CTL_ELEM_ACCESS_TLV_READ,
    ...
    .tlv.p = db_scale_my_control,
};
```

The `DECLARE_TLV_DB_SCALE()` макрос определяет информацию о элементе управления микшером, где каждый шаг в значении элемента управления изменяет значение dB на постоянную величину dB. Первый параметр - это имя определяемой переменной. Второй параметр - это минимальное значение в единицах измерения 0,01 дБ. Третий параметр - это величина шага в единицах измерения 0,01 дБ. Установите четвертому параметру значение 1, если минимальное значение действительно отключает управление. Команда `DECLARE_TLV_DB_LINEAR()` макрос определяет информацию о элементе управления микшером, где значение элемента управления линейно влияет на выходные данные. Первый параметр - это имя определяемой переменной. Второй параметр - это минимальное значение в единицах измерения, равное 0,01 дБ. Третий параметр - это максимальное значение в единицах измерения, равное 0,01 дБ. Если минимальное значение отключает управление, установите для второго параметра значение `TLV_DB_GAIN_MUTE`.

1.2.8 API для кодека AC97

Общая информация

Уровень кодека ALSA AC97 является четко определенным, и вам не нужно писать много кода для управления им. Необходимы только низкоуровневые процедуры управления. API-интерфейс AC97 codec определен в `<sound/ac97_codec.h>`.

Полный пример кода

```
struct mychip {
    .... структура
    snd_ac97
    *ac97; ....
};

статический короткий snd_mychip_ac97_read без знака (структура snd_ac97 *ac97,
                                                    беззнаковый короткий reg)
{
    struct mychip *chip = ac97->private_data;
    .... /*
    считываем здесь значение регистра из кодека */
    возвращаем значение_register_value;
}

статическая пустота snd_mychip_ac97_write(структура snd_ac97 *ac97,
                                           беззнаковый короткий reg, беззнаковый короткий val)
{
    struct mychip *chip = ac97->private_data;
    .... /*
    записать заданное значение регистра в кодек */
}

статический int snd_mychip_ac97 (структура
mychip *чип) {
    struct snd_ac97_bus *bus;
    struct snd_ac97_template ac97;
    ошибка ввода; статическая
    структура snd_ac97_bus_ops ops = {
        .write = snd_mychip_ac97_write,
        .read = snd_mychip_ac97_read,
    };

    err = snd_ac97_bus(чип->карта, 0, &
операции, НУЛЬ, &шина); если (ошибка < 0)
        возвращает ошибку;

    memset(&ac97, 0, sizeof(ac97));
    ac97.private_data = chip; возвращает snd_ac97_mixer(шина,
&ac97, &чип->ac97);
}
```

Конструктор AC97

Чтобы создать экземпляр ac97, сначала вызовите `snd_ac97_bus()` с помощью `ac97_bus_ops_t` записи с функциями обратного вызова.

```
struct snd_ac97_bus *шина;

статическая структура snd_ac97_bus_ops ops = {

    .write = snd_mychip_ac97_write,
    .read = snd_mychip_ac97_read,
};

snd_ac97_bus (card, 0, & ops, NULL, & pbus);
```

Запись шины является общей для всех принадлежащих экземпляров ac97.

А затем вызовите `snd_ac97_mixer()` с помощью структуры `snd_ac97_template` запишите вместе с указателем шины, созданным выше.

```
struct snd_ac97_template ac97;

ошибка ввода;

memset(&ac97, 0, sizeof(ac97));

ac97.private_data = chip;

snd_ac97_mixer(шина, &ac97, &чип->ac97);
```

где `чип->ac97` - это указатель на только что созданный экземпляр `ac97_t`. В этом случае указатель чипа устанавливается как личные данные, так что функции обратного вызова чтения / записи могут ссылаться на этот экземпляр чипа. Этот экземпляр не обязательно хранится в записи чипа. Если вам нужно изменить значения регистров в драйвере или требуется приостановка / возобновление работы кодеков ac97, сохраните этот указатель для передачи соответствующим функциям.

Обратные вызовы AC97

Стандартными обратными вызовами являются `читать` и `запись`. Очевидно, они соответствуют функциям для чтения и записи доступа к аппаратным низкоуровневым кодам.

Функции `читать` обратный вызов возвращает значение регистра, указанное в аргументе.

```
статический беззнаковый короткий snd_mychip_ac97_read(структура snd_ac97 *ac97, unsigned short reg)

{

    struct mychip *chip = ac97->private_data;
    ....
    возвращает значение the_register_value;
}
```

Здесь микросхеме можно использовать из `ac97->private_data`.

Тем временем, обратный вызов `write` используется для установки

```
значения регистра static void snd_mychip_ac97_write(структура snd_ac97 *ac97, беззнаковый короткий reg, беззнаковый короткий val)
```

Эти обратные вызовы неатомны, как и обратные вызовы control API.

Существуют также другие обратные вызовы: `Сброс`, `подождите` и `инициализируйте`.

В `Сброс` обратный вызов используется для сброса кодека. Если микросхеме требуется особый вид сброса, вы можете определить этот обратный вызов.

Обратный вызов `wait` используется для добавления некоторого времени ожидания при стандартной инициализации кодека. Если микросхеме требуется дополнительное время ожидания, определите этот обратный вызов. Обратный вызов `init` используется для дополнительной инициализации кодека.

Обновление регистров в драйвере

Если вам необходимо получить доступ к кодеку из драйвера, вы можете вызвать следующие функции: `snd_ac97_write()`,

`snd_ac97_read()`, `snd_ac97_update()` и `snd_ac97_update_bits()`.

Оба `snd_ac97_write()` и `snd_ac97_update()` функции используются для установки значения в данный регистр

(`AC97_XXX`). Разница между ними в том, что `snd_ac97_update()` не записывает значение, если заданное

значение уже установлено, в то время как `snd_ac97_write()` всегда переписывает значение.

```
snd_ac97_write(ac97, AC97_MASTER, 0x8080);
```

```
snd_ac97_update(ac97, AC97_MASTER, 0x8080);
```

`snd_ac97_read()` используется для считывания значения данного регистра. Например,

```
значение = snd_ac97_read(ac97, AC97_MASTER);
```

`snd_ac97_update_bits()` используется для обновления некоторых битов в данном регистре.

```
snd_ac97_update_bits(ac97, reg, маска, значение);
```

Также есть функция для изменения частоты дискретизации (данного регистра, такого как `AC97_PCM_FRONT_DAC_RATE`)

когда кодек поддерживает VRA или DRA: `snd_ac97_set_rate()`.

```
snd_ac97_set_rate(ac97, AC97_PCM_FRONT_DAC_RATE, 44100);
```

Для установки скорости доступны следующие регистры:

`AC97_PCM_FRONT_DAC_RATE`, `AC97_PCM_LR_ADC_RATE`, `AC97_SPDIF`, `AC97_PCM_MIC_ADC_RATE`,
Когда указано значение `AC97_SPDIF`, регистр на самом деле не изменяется, но соответствующие биты состояния IEC958 будут обновлены.

Настройка синхронизации.

В некоторых чипах тактовая частота кодека не 48000, а используется тактовая частота PCI (для экономии кварца!). В этом случае

измените поле `шина->тактовая частота` к соответствующему значению. Например, драйверы `intel8x0` и `es1968` имеют

свою собственную функцию для чтения с часов.

Прос-файлы

Интерфейс ALSA AC97 создаст файл `прос`, такой как `/proc/asound/card0/codec97#0/ac97#0-0` и

`ac97#0-0+regs`. Вы можете обратиться к этим файлам, чтобы увидеть текущее состояние и регистры кодека.

Несколько кодеков

Когда есть несколько кодеков на ту же карту, вам нужно позвонить `snd_ac97_mixer()` несколько раз с

`ac97.кол-во=1` или больше. В `num` в поле указывается номер кодека.

Если вы настроили несколько кодеков, вам нужно либо написать разные обратные вызовы для каждого кодека, либо установить флажок `ac97-`

`> num` в процедурах обратного вызова.

Интерфейс 1.2.9 MIDI (MPU401-UART)

Общая информация

Многие звуковые карты имеют встроенные интерфейсы MIDI (MPU401-UART). Если звуковая карта

поддерживает стандарт MPU401-UART интерфейс, скорее всего, можно использовать ALSA

MPU401-UART с API. API MPU401-UART определен в `<sound/mpu401.h>`.

Некоторые звуковые чипы имеют похожую, но немного отличающуюся реализацию материала MPU401.

Например, emu10k1 имеет свои собственные подпрограммы MPU401.

MIDI конструктор

Чтобы создать объект rawmidi, вызовите `snd_mpu401_uart_new()`.

```
структура snd_rawmidi *rmidi;

snd_mpu401_uart_new(карта, 0, MPU401_HW_MPU401, порт, info_flags,
                    irq, &rmidi);
```

Первым аргументом является указатель карты, а вторым - индекс этого компонента. Вы можете создать

до 8 устройств rawmidi. Третий аргумент

- это тип оборудования, MPU401_HW_XXX. Если это не специальное оборудование, вы можете использовать

MPU401_HW_MPU401. 4-й аргумент - это адрес порта ввода-вывода. Многие обратно

совместимые MPU401 имеют порт ввода-вывода, такой как 0x330. Или это может быть часть его собственной

области ввода-вывода PCI. Это зависит от конструкции чипа. Пятый аргумент - это битовый флаг для получения

дополнительной информации. Если указанный выше адрес порта ввода-вывода является частью области

ввода-вывода PCI, порт ввода-вывода MPU401, возможно, уже выделен (зарезервирован) самим драйвером. В

таком случае передайте битовый флаг MPU401_INFO_INTEGRATED, и уровень MPU401-UART сам распределит порты

ввода-вывода.

Если контроллер поддерживает только входной или выходной MIDI-поток, передайте MPU401_INFO_INPUT или

MPU401_INFO_OUTPUT битовый флаг, соответственно. Затем экземпляр rawmidi создается как единый поток.

MPU401_INFO_MMIO битфлаг используется для изменения метода доступа на MMIO (через readb и writeb) вместо

ioread и iowrite. В этом случае вы должны передать ioremaped адрес в `snd_mpu401_uart_new()`. Когда

MPU401_INFO_TX_IRQ установлен, выходной поток не проверяется в обработчике прерываний по умолчанию.

Драйверу необходимо вызвать `snd_mpu401_uart_interrupt_tx()` самостоятельно, чтобы начать обработку

выходного потока в обработчике irq. Если интерфейс

MPU401 совместно использует свое прерывание с другими логическими устройствами на плате, установите

MPU401_INFO_IRQ_HOOK (см. ниже).

Обычно адрес порта соответствует командному порту, а порт + 1 соответствует порту данных.

Если нет, вы можете изменить поле cport в файле struct snd_mpu401 позже вручную. Однако, struct

Вам нужно выполнить кастинг rmidi -

snd_mpu401 указатель не возвращается явно с помощью функции `snd_mpu401_uart_new()`.

```
>приватные данные Дляструктура snd_mpu401 ЯВНО,

struct snd_mpu401 *mpu;

mpu = rmidi->private_data;
```

и сбросьте значение cport как вам угодно:

```
mpu-> cport = my_own_control_port;
```

6-й аргумент указывает номер irq ISA, который будет выделен. Если прерывание не должно выделяться (потому

что ваш код уже выделяет общее прерывание или потому что устройство не использует прерывания),

передайте -1 вместо этого. Для устройства MPU401 без прерывания вместо него будет использоваться таймер опроса.

Обработчик MIDI-прерываний

Когда прерывание выделяется в `snd_mpu 4 0 1 _uart_new()`, автоматически используется эксклюзивный обработчик прерываний ISA, следовательно, вам не остается ничего другого, как создать материал три401. В противном случае вы должны установить `MPU 4 0 1 _INFO_IRQ_HOOK` и вызываем `snd_mpu 4 0 1 _uart_interrupt()` явно из вашего собственного обработчика прерываний, когда он определил, что произошло прерывание UART.

В этом случае вам нужно передать `private_data` возвращаемого объекта `rawmidi` из

`snd_mpu 4 0 1 _uart_new()` в качестве второго аргумента `snd_mpu 4 0 1 _uart_interrupt()`.

```
snd_mpu 4 0 1 _uart_interrupt(irq, rmidi-> private_data, regs);
```

1.2.10 Интерфейс RawMIDI

Обзор

Необработанный MIDI-интерфейс используется для аппаратных MIDI-портов, к которым можно получить доступ в виде потока байтов. Он не используется для микросхем синтезатора, которые напрямую не понимают MIDI. ALSA управляет файлами и буферами. Все, что вам нужно сделать, это написать некоторый код для перемещения данных между буфером и оборудованием.

API `rawmidi` определен в `<звук/rawmidi.h>`.

Конструктор RawMIDI

Чтобы создать устройство `rawmidi`, вызовите команду `snd_rawmidi_new()` функция:

```
struct snd_rawmidi *rmidi;

ошибка = snd_rawmidi_new(чип-> карта, "MyMIDI", 0, выходы, входы и rmidi);

если (ошибка < 0)

    возвращать ошибку;

rmidi-> private_data = chip;
strcpy(rmidi-> name, "Мой MIDI");
rmidi-> info_flags = SNDRV_RAWMIDI_INFO_OUTPUT |

                    SNDRV_RAWMIDI_INFO_INPUT |

                    SNDRV_RAWMIDI_INFO_DUPLEX;
```

Первым аргументом является указатель на карточку, вторым аргументом является строка идентификатора.

Третьим аргументом является индекс этого компонента. Вы можете создать до 8 устройств `rawmidi`.

Четвертый и пятый аргументы - это количество выходных и входных подпотоков, соответственно, данного устройства (подпоток эквивалентен MIDI-порту). Установите `info_flags` поле для указания возможностей устройства. Установите `SNDRV_RAWMIDI_INFO_OUTPUT` если имеется хотя бы один выходной порт., `SNDRV_RAWMIDI_INFO_INPUT` если имеется хотя бы один входной порт, и `SNDRV_RAWMIDI_INFO_DUPLEX` если устройство может обрабатывать вывод и ввод одновременно.

После создания устройства `rawmidi` вам необходимо установить операторы (обратные вызовы) для каждого подпотока. Существуют вспомогательные функции для установки операторов для всех подпотоков устройства:

```
snd_rawmidi_set_ops(rmidi, SNDRV_RAWMIDI_STREAM_OUTPUT,
&snd_myrawmidi_output_ops); snd_rawmidi_set_ops(rmidi, SNDRV_RAWMIDI_STREAM_INPUT,
&snd_myrawmidi_input_ops); Операторы обычно определяются следующим образом:
```

```
статическая структура snd_rawmidi_ops snd_myrawmidi_output_ops = {

    snd_myrawmidi_output_open,
    snd_myrawmidi_output_close,
    .trigger = snd_myrawmidi_output_trigger,
};
```

Эти обратные вызовы описаны в разделе *Обратные вызовы RawMIDI* Раздел.

Если существует более одного подпотока, вы должны присвоить каждому из них

```
уникальное имя: struct snd_rawmidi_substream *подпоток;

список_for_each_entry(подпоток,
    &rmidi->
        потоки[SNDRV_RAWMIDI_STREAM_OUTPUT].подпотоки, список { sprintf(подпоток->
            имя, "Мой MIDI-порт %d", подпоток-> номер + 1 );
} /* то же самое для SNDRV_RAWMIDI_STREAM_INPUT
*/
```

Обратные вызовы RawMIDI

Во всех обратных вызовах к личным данным, которые вы установили для устройства rawmidi, можно получить доступ как подпоток-

```
> rmidi-> private_data.
```

Если имеется более одного порта, ваши обратные вызовы могут определять индекс порта из структуры

данные snd_rawmidi_substream, передаваемые каждому обратному вызову:

```
struct snd_rawmidi_substream *подпоток;

int index = подпоток-> номер;
```

RawMIDI открывает обратный вызов

```
статический int snd_xxx_open(struct snd_rawmidi_substream *подпоток);
```

Вызывается при открытии подпотока. Вы можете инициализировать оборудование здесь, но вам пока

не следует начинать передачу / прием данных.

RawMIDI завершает обратный вызов

```
статический int snd_xxx_close(структура snd_rawmidi_substream *субпоток);
```

Угадайте, что.

В Открыть и Закрыть обратные вызовы устройства rawmidi сериализуются с помощью мьютекса и могут переходить в режим ожидания.

Rawmidi запускает обратный вызов для выходных подпотоков.

```
static void snd_xxx_output_trigger(struct snd_rawmidi_substream *подпоток, int up);
```

Это вызывается с ненулевым значением up параметр, когда в буфере подпотока есть какие-либо данные,

которые должны быть переданы. Чтобы

прочитать данные из буфера, вызовите `snd_rawmidi_transmit_peek()`. Он вернет количество байт, которые

были прочитаны; это будет меньше, чем количество запрошенных байт, когда в

буфере больше нет данных. После успешной передачи данных вызовите `snd_rawmidi_transmit_ack()` чтобы

удалить данные из буфера подпотока:

```
данные символа без знака;

while (snd_rawmidi_transmit_peek(подпоток, &данные, 1) == 1) {
    if (snd_mychip_try_to_transmit(данные))
        snd_rawmidi_transmit_ack(подпоток, 1);
    ещё
```

```
перерыв; / * аппаратная поддержка FIFO завершена */
}
```

Если вы заранее знаете, что аппаратное обеспечение будет принимать данные, вы можете использовать `snd_rawmidi_transmit()` функция, которая считывает некоторые данные и сразу удаляет их из буфера:

```
while (snd_mychip_transmit_possible()) {
    данные символа без знака; if (snd_rawmidi_transmit(подпоток,
&data, 1) != 1)
        break; / * больше нет
данных */ snd_mychip_transmit(данные);
}
```

Если вы заранее знаете, сколько байт вы можете принять, вы можете использовать размер буфера больше единицы с помощью функций `snd_rawmidi_transmit * ()` . Кнопка запуска обратного вызова не должна находиться в режиме ожидания. Если аппаратный FIFO заполнен до того, как был опорожнен буфер подпотока, вам придется продолжить передачу данных позже, либо в обработчике прерываний, либо с помощью таймера, если на оборудовании нет прерывания передачи MIDI. Обратный вызов `trigger` вызывается с нулевым параметром `up` , когда передача данных должна быть прервана.

RawMIDI запускает обратный вызов для входных подпотоков

```
статический void snd_xxx_input_trigger(struct snd_rawmidi_substream *подпоток, int up);
```

Это вызывается с ненулевым параметром `up` , чтобы включить прием данных, или с нулевым параметром `up` , чтобы отключить прием данных. В триггер обратный вызов не должен переходить в режим ожидания; фактическое считывание данных с устройства обычно выполняется в обработчике прерываний. Когда прием данных включен, ваш обработчик прерываний должен вызвать `snd_rawmidi_receive()` для всех повторно полученных данных:

```
аннулирует snd_mychip_midi_interrupt(...)
{
    while (mychip_midi_available()) {
        данные символа без знака;
        data = mychip_midi_read();
        snd_rawmidi_receive(подпоток и данные, 1);
    }
}
```

обратный вызов drain

```
статическая пустота snd_xxx_drain(struct snd_rawmidi_substream *подпоток);
```

Используется только с подпотоками вывода. Эта функция должна дождаться передачи всех данных, считанных из подпотока буфера. Это гарантирует, что устройство можно закрыть и выгрузить драйвер без потери данных. Этот обратный вызов необязателен. Если вы не установите параметр `drain` в структуре `snd_rawmidi_ops structure`, ALSA просто будет ждать 50 миллисекунд вместо этого.

1.2.11 Прочие Устройства

FM OPL3

FM OPL3 по-прежнему используется во многих чипах (в основном для обеспечения обратной совместимости).

У ALSA также есть хороший уровень управления OPL3 FM. API OPL3 определен в `<sound/opl3.h>`.

К регистрам FM можно получить прямой доступ через API direct-FM, определенный в `<sound/asound_fm.h>`.

В ALSA собственный режим доступа к регистрам FM осуществляется через аппаратно-зависимое устройство

direct-FM extension API, тогда как в режиме, совместимом с OSS, доступ к регистрам FM можно получить с

помощью OSS direct-FM compatible API в `/dev/dmfmX` устройство.

Чтобы создать компонент OPL3, вам нужно вызвать две функции. Первая - это конструктор для `opl3_t` экземпляра.

```
struct snd_opl3 *opl3;

snd_opl3_create(карта, lport, rport, OPL3_HW_OPL3_XXX,
                интегрированный, &opl3);
```

Первый аргумент - это указатель карты, второй - адрес левого порта, а третий - адрес правого порта. В большинстве случаев правый порт помещается рядом с левым портом + 2.

Четвертый аргумент - это тип оборудования. Когда

левый и правый порты уже выделены драйвером карты, передайте ненулевое значение пятому

аргументу (`встроено`). В противном случае модуль `opl3` выделит указанные порты сам.

Если для доступа к оборудованию требуется специальный метод вместо стандартного доступа

ввода-вывода, вы можете создать экземпляр `opl3` отдельно с помощью `snd_opl3_new()`.

```
struct snd_opl3 *opl3;

snd_opl3_new(карточка, OPL3_HW_OPL3_XXX, &opl3);
```

Затем установите `command`, `private_data` и `private_free` для функции частного доступа, личных данных и деструктора. В `l_port` и `r_port` не обязательно установлены. Только команда должна быть установлена правильно. Вы можете получить данные из поля `opl3 -> private_data`. После создания экземпляра `opl3` с помощью `snd_opl3_new()` вызовите `snd_opl3_init()` для инициализации чипа в надлежащее состояние. Обратите внимание, что `snd_opl3_create()` всегда вызывает его внутренне. Если экземпляр `opl3` создан успешно, создайте устройство `hwdep` для этого `opl3`.

```
структура snd_hwdep *opl3 hwdep;

snd_opl3_hwdep_new(opl3, 0, 1, &opl3 hwdep);
```

Первым аргументом является созданный вами экземпляр `opl3_t`, а вторым - порядковый номер,

обычно 0. Третий аргумент - это смещение индекса для клиента секвенсора, назначенного порту

OPL3. Если есть MPU401-UART, укажите здесь значение 1 (UART всегда принимает значение 0).

Устройства, зависящие от оборудования.

Некоторым чипам требуется доступ к пользовательскому пространству для специальных элементов управления или для загрузки

микрокодов. В таком случае вы можете создать устройство `hwdep` (зависящее от оборудования). API `hwdep` определен в `<sound/hwdep.h>`.

Вы можете найти примеры в драйвере `opl3` или `isa/sb/sb16_csp.c`.

Создание экземпляра `hwdep` выполняется с помощью `snd_hwdep_new()`.

```
struct snd_hwdep *hw;

snd_hwdep_new(карточка, "Мой HWDEP", 0, &hw);
```

где третьим аргументом является номер индекса. Затем

вы можете передать любое значение указателя в файл `private_data`. Если вы назначаете личные данные, вам также следует определить

деструктор. Функция деструктора установлена в поле `private_free`.

```
struct mydata *p = kmalloc(sizeof(*p), GFP_KERNEL);

hw-> private_data = p;

hw-> private_free = mydata_free;
```

и реализация деструктора была бы такой:

```
static void mydata_free(struct snd_hwdep *hw)
{
    struct mydata *p = hw->private_data;
    kfree(p);
}
```

Для этого экземпляра могут быть определены произвольные файловые операции. Файловые операторы определены в операции таблица. Например, предположим, что для этого чипа требуется ioctl.

```
hw-> ops.open = mydata_open;

hw-> ops.ioctl = mydata_ioctl;

hw-> ops.release = mydata_release;
```

И реализуйте функции обратного вызова по своему усмотрению.

IEC958 (S / PDIF)

Обычно элементы управления для устройств IEC958 реализуются через интерфейс управления. Существует макрос для составления строки имени для элементов управления IEC958, SNDRV_CTL_NAME_IEC 9 5 8 () определено в <включить/asound.h>.

Существуют некоторые стандартные элементы управления для битов состояния IEC958. Эти элементы управления используют тип

SNDRV_CTL_ELEM_TYPE_IEC 9 5 8 , а размер элемента фиксируется как массив размером 4 байта (value.iec958.status[x]).

Для Информация при обратном вызове вы не указываете поле значения для этого типа (однако поле count должно быть установлено). "Маска разрешения воспроизведения IEC958" используется для возврата битовой маски для битов состояния IEC958 в режиме потребителя. Аналогично, "Маска IEC958 Playback Pro Mask" возвращает битовую маску для профессионального режима. Они доступны только для чтения элементы управления и определены как элементы управления МИКСЕРОМ (iface = SNDRV_CTL_ELEM_IFACE_MIXER - УСТРОЙСТВО для СМЕШИВАНИЯ). Между тем, элемент управления "Воспроизведение по умолчанию IEC958" определен для получения и установки текущих битов IEC958 по умолчанию . Обратите внимание, что этот элемент управления обычно определяется как элемент управления PCM (iface = SNDRV_CTL_ELEM_IFACE_PCM), хотя в некоторых местах это определяется как элемент управления микшером. Кроме того, вы можете определить управляющие переключатели для включения / выключения или для установки режима raw bit. Реализация будет зависеть от чипа , но элемент управления должен быть назван как "IEC958 xxx", предпочтительно с использованием макроса SNDRV_CTL_NAME_IEC 9 5 8 () .

Вы можете найти несколько вариантов, например, pci/emu 1 0 k 1, pci / ice 1 7 1 2 или pci/ cmipci.c.

1.2.12 Управление буферами и памятью

Типы буферов

ALSA предоставляет несколько различных функций выделения буфера в зависимости от шины и Все они имеют согласованный API. Выделение физически смежных страниц осуществляется с помощью архитектуры.

snd_malloc_xxx_pages () функция, где xxx - тип шины.

Выделение страниц с резервным вариантом равно snd_malloc_xxx_pages_fallback (). Эта функция пытается выделить указанные страницы, но если страницы недоступны, она пытается уменьшить размеры страниц до тех пор, пока не будет найдено достаточно места .

Чтобы освободить страницы, вызовите функцию snd_free_xxx_pages () . Обычно драйверы ALSA пытаются выделить и зарезервировать большое непрерывное физическое пространство во время загрузки модуля для последующего использования. Это называется "предварительным выделением". Как уже было написано, вы можете вызвать следующую функцию во время создания экземпляра rst (в случае шины PCI).

```
snd_pcm_lib_preallocate_pages_for_all(pcm, SNDRV_DMA_TYPE_DEV,
                                     snd_dma_pci_data(pci), максимальный размер);
```

где `размер` должен ли быть предварительно выделен размер в байтах и `макс` - максимальный размер, который можно изменить с помощью `prealloc` -грос-файл. Распределитель попытается получить область как можно большего размера в пределах заданного размера. Второй аргумент (тип) и третий аргумент (указатель устройства) зависят от шины. В случае шины ISA передайте `snd_dma_isa_data()` в качестве третьего аргумента с `SNDRV_DMA_TYPE_DEV` Тип. Для непрерывный буфер, не связанный с шиной, может быть предварительно выделен с помощью `SNDRV_DMA_TYPE_CONTINUOUS` введите и `snd_dma_continuous_data(GFP_KERNEL)` указатель устройства, где `GFP_KERNEL` используется ли флаг распределения ядра. Для буферов рассеяния PCI используйте `SNDRV_DMA_TYPE_DEV_SG` с `snd_dma_pci_data(pci)` (смотрите раздел [Несмежные буферы](#)). Как

только буфер будет предварительно выделен, вы можете использовать распределитель в `hw_params` обратный вызов:

```
snd_pcm_lib_malloc_pages (подпоток, размер);
```

Обратите внимание, что для использования этой функции необходимо предварительно выделить ресурсы.

Внешние аппаратные буферы

Некоторые чипы имеют свои собственные аппаратные буферы, и передача DMA из памяти хоста недоступна.

В таком случае вам нужно либо 1) скопировать / установить аудиоданные непосредственно во внешний аппаратный буфер, либо 2) создать промежуточный буфер и скопировать / установить данные из него во внешний аппаратный буфер в прерываниях (или, предпочтительно, в тасклетах). Первый случай работает нормально, если внешний аппаратный буфер достаточно велик. Этот метод не требует дополнительных буферов и, следовательно, более эффективен. Вам нужно определить `copy_user` и `copy_kernel` обратные вызовы для передачи данных, в дополнение к `fill_silence` обратный вызов для воспроизведения. Однако есть недостаток: его нельзя отобразить на карту mm. Примерами являются PCM GF1 от GUS или PCM wavetable от emu8000. Второй случай допускает mmap в буфере, хотя вам необходимо обработать прерывание или тасклет для передачи данных из промежуточного буфера в аппаратный буфер. Вы можете найти пример в драйвере vxrocket.

Другой случай, когда чип использует область карты памяти PCI для буфера вместо основной памяти. В этом случае mmap доступен только на определенных архитектурах, таких как Intel one. В режиме без mmap данные не могут быть переданы обычным способом. Таким образом, вам необходимо определить `copy_user`, `copy_kernel` и `fill_silence` также обратные вызовы, как и в приведенных выше случаях. Примеры находятся в `rme 3 2 .c` и `rme 9 6 .c`.

Реализация `copy_user`, `copy_kernel` и `silence` обратные вызовы зависят от того, поддерживает ли оборудование чередующиеся или неперемежающиеся выборки. The `copy_user` обратный вызов определяется, как показано ниже, немного по-разному в зависимости от того, является ли направление воспроизведением или захватом:

```
статический int playback_copy_user(структура snd_pcm_substream *подпоток,
                                  внутренний канал, беззнаковый длинный pos,
                                  void __user *src, беззнаковый длинный счет);
статический int capture_copy_user(struct snd_pcm_substream *подпоток,
                                  внутренний канал, беззнаковый длинный pos,
                                  void __user *dst, беззнаковый длинный счет);
```

В случае чередующихся выборок второй аргумент (`канал`) не используется. Третий аргумент (`позиция`) указывает текущее смещение позиции в байтах. Значение четвертого аргумента различается при воспроизведении и захвате. Для воспроизведения он содержит указатель исходных данных, а для захвата - указатель данных назначения. Последний аргумент - это количество байтов, которые необходимо скопировать. То, что вам нужно сделать в этом обратном вызове, снова отличается в направлениях воспроизведения и захвата. В случае воспроизведения вы копируете заданный объем данных (`count`) по указанному указателю (`src`) к указанному смещению (`позиция`) в аппаратном буфере. При кодировании подобным тетспру способом копия будет выглядеть так::

```
my_memcpy_from_user(my_buffer + pos, src, count);
```

Для направления захвата вы копируете заданный объем данных (`count`) с указанным смещением (`позиция`) в аппаратном буфере до указанного указателя (`летнее время`).

```
my_memcpy_to_user(dst, my_buffer + pos, count);
```

Здесь функции названы как `from_user` и `to_user` потому что этим обратным вызовам передается буфер пользовательского пространства. То есть предполагается, что обратный вызов копирует данные из пользовательского пространства непосредственно в / из аппаратного буфера. Внимательные читатели могут заметить, что эти обратные вызовы получают аргументы в байтах, а не в кадрах, как другие обратные вызовы. Это потому, что это упростило бы кодирование, как в приведенных выше примерах, а также упростило бы унификацию как чередующихся, так и не чередующихся вариантов, как описано ниже. В случае выборки без чередования реализация будет немного сложнее. Обратный вызов вызывается для каждого канала, передаваемого вторым аргументом, так что в общей сложности он вызывается для N каналов раз за передачу. Значение других аргументов почти такое же, как и в случае с чередованием. Предполагается, что обратный вызов копирует данные из / в данный буфер пользовательского пространства, но только для данного канала. Подробные реализации, пожалуйста, ознакомьтесь `isa/gus/gus_pcm.c` или `pci/rme9652/rme9652.c` в качестве примеров. Приведенные выше обратные вызовы представляют собой копирование из / в буфер пользовательского пространства. В некоторых случаях мы хотим вместо этого скопировать из / в буфер пространства ядра. В таком случае, `copy_kernel` вызывается обратный вызов. Это будет выглядеть так:

```
статический int playback_copy_kernel(структура snd_pcm_substream *подпоток,
                                     int channel, беззнаковый длинный pos,
                                     void *src, беззнаковый длинный счет);
статический int capture_copy_kernel(структура snd_pcm_substream *подпоток,
                                    int channel, беззнаковый длинный pos,
                                    void *dst, беззнаковый длинный счет);
```

Как легко найти, единственное отличие состоит в том, что указатель буфера не содержит `__user` префикс; то есть в четвертом аргументе передается указатель буфера ядра на. Соответственно, реализацией будет версия без пользовательской копии, такая как:

```
my_memcpy(my_buffer + pos, src, count);
```

Обычно для воспроизведения определяется другой обратный вызов `fill_silence`. Это реализовано аналогично обратным вызовам копирования, описанным выше:

```
статическое молчание int (структура snd_pcm_substream *подпоток, канал int,
                           беззнаковый длинный pos, беззнаковый длинный счетчик);
```

Значения аргументов такие же, как в `copy_user` и `copy_kernel` обратные вызовы, хотя в них нет аргумента указателя буфера. В случае чередующихся выборки аргумент канала не имеет значения, также как и при `Копировать_*` обратные вызовы. Роль функции `fill_silence` обратного вызова заключается в установке заданного количества (`count`) данных тишины с указанным смещением (`позиция`) в аппаратном буфере. Предположим, что формат данных подписан (то есть данные, хранящиеся в режиме ожидания, равны 0), и реализация с использованием функции, подобной `memset`, будет выглядеть следующим образом: `my_memset(my_buffer + pos, 0, количество);`

В случае выборки без чередования, опять же, реализация становится немного сложнее, поскольку это вызывается N раз за передачу для каждого канала. Смотрите, например, `isa/gus/gus_pcm.c`.

Несмежные буферы

Если ваше оборудование поддерживает таблицу страниц, как в етu10k1, или дескрипторы буфера, как в v1a82xx, вы можете использовать модуль сбора данных (SG) DMA. ALSA предоставляет интерфейс для обработки SG-буферов. API предоставляется в <sound/pcm.h>.

Для создания обработчика SG-буфера вызовите `snd_pcm_lib_preallocate_pages()`

или

`snd_pcm_lib_preallocate_pages_for_all()` с помощью `SNDRV_DMA_TYPE_DEV_SG` в конструкторе PCM, как в другом предварительном распределителе PCI. Вам нужно передать `snd_dma_pci_data(pci)`, где `pci` - это `struct pci_dev` также указатель чипа. Экземпляр `struct snd_sg_buf` создается как `подпоток-> dma_private`. Вы можете преобразовать указатель следующим образом:

```
struct snd_sg_buf *sgbuf = (struct snd_sg_buf *)подпоток->dma_private;
```

Затем вызовите `snd_pcm_lib_malloc_pages()` в обратном вызове `hw_params`, а также в случае обычного буфера PCI

. Обработчик SG-буфера выделит несмежные страницы ядра заданного размера и отобразит

их в практически непрерывной памяти. Виртуальный указатель адресуется во время выполнения-> `dma_area`.

Физический адрес (время выполнения-> `dma_addr`) устанавливается равным нулю, поскольку буфер физически не является непрерывным.

Таблица физических адресов настраивается в `sgbuf->` таблица. Вы можете получить физический адрес с определенным смещением

через `snd_pcm_sgbuf_get_addr()`.

При использовании SG-обработчика необходимо установить `snd_pcm_sgbuf_ops_page()` в качестве Страница обратный вызов. (См. [страница обратного вызова](#) Раздел.)

Чтобы освободить данные, вызовите `snd_pcm_lib_free_pages()` в `hw_free` обратный вызов как обычно.

Буферы с выделением Vmalloc

Можно использовать буфер, выделенный через `vmalloc()`, например, для промежуточного буфера. Поскольку

выделенные страницы не являются смежными, вам необходимо установить Страница обратный вызов для получения физического адреса при каждом смещении.

Реализация Страница обратный вызов будет выглядеть следующим образом:

```
# включить <linux/vmalloc.h>

/* получить указатель физической страницы с заданным смещением */

страница статической структуры *mychip_page(структура snd_pcm_substream *подпоток,
                                           длинное смещение без знака)
{
    недействительно *pageptr = подпоток-> среда выполнения-> dma_area
    + смещение; возвращает vmalloc_to_page(pageptr);
}
```

1.2.13 Интерфейс Proc

ALSA предоставляет простой интерфейс для `procfs`. Файлы `proc` очень полезны для отладки. Я рекомендую вам настроить файлы `proc`, если вы пишете драйвер и хотите получать статус запуска или регистрировать дампы. API находится в <звук /информация.h>.

Чтобы создать файл `proc`, вызовите `snd_card_proc_new()`.

```
struct snd_info_entry *запись;

int err = snd_card_proc_new(карточка, "мой файл", &запись);
```

где второй аргумент указывает имя создаваемого файла `proc`. Приведенный выше пример

создаст файл `my-file` в каталоге `card`, например `/proc/asound/card 0 /my-file`.

Как и другие компоненты, запись `proc` создается с помощью `snd_card_proc_new()` будет зарегистрирован и выпущен автоматически в функциях регистрации и выпуска карты.

Когда создание проходит успешно, функция сохраняет новый экземпляр в указателе, указанном в третьем аргументе . Он инициализируется как текстовый файл процедуры, доступный только для чтения. Чтобы использовать этот прос-файл как текстовый файл,

доступный только для чтения, как он есть, установите обратный вызов read с закрытыми данными через snd_info_set_text_ops().

```
snd_info_set_text_ops(запись, чип, my_proc_read);
```

где второй аргумент (chip) - это личные данные, которые будут использоваться в обратных вызовах. Третий параметр

определяет размер буфера чтения, а четвертый (my_proc_read) является функцией обратного вызова, которая определяется как

```
static void my_proc_read(struct snd_info_entry * запись,
                        struct snd_info_buffer * буфер);
```

В обратном вызове read используйте snd_iprintf() для выходных строк, который работает так же, как обычный printf().

Для примера,

```
static аннулирует my_proc_read(структура snd_info_entry * запись,
                               структура snd_info_buffer * буфер)
{
    struct my_chip * чип = запись-> приватные_данные;

    snd_iprintf(буфер, "Это мой чип!\n");
    snd_iprintf(буфер, "Порт = %ld\n", чип-> порт);
}
```

Права доступа к файлам могут быть изменены впоследствии. По умолчанию для всех пользователей установлено значение "только

для чтения". Если вы хотите добавить разрешение на запись для пользователя (root по умолчанию), сделайте следующее:

```
entry-> mode = S_IFREG | S_IRUGO | S_IWUSR;
```

и установите размер буфера записи и обратный вызов

```
entry-> c.text.write = my_proc_write;
```

Для обратного вызова write вы можете использовать snd_info_get_line() чтобы получить текстовую строку, и snd_info_get_str()

для извлечения строки из строки. Некоторые примеры можно найти в core/oss/ mixer_oss.c, core/oss/и

pcm_oss.c.

Для прос-файла с исходными данными задайте атрибуты следующим образом:

```
статическая структура snd_info_entry_ops my_file_io_ops = {

    .read = my_file_io_read,
};

запись-> content = SNDRV_INFO_CONTENT_DATA;
запись-> private_data = чип;
запись-> c.ops = &my_file_io_ops;
запись-> size = 4096;
запись-> mode = S_IFREG | S_IRUGO;
```

Для получения необработанных данных, размер поле должно быть настроено правильно. Это определяет максимальный

размер доступа к файлу прос. Обратные вызовы чтения / записи в режиме raw являются более прямыми, чем в текстовом режиме. Вам

необходимо использовать низкоуровневые функции ввода-вывода, такие как copy_from/to_user() для передачи данных.

```
статический ssize_t my_file_io_read(struct snd_info_entry * запись,
                                   void * file_private_data,
                                   struct file * файл,
                                   символ * buf,
                                   размер_t кол-во,
                                   loff_t pos)
{
    if (copy_to_user(buf, local_data + pos, count))
        возврат -EFAULT;
```

```
        возвращает количество результатов;
    }
}
```

Если размер информационной записи был настроен правильно, количество результатов и позиция гарантированно укладываются в 0 и заданный размер. Вам не нужно проверять диапазон в обратных вызовах, если не требуется выполнение каких-либо других условий.

1.2.14 Управление питанием.

Если предполагается, что микросхема будет работать с функциями приостановки / возобновления, вам необходимо добавить код управления питанием в драйвер. Дополнительный код для управления питанием должен быть, если он определен, равен CONFIG_PM. Если драйвер полностью поддерживает приостановку / возобновление, то есть устройство может быть надлежащим образом возвращено в состояние, в котором оно находилось при вызове приостановки, вы можете установить SNDRV_PCM_INFO_RESUME установите флажок в поле pcm info. Обычно это возможно, когда регистры микросхемы могут быть безопасно сохранены и восстановлены в оперативной памяти. Если этот параметр установлен, обратный вызов триггера вызывается с помощью команды SNDRV_PCM_TRIGGER_RESUME после завершения обратного вызова resume. Даже если драйвер не поддерживает PM полностью, но частичная приостановка / возобновление все еще возможна, все равно стоит реализовать обратные вызовы suspend / resume. В таком случае приложения сбросили бы статус, вызвав snd_pcm_prepare() и перезапустили бы поток соответствующим образом. Следовательно, вы можете определить обратные вызовы приостановки / возобновления ниже, но не устанавливайте SNDRV_PCM_INFO_RESUME информационный флаг для PCM. Обратите внимание, что триггер с функцией SUSPEND всегда может быть вызван при вызове snd_pcm_suspend_all(), учитывая меньшую часть SNDRV_PCM_INFO_RESUME Отметить. Флаг RESUME влияет только на поведение snd_pcm_resume(). (Таким образом, теоретически, SNDRV_PCM_TRIGGER_RESUME не требуется обрабатывать в обратном вызове триггера, когда нет SNDRV_PCM_INFO_RESUME установлен флаг. Но лучше сохранить его по соображениям совместимости.) В более ранней версии драйверов ALSA был предусмотрен общий уровень управления питанием, но он был удален. Драйверу необходимо определить перехваты приостановки / возобновления в соответствии с шиной, к которой подключено устройство. В случае драйверов PCI обратные вызовы выглядят следующим образом:

```
#ifdef CONFIG_PM

статический int snd_my_suspend(struct pci_dev *pci, pm_message_t state)
{
    .... /* выполняем действия
для suspend */ возвращаем
    0; } static int
snd_my_resume(struct
pci_dev *pci) {
    .... /* выполните действия для
приостановки */ верните 0;
}
#endif
```

Схема реального задания приостановки следующая.

- 1. Извлеките карту и данные чипа.
- 2. Вызовите snd_power_change_state() с помощью SNDRV_CTL_POWER_D3hot чтобы изменить состояние питания.
- 3. Вызовите snd_pcm_suspend_all() чтобы приостановить текущие потоки PCM.
- 4. Если используются кодеки AC97, вызовите snd_ac97_suspend() для каждого кодека.
- 5. При необходимости сохраните значения регистров.
- 6. При необходимости остановите аппаратное обеспечение.
- 7. Отключите устройство PCI, вызвав pci_disable_device(). Затем, наконец, вызовите pci_save_state() .

Типичный код будет выглядеть так:

```
статический int mychip_suspend(структура pci_dev *pci, pm_message_t состояние)
{
    ....
    /* ( 1 ) */
    ....
}
```

```
    struct snd_card *card =
pci_get_drvdata(pci); struct
mychip *chip = card-> private_data;
/* (2) */ snd_power_change_state(card,
SNDRV_CTL_POWER_D3hot); /* (3) */
snd_pcm_suspend_all(chip->pcm);
/* (4) */
snd_ac97_suspend(chip->ac97);
/* (5) */ snd_mychip_save_registers(микросхема);
/* (6) */
snd_mychip_stop_hardware(микросхема);
/* (7) */
pci_disable_device(pci);
pci_save_state(pci);
возвращает 0;
}
```

Схема реального задания по возобновлению работы следующая.

1. Извлеките карту и данные чипа.

2. Настройте PCI. Сначала вызовите `pci_restore_state()`.

3. Повторно инициализируйте микросхему.

4. Восстановите сохраненные регистры, если необходимо.

5. Возобновите работу микшера, например, вызвав `snd_ac97_resume()`.

6. Перезапустите оборудование (если оно есть).

7. Вызовите `snd_power_change_state()` с помощью `SNDRV_CTL_POWER_D0` для
- Затем снова включите устройство pci, вызвав

уведомления процессов. Типичный код был бы таким:

```
статический int mychip_resume(struct pci_dev *pci)
{
    /* (1) */ структура
snd_card *card =
pci_get_drvdata(pci); struct
mychip *chip = card-> private_data;
/* (2) */ pci_restore_state(pci);
pci_enable_device(pci);
pci_set_master(pci);
/* (3) */
snd_mychip_reinit_chip(чип);
/* (4) */
snd_mychip_restore_registers(чип);
/* (5) */ snd_ac97_resume(чип->ac97);
/* (6) */ snd_mychip_restart_chip(микросхема);
/* (7) */ snd_power_change_state(карта,
SNDRV_CTL_POWER_D0);
возвращает 0;
}
```

Как показано выше, лучше сохранять регистры после приостановки операций PCM через `snd_pcm_suspend_all()` или `snd_pcm_suspend()`. Это означает, что потоки PCM уже остановлены при создании снимка регистра. Но помните, что вам не нужно перезапускать поток PCM в возобновлении обратного вызова. Он будет перезапущен с помощью триггерного вызова с помощью `SNDRV_PCM_TRIGGER_RESUME` при необходимости. Хорошо, теперь у нас есть все обратные вызовы. Давайте настроим их. При инициализации карты убедитесь, что вы можете

получайте данные чипа из экземпляра карты, обычно через `private_data` поле, если вы создали данные чипа индивидуально.

```
статический int snd_mychip_probe(struct pci_dev *pci,
                                постоянная структура pci_device_id *pci_id)
{
    ....
    struct snd_card *карта;
    struct mychip * чип;
    ошибка ввода; ....
    err = snd_card_new(&pci->dev, индекс[dev],
                      идентификатор[dev], THIS_MODULE,
                      0, &карточка);

    .... чип = kzalloc(sizeof( * чип),
                      GFP_KERNEL); ....

    card-> private_data =
    чип; ....
}
```

Когда вы создали данные чипа с помощью `snd_card_new()`, они в любом случае доступны через `private_data` поле.

```
статический int snd_mychip_probe(struct pci_dev *pci,
                                const struct pci_device_id *pci_id)
{
    ....
    struct snd_card *карта;
    struct mychip * чип;
    ошибка ввода; ....
    err = snd_card_new(&pci->dev, индекс[dev],
                      идентификатор[dev], THIS_MODULE,
                      sizeof(структура mychip), &card);

    .... чип
    =
    card->private_data; ....
}
```

Если вам нужно место для сохранения регистров, выделите для него буфер и здесь, поскольку это было бы фатально, если вы не сможете выделить память на этапе приостановки. Выделенный буфер должен быть освобожден в соответствующем деструкторе. И затем установите для `pci_driver` функцию приостановки / возобновления обратных вызовов.

```
статическая структура pci_driver driver = {
.name = KBUILD_MODNAME,
.id_table = snd_my_ids,
.probe = snd_my_probe,
.remove = snd_my_remove,
#ifdef CONFIG_PM
    .приостановить = snd_my_suspend,
    .возобновить = snd_my_resume,
#endif
};
```

1.2.15 Параметры модуля

Существуют стандартные варианты модулей для ALSA. По крайней мере, каждый модуль должен иметь указатель, ID и включить Опции. Если модуль поддерживает несколько карт (обычно до 8 = `SNDRV_CARDS` карты), они должны быть массивами. Начальные значения по умолчанию уже определены как константы для упрощения программирования:

```
статический int-индекс[SNDRV_CARDS] = SNDRV_DEFAULT_IDX;
статический char * id[SNDRV_CARDS] = SNDRV_DEFAULT_STR;
статический int-enable[SNDRV_CARDS] = SNDRV_DEFAULT_ENABLE_PNP;
```

Если модуль поддерживает только одну карту, вместо этого они могут быть отдельными переменными. `включить` опция не всегда необходима в этом случае, но было бы лучше иметь фиктивную опцию для совместимости.

Параметры модуля должны быть объявлены в соответствии со стандартом `модуль_парама()`, `мод-ule_param_array()` и `MODULE_PARM_DESC()` макросы.

Типичная кодировка была бы такой, как показано ниже:

```
# определение ИМЕНИ карты "Мой чип"

module_param_array(индекс, int, NULL, 0 4 4 4);

MODULE_PARM_DESC(индекс, "Значение индекса для звуковой карты "CARD_NAME ".");

module_param_array(идентификатор, charp, NULL, 0 4 4 4);

MODULE_PARM_DESC(идентификатор, "строка идентификатора для звуковой карты "CARD_NAME ".");

module_param_array (включить, bool, NULL, 0 4 4 4);

MODULE_PARM_DESC(включить, "Включить " ИМЯ_КАРТЫ " soundcard.");
```

Также не забудьте указать описание модуля, классы, лицензию и устройства. В частности, недавний `modprobe` требует определить лицензию модуля как GPL и т.д., В противном случае система отображается как

```
"испорченная".MODULE_DESCRIPTION("Мой чип");

MODULE_LICENSE("GPL");

MODULE_SUPPORTED_DEVICE("{Поставщик, название моего чипа}");
```

1.2.16 Как поместить ваш драйвер в дерево ALSA

Общая информация

К настоящему моменту вы научились писать коды драйверов. И теперь у вас может возникнуть вопрос: как поместить мой собственный драйвер в дерево драйверов ALSA? Здесь (наконец :) кратко описана стандартная процедура. Предположим, что вы создаете новый драйвер PCI для карты "xyz". Имя модуля карты будет `snd-xyz`. Новый драйвер обычно помещается в дерево `alsa-driver`, `alsa-драйвер/pci` каталог в случае PCI-карт. Затем драйвер оценивается, аудируется и тестируется разработчиками и пользователями. Через определенное время драйвер перейдет в дерево `alsa-kernel` (в соответствующий каталог, например `alsa-kernel/pci`) и в конечном итоге будет интегрирован в дерево Linux 2.6 (каталог будет `linux / sound / pci`). В следующих разделах предполагается, что код драйвера будет помещен в `alsa-дерево драйверов`. Рассмотрены два случая: драйвер, состоящий из одного исходного файла, и драйвер, состоящий из нескольких исходных файлов.

Драйвер с одним исходным файлом

- 1. Измените `alsa-driver/pci/Makefile`

Предположим, у вас есть файл `xyz.c`. Добавьте следующие две строки

```
snd-xyz-objs := xyz.o
obj-$(CONFIG_SND_XYZ) += snd-xyz.o
```

- 2. Создайте запись `Kconfig`

Добавьте новую запись `Kconfig` для вашего драйвера `xyz`. конфигурация `SND_XYZ tristate "Foobar XYZ"` зависит от `SND` выберите `SND_PCM`, в справке здесь указано, что `Y` включает поддержку звуковой карты `Foobar XYZ`. Чтобы скомпилировать этот драйвер в виде модуля, выберите здесь `M`: модуль будет называться `snd-xyz`. строка, выберите `SND_PCM`, указывает, что драйвер `xyz` поддерживает `PCM`. Помимо `SND_PCM`, для команды `select` поддерживаются следующие компоненты : `SND_RAWMIDI`, `SND_TIMER`, `SND_HWDEP`, `SND_MPU401_UART`,

SND_OPL3_LIB, SND_OPL4_LIB, SND_VX_LIB, SND_AC97_CODEC. Добавьте команду select для каждого поддерживаемого компонента. Обратите внимание, что некоторые варианты выбора подразумевают низкоуровневые варианты. включает ТАЙМЕР, MPU401_UART включает RAWMIDI, AC97_CODEC включает PCM, а OPL3_LIB включает HWDEP. Вам не нужно снова указывать низкоуровневые параметры.

Например, PCM

Для получения подробной информации о скрипте Kconfig обратитесь к документации kbuild.

3. Запустите cvscompile script, чтобы повторно сгенерировать configure script и собрать все заново.

Драйверы с несколькими исходными файлами

Предположим, что драйвер snd-xyz имеет несколько исходных файлов. Они расположены в новом подкаталоге, pci/xyz.

1. Добавьте новый каталог (xyz) в alsa-driver/pci/Makefile как показано ниже

obj-\$(CONFIG_SND) += xyz/

2. В каталоге xyz создайте Makefile

```
ifndef SND_TOPDIR

SND_TOPDIR=../..

endif

включить $(SND_TOPDIR)/toplevel.config

включить $(SND_TOPDIR)/Makefile.conf

snd-xyz-objs := xyz.o abc.o def.o

obj-$(CONFIG_SND_XYZ) += snd-xyz.o

включить $(SND_TOPDIR)/Rules.make
```

3. Создайте запись Kconfig

Эта процедура такая же, как в предыдущем разделе.

4. Запустите cvscompile script, чтобы повторно сгенерировать configure script и собрать все заново.

1.2.17 Полезные функции

snd_printk() и друзья

ALSA предоставляет подробную версию функции printk(). Если конфигурация ядра CONFIG_SND_VERBOSE_PRINTK если задано, эта функция печатает данное сообщение вместе с именем файла и строкой вызывающего объекта. KERN_XXX префикс обрабатывается так же, как и оригинал printk() делает, поэтому рекомендуется добавить этот префикс, например, snd_printk(KERN_ERR "О боже, извините, это крайне плохо!\n"); Существуют также printk() для отладки. snd_printd() может использоваться для общих целей отладки. Если CONFIG_SND_DEBUG установлен, эта функция скомпилирована и работает точно так же, как snd_printk(). Если ALSA компилируется без флага отладки, он игнорируется.

snd_printdd() компилируется только тогда, когда CONFIG_SND_DEBUG_VERBOSE установлен.

Пожалуйста, обратите внимание, что CON-

FIG_SND_DEBUG_VERBOSE не устанавливается по умолчанию, даже если вы настраиваете alsa-драйвер с параметром --with-debug=full. Вместо этого вам нужно явно указать параметр --with-debug=detect.

`snd_BUG ( )`

Она показывает сообщение `ОШИБКА?` и трассировку стека, а также `snd_BUG_ON( )` в точке. Полезно показать, что там происходит фатальная ошибка.

Когда флаг отладки не установлен, этот макрос игнорируется.

`snd_BUG_ON( )`

`snd_BUG_ON( )` макрос аналогичен с `WARN_ON( )` макрос. Например, `snd_BUG_ON(!указатель);` или это может быть использовано в качестве условия, `if (snd_BUG_ON(non_zero_is_bug)) return -EINVAL;` Макрос принимает условное выражение для вычисления. Когда установлено значение `CONFIG_SND_DEBUG`, если выражение не равно нулю, отображается предупреждающее сообщение, такое как `ОШИБКА? (xxx)` обычно за ним следует трассировка стека. В обоих случаях возвращается оцененное значение.

1.2.18 Благодарности

Я хотел бы поблагодарить Фила Керра за его помощь в улучшении и исправлении этого документа.

Кевин Кондер переформатировал исходный обычный текст в формат DocBook.

Джулиано Почини исправил опечатки и добавил примеры кодов в раздел "Аппаратные ограничения".

ПРОЕКТЫ И РЕАЛИЗАЦИИ

2.1 Стандартные названия элементов управления ALSA

В этом документе описаны стандартные названия элементов управления микшером.

2.1.1 Стандартный синтаксис

Синтаксис: [МЕСТОПОЛОЖЕНИЕ] ИСТОЧНИК [КАНАЛ] [НАПРАВЛЕНИЕ] ФУНКЦИЯ

НАПРАВЛЕНИЕ

<ничего> Воспроизведение	в обоих направлениях
, Захват	в одном направлении
в обход воспроизведения,	в одном направлении
Захват в обход	one direction

ФУНКЦИЯ

Переключение	переключатель включения/выключения
маршрута	усилитель
громкости	управление маршрутом, зависящее от оборудования

КАНАЛ

<ничего>	не зависит от канала или применяется
Спереди	ко всем каналам, передние левые / правые каналы, задние левые /
Объемный	правые для объемного звучания 4.0 /
Центр	5.1, каналы C / LFE, центральная канавка, канал LFE, боковые левые
Сбоку	/ правые для объемного звучания 7.1

МЕСТОПОЛОЖЕНИЕ (физическое местоположение источника)

Передняя/задняя	передняя позиция задняя позиция
док-станция	на док-станции
внутренняя	внутренняя

Источник

Мастер -	
мастер, моно -	
мастер	
аппаратного	внутренний динамик
динамик	внутренний динамик LFE
НЧ-динамик для	
наушников,	генератор звуковых сигналов
линейный выход	
звукового	
сигнала телефон	
телефон введите	
телефон выходе	
синтезатора	
FM-радио	микрофонная часть комбинированного разъема для гарнитуры - 4-контактный
микрофон	разъем для наушников + микрофон микрофонная часть либо / или - 3-контактный разъем для
гарнитура	наушников или микрофона только вход, для вывода используйте "Линейный выход"
микрофон	
наушники	
микрофон линии	
CD с видео - зум	
видео - вход AUX	
PCM PCM	
Пан замыкания	D / A -> A / D loopback
аналогового	воспроизведение -> захват loopback - без аналогового канала
шлейфа	
шлейфа	
цифровых моно	
моно выход многоканального	
АЦП	
волны музыки	
I2S для IEC958	
вход HDMI на	
SPDIF вход	только выходной сигнал
цифровой	
выход HDMI/ДП	
	либо HDMI, либо DisplayPort

2.1.2 Исключения (устарели)

[Аналоговый цифровой] Источник захвата [Аналоговый цифровой]	он же переключатель входного усиления, он же громкость входного усиления, он же переключатель выходного усиления, он же громкость выходного усиления.
Переключатель захвата	
[Аналоговый цифровой] Громкость захвата [Аналоговый цифровой]	
Переключатель воспроизведения	
[Аналоговый Цифровой] Регулятор громкости воспроизведения	
- Переключатель Регулятор тона - Bass Регулятор тона	
- Treble 3D Control - Переключатель 3D Control - Center 3D Control	
- Depth 3D Control - Wide 3D Control	
- Space 3D Control - Level	
Усиление микрофона [(?дБ)]	

2.1.3 Интерфейс PCM

Образец источника тактовых импульсов, Состояние синхронизации тактовых импульсов, Внешняя скорость	{ "Word", "Internal", "AutoSync" } {"Блокировка", "Синхронизация", "Без блокировки"}
Скорость захвата	скорость захвата, полученная из внешнего источника

2.1.4 Интерфейс IEC958 (S/PDIF)

IEC958 [...] [Воспроизведение захват] Переключатель	включение/выключение интерфейса
IEC958 [...] [Воспроизведение захват] Громкость	IEC958 цифровой регулятор громкости
IEC958 [...] [Воспроизведение захват] По умолчанию	по умолчанию или глобальное значение -
IEC958 [...] [Воспроизведение захват] Маска IEC958 [...]	чтение/запись потребительская и профессиональная маска
[Воспроизведение захват] Против маски IEC958 [...]	
[Воспроизведение захват] Профессиональная маска	потребительская маска профессиональная маска
IEC958 [...] [Воспроизведение захват] Поток PCM	настройки, назначенные потоку
Q-подкод IEC958 [Воспроизведение захват] По умолчанию	PCM биты Q-субкодирования слова
Преамбула IEC958 [Воспроизведение захват] По умолчанию	преамбулы пакета (4*16 бит)

2.2 API отображения каналов ALSA PCM

Такаши Иваи <tiwai@suse.de>

2.2.1 Общие положения

API отображения каналов позволяет пользователю запрашивать возможные карты каналов и текущую карту каналов, также необязательно изменять карту каналов текущего потока.

Карта каналов - это массив позиций для каждого PCM-канала. Как правило, стереопоток PCM имеет

карту каналов { front_left, front_right } в то время как поток PCM объемного звучания 4.0 имеет карту каналов { передний левый, передний правый, задний левый, задний правый }.

Проблема до сих пор заключалась в том, что у нас не было явной стандартной карты каналов, и у приложений не было возможности узнать, какой канал соответствует какой позиции (динамика). Таким образом, приложения применяли неправильные каналы

для выходов 5.1, и вы внезапно слышите странный звук сзади. Или же некоторые устройства тайно предполагают, что center / LFE - это третий / четвертый каналы, в то время как другие используют / LFE как 5-й / 6-й каналы. Кроме того, некоторые устройства, такие как HDMI, настраиваются для разных положений динамиков даже при одинаковом количестве общих каналов. Однако указать это было невозможно из-за отсутствия спецификации карты каналов . Это основные мотивы для создания нового API отображения каналов.

2.2.2Дизайн

На самом деле, "API сопоставления каналов" не приносит ничего нового с точки зрения ABI ядра / пользовательского пространства . Он использует только существующие функции элемента управления. В качестве наземной конструкции каждый подпоток PCM может содержать элемент управления, обеспечивающий отображение канала информацию и конфигурацию. Этот элемент определяется:

• iface = SNDRV_CTL_ELEM_IFACE_PCM • name = "Карта канала воспроизведения" или "Карта канала захвата" • device = тот же номер устройства для назначенного подпотока PCM • index = тот же номер индекса для назначенного подпотока PCM Обратите внимание, что имя отличается в зависимости от направления подпотока PCM.

Каждый элемент управления обеспечивает, по меньшей мере, операцию чтения TLV и операцию считывания. Опционально может быть предусмотрена операция записи, позволяющая пользователю динамически изменять карту каналов. TLV

Операция TLV выдает список доступных карт каналов. Элемент списка карты каналов обычно представляет собой TLV из `type` `ch 0` , `ch 1` , `ch 2` ... где `type` - это значение типа TLV, второй аргумент - это общее количество байтов (не чисел) значений канала, а остальные - это значения позиции для каждого канала. Как тип TLV, либо `SNDRV_CTL_TLV_T_CHMAP_FIXED`, `SNDRV_CTL_TLV_CHMAP_VAR`

или

`SNDRV_CTL_TLV_T_CHMAP_PARED` может использоваться. The `_FIXED` тип предназначен для карты каналов с фиксированным положением канала, в то время как последние два предназначены для гибкого положения канала. `_VAR` тип предназначен для карты каналов, где все каналы можно свободно менять местами и `_PARED` тип - это когда парные каналы можно менять местами. Например, когда у вас есть карта каналов {FL / FR / RL / RR}, `_PARED` тип позволит вам менять местами только {RL / RR / FL / FR}, в то время как `_VAR` тип позволил бы даже менять местами FL и RR.

Эти новые типы TLV определены в `звук/ tlv.h`.

Доступные значения положения канала определены в разделе `звук/asound.h`,

```
вот фрагмент: / * позиции каналов */
перечисление {
    SNDRV_CHMAP_UNKNOWN
    = 0, SNDRV_CHMAP_NA, /* N /A, беззвучный */
    SNDRV_CHMAP_MONO, /* монофонический поток */

    /* это следует за значением канала микшера alsa-lib +
    3 */ SNDRV_CHMAP_FL, /* спереди слева */
    SNDRV_CHMAP_FR, /* спереди справа */
    SNDRV_CHMAP_RL, /* сзади слева */
    SNDRV_CHMAP_RR, /* сзади справа */
    SNDRV_CHMAP_FC, /* спереди по центру */
    SNDRV_CHMAP_LFE, /* LFE */
    SNDRV_CHMAP_SL, /* сбоку слева */
    SNDRV_CHMAP_SR, /* сбоку справа */
    SNDRV_CHMAP_RC, /* сзади по центру */

    /* новые определения */
    SNDRV_CHMAP_FLC, /* спереди слева по центру */
    SNDRV_CHMAP_FRC, /* спереди справа по центру */
    SNDRV_CHMAP_RL , /* сзади слева по центру */
```



```

    SNDRV_CHMAP_RRC,                /* задний правый центральный */
    SNDRV_CHMAP_FLW,                /* передний левый широкий */
    SNDRV_CHMAP_FRW,                /* передний правый широкий */
    SNDRV_CHMAP_FLH,                /* передний левый высокий */
    SNDRV_CHMAP_FCH,                /* передний центральный высокий
    SNDRV_CHMAP_FRH,                /* */
    SNDRV_CHMAP_TC,                 /* */
    SNDRV_CHMAP_TFL,                /* верхний передний левый */
    SNDRV_CHMAP_TFR,                /* верхний передний правый */
    SNDRV_CHMAP_TFC,                /* верхний передний центральный
    SNDRV_CHMAP_TRL,                /* */
    SNDRV_CHMAP_TRR,                /* */
    SNDRV_CHMAP_TRC,                /* верхний задний центральный */

    SNDRV_CHMAP_LAST = SNDRV_CHMAP_TRC,
};
```

Когда поток PCM может предоставлять более одной карты каналов, вы можете предоставить карты нескольких каналов в контейнерном типе TLV. Возвращаемые

```

данные TLV будут содержать такие данные, как: SNDRV_CTL_TLVT_КОНТЕЙНЕР 9 6

    SNDRV_CTL_TLVT_CHMAP_FIXED 4 SNDRV_CHMAP_FC
    SNDRV_CTL_TLVT_CHMAP_FIXED 8 SNDRV_CHMAP_FL SNDRV_CHMAP_FR
    SNDRV_CTL_TLVT_CHMAP_FIXED 1 6 SNDRV_CHMAP_FL SNDRV_CHMAP_FR \
    SNDRV_CHMAP_RL SNDRV_CHMAP_RR
```

Позиция канала указывается в 16 битах LSB. Старшие биты

```

используются для битовых флагов. # определить

SNDRV_CHMAP_POSITION_MASK 0xffff # определить SNDRV_CHMAP_PHASE_INVERSE (0x01 << 16)

# определить SNDRV_CHMAP_DRIVER_SPEC (0x02 << 16)
```

SNDRV_CHMAP_PHASE_INVERSE указывает, что канал инвертирован по фазе (таким образом, суммирование левого и правого каналов привело бы к почти полной тишине). Это есть в некоторых цифровых микрофонных устройствах. Когда SNDRV_CHMAP_DRIVER_SPEC если параметр установлен, все значения положения канала не соответствуют стандартному определению, приведенному выше, а зависят от драйвера.

Операция чтения

Операция чтения управления предназначена для предоставления текущей карты каналов данного потока.

Элемент control возвращает массив целых чисел, содержащий позицию каждого канала.

Если это выполняется до того, как указан номер канала (т. Е. Установлен параметр hw_params), он должен вернуть все каналы, для которых установлено значение неизвестно.

Операция записи

Операция управляющей записи необязательна и предназначена только для устройств, которые могут изменять конфигурацию канала "на лету", таких как HDMI. Пользователю необходимо передать целое значение, содержащее допустимые позиции каналов для всех каналов назначенного подпотока PCM. Эта операция разрешена только в ПОДГОТОВЛЕННОМ состоянии PCM. При вызове в других состояниях он должен возвращать ошибку.

2.3 API сжатия-разгрузки ALSA

Pierre-Louis.Боссарт <pierre-louis.bossart@linux.intel.com>
Винод Кул <vinod.koul@linux.intel.com>

Обзор 2.3.1

С первых дней своего существования API ALSA определялся с учетом поддержки PCM или полезных нагрузок с постоянными скоростями передачи данных, таких как IEC61937. Аргументы и возвращаемые значения во фреймах являются нормой, что затрудняет расширение существующего API для потоков сжатых данных. В последние годы процессоры цифровых аудиосигналов (DSP) были интегрированы в конструкции "система на кристалле", а DSP также интегрированы в аудиокодеки. Обработка сжатых данных на таких DSP приводит к резкому снижению энергопотребления по сравнению с обработкой на базе хоста. Поддержка такого оборудования в Linux была не очень хорошей, в основном из-за отсутствия универсального API, доступного в основном ядре. Вместо того, чтобы требовать нарушения совместимости с изменением API интерфейса ALSA PCM, представлен новый API Compressed Data, обеспечивающий интерфейс управления и потоковой передачи данных для аудиосистем DSP. Дизайн этого API был вдохновлен 2-летним опытом работы с Intel Moorestown SOC, при этом потребовалось множество исправлений, чтобы внедрить API в основное ядро вместо промежуточного дерева и сделать его доступным для других пользователей.

2.3.2 Требования

Основными требованиями являются:

- разделение количества байтов и времени. Сжатые форматы могут иметь заголовок для каждого файла, для каждого фрейма или вообще не иметь заголовка. Размер полезной нагрузки может варьироваться от кадра к кадру. В результате невозможно достоверно оценить длительность аудиобуферов при обработке сжатых данных. Требуются специальные механизмы для обеспечения надежной синхронизации аудио-видео, что требует точного отчета о количестве сэмплов, отрисованных в любой момент времени.
- Работа с несколькими форматами. Для данных PCM требуется только указание частоты дискретизации, количества каналов и битов на выборку. Напротив, сжатые данные поставляются в различных форматах. Аудио DSP могут также обеспечивать поддержку ограниченного числа аудиокодеров и декодеров, встроенных в встроенное ПО, или могут поддерживать больший выбор за счет динамической загрузки библиотек.
- Сосредоточьтесь на основных форматах. Этот API обеспечивает поддержку наиболее популярных форматов, используемых для аудио и захвата и воспроизведения видео. Вполне вероятно, что по мере развития технологии сжатия звука будут добавляться новые форматы.
- Обработка нескольких конфигураций. Даже для данного формата, такого как AAC, некоторые реализации могут поддерживать многоканальный AAC, но стереосистемы HE-AAC. Аналогично, WMA10 уровня M3 может требовать слишком много циклов памяти и процессора. Новый API должен предоставлять общий способ перечисления этих форматов.
- Только рендеринг / захват. Этот API не предоставляет никаких средств аппаратного ускорения, где образцы PCM передаются обратно в пространство пользователя для дополнительной обработки. Вместо этого этот API фокусируется на потоковой передаче сжатых данных в DSP, предполагая, что декодированные выборки направляются на физический вывод или логический сервер.
- Скрытие сложности. Все существующие мультимедийные фреймворки пользовательского пространства имеют существующие перечисления / структуры для

каждого сжатого формата. Этот новый API предполагает существование уровня совместимости

для конкретной платформы для предоставления, перевода и использования

возможностей аудио DSP, например. Android HAL или приемники PulseAudio. По

конструкции обычные приложения не должны использовать этот API.

2.3.3 Дизайн

Новый API разделяет ряд концепций с PCM API для управления потоком. Команды

Start, pause, resume, drain и stop имеют одинаковую семантику независимо от

содержания. Концепция кольцевого буфера памяти, разделенного на набор

фрагментов, заимствована из API ALSA PCM. Однако могут быть указаны только размеры в

байтах. Предполагается, что режимы поиска / уловки обрабатываются хостом.

Понятие перемотки назад / пересылки вперед не поддерживается. Данные, сохраненные в кольцевом буфере, не могут быть признаны недействительными, за исключением удаления всех буферов.

API сжатых данных не делает никаких предположений о том, как данные передаются в аудио

DSP. Возможна передача данных DMA из основной памяти во встроенный аудиокластер или на интерфейс SPI

для внешних DSP. Как и в случае с ALSA PCM, предоставляется базовый набор подпрограмм; каждый

разработчик драйвера должен будет написать поддержку для набора обязательных подпрограмм и, возможно,

использовать необязательные. Основными дополнениями являются

get_caps Эта процедура возвращает список поддерживаемых аудиоформатов. Запрашивает кодеки при записи.

stream вернет кодеки, декодеры будут перечислены для потоков воспроизведения.

get_codec_caps Для каждого кодека эта процедура возвращает список возможностей. Цель состоит в том, чтобы убедиться, что

все возможности соответствуют действительным настройкам и сводят к минимуму риски сбоев конфигурации.

Например, для сложного кодека, такого как AAC, количество поддерживаемых каналов может зависеть от конкретного

профиля. Если возможности были предоставлены с помощью одного дескриптора, может случиться так, что определенная

комбинация профилей / каналов / форматов может не поддерживаться. Аналогично, встроенные DSP имеют

ограниченные циклы памяти и процессора, вероятно, что некоторые реализации делают список возможностей динамичным

и зависящим от существующих рабочих нагрузок. В дополнение к настройкам кодека, эта процедура возвращает

минимальный размер буфера, обрабатываемый реализацией. Эта информация может зависеть от размеров буфера

DMA, количества байт, необходимых для синхронизации, и т.д. И может использоваться пользовательским пространством

для определения объема, который необходимо записать в кольцевой буфер перед началом воспроизведения.

set_params Эта процедура устанавливает конфигурацию, выбранную для конкретного кодека. Наиболее важное поле в

параметрами является тип кодека; в большинстве случаев декодеры будут игнорировать другие поля, в то время как

кодировщики будут строго соответствовать настройкам

get_params Эта процедура возвращает фактические настройки, используемые DSP. Изменения в настройках должны

оставаться исключением.

get_timestamp Временная метка становится структурой с несколькими полями. Он отображает количество байтов, транс-

вिलегированных, количество обработанных сэмплов и количество визуализированных / захваченных сэмплов.

Все эти значения могут быть использованы для определения среднего битрейта, определения необходимости

повторного заполнения кольцевого буфера или задержки из-за декодирования / encoding / io в DSP. Обратите

внимание, что список кодеков / профилей / режимов был получен из спецификации OpenMAX AL, а не из того,

что позволило повернуть колесо вспять. Модификации включают: - Добавление форматов FLAC и IEC -

Объединение возможностей кодера / декодера - Профили / режимы, перечисленные в виде битовых масок, чтобы

сделать дескрипторы более компактными - Добавление set_params для декодеров (отсутствует в OpenMAX AL) -

Добавление режимов кодирования AMR / AMR-WB (отсутствует в OpenMAX AL) - Добавление информации о

формате для WMA - Добавление параметров кодирования при необходимости (производных от OpenMAX IL) -

Добавление rateControlSupported (отсутствует в OpenMAX AL) **2.3.4 Воспроизведение без пауз**

При воспроизведении альбома декодеры имеют возможность пропускать задержку кодировщика и отступы и

напрямую переходить от содержимого одной дорожки к другой. Конечный пользователь может воспринять это

как воспроизведение без задержек, поскольку у нас нет тишины при переключении с одной дорожки на другую

Кроме того, могут быть шумы низкой интенсивности из-за кодирования. Идеальный режим без пробелов

труднодостижим для всех типов сжатых данных, но отлично работает с большинством музыкального контента.

Декодеру необходимо знать задержку кодера и заполнение кодера. Итак, нам нужно передать это в DSP. Эти

метаданные извлекаются из заголовков ID3 / MP4 и по умолчанию не присутствуют в потоке битов, следовательно,

необходим новый интерфейс для передачи этой информации в DSP. Также DSP и пользовательскому пространству

необходимо переключиться с одной дорожки на другую и начать использовать данные для второй дорожки.

Основными дополнениями являются:

Основными дополнениями являются:

set_metadata Эта процедура устанавливает задержку кодера и заполнение кодера. Это может использоваться декодером

для отключения тишины. Это необходимо установить перед записью данных в дорожке.

set_next_track Эта процедура сообщает DSP, что метаданные и операция записи, отправленные после этого, будут соответствовать

последующему треку

частичная утечка Это вызывается при достижении конца файла. Пользовательское пространство может сообщить DSP, что EOF достигнуто

и теперь DSP может начать пропускать задержку заполнения. Также следующие данные записи будут принадлежать следующему треку, Поток последовательности для garless был бы следующим: - Открыть - Получить заглавные буквы / кодеки - Установить параметры - Установить метаданные первого трека - Заполнить данные первого трека - Запустить триггер - Пользовательское пространство завершило отправку всех, - Указать данные следующего трека отправкой set_next_track - Установить метаданные следующего трека - затем вызвать partial_drain для очистки большей части буфера в DSP - Заполнить данные следующего трека - DSP переключается на второй трек (примечание: заказ partial_drain и запись для следующего трека могут быть изменены). тоже в обратном порядке) **2.3.5 Не поддерживается**

- Поддержка вызовов VoIP / с коммутацией каналов не является целью этого API. Поддержка динамической скорости передачи данных.

изменения потребовали бы тесной связи между DSP и стеком хостов, что ограничило бы экономию энергии . • Скрытие потери пакетов не поддерживается.

Для этого потребуется дополнительный интерфейс, позволяющий de-

кодеру синтезировать данные, когда кадры теряются во время передачи. Это может быть добавлено в будущем. • Управление громкостью / маршрутизация не обрабатываются этим API. Устройства, использующие интерфейс со сжатыми данными, будут

будут рассматриваться как обычные устройства ALSA; изменения громкости и информация о маршруте

будут предоставляться обычными контроллерами ALSA kcontrols. • Встроенные звуковые эффекты.

Такие эффекты должны быть активированы одинаковым образом, независимо от того, был ли входной сигнал PCM или сжатым.

- многоканальное кодирование IEC. Неясно, требуется ли это.

- Ускорение кодирования / декодирования не поддерживается, как упоминалось выше. Можно направить вывод

декодера в поток записи или даже реализовать возможности перекодирования. Эта

маршрутизация была бы включена с помощью ALSA kcontrols. • Политика аудио / управление

ресурсами. Этот API не предоставляет никаких перехватов для запроса использования

аудио DSP, ни каких-либо механизмов вытеснения.

- Нет понятия недозагрузки / переполнения. Поскольку записываемые байты по своей природе сжаты, а данные записываются-

ten / read не преобразуется напрямую в отрисованный результат во времени, это не связано с

недостаточным запуском / переполнением и, возможно, обрабатывается в пользовательской

библиотеке **2.3.6 Кредитов**

- Марку Брауну и Лиаму Гирдвуду за обсуждение необходимости этого

API • Харше Прие за ее работу над сжатым API intel_sst • Ракешу

Угредже за ценный отзыв • Сингу Налласеллану, Сиккандару Мадару

и Прасанне Самаге за демонстрацию и количественную оценку бен-

эффективность разгрузки звука на реальной платформе.

Временная метка 2.4 ALSA PCM

API ALSA может предоставлять две разные системные временные метки:

- Trigger_timestamp - это снимок системного времени, сделанный при вызове обратного вызова .trigger. Эта привязка-

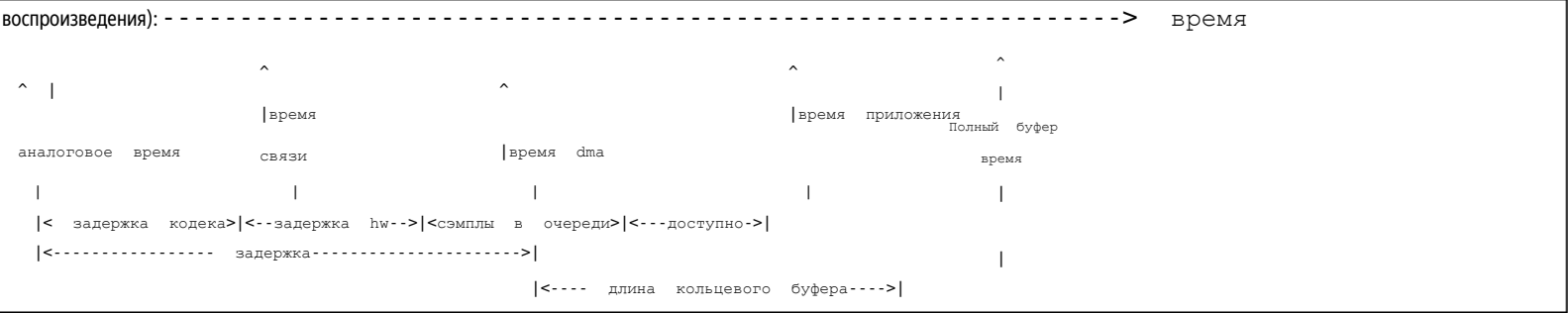
в общем случае снимок выполняется ядром ALSA, но конкретное аппаратное обеспечение может иметь возможности синхронизации или, наоборот, может выдавать правильную оценку только с задержкой. В последних двух случаях низкоуровневый драйвер отвечает за обновление trigger_timestamp в наиболее подходящий и точный момент. Приложениям не следует полагаться исключительно на первый параметр trigger_timestamp, но обновлять свои внутренние вычисления, если драйвер предоставляет уточненную оценку с задержкой.

- `tstamp` - это текущая системная временная метка, обновленная во время последнего события или запроса приложения. The разница (`tstamp - trigger_tstamp`) определяет прошедшее время.

API ALSA предоставляет две основные части информации: доступность и задержка, которые в сочетании с триггером и текущими системными временными метками позволяют приложениям отслеживать "заполненность" кольцевого буфера и количество выборок, поставленных в очередь. Использование этих различных указателей и информации о времени зависит от потребностей приложения:

- `доступно` сообщает, сколько данных может быть записано в кольцевой буфер
- `задержка` сообщает время, необходимое для прослушивания нового сэмпла после воспроизведения всех сэмплов, поставленных в очередь
- `длина буфера` сообщает длину кольцевого буфера

Когда временные метки включены, отображается информация о доступности / задержке вместе со снимком системного времени. Приложения могут выбирать из `CLOCK_REALTIME` (коррекция NTP, включая возврат назад), `CLOCK_MONOTONIC` (коррекция NTP, но никогда не возврат назад), `CLOCK_MONOTIC_RAW` (без исправлений NTP) и динамического изменения режима с помощью `sw_params` API ALSA также предоставляет `audio_tstamp`, который отражает течение времени, измеряемое различными компонентами аудиооборудования. В формате `ascii` это может быть представлено следующим образом (для случая



Аналоговое время отсчитывается на последнем этапе воспроизведения, как можно ближе к фактическому преобразователю. Время соединения отсчитывается на выходе SoC/чипсета по мере того, как сэмплы передаются по каналу связи. Время соединения может быть измерено напрямую, если поддерживается аппаратно, с помощью счетчиков выборок или настенных блокировок (например, с HDAudio 24 МГц или тактовой частотой PTP для сетевых решений) или оценено косвенно (например, с помощью счетчика кадров в USB). Время DMA измеряется с помощью счетчиков - обычно это наименее надежное из всех измерений из-за периодичности передачи данных DMA. Время приложения соответствует времени, отслеживаемому приложением после записи в кольцевой буфер. Приложение может запросить аппаратные возможности, определить, какое время воспроизведения звука оно хочет сообщить, выбрав соответствующие настройки в полях `audio_tstamp_config`, таким образом, получить оценку точности временной метки. Он также может запросить включение в измерение задержки преобразования в аналог. Прямой доступ к времени соединения очень интересен на платформах, которые предоставляют встроенный DSP; прямое измерение времени соединения с помощью выделенного оборудования, возможно синхронизированного с системным временем, устраняет необходимость отслеживать внутреннее время обработки DSP и задержку. В случае, если приложение запрашивает временную метку звука, которая не поддерживается аппаратным обеспечением / низкоуровневым драйвером, тип переопределяется по УМОЛЧАНИЮ, и временная метка будет сообщать время DMA на основе значения `hw_pointer`. Для обеспечения обратной совместимости с предыдущими реализациями, которые не предоставляли выбор временной метки, с типом временной метки COMPAT с нулевым значением результаты по умолчанию будут соответствовать настенным часам HDAudio для воспроизведения потоков и времени DMA (`hw_ptr`) во всех остальных случаях. Точность временной метки звука может быть возвращена в пространство пользователя, чтобы принимались соответствующие решения:

- для времени dma (по умолчанию) детализацию передач можно определить по шагам между ир-даты и, в свою очередь, предоставляют информацию о том, насколько безопасно можно перемотать указатель приложения назад. • время перехода можно использовать для отслеживания долгосрочных переходов между звуковым и системным временем с помощью (`tstamp-trigger_tstamp`) / `audio_tstamp`, точность помогает определить, насколько сильно требуется сглаживание / настройка фильтра нижних частот. Время соединения может быть либо сброшено при запуске, либо указано как есть (последнее полезно для сравнения прогресса разных потоков, но может потребоваться, чтобы `wallclock` был всегда запущен и

не переносится в периоды простоя). Если поддерживается аппаратно, абсолютное время соединения также может быть использовано для определения точного времени начала (исправления WIP) - включение задержки в временную метку аудио может нелогично не увеличивать точность времени-сбои, например, если кодек включает DSP-обработку с переменной задержкой или цепочку аппаратных компонентов задержка обычно точно неизвестна.

Точность указывается в наносекундных единицах (с использованием 32-битного слова без знака), что дает максимальную точность 4,29 с, чего более чем достаточно для аудиоприложений ...

Из-за различного характера потребностей в временных метках, даже для одного приложения, audio_timestamp_config может быть изменен динамически. В Статус ioctl, параметры доступны только для чтения и не позволяют выбирать какое-либо приложение. Чтобы обойти это ограничение, не затрагивая устаревшие приложения, новый STATUS_EXT ioctl представлен с параметрами чтения / записи. ALSA-lib будет изменен для использования STATUS_EXT и фактически устареет. Статус.

API ALSA позволяет сообщать только об одной временной метке звука одновременно. Это сознательное проектное решение, считывание временных меток звука из аппаратных регистров или из IPC требует времени, чем больше временных меток считывается, тем более неточными получаются объединенные измерения. Чтобы избежать каких-либо проблем с интерпретацией, указывается единственная временная метка (системная, звуковая). Приложениям, которым требуются разные временные метки, потребуется выдавать несколько запросов и выполнять интерполяцию результатов

В некоторой конфигурации, зависящей от оборудования, системная временная метка фиксируется аудиоподсистемой низкого уровня, и информация передается обратно в драйвер. Из-за потенциальных задержек в обмене данными с оборудованием существует риск несоответствия информации о доступности и задержке. Чтобы убедиться, что приложения не перепутаются, в структуру snd_pcm_status добавлено поле driver_timestamp; эта временная метка показывает, когда драйвер собирает информацию перед возвратом из Статус и STATUS_EXT ioctl.

в большинстве случаев эта метка driver_timestamp будет идентична обычной системной tstamp.

Примеры меток типов с HDAudio:

1. Временная метка DMA, без компенсации DMA + аналоговой задержки

\$./audio_time	
воспроизведение: системное время: -p --ts_type=1	1 1 2 1 3 3 8 нсек, время звучания 3 4 2 0 0 0 0 0 нсек, системное время дельта - 8 7 8 6 6 2
воспроизведение: системное время: 4 2 6 2 3 6 6 3	нсек, время звучания 4 2 7 1 8 7 5 0 0 нсек, системное время дельта - 9 5 0 8 3 7
воспроизведение: системное время: 5 9 7 0 8 0 5 8 0	нсек, время звучания 5 9 8 0 0 0 0 0 0 нсек, системное время дельта - 9 1 9 4 2 0
воспроизведение: системное время: 6 8 2 0 5 9 7 8 2	нсек, время звучания 6 8 3 0 2 0 8 3 3 нсек, системное время дельта - 9 6 1 0 5 1
воспроизведение: системное время: 8 5 2 8 9 6 4 1 5	нсек, время звучания 8 5 3 8 5 4 1 6 6 нсек, разница во времени системы - 9 5 7 7 5 1
воспроизведение: системное время: 9 3 7 9 0 3 3 4 4	нсек, время звучания 9 3 8 8 5 4 1 6 6 нсек, разница во времени системы - 9 5 0 8 2 2

2. Временная метка DMA, компенсация DMA + аналоговая задержка

\$./ audio_time	
воспроизведение: system: -p --ts_type=2	3 4 1 0 6 2 5 0 0 нсек, время звучания 3 4 1 0 6 2 5 0 0 нсек, системное время дельта - 9 1 5 3
воспроизведение: системное время: 4 2 6 0 7 2 4 4 7	нсек, время звучания 4 2 6 0 6 2 5 0 0 нсек, системное время дельта 9 9 4 7
воспроизведение: системное время: 5 9 6 8 9 9 5 1 8	нсек, время звучания 5 9 6 8 9 5 8 3 3 нсек, системное время дельта 3 6 8 5
воспроизведение: системное время: 6 8 1 9 1 5 3 1 7	нсек, время звучания 6 8 1 9 1 6 6 6 6 нсек, системное время дельта - 1 3 4 9
воспроизведение: системное время: 8 5 2 7 4 1 3 0 6	нсек, время звучания 8 5 2 7 5 0 0 0 0 нсек, системное время delta - 8 6 9 4

3. временная метка канала связи, компенсация DMA + аналоговой задержки

\$./audio_time -p --ts_type = 2 -d	
воспроизведение: системное время: 3 4 1 0 6 0 0 0 4	нсек, время звучания 3 4 1 0 6 2 7 9 1 нсек, системное время дельта - 2 7 8 7
воспроизведение: системное время: 4 2 6 2 4 2 0 7 4	нсек, время звучания 4 2 6 2 4 4 8 7 5 нсек, системное время дельта - 2 8 0 1
воспроизведение: системное время: 5 9 7 0 8 0 9 9 2	нсек, время звучания 5 9 7 0 8 4 5 8 3 нсек, системное время дельта - 3 5 9 1
воспроизведение: системное время: 6 8 2 0 8 4 5 1 2	нсек, время звучания 6 8 2 0 8 8 2 9 1 нсек, системное время разности - 3 7 7 9
воспроизведение: системное время: 8 5 2 9 3 6 2 2 9	нсек, время звучания 8 5 2 9 4 0 9 1 6 нсек, системное время разности - 4 6 8 7
воспроизведение: системное время: 9 3 8 1 0 7 5 6 2	нсек, время звучания 9 3 8 1 1 2 7 0 8 нсек, системное время разности - 5 1 4 6

Пример 1 показывает, что временная метка на уровне DMA примерно на 1 мс опережает фактическое время воспроизведения (в качестве дополнительного времени такого рода измерения могут помочь определить меры предосторожности при перемотке назад). Компенсация задержки канала DMA- в примере 2 помогает устранить аппаратную буферизацию, но информация по-прежнему очень неустойчивая, поскольку

до одного примера ошибки. В примере 3, где временные метки измеряются с помощью настенных часов link, временные метки демонстрируют монотонное поведение и меньшую дисперсию.

Примеры 3 и 4 относятся к классу USB audio. В примере 3 показано большое смещение между временем воспроизведения звука и системным временем из-за буферизации. Пример 4 показывает, как компенсация задержки обеспечивает точность в 1 мс (из-за использования драйвером счетчика кадров)

Пример 3: временная метка DMA, без компенсации задержки, дельта ~ 5 мс

```
$ ./audio_time -p -fbc: 1 -t 1 воспроизведение: системное время: 1 2 0 1 7 4 0 1 9 нсек, время звучания 1 2 5 0 0 0 0 0
нсек, системное время дельта - 4 8 2 5 9 8 1 воспроизведение: системное время: 2 4 5 0 4 1 1 3 6 нсек, время
звучания 2 5 0 0 0 0 0 0 нсек, системное время дельта - 4 9 5 8 8 6 4 воспроизведение: системное время: 3 7 0 1 0 6 0 8 8
нсек, время звучания 3 7 5 0 0 0 0 нсек, системное время дельта - 4 8 9 3 9 1 2 воспроизведение: системное
время: 4 9 5 0 4 0 0 6 5 нсек, время звучания 5 0 0 0 0 0 0 0 нсек, системное время delta - 4 9 5 9 9 3 5 воспроизведение:
системное время: 6 2 0 0 3 8 1 7 9 нсек, время звучания 6 2 5 0 0 0 0 0 нсек, системное время delta - 4 9 6 1 8 2 1
воспроизведение: системное время: 7 4 5 0 8 7 7 4 1 нсек, время звучания 7 5 0 0 0 0 0 0 нсек, системное время
delta - 4 9 1 2 2 5 9 воспроизведение: системное время: 8 7 0 0 3 7 3 3 6 нсек, время звучания 8 7 5 0 0 0 0 0 нсек,
```

Пример 4: Временная метка DMA, компенсация задержки, задержка ~1 мс

```
$ ./audio_time -p -fbc: 1 -t 1 -d воспроизведение: системное время: 1 2 0 1 9 0 5 2 0 нсек, время
звучания 1 2 0 0 0 0 0 0 нсек, системная разница 1 9 0 5 2 0 воспроизведение: системное время: 2 4 5 0 3 6 7 4 0 нсек,
время звучания 2 4 4 0 0 0 0 0 нсек, системная разница 1 0 3 6 7 4 0 воспроизведение: системное время: 3 7 0 0 3 4 0 8 1
нсек, время звучания 3 6 9 0 0 0 0 0 нсек, системная разница 1 0 3 4 0 8 1 воспроизведение: системное время:
4 9 5 1 5 9 9 0 7 нсек, время звучания 4 9 4 0 0 0 0 0 нсек, системное время 1 1 5 9 9 0 7 воспроизведение: системное
время: 6 2 0 0 9 8 8 2 4 нсек, время звучания 6 1 9 0 0 0 0 0 нсек, системное время: 1 0 9 8 8 2 4 воспроизведение:
системное время: 7 4 5 0 3 1 8 4 7 нсек, время звучания: 7 4 4 0 0 0 0 0 нсек, systime delta 1 0 3 1 8 4 7
```

2.5 Элементы управления ALSA Jack

2.5.1 Зачем нам нужны Jack kcontrols

Этот раздел описывает, как ALSA Jack интегрируется с пользовательским пространством. Это означает, что ALSA Jack предоставляет набор элементов управления звуком (переключение, громкость, мультиплексирование, ...) в пространство пользователя. приложения для пользовательского пространства, такие как pulseaudio, могут отключать наушники и включать динамики, когда не подключены головные телефоны. Старый код ALSA jack создавал устройства ввода только для каждого зарегистрированного разъема. Эти устройства ввода jack не читаются устройствами пользовательского пространства, которые работают не от имени root. Новый код jack создает встроенные kcontrols jack для каждого разъема, которые могут быть прочитаны любым процессом. Это может быть объединено с UCM, чтобы позволить пользовательскому пространству более разумно маршрутизировать звук на основе событий вставки разъема или удаления.

2.5.2 Внутренние компоненты Jack Kcontrol.

У каждого гнезда будет список kcontrol, так что мы можем создать kcontrol и прикрепить его к гнезду на этапе создания гнезда . Мы также можем добавить kcontrol к существующему разъему в любое время, когда потребуется. Эти kcontrol будут освобождены автоматически при освобождении гнезда.

2.5.3 Как использовать jack kcontrols

Чтобы сохранить совместимость, snd_jack_new() был изменен путем добавления двух параметров: initial_kctl если true, создайте kcontrol и добавьте его в список разъемов. phantom_jack Не создавайте устройство ввода для фантомных разъемов.

Гнезда HDA могут установить phantom_jack в значение true, чтобы создать фантомный разъем, и установить initial_kctl в значение true, чтобы создать начальный kcontrol с правильным идентификатором. Разъемы ASoC должны устанавливать initial_kctl равным false. Имя pin-кода будет присвоено в качестве имени разъема kcontrol.

2.6 Точек трассировки в ALSA

2017/07/02 Такасахи Сакамото

2.6.1 Точки трассировки в ядре ALSA PCM

Эта подсистема включает регистры ядра ALSA PCM `snd_pcm` для системы трассировки ядра. две категории точек трассировки; для состояния буфера PCM и для обработки аппаратных параметров PCM. Эти точки трассировки доступны, когда включены соответствующие конфигурации ядра. Когда включен параметр `CONV-FIG_SND_DEBUG`, доступны последние точки трассировки. При дополнительных `SND_PCM_XRUN_DEBUG` также включен, прежние точки трассировки включены.

Точки трассировки для состояния буфера PCM

В эту категорию входят четыре точки трассировки; `hwptr`, `applptr`, `xrun` и `hw_ptr_error`.

Точки трассировки для обработки аппаратных параметров PCM

В эту категорию входят две точки трассировки; `hw_mask_param` и `hw_interval_param`.

В конструкции ядра ALSA PCM передача данных абстрагируется как подпоток PCM. Приложения управляют подпоток PCM для поддержки передачи данных для кадров PCM. Перед началом передачи данных, приложениям необходимо настроить подпоток PCM. В этой процедуре аппаратные параметры PCM определяются взаимодействием между приложениями и ядром ALSA PCM core. После принятия решения среда выполнения подпотока PCM сохраняет параметры. Параметры описаны в разделе `struct snd_pcm_hw_params`. Эта структура включает в себя несколько типов параметров. Приложения устанавливают предпочтительное значение для этих параметров, затем выполняют `ioctl(2)` с `SNDRV_PCM_IOCTL_HW_REFINE` или `SNDRV_PCM_IOCTL_HW_PARAMS`. Первый используется только для уточнения доступного набора параметров. Последний используется для фактического выбора параметров.

В `struct snd_pcm_hw_params` структура имеет следующие элементы:

- флаги** Настраиваемые. Ядро ALSA PCM и некоторые драйверы используют этот флаг для выбора удобных параметров или изменения их поведения.
- маски** Настраиваемые. Этот тип параметров описан в `структурируйте snd_mask` и представляйте значения маски. Начиная с PCM protocol версии v2.0.13, определены три типа.

- `SNDRV_PCM_HW_PARAM_ACCESS`
- `SNDRV_PCM_HW_PARAM_FORMAT`
- `SNDRV_PCM_HW_PARAM_SUBFORMAT`

интервалы Настраиваемые. Этот тип параметра описан в разделе `struct snd_interval` и представляет собой значения с диапазоном. Начиная с PCM protocol v2.0.13, определено двенадцать типов.

- `SNDRV_PCM_HW_PARAM_SAMPLE_BITS`
- `SNDRV_PCM_HW_PARAM_FRAME_BITS`
- `SNDRV_PCM_HW_PARAM_CHANNELS`
- `SNDRV_PCM_HW_PARAM_RATE`
- `SNDRV_PCM_HW_PARAM_PERIOD_TIME`

- SNDRV_PCM_HW_PARAM_PERIOD_SIZE
- SNDRV_PCM_HW_PARAM_PERIOD_BYTES
- SNDRV_PCM_HW_PARAM_PERIODS
- SNDRV_PCM_HW_PARAM_BUFFER_TIME
- SNDRV_PCM_HW_PARAM_BUFFER_SIZE
- SNDRV_PCM_HW_PARAM_BUFFER_BYTES
- SNDRV_PCM_HW_PARAM_TICK_TIME

<code>mask</code>	Настраивается.	Это вычисляется в <code>ioctl(2)</code> только с помощью <code>SNDRV_PCM_IOCTL_HW_REFINE</code> .	Примените-
			Для катионов можно выбра

маски / интервала может быть изменен ядром ALSA PCM core. `SNDRV_PCM_IOCTL_HW_PARAMS`, эта маска игнорируется, и все параметры будут изменены.

`mask` Доступен только для чтения. После возврата из `ioctl(2)` создайте буфер в пользовательском пространстве для `struct snd_pcm_hw_params` и содержит результат каждой операции. Эта маска показывает, какой параметр маски / интервала фактически изменен.

Информация Доступен только для чтения. Это представляет возможности оборудования / драйвера в виде битовых флагов с помощью `SNDRV_PCM_INFO_XXX`.

Обычно приложения выполняют `ioctl(2)` с `SNDRV_PCM_IOCTL_HW_REFINE` для получения этого флага, затем определяют возможные параметры и выполняют `ioctl(2)` с `SNDRV_PCM_IOCTL_HW_PARAMS` для получения подсчета PCM-потока.

биты Доступно только для чтения. Это значение представляет доступную разрядность на стороне MSB выборки PCM. Когда параметр `SNDRV_PCM_HW_PARAM_SAMPLE_BITS` был определен как фиксированное число, это значение также вычисляется в соответствии с ним. В противном случае - ноль. Но это поведение зависит от реализаций на стороне драйвера.

rate_num Доступно только для чтения. Это значение представляет числитель частоты дискретизации в виде дробей. В основном, когда параметр `SNDRV_PCM_HW_PARAM_RATE` был определен как единственное значение, это значение также вычисляется в соответствии с ним. В противном случае - ноль. Но это поведение зависит от реализаций на стороне драйвера.

rate_den Доступно только для чтения. Это значение представляет знаменатель частоты дискретизации в виде дробей. В основном, когда параметр `SNDRV_PCM_HW_PARAM_RATE` был определен как единственное значение, это значение также вычисляется в соответствии с ним. В противном случае - ноль. Но это поведение зависит от реализаций на стороне драйвера.

fifo_size Доступно только для чтения. Это значение представляет размер FIFO в последовательном звуковом интерфейсе аппаратного обеспечения. Басically, каждый водитель может присваивает соответствующее значение для этого параметра, но некоторые водители намеренно устанавливают ноль при помощи аппаратного дизайна или протокол передачи данных.

Ядро ALSA PCM обрабатывает буфер для `struct snd_pcm_hw_params` когда приложения выполняют `ioctl(2)` с параметрами `SNDRV_PCM_HW_REFINE` или `SNDRV_PCM_HW_PARAMS`. Параметры в буфере изменяются в соответствии с структурой `snd_pcm_hardware` и правилами ограничений во время выполнения. Структура описывает возможности обрабатываемого оборудования. Правила описывают зависимости, на основе которых определяется параметр в соответствии с несколькими параметрами. Правило имеет функцию обратного вызова, и драйверы могут регистрировать произвольные функции для вычисления целевого параметра. Ядро ALSA PCM по умолчанию регистрирует некоторые правила во время выполнения. Каждый драйвер может присоединиться к взаимодействию, если он подготовлен для двух компонентов при обратном вызове `struct`

`snd_pcm_ops`. открыть.

1. Ожидается, что при обратном вызове драйверы изменят элемент `struct snd_pcm_hardware` введете время выполнения в соответствии с возможностями соответствующего оборудования.
2. Ожидается, что при том же обратном вызове драйверы также зарегистрируют дополнительные правила ограничений в среде выполнения. когда несколько параметров имеют зависимости из-за конструкции оборудования.

Драйвер может ссылаться на результат взаимодействия в обратном вызове `struct snd_pcm_ops.hw_params`, однако это не должно изменять содержимое. Точки трассировки в этой категории предназначены для отслеживания изменений параметров маски / интервала. Когда ALSA PCM core изменяет их, `hw_mask_param` или `hw_interval_param` событие проверяется в соответствии с типом измененного параметра.

Ядро ALSA PCM также имеет красивый формат печати для каждой из точек трассировки. Ниже приведен пример для `hw_mask_param`.

2.7.3 Файлы, относящиеся к конкретной карте

Файлы, относящиеся к конкретной карте, находятся в каталогах `/proc/asound/card *`. Некоторые драйверы (например, `stirpci`) имеют свои собственные записи `proc` для дампа реестра и т.д. (например `/proc/asound/card */cmipci` показывает дамп регистра). Эти файлы были бы действительно полезны для отладки. Когда устройства PCM доступны на этой карте, вы можете просмотреть каталоги, такие как `pcm0p` или `pcm1c`. Они содержат информацию PCM для каждого потока PCM. Номер после `pcm` - номер устройства PCM, начинающийся с 0, и последний `p` или `c` означает направление воспроизведения или захвата. Файлы в этом поддереве описаны позже. Состояние ввода-вывода MIDI находится в разделе `midid *`. Файлы. Он показывает имя устройства и принятые / переданные байты через MIDI-устройство. Если карта оснащена кодеками AC97, то существуют `codec 9 7 # *` подкаталоги (описаны позже). Когда эмуляция OSS mixer включена (и модуль загружен), файл `oss_mixer` также появляется здесь. Здесь показано текущее сопоставление элементов микшера OSS с элементами управления ALSA. Вы можете изменить сопоставление, выполнив запись на это устройство. Прочитайте `OSS-Emulation.txt` для получения подробной информации.

2.7.4 Файлы PCM Proc.

`card */pcm */info` Общая информация об этом устройстве PCM: номер карты, номер устройства, подпотоки и т.д.

`card */pcm */xrun_debug` Этот файл появляется, когда `CONFIG_SND_DEBUG=y` и `CONFIG_PCM_XRUN_DEBUG=y`.

Это показывает состояние `xrun` (= переполнение буфера / `xrun`) и недопустимую отладку / проверку

положения PCM в ALSA на среднем уровне PCM. Он принимает целочисленное значение, которое

может быть изменено путем записи в этот файл, например: `# echo 5 >`

`/proc/asound/card 0 /pcm 0 p/xrun_debug` Значение состоит из следующих битовых флагов:

- бит 0 = Включить сообщения отладки XRUN / jiffies •
- бит 1 = Показывать трассировку стека при проверке XRUN /
- jiffies • бит 2 = Включить дополнительную проверку jiffies

Когда установлен бит 0, драйвер будет отображать сообщения в журнале ядра при обнаружении `xrun`. Сообщение отладки отображается также при обнаружении недопустимого указателя H / W при обновлении периодов (обычно вызывается из обработчика прерываний). Когда установлен бит 1, драйвер дополнительно отобразит трассировку стека. Это может помочь в отладке. Начиная с версии 2.6.30, эта опция может включать проверку `hwptr` с помощью `jiffies`. Это обнаруживает спонтанные недопустимые значения обратного вызова указателя, но может привести к слишком большому количеству исправлений для (в основном глючащего) оборудования, которое не обеспечивает плавного обновления указателя. Эта функция активируется с помощью бита 2.

`карта */pcm */sub */info` Общая информация этого подпотока PCM.

`карта */pcm */sub */status` Текущее состояние этого подпотока PCM, прошедшее время,

положение H / W и т.д. `card */pcm */sub */hw_params` Аппаратные параметры, установленные для этого

подпотока. `card */pcm */sub */sw_params` Программные параметры, установленные для этого

подпотока. `card */pcm */sub */prealloc` Информация о предварительном распределении буфера.

`card */pcm */sub */xrun_injection` Вызывает XRUN на набегающего потока при любом значении пишется этот файл тез.Докл. Применяется для инъекций вина. Эта запись только для записи.

2.7.5 информацию кодек AC97

`карта */codec 9 7 # */ac 9 7 # ?-?` Отображает общую информацию об этом чипе кодека AC97, такую как название, са-возможности, настройка.

`карта * /codec 9 7 # 0 /ac 9 7 # ?-?+regs` Показывает дамп регистра AC97. Полезно для отладки.

Когда включена функция CONFIG_SND_DEBUG, вы можете напрямую выполнять запись в этот файл для изменения регистра AC97. Передайте два шестнадцатеричных

числа. Например, `# echo 0 2 9 f 1 f > /proc/asound/card 0 /codec 9 7 # 0 /ac 9 7 # 0 - 0 +regs`

2.7.6 Аудиопотоки по USB

`карта * /поток *` Показывает назначение и текущее состояние каждого аудиопотока данной карты. Эта информация очень полезна для отладки.

2.7.7 HD-аудиокодеки

`карта * /кодек # *` Показывает общую информацию о кодеке и атрибуте каждого узла виджета.

`карта * / поле # *` Доступно для интерфейсов HDMI или DisplayPort. Показывает информацию о поле (данные, подобные EDID), полученную из подключенный приемник HDMI и описывает его звуковые возможности и конфигурации.

Некоторые поля ПОЛЯ можно изменить, выполнив команду `шестнадцатеричное значение имени эхо-сигнала > поле # * .`

Делайте это только в том случае, если вы уверены, что указанное значение приемника HDMI неверно. И если из-за этого ваш звук с HDMI работает, пожалуйста, сообщите нам, чтобы мы могли исправить это в будущих выпусках ядра.

Информация о секвенсоре 2.7.8

последовательность / драйверы Выводит список доступных в данный момент драйверов ALSA sequencer.

seq / clients Показывает список доступных в данный момент клиентов и портов sequencer. Состояние подключения и запущенный статус также показаны в этом файле.

последовательность / очереди Перечисляет выделенные в данный момент / запущенные очереди секвенсора.

последовательность / таймер Перечисляет выделенные в данный момент / запущенные таймеры секвенсора.

последовательность / операционные системы Содержит список совместимых с OSS компонентов секвенсора.

2.7.9 Справка по отладке?

Если проблема связана с PCM, сначала попробуйте включить режим `xrun_debug`. Это выдаст вам сообщения ядра, когда и где произошел `xrun`. Если это действительно проблемы, отправьте сообщение со следующей информацией:

- имя водителя/карта, показать в `в /proc/asound/карты`
- дампы реестра, если таковой имеется (например, `card * / смipci`)

если это проблема с PCM,

- настройка PCM, показанная в `hw_params`, `sw_params` и статусе в каталоге подпотока PCM, когда это проблема с микшером,

- Файлы AC97 `proc, codec 9 7 # * / *` файлы

для USB audio / midi,

- вывод `lsusb -v`
- `поток *` файлы в каталоге карты

Система отслеживания ошибок ALSA находится по адресу: <https://bugtrack.alsa-project.org/alsa-bug/>

2.8 Примечания по режиму энергосбережения

Драйверы AC97 и HD-audio имеют автоматический режим энергосбережения. Эта функция включена через Kconfig `CONFIG_SND_AC97_POWER_SAVE` и `CONFIG_SND_HDA_POWER_SAVE` параметры, соответственно. При автоматическом энергосбережении драйвер соответствующим образом отключает питание кодека, когда никаких операций не требуется. Если ни одно приложение не использует устройство и / или аналоговая обратная связь не установлена, отключение питания выполняется полностью или частично. Это позволит снизить определенное энергопотребление, что хорошо для ноутбуков (даже для настольных компьютеров). Время ожидания для автоматического отключения питания можно указать через `power_save` опция модуля для модулей `snd-ac97-codec` и `snd-hda-intel`. Укажите значение тайм-аута в секундах. 0 означает отключение автоматического энергосбережения. Значение тайм-аута по умолчанию задается через `CONFIG_SND_AC97_POWER_SAVE_DEFAULT` и `CONFIG_SND_HDA_POWER_SAVE_DEFAULT` Параметры Kconfig. Установка этого значения в 1 (минимальное значение) - это не отдых-рекомендуемые уровни, потому что многие приложения пытаются открыть устройство часто. 10 было бы хорошим выбором для обычных операций.

Параметр `power_save` экспортируется как доступный для записи. Это означает, что вы можете настроить значение через sysfs "на лету". Например, чтобы включить режим автоматического энергосбережения на 10 секунд, напишите в `/sys/модули/snd_ac97_codec/параметры/power_save` (обычно от имени пользователя root):

```
# echo 10 > /sys/модули/snd_ac97_codec/параметры/power_save
```

Обратите внимание, что при изменении режима питания может быть слышен щелчок / хлопанье. Кроме того, часто требуется определенное время для перехода из режима отключения питания в активное состояние. Часто их трудно исправить, поэтому не сообщайте дополнительных отчетов об ошибках, если у вас нет исправленного патча ;-)

Для интерфейса HD-audio есть другой вариант модуля, `power_save_controller`. Это включает / отключает режим энергосбережения на стороне контроллера. Включение этого параметра может немного снизить энергопотребление, но может привести к увеличению времени пробуждения и появлению шума щелчка. Попробуйте отключить его, когда вы сталкиваетесь с такой проблемой слишком часто.

2.9 Примечания по эмуляции операционной системы ядра

22 января 2004 г. Такаши Иваи <tiwai@suse.de>

2.9.1 Модули

ALSA обеспечивает мощную эмуляцию операционной системы в ядре. Эмуляция OSS для устройств PCM, микшера и секвенсора реализована в виде дополнительных модулей ядра, `snd-pcm-oss`, `snd-mixer-oss` и `snd-seq-oss`. Когда вам необходимо получить доступ к устройствам OSS PCM, микшера или секвенсора, должен быть загружен соответствующий модуль. Эти модули загружаются автоматически при вызове соответствующей службы. Определен псевдоним `sound-service-x-y`, где `x` и `y` - номер карты и второстепенный номер устройства. Обычно вам не нужно определять эти псевдонимы самостоятельно. Единственным необходимым шагом для автоматической загрузки модулей OSS является определение псевдонима карты в `/etc/modprobe.d/alsa.conf`, например:

```
псевдоним sound-slot-0 snd-emu 1 0 k 1
```

В качестве второй карты определите звуковой слот-1 также. Обратите внимание, что вы не можете использовать псевдонимное имя в качестве целевого имени (т.е. псевдоним `sound-slot-0` `snd-карты-0` больше не работает как старые модули). Доступная в настоящее время конфигурация OSS показана в `/proc/asound/oss/sndstat`. Это отображается в том же синтаксисе `/dev/sndstat`, который доступен в коммерческом драйвере OSS. В ALSA вы можете создать символическую ссылку `/dev/sndstat` на этот программный файл. Пожалуйста, обратите внимание, что устройства, перечисленные в этом файле процедуры, появляются только после загрузки соответствующего модуля эмуляции операционной системы. Не волнуйтесь, даже если в нем указано "НЕ ВКЛЮЧЕНО В КОНФИГУРАЦИИ".

2.9.2 Отображение устройств

ALSA поддерживает

следующие файлы устройств OSS: `PCM`:

```
    / dev / dspX
    / dev / adspX
```

Микшер:

```
    / dev / mixerX
```

MIDI:

```
/dev/midi 0 X
```

```
/dev/amidi 0 X
```

Секвенсор:

```
/dev/sequencer
```

```
/dev/sequencer 2 (он же /dev/music)
```

где X - номер карты от 0 до 7.

(ПРИМЕЧАНИЕ: некоторые дистрибутивы содержат файлы устройств, такие как `/dev/midi0` и `/dev/midi1`.

Они ПРЕДНАЗНАЧЕНЫ НЕ для OSS, а для `tcldmidi`, что совершенно другое дело.) В отличие от реальной

операционной системы, ALSA не может использовать файлы устройства больше, чем назначенные.

Например, первая карта не может использовать `/dev/dsp1` или `/dev/dsp2`, а только `/dev/dsp0` и `/dev/adsp0`. Как

видно выше, PCM и MIDI могут иметь два устройства. Обычно первое PCM-устройство (`hw: 0 , 0` в ALSA)

отображается в `/dev/dsp`, а вторичное устройство (`hw: 0 , 1`) - в `/dev/adsp` (если доступно). Для MIDI -

`/dev/midi` и `/dev/amidi` соответственно. Вы можете

изменить отображение этого устройства с помощью опций модуля `snd-pcm-oss` и `snd-rawmidi`. В случае

PCM для `snd-pcm-oss` доступны следующие опции:

dsp_map Номер устройства PCM, присвоенный `/dev/dspX` (по умолчанию = 0)

adsp_map Номер устройства PCM, присвоенный `/dev/adspX` (по умолчанию = 1)

Например, для сопоставления третьего устройства PCM (`hw: 0 , 2`) для `/dev/adsp0` определите следующим

образом: параметры `snd-pcm-oss adsp_map= 2`

Параметры принимают массивы. Для настройки второй карты укажите две записи, разделенные запятой.

Для например, чтобы отобразить третье PCM-устройство на второй карте в `/dev/adsp1`, определите, как

показано ниже: опции `snd-pcm-oss adsp_map= 0 , 2`

Для изменения отображения MIDI-устройств в `snd-rawmidi` доступны следующие опции:

midi_map номер MIDI-устройства, присвоенный `/dev/midi0X` (по умолчанию = 0)

amidi_map Номер MIDI-устройства, присвоенный `/dev/amidi0X` (по умолчанию = 1)

Например, чтобы присвоить третьему MIDI-устройству на первой карте `/dev/midi00`, определите

следующим образом: параметры `snd-rawmidi midi_map= 2`

2.9.3 Режим PCM

По умолчанию ALSA эмулирует OSS PCM с помощью так называемого `plugin layer`, т. е. Пытается автоматически

преобразовать формат выборки, скорость или каналы, когда карта не поддерживает это изначально. Это приведет к некоторым

проблемам для некоторых приложений, таких как `quake` или `wine`, особенно если они используют карту только

в режиме `MMAP`. В таком случае вы можете изменить поведение PCM для каждого приложения, записав команду

в файл `proc`. Для каждого потока PCM существует файл `proc, /proc/asound/cardX/pcmY[cp]/oss`, где X - карта

номер (на основе нуля), Y номер устройства PCM (на основе нуля) и `p` предназначен для воспроизведения и `c` для захвата, соответственно. Обратите внимание, что этот файл `proc` существует только после загрузки модуля

`snd-pcm-oss`. Последовательность команд имеет следующий синтаксис:

```
фрагменты имя_приложения fragment_size [параметры]
```

`имя_приложения` - это имя приложения с (более высоким приоритетом) или без пути. `фрагменты` указывает количество фрагментов или ноль, если конкретное число не указано. `fragment_size` размер фрагмента в байтах или ноль, если не указан. `Опции` это необязательные параметры. Доступны следующие опции:

отключить приложение пытается открыть `pcm`-устройство для этого канала, но не хочет его использовать.

направить не используйте плагины

блокируйте принудительно блокируйте открытый режим

неблокируйте принудительно неблокируйте открытый режим

частичный фрагмент записывайте также частичные фрагменты (влияет только на воспроизведение)

без паузы не заполняйте паузу впереди, чтобы избежать щелчков

`отключить` опция полезна, когда одно направление потока (воспроизведение или захват) неправильно обрабатывается приложением, хотя само оборудование поддерживает оба направления. The `direct` опция используется, как упоминалось выше, для обхода автоматического преобразования и полезна для

MMAP-приложений. Например, чтобы воспроизвести первое `PCM`-устройство без плагинов для `quake`, отправьте

```
команду через echo следующим образом: % echo "quake 0 0 direct" > /proc/asound/card 0 /pcm 0 p/oss
```

Пока `quake` хочет только воспроизведение, вы можете добавить вторую команду, чтобы уведомить драйвер о том, что будет выделено только это направление:

```
% echo "quake 0 0 отключить" > /proc/asound/card 0 /pcm 0 c/oss
```

Разрешение файлов `proc` зависит от параметров модуля `snd`. По умолчанию он установлен как `root`, поэтому вам, вероятно, потребуется быть суперпользователем для отправки приведенной выше команды.

Опции блокировать и неблокировать используются для изменения режима открытия файла устройства.

По умолчанию ALSA ведет себя как оригинальные драйверы OSS, то есть не блокирует файл, когда он занят. В

этом случае возвращается ошибка `-EBUSY`. Это

поведение блокировки может быть изменено глобально с помощью опции модуля `nonblock_open` в `snd-pcm-oss`.

Для использования режима блокировки по умолчанию для устройств с OSS определите следующим образом:

```
параметры snd-pcm-oss nonblock_open=0
```

Недавно были добавлены команды `частичное разрушение` и `без тишины`. Обе команды предназначены

только для оптимизации. Первая команда указывает на запуск передачи записи только

тогда, когда заполнен весь фрагмент. Вторая автоматически прекращает запись данных

тишины вперед. Обе команды отключены по умолчанию. Вы можете проверить

текущую конфигурацию, прочитав файл `proc`. Прочитанное изображение может быть

отправлено в файл `proc` снова, следовательно, вы можете сохранить текущую

```
конфигурацию % cat /proc / asound/card 0 /pcm 0 p /oss > /где-нибудь/ oss-cfg
```

и восстановите его как

```
% cat /somewhere/oss-cfg > /proc/asound/card 0 /pcm 0 p/oss
```

Кроме того, для очистки всей текущей конфигурации отправьте `стереть` команда, как показано ниже:

```
% echo "erase" > /proc/asound/card 0 /pcm 0 p/oss
```

2.9.4 Элементы микширования

Поскольку ALSA имеет совершенно другой интерфейс микширования, эмуляция микшера OSS относительно сложна. ALSA создает элемент микшера из нескольких различных элементов управления ALSA (mixer) на основе строки имени. Для примера, элемент громкости SOUND_MIXER_PCM состоит из элементов управления "Громкость воспроизведения PCM" и "Переключатель воспроизведения PCM" для направления воспроизведения и из "Громкость записи PCM" и "Переключатель записи PCM" для каталога записи (если он существует). При изменении громкости PCM OSS все перечисленные выше элементы управления громкостью и переключателями настраиваются автоматически. По умолчанию ALSA использует следующий элемент управления громкостью OSS:

Громкость OSS	Контроль	Индекс
SOUND_MIXER_VOLUME	тембра ALSA Master	0
SOUND_MIXER_BASS	- Басы	0
SOUND_MIXER_TREBLE	Тон-контроль	0
SOUND_MIXER_SYNTH	высоких частот	0
SOUND_MIXER_PCM	синтезатора PCM	0
SOUND_MIXER_SPEAKER	и ПК динамик	0
SOUND_MIXER_LINE	линия	0
SOUND_MIXER_MIC	микрофон	0
SOUND_MIXER_CD	компакт	0
SOUND_MIXER_IMIX	мониторный	0
SOUND_MIXER_ALTPCM	микс ПКМ	0
SOUND_MIXER_RECLEV	(не назначен)	1
SOUND_MIXER_IGAIN	захват воспроизведения	0
SOUND_MIXER_OGAIN	Окс	0
SOUND_MIXER_LINE1	Окс Окс	0
SOUND_MIXER_LINE2	цифровой	1
SOUND_MIXER_LINE3	цифровой	2
SOUND_MIXER_DIGITAL1	цифровой	0
SOUND_MIXER_DIGITAL2	цифровой	1
SOUND_MIXER_DIGITAL3	мобильном	0
SOUND_MIXER_PHONEIN	телефоне	0
SOUND_MIXER_PHONEOUT	видео	0
SOUND_MIXER_VIDEO	Радио	0
SOUND_MIXER_RADIO	монитор	
SOUND_MIXER_MONITOR		

Второй столбец является базовой строкой соответствующего элемента управления ALSA. Фактически, элементы управления с помощью xxx

[Воспроизведение | Захват] [Громкость | Переключатель] будет проверяться дополнительно.

Текущее назначение этих элементов микшера указано в файле /proc/asound/cardX/oss_mixer, который будет выглядеть следующим образом

```
ГРОМКОСТЬ "Master" 0
БАС "" 0
ВЫСОКИЕ частоты" " 0
СИНТЕЗАТОР" " 0
PCM "PCM" 0
...
```

где первый столбец - элемент тома OSS, второй столбец - базовая строка соответствующего элемента управления ALSA, а третий - индекс элемента управления. Когда строка пуста, это означает, что соответствующий элемент управления OSS недоступен.

Для изменения назначения вы можете записать конфигурацию в этот прос-файл. Например, чтобы сопоставить "Волновое воспроизведение" с громкостью PCM, отправьте команду следующим образом:

```
% echo 'ГРОМКОСТЬ "Воспроизведения волны" 0' > /proc/asound/card 0 /oss_mixer
```


Команда точно такая же, как указана в файле `rgos`. Вы можете изменить один или несколько элементов, по громкости в строке. В последнем примере как "Громкость воспроизведения волны", так и "Переключатель воспроизведения волны" будут изменены при изменении громкости PCM. Как и в случае с файлом PCM `rgos`, разрешение файлов `rgos` зависит от параметров модуля `snd`. вам, вероятно, потребуется быть суперпользователем для отправки приведенной выше команды. Как и в случае с файлом PCM `rgos`, вы можете сохранить и восстановить текущую конфигурацию микшера путем чтения и записи всего образа файла.

2.9.5 Дуплексные потоки.

Обратите внимание, что при попытке использовать один файл устройства для воспроизведения и

захвата, OSS API не предоставляет способа установить формат, частоту дискретизации или количество

каналов, различающихся в каждом направлении. Таким образом, `io_handle = open("устройство",`

`O_RDWR)` будет корректно работать только в том случае, если значения одинаковы в каждом

направлении. Чтобы использовать разные значения в двух направлениях, используйте оба

```
input_handle = открыть("устройство", O_RDONLY)
output_handle = открыть("устройство", O_WRONLY)
```

и установите значения для соответствующего дескриптора.

2.9.6 Неподдерживаемые функции

MMAP в драйвере ICE1712

ICE1712 поддерживает только нетрадиционный формат, чередующийся 10-канальный 24-битный формат (упакованный в 32-битный формат). Следовательно, вы не можете отобразить буфер в обычном формате (моно или 2 канала, 8 или 16 бит) в OSS.

Эмуляция секвенсора 2.10 OSS в ALSA

Авторское право (с) 1998,1999

Такаши Иваи версия.0.1.8; 16 ноября 1999 г.

2.10.1 Описание

Этот каталог содержит драйвер эмуляции OSS sequencer на ALSA. Обратите внимание, что эта программа все еще находится в состоянии разработки. Что это делает

- это обеспечивает эмуляцию секвенсора OSS, доступ через `/dev/ sequencer` и

`/dev / music` устройства. Большинство приложений, использующих OSS, могут

запускаться, если подготовлен соответствующий секвенсор ALSA .

Этот драйвер эмулирует следующие функции:

- Обычный секвенсор и MIDI-события:

- Они преобразуются в события ALSA sequencer и

- отправляются на соответствующий порт. • События таймера:

Таймер не выбирается `ioctl`. Частота управления фиксирована на 100 независимо от частоты Гц. Это означает, что даже в системе Alpha такт всегда равен 1/100 секунды. Базовую частоту и темп можно изменить в `/dev/muslc`.

- Загрузка патча:

Поддерживается ли это исключительно драйверами синтезатора, зависит от того, поддерживается ли это, поскольку загрузка патча осуществляется обратным вызовом драйвера синтезатора. • Элементы управления вводом / выводом:

Принимаются большинство элементов управления. Некоторые элементы управления зависят от драйвера синтезатора, а также даже от оригинальной операционной системы. Кроме того, вы можете найти следующие расширенные функции:

- Улучшенный механизм очереди:
События помещаются в очередь перед их обработкой. • Несколько приложений:

Вы можете запускать два или более приложений одновременно (даже для OSS sequencer)! Однако, каждое MIDI-устройство является эксклюзивным, то есть, если какое-либо приложение открывает MIDI-устройство один раз, другие приложения не могут его использовать. В синтезаторных устройствах такого ограничения нет. • Обработка событий в реальном времени:

События могут обрабатываться в реальном времени без использования `out of bounded ioctl`. Чтобы переключиться в режим реального времени, отправьте событие `ABSTIME 0`. Последующие события будут обрабатываться в режиме реального времени без постановки в очередь. Чтобы отключить режим реального времени, отправьте событие `RELTIME 0`. • `/proc` интерфейс:

Состояние приложений и устройств может быть отображено с помощью `/proc/asound/seq/oss` в любое время. В более поздней версии конфигурация также будет изменена через `/proc` интерфейс.

2.10.2 Установка

Запустите сценарий настройки как с поддержкой секвенсора (`-- с помощью-sequencer = да`), так и с эмуляцией OSS (`-- с помощью-oss= да`). Модуль `snd-seq-oss.o` будет создан. Если синтезаторный модуль вашей звуковой карты поддерживает эмуляцию OSS (пока только драйвер `Emu8000`), этот модуль будет загружен автоматически. В противном случае, вам необходимо загрузить этот модуль вручную.

Вначале этот модуль проверяет все MIDI-порты, которые уже были подключены к секвенсору. Сразу после этого создание и удаление портов отслеживаются механизмом объявления ALSA sequencer. Доступные синтезаторные и MIDI-устройства можно найти в интерфейсе `proc`. Выполнить `cat /proc/asound/seq/oss`, и проверьте устройства. Например, если вы используете карту AWE64, вы увидите следующее:

```
Версия эмуляции OSS sequencer 0.1.8
Номер клиента ALSA 63
Порт приемника ALSA 0
Количество приложений: 0
Количество синтезаторных устройств: 1
синтезатор 0: [EMU 8000]
тип 0x1 : подтип 0x20: voices 32
возможности: включен ioctl / включена load_patch
Количество MIDI-устройств: 3
midi 0: [Порт Emu 8000-0] Порт ALSA 65: 0
возможность записи / открытия отсутствует
```

```
midi 1 : [ Порт Emu 8 0 0 0 - 1 ] Порт 6 5 : 1 ALSA
          возможность записи / нет открытых данных

midi 2 : [ 0 : MPU- 4 0 1 (UART) ] Порт 6 4 ALSA: 0
          возможность чтения / записи / нет открытых данных
```

Обратите внимание, что количество устройств может отличаться от информации в `/proc/asound/OSS-устройств` или те оригинальной ОС драйвер. Используйте номер устройства, перечисленные в `/proc / asound /seq / oss` для воспроизведения через OSS эмуляция секвенсора.

2.10.3 Использование синтезаторных устройств

Запустите вашу любимую программу. Я протестировал playmidi-2.4, awemidi-0.4.3, gmod-3.1 и хтр-1.1.5. Вы можете загрузить сэмплы через `/dev/sequencer` мне тоже нравится sfxload. Если низкоуровневый драйвер поддерживает множественный доступ к синтезирующим устройствам (например, драйвер Emu8000), двум или более приложениям разрешается запускаться одновременно.

2.10.4 Использование MIDI-устройств

Пока тестировался только MIDI-выход. MIDI-вход вообще не проверялся, но, надеюсь, он будет работать. Используйте номер устройства, указанный в `/proc/ asound/seq/oss`. Имейте в виду, что эти номера в основном отличаются от списка в `/proc /asound /oss-устройства`.

2.10.5 Параметры модуля

Доступны следующие опции модуля:

- maxqlen** задает максимальную длину очереди чтения / записи. Эта очередь является частной для OSS sequencer., так что это не зависит от длины очереди ALSA sequencer. Значение по умолчанию равно 1024.
- seq_oss_debug** задает уровень отладки и принимает ноль (= нет сообщения отладки) или положительное целое число. Значение по умолчанию равно 0.

2.10.6 Механизм очереди

При эмуляции OSS sequencer используется очередь с приоритетом ALSA. События из `/dev/sequencer` обрабатываются и помещаются в очередь, указанную параметром модуля. Все события из `/dev/sequencer` анализируются в начале. Временные события также анализируются в этот момент, так что события могут обрабатываться в режиме реального времени. Отправка события ABSTIME 0 переключает рабочий режим в режим реального времени, а отправка события RELTIME 0 отключает его. В режиме реального времени все события отправляются немедленно. События, помещенные в очередь, отправляются на соответствующие порты ALSA sequencer по истечении запланированного времени диспетчером ALSA sequencer dispatcher. Если очередь записи заполнена, приложение переходит в спящий режим до тех пор, пока определенный объем (по умолчанию половина) не станет пустым в режиме блокировки. Также была реализована синхронизация по времени записи. Входные данные с MIDI-устройств или события обратного эха сохраняются в очереди чтения FIFO. Если приложение читает `/ dev / sequencer` в режиме блокировки процесс будет запущен.

2.10.7 Интерфейс к синтезаторному устройству

Регистрация

Чтобы зарегистрировать устройство синтезатора OSS, используйте функцию `snd_seq_oss_synth_register()`:

```
int snd_seq_oss_synth_register(имя символа, тип int, подтип int, nvoices int,
                               snd_seq_oss_callback_t *операция, недействительность *private_data)
```

Аргументы `имя`, `Тип`, `подтип` и `nvoices` используются для создания соответствующей структуры `synth_info` для `ioctl`. Возвращаемое значение представляет собой порядковый номер этого устройства.

Этот индекс необходимо запомнить для отмены регистрации. Если регистрация не удалась, будет

возвращено значение `-errno`. Чтобы освободить это устройство, вызовите функцию

```
snd_seq_oss_synth_unregister(): int snd_seq_oss_synth_unregister(индекс int)
```

где `Указатель` является порядковым номером, возвращаемым функцией `register`.

Обратные вызовы

Устройства синтезатора OSS имеют возможность загрузки сэмплов и `ioctls`, таких как сброс сэмпла. В

эмуляции OSS эти специальные функции реализуются с помощью обратных вызовов. Параметр

регистрации `opreg` используется для указания этих обратных вызовов. Должны быть определены

следующие функции обратного вызова. `snd_seq_oss_callback_t`:

```
int (* open)(snd_seq_oss_arg_t *p, void * closure);
int (* close)(snd_seq_oss_arg_t *p);
int (* ioctl)(snd_seq_oss_arg_t * p, unsigned int cmd, unsigned long arg);
int (* load_patch)(snd_seq_oss_arg_t * p, int format, const char * buf, int offs, int count) ;
int (* сброс)(snd_seq_oss_arg_t *p);
```

За исключением `Открыть` и `Закрыть` обратные вызовы, им разрешено иметь значение `NULL`.

Каждая функция обратного вызова принимает тип аргумента `snd_seq_oss_arg_t` в качестве

первого аргумента. `struct snd_seq_oss_arg_t` {

```
int app_index;
int file_mode;
int seq_mode;
snd_seq_addr_t addr;
void *private_data;
int event_passing;
};
```

Первые три поля, `app_index`, `file_mode` и `seq_mode` инициализируются OSS sequencer.

В

`app_index` это индекс приложения, который уникален для каждого приложения, открывающего OSS sequencer.

`file_mode` это битовые флаги, указывающие режим работы файла. Смотрите `seq_oss.h` для его значения. Параметр `seq_mode`

- это режим работы секвенсора. Только в текущей версии `SNД_OSSSEQ_MODE_SYNTH` используется. Следующие

два поля, `addr` и `private_data` должны быть заполнены драйвером синтезатора при открытом обратном вызове. В

`addr` содержит адрес порта ALSA sequencer, который назначен этому устройству. Если драйвер выделяет

память для `private_data`, он должен быть выпущен при закрытом обратном вызове сам по себе. Последнее

поле, `event_passing` указывает, как преобразовать события включения / выключения примечания. В `PROCESS_EVENTS` режим,

примечание 255 рассматривается как изменение скорости, и ключевое событие давления передается в порт. В `PASS_EVENTS`

в этом режиме все события включения / выключения `pote` передаются на порт без изменений. `PROCESS_KEYPRESS` режим

проверяет примечание выше 128 и рассматривает его как ключевое событие давления (в основном для драйвера `Emu8000`).

Открой Обратный Звонок

В `Открыть` вызывается каждый раз, когда это устройство открывается приложением, использующим OSS sequencer. Это значение должно не быть нулевым. Обычно открытый обратный вызов выполняет следующую процедуру:

1. Выделите запись личных данных.
2. Создайте порт секвенсора ALSA.
3. Установите новый адрес порта на `arg-> addr`.
4. Установите указатель записи личных данных на `arg-> private_data`.

Обратите внимание, что битовые флаги типа в `port_info` этого порта синтезатора не должны содержать `TYPE_MIDI_GENERIC` бит.

Вместо этого, `Следует` использовать `TYPE_SPECIFIC`. Также, `CAP_SUBSCRIPTION` бит также не должен включаться. Это необходимо, чтобы отличить его от других обычных MIDI-устройств. Если процедура открытия прошла успешно, верните ноль. В противном случае верните `-errno`.

Обратный вызов ioctl

Обратный вызов `ioctl` вызывается, когда секвенсор получает `ioctls` для конкретного устройства. Следующие два `ioctls` должны быть обработаны этим обратным вызовом:

IOCTL_SEQ_RESET_SAMPLES сброс всех выборок в памяти - возвращает 0

IOCTL_SYNTH_MEMAVL возвращает доступный размер памяти

FM_4OP_ENABLE могут быть проигнорированы, как правило,

других устройств обрабатываются внутри секвенсора без прохождения в низкоуровневые драйвера.

Load_Patch Обратный Звонок

В `load_patch` обратный вызов используется для загрузки образцов. Этот обратный вызов должен считывать данные в пользовательском пространстве и передавать на каждое устройство. Возвращает 0 в случае успеха и `-errno` в случае неудачи.

Аргумент `format` - это ключ `patch` в записи `patch_info`. `Buf` - это указатель пользовательского пространства, где хранится запись `patch_info`. На офф могут быть проигнорированы. Счетчик является общим размером данных этой выборки данных.

Закрыть Обратный Звонок

В `Закрыть` обратный вызов вызывается, когда это устройство закрывается приложением. Если какие-либо личные данные были выделены при открытом обратном вызове, они должны быть освобождены при закрытом обратном вызове. Здесь также следует выполнить удаление порта ALSA. Этот обратный вызов не должен быть нулевым.

Сбросить обратный вызов

The `сброс` обратный вызов вызывается, когда устройство секвенсора сбрасывается или закрывается приложениями.

Обратный вызов должен немедленно отключить звуковые сигналы на соответствующем порту и инициализировать состояние порта. Если этот обратный вызов не определен, OSS seq отправляет `СЕРДЦЕБИЕНИЕ` событие в порту.

2.10.8 События

Большинство событий обрабатываются sequencer и преобразуются в соответствующие события ALSA sequencer, так что каждое синтезаторное устройство может получать обратный вызов `input_event` порта ALSA sequencer. Следующие события ALSA должны быть реализованы драйвером:

Событие ALSA	Исходные события OSS
ЗАМЕТКА О	SEQ_NOTEON, MIDI_NOTEON
ПРИМЕЧАНИИ	SEQ_NOTEOFF, MIDI_NOTEOFF
НАЖАТИЕ	MIDI_KEY_PRESSURE
КЛАВИШИ	SEQ_AFTERTOUCH, MIDI_CHN_PRESSURE
CHANPRESS	SEQ_PGMCHANGE, MIDI_PGM_CHANGE
PGMCHANGE	SEQ_CONTROLLER(CTRL_PITCH_BENDER),
PITCHBEND	MIDI_PITCH_BEND MIDI_CTL_CHANGE,
CONTROLLER	
CONTROL14 REGPARAM	SEQ_BALANCE (CTL_PAN) SEQ_CONTROLLER SEQ_CONTROLLER(CTRL_PITCH_BENDER_RANGE)
SYSEX	SEQ_SYSEX

Большинство из этих функций может быть реализовано драйвером эмуляции MIDI, включенным в низкоуровневый драйвер Emu8000. В будущей версии этот модуль будет независимым.

Некоторые события OSS (события `SEQ_PRIVATE` и `SEQ_VOLUME`) передаются как событие `SND_SEQ_OSS_PRIVATE`. Секвенсор OSS передает эти 8-байтовые пакеты событий без каких-либо изменений. катион. Драйвер низкого уровня должен соответствующим образом обработать эти события. типа

2.10.9 Интерфейс к MIDI-устройству

Поскольку эмуляция OSS автоматически проверяет создание и удаление портов ALSA MIDI sequencer путем получения уведомления от ALSA sequencer, MIDI-устройства не нужно регистрировать явно, как синтезаторные устройства. Однако MIDI port_info, зарегистрированный в ALSA sequencer, должен содержать группу `имя SND_SEQ_GROUP_DEVICE` и бит возможностей `CAP_READ` или `CAP_WRITE`. Кроме того, возможности подписки, `CAP_SUBS_READ` или `CAP_SUBS_WRITE`, также должен быть определен. Если эти условия не выполняются, порт не регистрируется как MIDI-устройство OSS sequencer. События, передаваемые через MIDI-устройства, анализируются в OSS sequencer и преобразуются в соответствующие события ALSA sequencer. Входные данные из MIDI-секвенсора также преобразуются в MIDI-байтовые события с помощью OSS-секвенсора. Это работает в обратном направлении с модулем seq_midi.

2.10.10 Известные проблемы / ЗАДАЧИ

- Загрузка патча через инструментальный слой ALSA еще не реализована.

УРОВЕНЬ SOC ALSA

Документация разделена на следующие разделы:-

3.1 Обзор уровня SoC ALSA

Общая цель проекта уровня ALSA System on Chip (ASoC) - обеспечить лучшую поддержку ALSA для встроенных процессоров system-on-chip (например, pxa2xx, au1x00, iMX и т.д.) и портативных аудиокодеков. До подсистемы ASoC в ядре была определенная поддержка SoC audio, однако у нее были некоторые ограничения:-

- Драйверы кодеков часто были тесно связаны с базовым процессором SoC. Это не идеально и приводит к дублированию кода - например, в Linux были разные драйверы wm8731 для 4 разных платформ SoC. • Не существовало стандартного метода для сигнализации инициированных пользователем звуковых событий (например, вставка наушников / микрофона, Обнаружение наушников / микрофона после события вставки). Это довольно распространенные события на портативных устройствах, и часто требуется машинный код для перенаправления звука, включения усилителей и т.д. После такого события. • Драйверы, как правило, включают питание всего кодека при воспроизведении (или записи) звука. Это нормально для ПК, но имеет тенденцию расходовать много энергии на портативных устройствах. Также отсутствовала поддержка экономии электроэнергии за счет изменения частоты передискретизации кодеков, токов смещения и т.д.

3.1.1 Дизайн ASoC

Уровень ASoC разработан для решения этих проблем и предоставляет следующие функции :-

- Независимость от кодека. Позволяет повторно использовать драйверы кодека на других платформах и компьютерах.
- Простая настройка аудиоинтерфейса I2S / PCM между кодеком и SoC. Каждый интерфейс SoC и кодек регистрируются возможности его аудиоинтерфейса согласуются с ядром и впоследствии настраиваются, когда известны параметры аппаратного обеспечения приложения. • Динамическое управление питанием звука (DAPM). DAPM автоматически переводит кодек в режим минимальной мощности постоянно. Это включает в себя включение / выключение внутренних блоков питания в зависимости от внутреннего маршрутизации звука кодеком и любых активных потоков. • Уменьшение количества щелчков. Хлопки и щелчки можно уменьшить, включив кодек вверх / вниз в правильной последовательности (включая использование цифрового отключения звука). ASoC сигнализирует кодеку о необходимости изменения режима питания. • Элементы управления для конкретного устройства: позволяют устройствам добавлять элементы управления к звуковой карте (например, регулятор громкости для усилителя динамиков).

Чтобы достичь всего этого, ASoC в основном разбивает встроенную аудиосистему на несколько компонентов, которые можно использовать повторно драйверы :-

- Драйверы класса кодеков: драйвер класса кодеков не зависит от платформы и содержит элементы управления звуком, возможности аудиоинтерфейса, определение DAPM кодека и функции ввода-вывода кодека. Этот класс распространяется на BT, FM и модемные микросхемы, если требуется. Драйверы класса Codec должны представлять собой универсальный код, который может выполняться на любой архитектуре и компьютере.

- Драйверы класса платформы: Драйвер класса платформы включает драйвер audio DMA engine, драйверы цифрового аудио интерфейса (DAI) (например, I2S, AC97, PCM) и любые драйверы аудио DSP для этой платформы.
- Машина класса водителя: машина водителя-класс действует как клей, который описывает и связывает другие драйверы компонентов вместе, чтобы сформировать алса "Звуковое устройство карта". Он обрабатывает любые элементы управления, относящиеся к конкретной машине, и звуковые события машинного уровня (например, включение усилителя при начале воспроизведения).

3.2 Драйвер класса кодеков ASoC.

Драйвер класса codec - это универсальный и аппаратно независимый код, который настраивает кодек, FM, модем, BT или внешний DSP для обеспечения захвата и воспроизведения звука. Он не должен содержать кода, специфичного для целевой платформы или компьютера. Весь код, специфичный для платформы и компьютера, должен быть добавлен в драйверы платформы и компьютера соответственно. Драйвер каждого класса кодеков *должен* обеспечивать следующие функции:-

1. Конфигурацию DAI кодека и PCM 2.

Ввод-вывод управления кодеком - с помощью

RegMap API 3. Микшеры и элементы

управления звуком 4. Операции со звуком в

кодеке 5. Описание DAPM. 6. Обработчик

событий DAPM. Дополнительно драйверы

кодеков также могут обеспечивать:-

7. Цифровое управление отключением звука DAC.

Вероятно, лучше всего использовать это руководство в сочетании с существующим кодом драйвера кодека в sound / soc / codecs/

3.2.1 Описание драйвера кодека ASoC

Конфигурация DAI кодека и PCM

Каждый драйвер кодека должен иметь структуру snd_soc_dai_driver для определения его возможностей и операций DAI и PCM. ations. Эта структура экспортируется, чтобы она могла быть зарегистрирована в ядре драйвером вашего компьютера. например

```
статическая структура snd_soc_dai_ops wm 8 7 3 1_dai_ops = {

    .подготовить          = wm 8 7 3 1_pcm_prepare,
    .параметры hw_params = wm 8 7 3 1_hw_params, = wm 8 7 3 1_shutdown,
    .завершение работы   = wm 8 7 3 1_mute,
    .digital_mute         = wm 8 7 3 1_set_dai_sysclk,
    .set_sysclk           = wm 8 7 3 1_set_dai_fmt,
    .set_fmt              = wm 8 7 3 1_set_dai_fmt,
};

struct snd_soc_dai_driver wm 8 7 3 1_dai = {
    .name = "wm 8 7 3 1-hifi",
    .воспроизведение = {
        .stream_name =
        "Воспроизведение",
        .channels_min = 1,
        .channels_max = 2, .rates =
        WM 8 7 3 1_RATES, .formats = WM 8 7 3 1_FORMATS,},
    .capture = {
        .stream_name = "Захват",
        .channels_min = 1,
```



```
.channels_max = 2, .rates
= WM 8 7 3 1 _RATES, .formats
= WM 8 7 3 1 _FORMATS,},
.ops = &wm 8 7 3 1 _dai_ops,
.symmetric_rates = 1,
};
```

Ввод-вывод управления кодеком

Обычно кодеком можно управлять через интерфейс в стиле I2C или SPI (AC97 объединяет управление с данными в DAI). Драйвер кодека должен использовать Regmap API для всех операций ввода-вывода кодеков. Пожалуйста, смотрите include/linux/regmap.h и существующие драйверы кодека для примера использования regmap. **Микшеры и элементы управления звуком**

Все микшеры кодеков и элементы управления звуком могут быть определены с помощью удобных

макросов, определенных в soc.h. #define SOC_SINGLE(xname, reg, shift, mask, инвертировать)

Определяет отдельный элемент управления следующим образом:-

```
xname = имя элемента управления, например, "Громкость воспроизведения"

reg = регистр кодека

shift = смещение управляющего бита (ов) в регистре

mask = размер управляющего бита (ов), например, маска из 7 = 3 бит

invert = элемент управления инвертирован
```

Другие макросы включают:-

```
#define SOC_DOUBLE(xname, reg, shift_left, shift_right, mask, invert)
```

Элемент управления стерео

```
#определение SOC_DOUBLE_R(xname, reg_left, reg_right, shift, mask, invert)
```

Стереуправление, охватывающее 2 регистра

```
#define SOC_ENUM_SINGLE(xreg, xshift, xmask, xtexts)
```

Определяет один перечисляемый элемент управления следующим образом:-

```
xreg = регистр

xshift = смещение управляющих битов в регистре

xmask = размер управляющих битов

xtexts = указатель на массив строк, описывающих каждый параметр

#define SOC_ENUM_DOUBLE(xreg, xshift_l, xshift_r, xmask, xtexts)
```

Определяет элемент управления стереосистемой с перечислением

Операции со звуком в кодеке

Драйвер кодека также поддерживает следующие операции ALSA PCM:-

```
/* SoC audio ops */

структура snd_soc_ops {

    int (*запуск)(структура snd_pcm_substream *); void (*
    завершение работы)(структура snd_pcm_substream *);
```

```
int (*hw_params)(struct snd_pcm_substream *, struct snd_pcm_hw_params *);
int (*hw_free)(структура snd_pcm_substream *);
int (*подготовить)(структура snd_pcm_substream *);
};
```

Пожалуйста, обратитесь к документации ALSA driver PCM для получения
подробной информации. [написание alsa-драйвера/](#)

<http://www.alsa-project.org/~iwai/>

Описание DAPM

Описание Dynamic Audio Power Management описывает компоненты питания кодека и их
взаимосвязи и регистры с ядром ASoC. Пожалуйста, прочтите dapm.txt для получения подробной
информации о построении описания. Пожалуйста, смотрите также примеры в других драйверах
кодеков. **Обработчик событий DAPM**

Эта функция представляет собой обратный вызов, который обрабатывает вызовы РМ домена кодека и
системные вызовы РМ домена (например, suspend и resume). Используется для перевода кодека в
спящий режим, когда он не используется. Состояния питания:-

```
SNDRV_CTL_POWER_D 0 : /* полностью включен */
/ * vref / mid, clk и оэс включены, активны */
SNDRV_CTL_POWER_D 1 : /* частично включен */
SNDRV_CTL_POWER_D 2 : /* частично включен */
SNDRV_CTL_POWER_D 3 hot: / * Выключен, при подаче питания */
/ * все выключено, кроме vref / vmid, неактивно */
SNDRV_CTL_POWER_D 3 cold: /* Все выключено, без питания */
```

Цифровое управление отключением звука Codec DAC

Большинство кодеков имеют цифровое отключение звука перед ЦАП, которое можно
использовать для минимизации любых системных шумов. Отключение звука
останавливает поступление любых цифровых данных в ЦАП. Может быть создан обратный вызов, который
вызывается ядром для каждого DAI кодека, когда применяется или освобождается

```
Отключение звука. т.е. статический int wm 8 9 7 4 _mute(struct snd_soc_dai *dai, int mute)
{
    struct snd_soc_codec *codec = dai->кодек;
    u 1 6 mute_reg = snd_soc_read(кодек, WM 8 9 7 4 _DAC) & 0 xffbfbf;
    if (отключить звук)
        snd_soc_write(кодек, WM 8 9 7 4 _DAC, mute_reg | 0 x 4 0);
    ещё
        snd_soc_write(кодек,
WM 8 9 7 4 _DAC, mute_reg); возвращает 0;
}
```

3.3 Цифровой аудиointерфейс ASoC (DAI)

В настоящее время ASoC поддерживает три основных цифровых аудиointерфейса (DAI), которые
сегодня используются в SoC-контроллерах и портативных аудиокодеках, а именно AC97, I2S и PCM.

3.3.1 AC97

AC97 - это пятипроводной интерфейс, обычно встречающийся на многих звуковых картах ПК. Сейчас он также популярен во многих портативных устройствах. Этот DAI имеет линию сброса и мультиплексирует свои данные по времени в своих линиях SDATA_OUT (воспроизведение) и SDATA_IN (захват). Тактовая частота (BCLK) всегда определяется кодеком (обычно 12,288 МГц), а кадр (FRAME) (обычно 48 кГц) всегда определяется контроллером. Каждый кадр AC97 имеет длину 21 МКС и разделен на 13 временных интервалов. Спецификацию AC97 можно найти на странице : http://www.intel.com/p/en_US/business/design

3.3.2 I2S

I2S - это обычный 4-проводной DAI, используемый в HiFi, STB и портативных устройствах. Линии Tx и Rx используются для передачи звука, в то время как битовые тактовые сигналы (BCLK) и левые/правые тактовые сигналы (LRC) синхронизируют канал связи. I2S отличается гибкостью в том, что либо контроллер, либо КОДЕК могут управлять (master) тактовыми линиями BCLK и LRC. Битовая частота обычно варьируется в зависимости от частоты дискретизации и тактовой частоты главной системы (SYSCLK). LRCLK совпадает с частотой дискретизации. Несколько устройств поддерживают отдельные LRCLK АЦП и ЦАП, что позволяет осуществлять одновременный захват и воспроизведение с разной частотой дискретизации. I2S имеет несколько различных режимов работы: **I2S MSB** передается по падающему фронту первого BCLK после перехода LRC.

Выровнено по левому краю MSB передается при переходе LRC.

Выровнено по правому краю MSB передается размером выборки BCLKs перед переходом LRC.

3.3.3 PCM

PCM - это еще один 4-проводной интерфейс, очень похожий на I2S, который может поддерживать более гибкий протокол. Он имеет битовые тактовые (BCLK) и синхронизирующие (SYNC) линии, которые используются для синхронизации канала, в то время как линии Tx и Rx используются для передачи и приема аудиоданных. Битовая частота обычно варьируется в зависимости от частоты дискретизации, в то время как синхронизация выполняется с этой частотой дискретизации. PCM также поддерживает мультиплексирование с временным разделением (TDM) в том смысле, что несколько устройств могут использовать шину одновременно (иногда это называют сетевым режимом). Обычные режимы работы PCM: **Режим А MSB** передается по падающему фронту первого BCLK после кадра/СИНХРОНИЗАЦИИ. **Режим В MSB** передается по переднему фронту кадра / СИНХРОНИЗАЦИИ.

3.4 Динамическое управление питанием звука для портативных устройств

3.4.1 Описание

Динамическое управление питанием звука (DAPM) предназначено для того, чтобы портативные устройства Linux могли постоянно использовать минимальное количество энергии в аудиоподсистеме. Он независим от других PM ядра и, как таковой, может легко сосуществовать с другими системами PM. DAPM также полностью прозрачен для всех приложений пользовательского пространства, поскольку все переключение питания выполняется внутри ядра ASoC. Для приложений пользовательского пространства не требуется никаких изменений кода или перекомпиляции. DAPM принимает решения о переключении питания на основе любых действий с аудиопотоком (захват / воспроизведение) и настроек аудиомикшера в устройстве. DAPM охватывает всю машину. Он охватывает управление мощностью во всей аудиоподсистеме, включая блоки питания внутренних кодеков и системы питания машинного уровня. В DAPM есть 4 области питания.

Домен смещения кодека VREF, VMID (базовый кодек и мощность звука)

Обычно управляется при проверке / удалении и приостановке / возобновлении работы кодека, хотя может быть установлен во время трансляции. Если питание не требуется для бокового звука и т.д. **Платформа / Домен компьютера** физически подключенные входы и выходы

Зависят от платформы / компьютера и действий пользователя, настраиваются драйвером компьютера и реагируют на асинхронные события, например, при вставке HP. **Домен пути** пути прохождения сигналов аудиоподсистемы

Устанавливаются автоматически при изменении пользователем настроек микшера и мультиплексирования, например, alsamixer, amixer. **Область потока** ЦАП и АЦП.

Включается и отключается при запуске и остановке воспроизведения / захвата потока соответственно, например, arplay, arescord. Все решения о переключении питания DAPM принимаются автоматически путем сверки с картой маршрутизации звука всей машины. Эта карта специфична для каждой машины и состоит из взаимосвязей между каждым аудиокомпонентом (включая внутренние компоненты кодека). Все аудиокомпоненты, влияющие на питание, в дальнейшем называются виджетами.

3.4.2 Виджеты DAPM

Аудио виджеты DAPM делятся на несколько типов:-

Микшер Микширует несколько аналоговых сигналов в один аналоговый сигнал.

Мультиплексор Аналоговый переключатель, который выводит только один из многих входов. **PGA** Программируемый усилитель усиления или виджет ослабления.

АЦП Аналого-цифровой преобразователь

ЦАП Цифроаналоговый преобразователь

Переключатель Аналоговый переключатель

Вход Входной контакт кодека

Выход Выходной контакт кодека

Наушники Наушники (и дополнительный разъем)

Микрофон Микрофон (и дополнительный разъем)

Линейный Линейный вход / выход (и дополнительный разъем)

Динамик Динамик

Источник питания Виджет питания или синхронизации, используемый другими виджетами.

Регулятор Внешний регулятор, который подает питание на аудиокомпоненты.

Часы Внешние часы, которые подают синхронизацию на аудиокомпоненты.

AIF В Вход аудиоинтерфейса (с маской слота TDM).

Выход AIF Выход аудиоинтерфейса (с маской слота TDM).

Siggen Генератор сигналов.

Вход DAI Вход цифрового аудиоинтерфейса.

Выход DAI Выход цифрового аудиоинтерфейса.

DAI Link DAI Link между двумя структурами DAI

Предварительный Специальный предварительный виджет (ехес перед всеми остальными)

Публикация Специальный виджет для ПУБЛИКАЦИИ (ехес после всех остальных)

Буфер Буфер аудиоданных между виджетами в DSP.

Планировщик Внутренний планировщик DSP, который планирует работу компонентов / конвейера обработки. **Эффект** Виджет, который выполняет эффект обработки звука. **Src** Преобразователь частоты дискретизации в DSP или КОДЕКЕ

ASRC Асинхронный преобразователь частоты дискретизации в DSP или КОДЕКЕ

Кодировщик Виджет, который кодирует аудиоданные из одного формата (обычно PCM) в другой, обычно более удобный-сжатый формат.

Декодер Виджет, который декодирует аудиоданные из сжатого формата в несжатый формат, такой как PCM.

(Виджеты определены в include/sound/soc-dapm.h)

Виджеты могут быть добавлены на звуковую карту любым из типов драйверов компонентов. В soc-dapm.h определены удобные макросы, которые можно использовать для быстрого создания списка виджетов кодеков и компьютеров DAPM widgets. Большинство виджетов имеют имя, регистр, сдвиг и инверсию. Некоторые виджеты имеют дополнительные параметры для stream name и kcontrols.

Виджеты домена Stream

Виджеты Stream относятся к области stream power и состоят только из АЦП (аналого-цифровых преобразователей), ЦАП (цифроаналоговых преобразователей), AIF IN и AIF OUT.

Виджеты Stream имеют следующий формат:-

```
SND_SOC_DAPM_DAC(имя, название потока, рег, сдвиг, инвертировать),
SND_SOC_DAPM_AIF_IN(имя, поток, слот, рег, сдвиг, инвертировать)
```

ПРИМЕЧАНИЕ: имя потока должно совпадать с соответствующим именем потока в вашем кодеке

snd_soc_codec_dai. например, виджеты stream для воспроизведения и захвата в формате HiFi

```
SND_SOC_DAPM_DAC("HiFi DAC", "Воспроизведение HiFi", REG, 3, 1),
SND_SOC_DAPM_ADC("HiFi ADC", "Захват HiFi", REG, 2, 1),
```

например, виджеты потокового вещания для AIF

```
SND_SOC_DAPM_AIF_IN("AIF 1 RX", "Воспроизведение AIF 1", 0, SND_SOC_NOPM, 0, 0),
SND_SOC_DAPM_AIF_OUT("AIF 1 TX", "Захват AIF 1", 0, SND_SOC_NOPM, 0, 0),
```

Виджеты домена пути

Виджеты домена пути имеют возможность управлять аудиосигналом или аудиотрассами в пределах аудио подсистемы или влиять на них. Они имеют следующий вид:-

```
SND_SOC_DAPM_PGA(имя, рег, сдвиг, инверсия, элементы управления, num_controls)
```

Любой виджет kcontrols может быть установлен с помощью элементов controls и num_controls.

например, виджет Mixer (kcontrols объявляются первыми)

```
/* Выходной микшер */

статическая константа snd_control_new_t wm 8 7 3 1_output_mixer_controls[] = {
SOC_DAPM_SINGLE("Переключатель обхода линии", WM 8 7 3 1_APANA, 3, 1, 0),
SOC_DAPM_SINGLE("Переключатель боковых тонов микрофона", WM 8 7 3 1_APANA, 5, 1, 0),
SOC_DAPM_SINGLE("Переключатель воспроизведения HiFi", WM 8 7 3 1_APANA, 4, 1, 0),
};
```

```
SND_SOC_DAPM_MIXER("Выходной микшер", WM 8 7 3 1_PWR, 4, 1, wm 8 7 3 1_output_mixer_controls,
ARRAY_SIZE(wm 8 7 3 1_output_mixer_controls)),
```

Если вы не хотите, чтобы к элементам микшера добавлялся префикс имени виджета микшера, вместо этого можно использовать параметры те же, что и для SND_SOC_DAPM_MIXER_NAMED_CTL. SND_SOC_DAPM_MIXER.

Виджетов машинного домена

Виджеты компьютера отличаются от виджетов кодека тем, что с ними не связан бит регистра кодека . Каждому аппаратному аудиокomпоненту (без кодека или DSP), который может питаться независимо, присваивается свой машинный виджет. например,

- Усилитель динамика
- Смещение микрофона
- Разъемные разъемы

Виджет устройства может иметь дополнительный обратный вызов.

например, виджет разъема Jack для внешнего микрофона, который обеспечивает смещение

```
микрофона при вставке микрофона:-: статический int spitz_mic_bias(struct
snd_soc_dapm_widget * w, событие int) {
    gpio_set_value(SPITZ_GPIO_MIC_BIAS, SND_SOC_DAPM_EVENT_ON(событие));
    возвращает 0 ;
}

SND_SOC_DAPM_MIC("Разъем для микрофона", spitz_mic_bias),
```

Домен кодека (BIAS)

Домен мощности смещения кодеков не имеет виджетов и обрабатывается обработчиком событий codecs DAPM. Этот обработчик вызывается, когда состояние мощности кодека изменяется на любое потоковое событие или с помощью событий PM ядра.

Виртуальные виджеты

Иногда в кодеке или звуковой карте компьютера существуют виджеты, которые не имеют соответствующего программного обеспечения. В этом случае необходимо создать виртуальный виджет - виджет без

```
управляющих битов, например, SND_SOC_DAPM_MIXER("Микшер AC 9 7 ", SND_SOC_DAPM_NOPM, 0, 0, NULL, 0 ),
```

Это может быть использовано для объединения сигнальных путей вместе в программном обеспечении.

После того, как все виджеты определены, их можно добавить в подсистему DAPM по отдельности с помощью вызова snd_soc_dapm_new_control().

3.4.3 Взаимосвязи виджетов Codec/DSP

Виджеты связаны друг с другом в кодеке, платформе и компьютере с помощью аудиотрансляций (называемых межсоединениями). Каждое межсоединение должно быть определено, чтобы создать карту всех звуковых путей между виджетами. Это проще всего сделать со схемой кодека или DSP (и схемой аудиосистемы компьютера), поскольку для этого требуется объединить виджеты по их каналам аудиосигнала. например, из выходного микшера WM8731 (wm8731.c)

Выходной микшер WM8731 имеет 3 входа (источника)

1. Линейный байпасный вход
2. ЦАП (воспроизведение в формате HiFi)
3. Микрофонный боковой вход

С каждым входом в этом примере связан kcontrol (определенный в примере выше), и он подключен к выходному микшеру через его имя kcontrol. Теперь мы можем подключить целевой виджет (wrt

```
аудиосигнал) с его исходными виджетами. / *   выходной микшер   */
{"Выходной микшер", "Переключатель обхода линии", "Линейный вход"}, {"Выходной микшер", "Переключатель воспроизведения HiFi", "ЦАП"}, {"Выходной микшер", "Переключатель боковых тонов микрофона", "Смещение микрофона"},
Итак, у нас есть :-
```

- Виджет назначения <=== Имя пути <=== Виджет источника, или
- Приемник, Путь, источник, или •

Выходной микшер подключен к DAC с помощью переключателя воспроизведения HiFi.

Когда нет имени пути, соединяющего виджеты (например, прямое соединение), мы передаем NULL для имени пути.

Межсоединения создаются с помощью вызова:-

```
snd_soc_dapm_connect_input(кодек, приемник, путь, источник);
```

Наконец, snd_soc_dapm_new_widgets(кодек) должен быть вызван после того, как все виджеты и взаимосвязи были зарегистрированы в ядре. Это заставляет ядро сканировать кодек и компьютер, чтобы внутреннее состояние DAPM соответствовало физическому состоянию компьютера.

Взаимосвязи между виджетами компьютера.

Соединения машинного виджета создаются таким же образом, как и соединения кодека, и напрямую подключают выводы кодека к виджетам машинного уровня. например, подключает выводы кодека выхода динамика к внутреннему динамику.

```
/ *   внешний динамик, подключенный к выводам кодека   LOUT 2 ,   ROUT 2   */
{"Ext Spk", NULL, "ROUT 2 "},
{"Ext Spk", NULL, "LOUT 2 "},
```

Это позволяет DAPM включать и выключать контакты, которые подключены (и используются), и контакты с ЧПУ соответственно.

3.4.4 Виджеты конечных точек.

Конечная точка - это начальная или конечная точка (виджет) аудиосигнала внутри устройства, которая включает кодек. например,

- Разъем для наушников
- Встроенный динамик
- Внутренний микрофон
- Разъем для микрофона
- Контакты кодека

Конечные точки добавляются в график DAPM, чтобы можно было определить их использование в целях экономии электроэнергии. например, контакты кодеков с ЧПУ будут отключены, неподключенные гнезда также могут быть отключены.

3.4.5 События виджета DAPM

Некоторые виджеты могут регистрировать свой интерес к событиям PM в ядре DAPM. например, динамик с усилителем регистрирует виджет, поэтому усилитель может быть включен только при использовании

```
spk. / * включение / выключение усилителя громкоговорителя в зависимости от использования */
static int corgi_amp_event(struct snd_soc_dapm_widget *w, событие int)
{
    gpio_set_value(CORGI_GPIO_APM_ON, SND_SOC_DAPM_EVENT_ON(событие));
    возвращает 0;
}

/* виджеты dapm машины corgi */ статическая постоянная
структура snd_soc_dapm_widget wm 8 7 3 1 _dapm_widgets =
    SND_SOC_DAPM_SPK("Ext Spk", corgi_amp_event);
```

Пожалуйста, смотрите soc-dapm.h для всех других виджетов, поддерживающих события.

Типы событий

Виджеты событий поддерживают следующие

```
типы событий. / * типы событий dapm */
# определить SND_SOC_DAPM_PRE_PMU 0 x 1
# определить SND_SOC_DAPM_POST_PMU 0 x 2
# определить SND_SOC_DAPM_PRE_PMD 0 x 4
# определить SND_SOC_DAPM_POST_PMD 0 x 8
# определить SND_SOC_DAPM_PRE_REG 0 x 1 0
# определить SND_SOC_DAPM_POST_REG 0 x 2 0

/* перед включением виджета */
/* после включения виджета */
/* перед выключением виджета */
/* после отключения питания виджета */
/* перед настройкой звукового тракта */
/* после настройки звукового тракта */
```

Драйвер платформы 3.5 ASoC

Класс драйверов платформы ASoC можно разделить на драйверы audio DMA, драйверы SoC DAI и драйверы DSP. Драйверы платформы предназначены только для процессора SoC и не должны содержать кода, специфичного для платы.

3.5.1 Аудио DMA

Драйвер платформы DMA опционально поддерживает следующие

```
операции ALSA:- / * SoC audio ops */
struct snd_soc_ops {
    int (*запуск)(структура snd_pcm_substream *); void (*
    завершение работы)(структура snd_pcm_substream *);
    int (* hw_params)(структура snd_pcm_substream *, структура
    snd_pcm_hw_params *); int (* hw_free)(структура snd_pcm_substream *);
    int (* подготовить)(структура snd_pcm_substream *);
    int (* запустить)(структура snd_pcm_substream *, int);
};
```

Драйвер платформы экспортирует свою функциональность DMA через struct snd_soc_platform_driver:-


```
struct snd_soc_platform_driver {  
  
    символ    * имя;  
  
  
    int  (* проверить)(struct platform_device *pdev);  
    int  (* удалить)(struct platform_device *pdev);  
    int  (* приостановить)(структура platform_device *pdev,  структура snd_soc_cpu_dai *cpu_dai);  
    int  (* возобновить)(структура platform_device *pdev,  структура snd_soc_cpu_dai *cpu_dai);  
  
    /* создание и уничтожение pcm */  
    int  (* pcm_new)(структура snd_card *,  структура snd_soc_codec_dai *,  структура  
snd_pcm *); void  (* pcm_free)(структура snd_pcm *);  
  
    /*  
        * Для отчетов о задержках, вызванных  
платформой. * Необязательно.  
        */ snd_pcm_sframes_t  
    (* задержка)(структура snd_pcm_substream *,  
                структура snd_soc_dai *);  
  
    /* платформа stream ops */  
    структура snd_pcm_ops * pcm_ops;  
};
```

Пожалуйста, обратитесь к документации драйвера ALSA для получения подробной информации о audio DMA. [http://www.alsa-project.org](http://www.alsa-project.org/~iwai/)

[/~iwai/](#) написание драйвера alsa/

Примером драйвера DMA является soc /pxa/pxa2xx-pcm.c

3.5.2 Драйверы SoC DAI

Каждый драйвер SoC DAI должен обеспечивать следующие функции:-

1. Описание цифрового аудиointерфейса (DAI)
2. Конфигурация цифрового аудиointерфейса
3. Описание PCM
4. Конфигурация SYSCLK
5. Приостановка и возобновление (необязательно)

Пожалуйста, смотрите codec.txt описание пунктов 1-4.

3.5.3 Драйверы SoC DSP

Каждый драйвер SoC DSP обычно предоставляет следующие функции :-

1. График DAPM
2. Элементы управления микшером
3. Ввод-вывод DMA в /из буферов DSP (если применимо)
4. Определение интерфейсных устройств PCM DSP (FE). Пожалуйста, ознакомьтесь с DPCM.txt описанием пункта 4.

3.6 Драйвер машины ASoC

Драйвер компьютера (или платы) ASoC - это код, который объединяет все драйверы компонентов

(например, кодеки, платформы и DAIs). В нем также описываются взаимосвязи между каждым

компонентом, включающим аудио

3.6. Драйвер компьютера ASoC

пути, графические процессоры, прерывания, синхронизация, разъемы и регуляторы напряжения.

Драйвер компьютера может содержать кодек и код, зависящий от платформы. Он регистрирует аудиоподсистему в ядре как устройство

```
платформы и представлен следующей структурой:- / *   SoC-машина   */

struct snd_soc_card {
    символ   *имя;

    ...

    int  (* проверить)(struct platform_device *pdev);
    int  (* удалить)(struct platform_device *pdev);

    /* функции pre и post PM используются для выполнения любой работы с PM до и после того, как кодек и DAIs выполнят любую работу с PM.
       * */
    int  (* suspend_pre)(структура platform_device *pdev, pm_message_t
    состояние); int  (* suspend_post)(структура platform_device *pdev,
    pm_message_t состояние); int  (* resume_pre)(структура platform_device
    *pdev); int  (* resume_post)(структура platform_device *pdev);

    ...

    /* Ссылки CPU <--> Codec DAI           */
    struct snd_soc_dai_link *dai_link;
    int  num_links;

    ...
};
```

3.6.1 проверять()/удалять()

проверка / удаление необязательны. Выполните проверку для любой конкретной машины здесь.

3.6.2 приостановить () / возобновить ()

Драйвер компьютера имеет предварительные и последующие версии suspend и resume, чтобы позаботиться о любых задачах машинного аудио, которые необходимо выполнить до или после приостановки и возобновления работы кодека, DAIs и DMA. Необязательно.

3.6.3 Конфигурация машинного интерфейса DAI

Конфигурация машинного интерфейса DAI объединяет все модули кодека и процессора. Его также можно использовать для настройки системных часов DAI и для любой связанной с машиной инициализации DAI, например, звуковая карта компьютера может быть подключена к звуковой карте кодека, как таковые могут быть установлены неподключенные контакты кодека. struct snd_soc_dai_link используется для настройки каждого DAI на

```
вашем компьютере. например, / *   цифровой аудиоинтерфейс corgi для склеивания кодеков <--> CPU   */

статическая структура snd_soc_dai_link corgi_dai = {

    .name = "WM 8 7 3 1",
    .имя_потока = "WM 8 7 3 1",
    .cpu_dai_name = "pxa-is 2 -dai",
    .codec_dai_name = "wm 8 7 3 1 -hifi",
    .platform_name = "pxa-pcm-audio",

    .codec_name = "wm 8 7 1 3 -codec. 0 - 0 0 1 a",
    .init = corgi_wm 8 7 3 1_init,
    .ops = &corgi_ops,

};
```

затем struct snd_soc_card настраивает машину с ее

```
платформой. например, / *  драйвер  аудиомашины  corgi  */
статическая  структура  snd_soc_card  snd_soc_corgi  =  {
    .name  =  "Корги",
    .dai_link  =  &corgi_dai,
    .num_links  =  1 ,
};
```

3.6.4 Карта мощности машины

Драйвер устройства может дополнительно расширить карту питания кодека и превратить ее в карту мощности звука аудиоподсистемы. Это позволяет автоматически включать / выключать питание динамиков / усилителей НР и т.д. Контакты кодека могут быть подключены к гнездам разъемов аппарата с помощью функции инициализации аппарата.

3.6.5 Элементы управления аппаратом.

В функции инициализации DAI можно добавить элементы управления аудиомикшером для конкретного аппарата.

3.7 Звуковые хлопки и щелчки

Хлопки и щелчки - это нежелательные звуковые артефакты, вызываемые включением и выключением компонентов внутри аудиоподсистемы. Это заметно на ПК, когда аудиомодуль загружен или выгружен (во время загрузки модуля звуковая карта включается и вызывает треск в динамиках). Хлопки и щелчки могут быть более частыми в портативных системах с DAPM. Это связано с тем, что компоненты внутри подсистемы получают динамическое питание в зависимости от использования звука, и это может впоследствии вызывать небольшой щелчок каждый раз, когда изменяется состояние питания компонента.

3.7.1 Сведение к минимуму хлопков при воспроизведении

Хлопки при воспроизведении в портативных аудиосистемах в настоящее время не могут быть полностью устранены, однако в будущем аппаратные средства аудиокодека будут лучше подавлять хлопки и щелчки. Хлопки можно уменьшить во время воспроизведения , включив аудиокомпоненты в

```
определенном порядке. Этот порядок отличается для запуска и выключения и соответствует некоторым
основным правилам:- Порядок  запуска:  - ЦАП ->  Микшеры  ->  Выходной  PGA  ->  Цифровое  включение
Порядок  выключения:  - Цифровое  отключение  звука  ->  Выходной  PGA  ->  Микшеры  ->  ЦАП
```

Это предполагает, что выходной канал PCM кодека от ЦАП проходит через микшер, а затем через PGA (программируемый усилитель усиления) перед выводом на динамики.

3.7.2 Сведение к минимуму всплывающих окон и щелчков при захвате

Избавиться от артефактов захвата несколько проще, поскольку мы можем отложить активацию АЦП до тех пор, пока не произойдут все всплывающие окна . Это соответствует правилам питания, аналогичным правилам воспроизведения, в том смысле, что компоненты включаются в определенной

```
последовательности в зависимости от запуска или выключения потока.Порядок  запуска  -  ввод  PGA  ->
Микшеры  ->  АЦП  Порядок  выключения  -  АЦП  ->  Микшеры  ->  Ввод  PGA
```

3.7.3 Шум молнии

Нежелательный шум молнии может возникать в потоке воспроизведения звука или записи, когда регулятор громкости изменяется вблизи максимального значения усиления. Шум застёжки-молнии слышен, когда увеличение или уменьшение усиления слишком быстро изменяет среднюю амплитуду аудиосигнала. Его можно свести к минимуму, включив перекрестие нуля для каждого регулятора громкости. ZC вызывает изменение усиления, когда сигнал пересекает нулевую линию амплитуды.

3.8 Синхронизация звука

В этом тексте описываются термины синхронизации звука в ASoC и цифровом аудио в целом. Примечание: Синхронизация звука может быть сложной задачей!

3.8.1 Основные часы

Каждая аудиоподсистема управляется главными часами (иногда называемыми MCLK или SYSCLK). Этот звуковой мастер-тактовый сигнал может быть получен из нескольких источников (например, crystal, PLL, CPU clock) и отвечает за правильное воспроизведение звука и фиксирование частоты дискретизации. Некоторые основные тактовые частоты (например, PLL и тактовые частоты на базе процессора) настраиваются таким образом, что их скорость может быть изменена программным обеспечением (в зависимости от использования системы и для экономии энергии). Другие основные тактовые частоты установлены на заданной частоте (т.е. кристаллы).

3.8.2 Тактовые частоты DAI.

Цифровой аудиоинтерфейс обычно управляется битовым тактовым сигналом (часто называемым BCLK).

Этот тактовый сигнал используется для передачи цифровых аудиоданных по каналу связи между декодом и процессором. DAI также имеет тактовый сигнал для подачи сигнала о начале каждого аудиокadra. Эти часы иногда называют LRC (left right clock) или FRAME. Эти часы работают точно с той же частотой дискретизации (LRC = Rate). Битовый тактовый сигнал может быть сгенерирован следующим образом:-

- $BCLK = MCLK / x$, или • $BCLK$
- = $LRC * x$, или • $BCLK = LRC$
- * Каналы * Размер слова

Это соотношение зависит, в частности, от кода или процессора SoC. Как правило, лучше всего настроить BCLK на минимально возможную скорость (в зависимости от вашего тарифа, количества каналов и размера word), чтобы сэкономить на питании. Также желательно использовать кодек (если возможно) для управления (или управления) звуковыми часами, поскольку он обычно дает более точную частоту дискретизации, чем центральный процессор.

3.9 Определение разъема ASoC

У ALSA есть стандартный API для представления физических разъемов в пространстве пользователя, часть ядра которого можно увидеть в `include / sound / jack.h`. ASoC предоставляет версию этого API, добавляющую две дополнительные функции:

- Это позволяет нескольким методам обнаружения гнезд работать вместе на одном видимом пользователем гнезде. Во встроенных системах обычно несколько устройств подключаются к одному разъему, но управляются отдельными частями аппаратного обеспечения.
- Интеграция с DAPM, позволяющая автоматически обновлять конечные точки DAPM на основе обнаруженного состояния разъема (например, отключение выходов для наушников, если наушников нет).

Это делается путем разделения разъемов на три части, работающие вместе: сам разъем, представленный структурой `snd_soc_jack`, наборами `snd_soc_jack_pins`, представляющими конечные точки DAPM для обновления, и блоками кода, предоставляющими механизмы отчетности `jack`. Например, система может иметь разъем для стереогарнитуры с двумя механизмами подачи сообщений, один для наушников и один для микрофона. Некоторые системы не смогут использовать выход громкоговорителя при подключении наушников, поэтому при изменении состояния разъема для наушников необходимо обязательно обновлять как громкоговоритель, так и наушники.

3.9.1 Гнездо - структура `snd_soc_jack`

Это представляет собой физическое гнездо в системе и является тем, что видно в пространстве пользователя. Само гнездо является полностью пассивным, оно настраивается драйвером машины и обновляется с помощью методов обнаружения гнезда. Гнезда создаются драйвером машины, вызывающим `snd_soc_jack_new()`.

3.9.2 `snd_soc_jack_pin`

Они представляют собой вывод DAPM для обновления в зависимости от некоторых битов состояния, поддерживаемых разъемом. Каждый `snd_soc_jack` имеет ноль или более из них, которые обновляются автоматически. Они создаются драйвером компьютера и связываются с разъемом с помощью `snd_soc_jack_add_pins()`. Состояние конечной точки при необходимости может быть настроено как противоположное состоянию разъема (например, включение встроенного микрофона, если микрофон не подключен через разъем).

3.9.3 Методы обнаружения домкрата

Фактическое обнаружение разъема выполняется кодом, который способен отслеживать некоторые входные данные в систему и обновлять разъем путем вызова `snd_soc_jack_report()`, указывая подмножество битов для обновления. Код обнаружения разъема должен быть настроен драйвером машины с учетом конфигурации для обновления разъема и набора параметров, о которых нужно сообщать при подключении разъема. Часто это делается на основе состояния GPIO - обработчик для этого предоставляется функцией `snd_soc_jack_add_gpio()`. Также доступны другие методы, например, интегрированные в кодеки. Один из примеров обнаружения встроенного в кодек разъема можно увидеть в драйвере WM8350. Каждый разъем может иметь несколько механизмов отчетности, хотя для его использования потребуется по крайней мере один.

3.9.4 Драйвера компьютера

Все они объединяются драйвером компьютера в зависимости от системного оборудования. Драйвер компьютера настроит `snd_soc_jack` и список контактов для обновления, затем настроит один или несколько механизмов обнаружения гнезд для обновления этого гнезда на основе их текущего состояния.

3.10 Динамический PCM

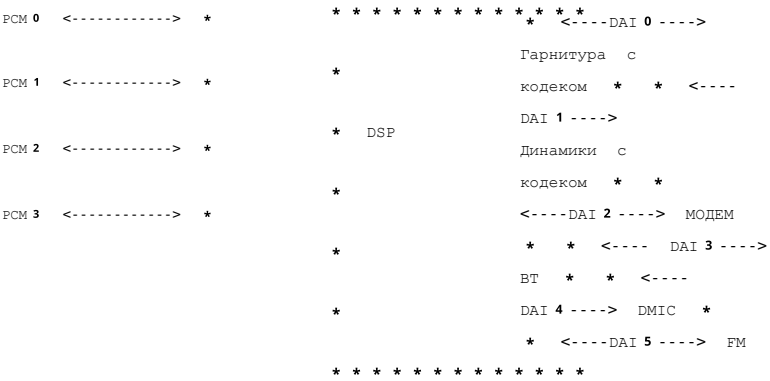
3.10.1 Описание

Dynamic PCM позволяет устройству ALSA PCM передавать свой PCM-звук в цифровом виде к различным цифровым конечным точкам во время выполнения потока PCM. например, PCM0 может направлять цифровой звук на I2S DAI0, I2S DAI1 или PDM DAI2. Это полезно для драйверов DSP на SoC, которые предоставляют доступ к нескольким PCM ALSA и могут маршрутизироваться к нескольким платформам. Маршрутизация во время выполнения DPCM определяется настройками микшера ALSA таким же образом, как аналоговый сигнал маршрутизируется в драйвере кодека ASoC. DPCM использует график DAPM, представляющий внутренние аудиотрассы DSP и использует настройки микшера для определения патча, используемого каждым ПКМ ALSA. DPCM повторно использует все существующие компонентные кодеки, платформу и драйверы DAI без каких-либо изменений.

Телефонная аудиосистема с DSP на базе SoC

Рассмотрим следующую телефонную аудиоподсистему. Это будет использоваться в этом

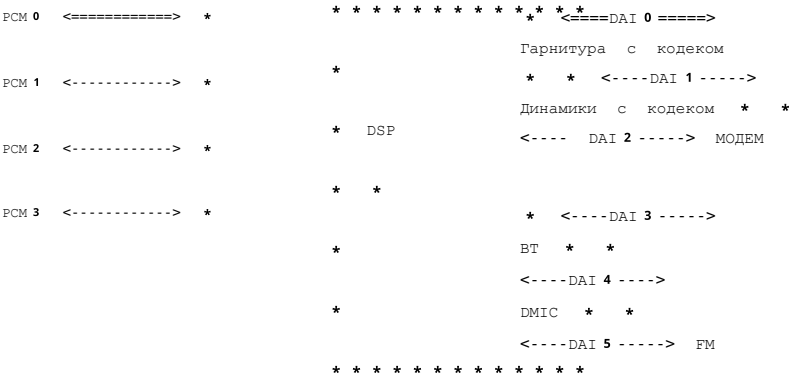
документе для всех примеров:- | Интерфейс PCMs SoC DSP | Серверная платформа | Аудиоустройства |



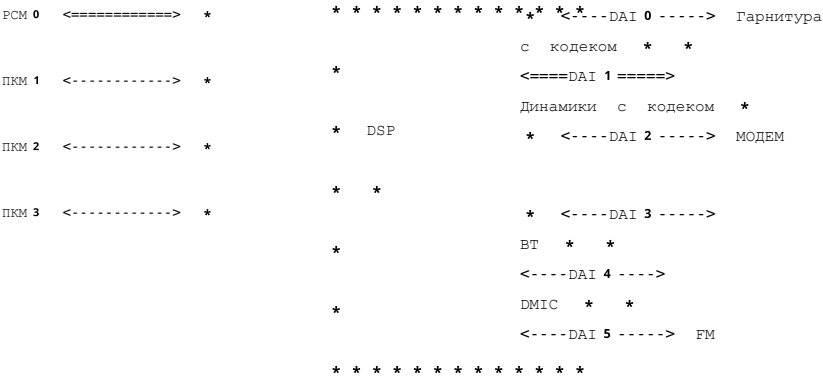
На этой схеме показана простая аудиоподсистема смартфона. Он поддерживает Bluetooth, цифровое FM-радио, динамики, разъем для гарнитуры, цифровые микрофоны и сотовый модем. Эта звуковая карта поддерживает 4 интерфейса DSP (FE) ALSA PCM и 6 интерфейсов back end (BE). Каждый FE PCM может передавать аудио в цифровом виде данные на любой из BE DAI. Устройства FE PCM также могут передавать аудио более чем на 1 BE DAI. **Пример - DPCM Переключает воспроизведение с DAI0 на DAI1**

На гарнитуру воспроизводится звук. Через некоторое время пользователь снимает гарнитуру, и звук продолжает воспроизводиться на динамиках.

Воспроизведение с PCM0 на гарнитуру будет выглядеть следующим образом :-



Пользователь вынимает гарнитуру из гнезда, поэтому теперь необходимо использовать динамики :-



Звуковой драйвер обрабатывает это следующим образом :-

- 1. Драйвер компьютера получает событие удаления разъема.
- 2. Драйвер компьютера Или аудио HAL отключает подключение к гарнитуре.
- 3. DPCM запускает операции PCM trigger (остановка), hw_free (), shutdown() на DAI0 для гарнитуры с момента путь теперь отключен.
- 4. Драйвер компьютера или audio HAL активирует путь динамика.
- 5. DPCM запускает PCM-операции для startup(), hw_params(), prepapre() и trigger(запуск) для динамиков DAI1 поскольку путь включен.

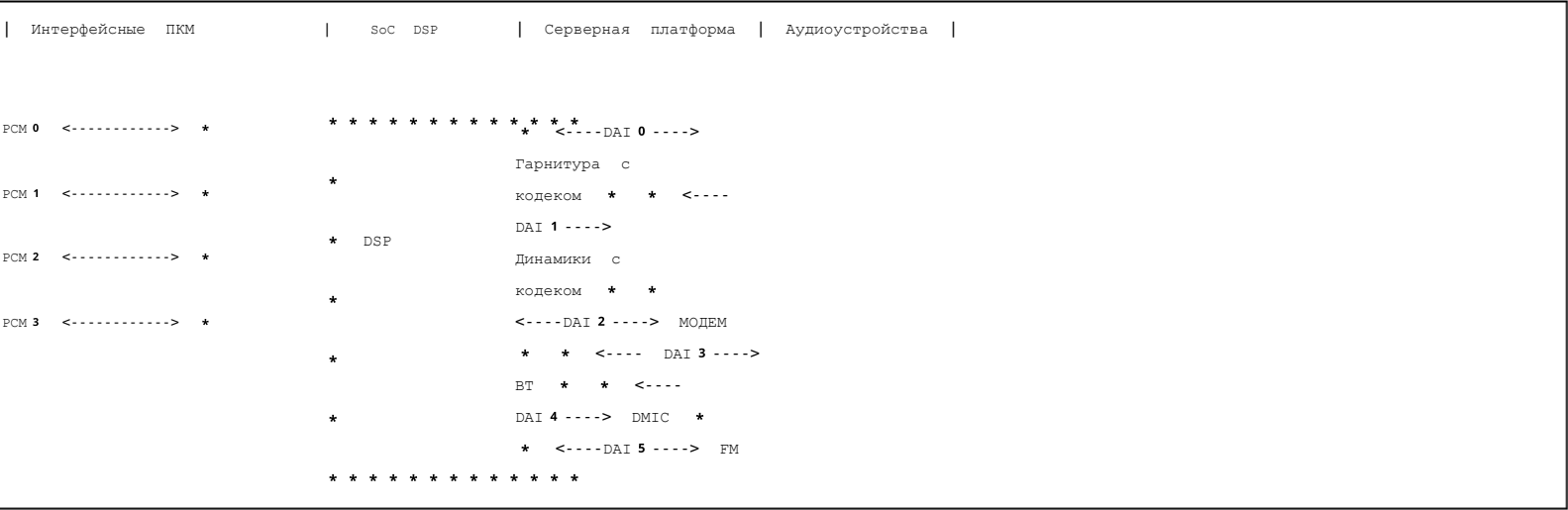
В этом примере драйвер компьютера или пользовательская аудиосистема HAL могут изменить маршрутизацию, и тогда DPCM возьмет на себя заботу об управлении операциями DAI PCM для увеличения или уменьшения связи. Воспроизведение звука не останавливается во время этого перехода.

3.10.2 Драйвер устройства DPCM

Драйвер устройства ASoC с поддержкой DPCM аналогичен обычным драйверам устройств, за исключением того, что мы также должны :-

- 1. Определите ссылки FE и BE DAI. 2.
- Определите любые операции FE / BE PCM. 3.
- Определите соединения графа виджета.

Ссылки FE и BE DAI



Для примера выше мы должны определить 4 ссылки FE DAI и 6 ссылок BE DAI. Ссылки FE DAI определены следующим

образом:-

```
статическая структура snd_soc_dai_link machine_dais[] = {
    {
        .name = "Система PCM 0 ",
        .stream_name = "Системное воспроизведение",
        .cpu_dai_name = "Системный PIN",
        .platform_name = "dsp-audio",
        .codec_name = "snd-soc-фиктивный",
        .codec_dai_name = "snd-soc-фиктивный-dai",
        .dynamic = 1, .trigger
    = {SND_SOC_DPCM_TRIGGER_POST,
      SND_SOC_DPCM_TRIGGER_POST}, . dpcm_playback = 1,
    },
}
```

```
.....< другие ссылки на
FE и BE DAI здесь > };
```

Эта ссылка FE DAI очень похожа на обычную ссылку DAI, за исключением того, что мы также устанавливаем ссылку DAI в DPCM FE с параметром `dynamic = 1`. Поддерживаемые направления потока FE также должны быть установлены с помощью флагов `dpcm_playback` и `dpcm_capture`. Существует также возможность указать порядок вызова триггера для каждого FE. Это позволяет ядру ASoC запускать DSP до или после других компонентов (поскольку некоторые DSP имеют строгие требования к порядку последовательностей запуска и остановки DAI/ DSP). Приведенный выше FE DAI устанавливает кодек и уровень кода для фиктивных устройств, поскольку BE является динамическим и будет меняться в зависимости от конфигурации среды выполнения. Платформы BE настроены следующим образом:-

```
статическая структура snd_soc_dai_link machine_dais[] = {

    .....< FE DAI ссылается
    здесь > {

.name = "Гарнитура с кодеком",
.cpu_dai_name = "ssp-dai.0",
.platform_name =
"snd-soc-dummy", .no_pcm = 1, .codec_name
= "rt5640.0-001c", .codec_dai_name
= "rt5640-aif1", .ignore_suspend
= 1, .ignore_pmdown_time
= 1, .be_hw_params_fixup
= hswult_ssp0_fixup,
.ops = &haswell_ops,
.dpcm_playback = 1,
.dpcm_capture = 1,
    .....< другие ссылки BE DAI
здесь > };
```

Эта ссылка BE DAI соединяет DAI0 с кодеком (в данном случае RT5460 AIF1). Он устанавливает значение `no_pcm` отметьте это имеет значение BE и устанавливает флаги для поддерживаемых направлений потока, используя `dpcm_playback` и `dpcm_capture` выше. В BE также установлены флаги для игнорирования времени ожидания и простоя PM. Это позволяет BE работать без хоста в режиме, когда центральный процессор хоста не передает данные, как при телефонном вызове BT. :-

```
PCM 0 <-----> *          * * * * * * * * * * * * * *
                                * <---DAI 0 --->
                                *
                                Гарнитура с
PCM 1 <-----> *          *          * * <---DAI 1 --->
                                *
                                * DSP          *
                                *          * Динамики с
PCM 2 <-----> *          *          * *
                                *          *
                                *          *
                                <===DAI 2 ===> МОДЕМ *
PCM 3 <-----> *          *          * <===DAI 3 ===> BT
                                *
                                *
                                * <---DAI 4 --->
                                *
                                *          * DMIC * *
                                *          *
                                <---DAI 5 ---> FM
                                * * * * * * * * * * *
```

Это позволяет центральному процессору хоста переходить в спящий режим, пока DSP, модемный DAI и BT DAI все еще работают. Ссылка BE DAI также может установить кодек на фиктивное устройство, если код представляет собой устройство, управляемое извне. Также будет DAI также можете установить дай процессора манекен, если дай процессора осуществляется прошивки ДСП.

Фе/быть операции PCM

Быть выше и экспортирует некоторые операции ПКМ и исправление обратный звонок. Обратный вызов fixup используется драйвером компьютера для (повторной) настройки DAI на основе параметров FE hw.

ASRC от будущей FE.

например, DSP преобразует все параметры FE hw для работы с фиксированной скоростью 48k, 16bit, стерео для DAI0. Это означает, что все параметры FE hw_params должны быть исправлены в драйвере компьютера для DAI0, чтобы DAI работал в желаемой конфигурации независимо от конфигурации FE.

```
статический int dai_0_fixup(struct snd_soc_pcm_runtime *rtd,
                           struct snd_pcm_hw_params *параметры)
{
    struct snd_interval *rate = hw_param_interval(параметры,
                                                  SNDRV_PCM_HW_PARAM_RATE);

    структура snd_interval *каналы = hw_param_interval(параметры,
                                                         SNDRV_PCM_HW_PARAM_CHANNELS);

    /* DSP уменьшит частоту FE до 48k,
    стерео */ rate->min = rate->max = 48000;
    каналы->min = каналы->max = 2;

    /* установить DAI0 равным 16 битам */
    snd_mask_set(&params->маски[SNDRV_PCM_HW_PARAM_FORMAT -
                                SNDRV_PCM_HW_PARAM_FIRST_MASK],
                SNDRV_PCM_FORMAT_S16_LE);

    возвращает 0;
}
```

Остальные операции PCM такие же, как и для обычных ссылок DAI. Используйте по мере необходимости.

Подключения к графику виджета

Ссылки BE DAI обычно подключаются к графику во время инициализации ядром ASoC DAPM. Однако, если кодек BE или BE DAI является фиктивным, то это

```
должно быть установлено явно в драйвере :- /* BE для гарнитуры с кодеком -
DAI0 является фиктивным и управляется DSP FW */
{"КОДЕК DAI0 ВКЛЮЧЕН", NULL, "Захват AIF1"},
{"Воспроизведение AIF1", NULL, "КОДЕК DAI0 ОТКЛЮЧЕН"},
```

3.10.3 Написание драйвера DPCM DSP

Драйвер DPCM DSP очень похож на стандартный драйвер ASoC класса платформы в сочетании с элементами из драйвера класса codec. Драйвер платформы DSP должен реализовывать :-

- 1. Интерфейс PCM DAIs - т.е. структура snd_soc_dai_driver.
- 2. График DAPM, показывающий маршрутизацию звука DSP от FE DAIs к BEs.
- 3. Виджеты DAPM из DSP graph.
- 4. Микшеры для усиления, маршрутизации и т.д.
- 5. Конфигурация DMA.
- 6. БУДЬТЕ виджетами AIF.

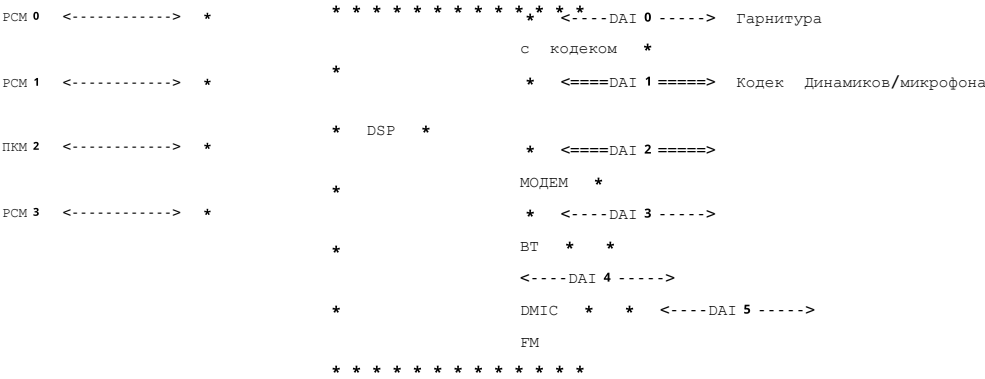
Пункты 6 важны для маршрутизации звука за пределы DSP. AIF необходимо определить для каждого BE и каждого направления потока. например, для BE DAI0 выше

```
мы имели бы:- SND_SOC_DAPM_AIF_IN("DAI0 RX", NULL, 0, SND_SOC_NOPM, 0, 0),
SND_SOC_DAPM_AIF_OUT("DAI0 TX", NULL, 0, SND_SOC_NOPM, 0, 0),
```

BE AIF используются для соединения графика DSP с графиками для других драйверов компонентов (например, для графа codec).

3.10.4 Поток PCM без хостинга

Поток PCM без хостинга - это поток, который не маршрутизируется через центральный процессор хоста. Примером этого может быть телефонный звонок с телефонной трубки на модем.



В этом случае данные PCM передаются через DSP. Центральный процессор хоста в этом варианте использования используется только для управления и может находиться в режиме ожидания во время выполнения потока. Хост может управлять каналом без хостинга либо с помощью :-

- 1. Настройка ссылки в виде ссылки в стиле кодека <->. В этом случае ссылка включается или отключается в зависимости от состояния графика DAPM. Обычно это означает наличие пульта управления микшером, который можно использовать для подключения или отключения канала между обоими помостами.
- 2. FE без хоста. Этот Fe виртуальное соединение в одной ДАИ на графике DAPM. Контроль

затем проводится Фе, как и обычные операции ПКМ. Этот метод дает больший контроль над ссылками

DAI , но требует гораздо больше пользовательского кода для управления ссылкой. Рекомендуется

использовать CODEC<-> CODEC, если только вашему HW не требуется более детальная

последовательность операций PCM. **КОДЕК <-> Ссылка на КОДЕК**

Эта ссылка DAI включается, когда DAPM обнаруживает допустимый путь в графе DAPM. Драйвер

компьютера устанавливает некоторые дополнительные параметры для ссылки DAI, т.е.

```
static const struct snd_soc_pcm_stream dai_params = {
    .formats = SNDRV_PCM_FMTBIT_S3_2_LE,
    .rate_min = 8000,
    .rate_max = 8000,
    .channels_min = 2,
    .channels_max = 2,
};

статическая структура snd_soc_dai_link dais[] = {
    < ... больше ссылок на
    DAI выше ... > {
        .name = "МОДЕМ", .stream_name
        = "МОДЕМ", .cpu_dai_name
        = "dai 2", .codec_dai_name =
        "модем-aif 1", .codec_name =
        "модем", .dai_fmt =
        SND_SOC_DAIFMT_I2S | SND_SOC_DAIFMT_NB_NF
        | SND_SOC_DAIFMT_CBM_CFM,
        .параметры =
        &dai_params, } < ... здесь
        больше ссылок на DAI ... >
```

Эти параметры используются для настройки DAI hw_params(), когда DAPM обнаруживает допустимый путь, а затем вызывает операции PCM для запуска ссылки. DAPM также вызовет соответствующие операции PCM для отключения

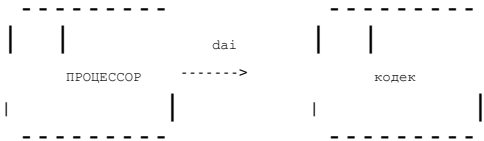
DAI, когда путь больше недействителен.

FE без хоста

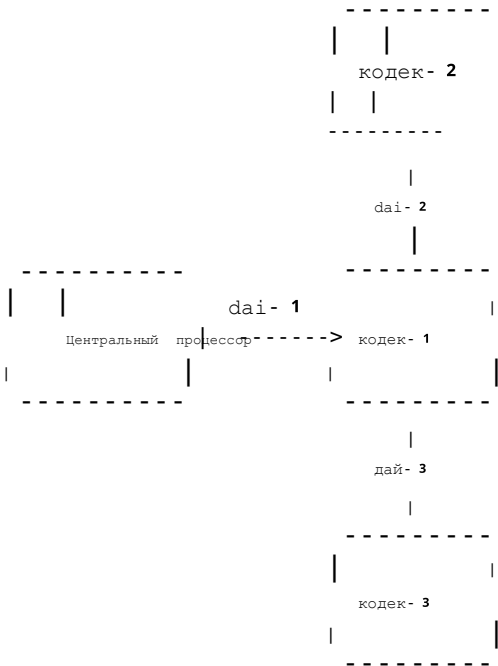
Ссылки DAI включаются FE, который не считывает и не записывает никаких PCM-данных. Это означает создание нового FE, который связан виртуальным путем с обоими ссылками DAI. Ссылки DAI будут запущены при запуске FE PCM и остановлены при остановке FE PCM. Обратите внимание, что FE PCM не может считывать или записывать данные в этой конфигурации.

3.11 Создание канала dai от кодека к кодеку для ALSA dapm

В основном поток звука всегда передается от процессора к кодеку, поэтому ваша система будет выглядеть следующим образом:



В случае, если ваша система выглядит следующим образом:



Предположим, что codec-2 - это чип Bluetooth, а codec-3 подключен к динамику, и у вас есть следующий сценарий: codec-2 будет принимать аудиоданные, и пользователь хочет воспроизвести этот звук через codec-3 без привлечения центрального процессора.Этот вышеупомянутый случай является идеальным случаем, когда следует использовать соединение

кодек-кодек. Ваша dai link должна отображаться как показано ниже в вашем машинном файле: / *

```
* этот поток pcm поддерживает только 2 4 бита, 2 канала и
частоту дискретизации 4 8 кб.
*/
статическая постоянная структура snd_soc_pcm_stream dsp_codec_params = {
    .formats = SNDRV_PCM_FMTBIT_S 2 4_LE,
    .rate_min = 4 8 0 0 0 ,
    .rate_max = 4 8 0 0 0 ,
    .channels_min = 2 ,
    .channels_max = 2 ,
};
```

```

{
    .name = "CPU-DSP", .stream_name
    = "CPU-DSP", .cpu_dai_name
    = "samsung-i 2 s. 0 ", .codec_name =
    "codec- 2 ", .codec_dai_name =
    "codec- 2 -dai_name", .platform_name =
    "samsung-i 2 s. 0 ", .dai_fmt =
    SND_SOC_DAIFMT_I 2 S | SND_SOC_DAIFMT_NB_NF
        | SND_SOC_DAIFMT_CBM_CFM,
    .ignore_suspend = 1,
    .params = &dsp_codec_params,
},
{
    .name = "DSP-CODEC", .stream_name
    = "DSP-CODEC", .cpu_dai_name
    = "wm 0 0 1 0 -sdi 2 ", .codec_name =
    "codec- 3 ", .codec_dai_name =
    "codec- 3 -dai_name", .dai_fmt =
    SND_SOC_DAIFMT_I 2 S | SND_SOC_DAIFMT_NB_NF
        | SND_SOC_DAIFMT_CBM_CFM,
    .ignore_suspend = 1,
    .params = &dsp_codec_params,
},
},

```

Приведенный выше фрагмент кода взят из sound/soc/samsung/speyside.c.

Обратите внимание на обратный вызов "params", который сообщает daprм, что эта dai_link является соединением кодек-кодек. В ядре daprм создается маршрут между виджетом воспроизведения cpu_dai и виджетом захвата codec_dai для воспроизведения- обратный путь, и наоборот, верно для пути захвата. Для того, чтобы сработал этот вышеупомянутый маршрут, DAPM необходимо найти допустимую конечную точку, которая может быть либо виджетом приемника, либо виджетом источника, соответствующим пути воспроизведения и захвата соответственно. Для того чтобы вызвать этот dai_link виджет, тонкий кодек драйвер для спикер усилитель может быть создан как демон- начав в wm8727.файл с, устанавливает определенные ограничения на устройство, даже если он не нуждается в контроле. Обязательно укажите соответствующие процессору и кодеку имена dai воспроизведения и захвата, заканчивающиеся на "Play- back" и "Capture" соответственно, поскольку ядро daprм будет связывать и питать эти dai на основе названия. Обратите внимание, что в текущем дереве устройств нет способа пометить dai_link как кодек к кодеку. Однако это может измениться в будущем.

РАСШИРЕННАЯ ЗВУКОВАЯ АРХИТЕКТУРА LINUX - ДРАЙВЕР РУКОВОДСТВО ПО НАСТРОЙКЕ

Конфигурация ядра 4.1

Чтобы включить поддержку ALSA, вам необходимо, по крайней мере, собрать ядро с поддержкой основной звуковой карты (`CONFIG_SOUND`). Поскольку ALSA может эмулировать OSS, вам не нужно выбирать какой-либо из модулей OSS.

Включите "Эмуляцию OSS API" (`CONFIG_SND_OSSEMUL`) и поддерживает как OSS mixer, так и PCM, если вы хотите запускать приложения OSS с помощью ALSA.

Если вы хотите поддерживать функциональность WaveTable на таких картах, как SB Live! затем вам нужно включить "Поддержку секвенсора" (`CONFIG_SND_SEQUENCER`).

Чтобы сделать отладочные сообщения ALSA более подробными, включите опции "Verbose printk" и "Debug". Чтобы проверить на наличие утечек памяти, включите также "Отлаживать память". "Обнаружение отладки" добавит проверки на обнаружение карт памяти. Пожалуйста, обратите внимание, что все драйверы ALSA ISA поддерживают Linux isapnp API (если карта поддерживает ISA PnP). Вам не нужно настраивать карты с помощью isapnptools.

4.2 Параметры модуля

Пользователь может загружать модули с опциями. Если модуль поддерживает более одной карты и у вас имеется более одной карты одного типа, то вы можете указать несколько значений для параметра через запятую.

4.2.1 SND модуля.

Основной модуль ALSA. Он используется всеми драйверами карт ALSA. Он использует следующие опции, которые имеют глобальные эффекты.

основные основной номер для звукового драйвера; По умолчанию: 116

cards_limit ограничение индекса карт для автоматической загрузки (1-8); По умолчанию: 1; Для автоматической загрузки более одной карты, укажите этот параметр вместе с псевдонимами snd-card-X.

слоты Зарезервируйте индекс слота для данного драйвера; Для этого параметра требуется несколько строк. Подробности см. в разделе [Автоматическая загрузка модуля-поддержка редактирования](#).

отладка Определяет уровень отладочных сообщений; (0 = отключить отладочные печати, 1 = обычные отладочные сообщения, 2 = подробные отладочные сообщения); Этот параметр появляется только тогда, когда `CONFIG_SND_DEBUG=` `y`. Этот параметр может быть динамически изменен через файл `sysfs /sys/modules/snd/parameters/debug`.

4.2.2 Модуль snd-pcm-oss.

Модуль эмуляции PCM OSS. Этот модуль использует опции, которые изменяют отображение устройств.

dsp_map Карты номеров устройств PCM, присвоенных 1-му устройству OSS; По умолчанию:

0 **adsp_map** карты номеров устройств PCM, присвоенных 2-му устройству OSS; по умолчанию:

1 **nonblock_open** Не блокировать открытие занятых устройств PCM; По умолчанию: 1

Например, когда `dsp_map= 2 , /dev/dsp` будет сопоставлен с PCM #2 карты #0. Аналогично, когда `adsp_map= 0 , /dev/adsp` будет сопоставлен PCM #0 карты #0. Для изменения второй или более поздней карты, укажите параметр через запятую, например `dsp_map= 0 , 1`.

`nonblock_open` опция используется для изменения поведения PCM при открытии устройства. Когда этот параметр не равен нулю, открытие занятого устройства OSS PCM не будет заблокировано, а будет немедленно возвращено с EAGAIN (точно так же, как флаг O_NONBLOCK).

4.2.3 Модуль snd-rawmidi

Этот модуль использует параметры, которые изменяют отображение устройств. аналогично параметрам модуля `snd-pcm-oss`.

midi_map карты номеров MIDI-устройств, присвоенных 1-му устройству OSS; По умолчанию: 0

amidi_map карты номеров MIDI-устройств, присвоенных 2-му устройству OSS; по умолчанию: 1

4.2.4 Общие параметры для модулей звуковой карты высшего уровня

Каждый модуль звуковой карты высшего уровня может использовать следующие опции.

Указатель индекс (номер слота) звуковой карты; Значения: от 0 до 31 или отрицательные; Если неотрицательный, присвоите этот индекс
число; если отрицательное, интерпретируется как битовая маска допустимых индексов; присваивается первый свободный разрешенный индекс; По умолчанию: -1
ю идентификатор карты (идентификатор или имя); Может содержать до 15 символов; По умолчанию: тип карты; Каталог по

это имя создается в `/proc/asound/`, содержащем информацию о карте; Этот идентификатор можно использовать вместо индексного номера при идентификации карты

включить включить карту; По умолчанию: включена для карт PCI и ISA PnP

Модуль 4.2.5 snd-adlib

Модуль для карт AdLib FM.

порт порт # для чипа OPL

Этот модуль поддерживает несколько карт. Он не поддерживает автозондирование, поэтому необходимо указать порт. Для реальных карт AdLib FM это будет 0x388. Обратите внимание, что у этой карты нет поддержки PCM и микшера; только FM-синтезатор.

Убедитесь, что у вас есть `sbiload` из доступного пакета `alsa-tools` и, после загрузки модуля, узнайте присвоенный номер порта ALSA sequencer через `sbiload -l`.

Пример вывода:

Имя клиента порта, название порта

6 4 : 0 OPL 2 FM-синтезатор, OPL 2 FM-порт

Загрузите `std.sb` и `drums.sb` исправления также поставляются компанией `sbiload`:

общая нагрузка - 6 4 : 0 std.sb drums.sb

Если вы используете этот драйвер для управления OPL3, вы можете использовать `std.o 3` и `drums.o 3` вместо этого. Чтобы карта воспроизводила звук, используйте `aplaymidi` от `alsa-утилиты`:

```
aplaymidi -p 64:0 foo.mid
```

Модуль 4.2.6 snd-ad1816a

Модуль для звуковых карт на базе микросхем Analog Devices AD1816A/AD1815

ISA. **тактовая частота** Тактовая частота чипа AD1816A (по умолчанию = 0,33000

Гц) Этот модуль поддерживает несколько карт, автозондирование и PnP.

4.2.7 Модуль snd-ad1848

Модуль для звуковых карт на базе чипов AD1848/AD1847/CS4248

ISA. **порт** номер порта

для чипа AD1848 **irq** IRQ # для чипа AD1848

dma1 DMA # для чипа AD1848 (0,1,3)

Этот модуль поддерживает несколько карт. Он не поддерживает

автозапуск, поэтому необходимо указать основной порт !!!

Другие порты необязательны. Поддерживается управление питанием.

4.2.8 Модуль snd-ad1889

Модуль для аналоговых устройств на микросхемах AD1889.

ac97_quirk Обходной путь AC'97 для странного оборудования; Подробнее смотрите

в описании модуля intel8x0. Этот модуль поддерживает несколько карт.

4.2.9 Модуль snd-ali5451

Модуль для чипа ALi M5451 PCI.

pcm_channels Количество аппаратных каналов, назначенных

для PCM **spdif** Поддержка ввода-вывода SPDIF; По умолчанию:

отключено Этот модуль поддерживает один чип и

автозондирование. Поддерживается управление питанием.

4.2.10 Модуль snd-als100

Модуль для звуковых карт на базе чипов Avance Logic

ALS100/ALS120 ISA. Этот модуль поддерживает несколько карт, автозондирование

и PnP. Поддерживается управление питанием.

4.2.11 Модуль snd-als300

Модуль для Avance Logic ALS300 и ALS300 +

Этот модуль поддерживает несколько плат.

Поддерживается управление питанием.

4.2.12 Модуль snd-als4000

Модуль для звуковых карт на базе чипа Avance Logic ALS4000 PCI.

joystick_port порт # для поддержки устаревшего джойстика; 0 = отключен (по умолчанию), 1 = автоматическое определение Этот модуль поддерживает несколько карт, автоматический поиск и PnP. Поддерживается управление питанием.

4.2.13 Модуль snd-asihpi

Модуль для звуковых карт AudioScience ASI

enable_hpi_hwdep включить HPI hwdep для звуковой карты AudioScience

Этот модуль поддерживает несколько плат. Драйвер требует поддержки загрузчика прошивки в ядре.

Модуль snd 4.2.14-atiixp

Модуль для контроллеров ATI IXP 150/200/250/400 AC97.

ac97_clock Часы AC'97 (по умолчанию = 48000)

ac97_quirk Обходной путь AC '97 для странного оборудования; Смотрите *Вариант AC97 Quirk* раздел ниже.

ac97_codec Обходной путь, позволяющий указать, какой кодек AC'97, вместо проверки. Если у вас это работает, отправьте сообщение об ошибке

с выводом `/spci -vn` . (-2 = Принудительное тестирование, -1 = Поведение по умолчанию, 0-2 = использование указанного кодека.)

spdif_aclink Передача

S / PDIF по AC-link (по умолчанию = 1) Этот модуль поддерживает одну карту и автоматический поиск.

ATI IXP использует два разных метода управления выходом SPDIF. Один - через AC-link, а другой - через

"прямой" выход SPDIF. Реализация зависит от материнской платы, и вам нужно будет выбрать

правильную с помощью опции модуля spdif_aclink.

Поддерживается управление питанием.

4.2.15 Модуль snd-atiixp-modem

Модуль для контроллеров модемов ATI IXP 150/200/250 AC97.

Этот модуль поддерживает одну карту и автоматический поиск.

Примечание: Значение индекса по умолчанию для этого модуля равно -2,

т.е. Первый слот исключен. Поддерживается управление питанием.

4.2.16 Модуль snd-au8810, snd-au8820, snd-au8830

Модуль для устройств Aureal Vortex, Vortex2 и Advantage.

pcifix Управляйте обходными путями PCI; 0 = Отключите все обходные пути, 1 = Увеличьте задержку PCI карты Aureal

к 0xff, 2 = Принудительно расширить внутренний мастер PCI # 2 для эффективной обработки фиктивных запросов на

ЧЕРЕЗ мост KT133 AGP, 3 = Принудительно выполнить обе настройки, 255 = автоматически определить, что требуется (по умолчанию) Этот модуль поддерживает все каналы ADB PCM, микшер ac97, SPDIF, аппаратный эквалайзер, три401, gameport.

Поддержка A3D и wavetable все еще находится в разработке. Разработка и обратный инжиниринг координируются

на <http://savannah.nongnu.org/проекты/openvortex/> Выходные данные SPDIF содержат копию выходных данных AC97

кодека, если только вы не используете `spdif_pcm`-устройство, которое позволяет передавать необработанные данные.

Аппаратный эквалайзер аппаратное обеспечение и SPDIF присутствуют только в Vortex2 и Advantage.

Примечание: Некоторые приложения ALSA mixer неправильно управляют частотой дискретизации SPDIF. Если у вас возникли проблемы с этим, попробуйте другой микшер, совместимый с ALSA (alsamixer работает).

4.2.17 Модуль snd-azt1605

Модуль для звуковых карт Aztech Sound Galaxy на базе чипсета Aztech AZT1605.

порт номер порта для базы (0x220, 0x240, 0x260, 0x280)

wss_port номер порта для WSS (0x530, 0x604, 0xe80, 0xf40)

irq Номер IRQ для WSS (7,9,10,11)

dma1 DMA # для воспроизведения WSS (0,1,3)

dma2 DMA # для записи WSS (0,1), -1 = отключено (по умолчанию)

mpu_port номер порта для MPU-401 UART (0x300, 0x330), -1 = отключен (по

умолчанию) **mpu_irq** IRQ # для MPU-401 UART (3,5,7,9), -1 = отключено (по

умолчанию) **fm_port** номер порта для OPL3 (0x388), -1 = отключено (по умолчанию)

Этот модуль поддерживает несколько карт. Он не поддерживает автоматический поиск: `порт`, `wss_port`, `irq` и `dma 1` должны быть указаны. Остальные значения необязательны.

`порт` должно соответствовать переключке БАЗОВОГО АДРЕСА на карте (0x220 или 0x240) или значению, хранящемуся в EEPROM карты для карт, у которых есть EEPROM и для их переключки "CONFIG MODE" установлено значение "EEPROM SETTING". Остальные значения можно свободно выбирать из перечисленных выше опций. Если указано

значение `dma 2`, отличное от значения `dma 1`, карта будет работать в полнодуплексном режиме. Когда `dma 1 = 3`, только

`dma 2 = 0` допустимо и является единственным способом включить захват, поскольку для захвата доступны только каналы

0 и 1. Общие настройки следующие: `порт = 0x220`, `wss_port=0x530`, `irq= 10`, `dma 1= 1`, `dma 2= 0`, `mpu_port=0x330`

`mpu_irq= 9`, `fm_port=0x388`.

Какие бы каналы IRQ и DMA вы ни выбрали, обязательно зарезервируйте их для устаревшего ISA в вашем BIOS.

4.2.18 Модуль snd-azt2316

Модуль для звуковых карт Aztech Sound Galaxy на базе чипсета Aztech AZT2316.

порт номер порта для базы (0x220, 0x240, 0x260, 0x280)

wss_port номер порта для WSS (0x530, 0x604, 0xe80, 0xf40)

irq Номер IRQ для WSS (7,9,10,11)

dma1 DMA # для воспроизведения WSS (0,1,3)

dma2 DMA # для записи WSS (0,1), -1 = отключено (по умолчанию)

mpu_port номер порта для MPU-401 UART (0x300, 0x330), -1 = отключен (по

умолчанию) **mpu_irq** IRQ # для MPU-401 UART (5,7,9,10), -1 = отключено (по

умолчанию) **fm_port** номер порта для OPL3 (0x388), -1 = отключено (по умолчанию)

Этот модуль поддерживает несколько карт. Он не поддерживает автоматический поиск: `порт`, `wss_port`, `irq` и `dma 1` должны быть указаны. Остальные значения необязательны.

`порт` должно соответствовать переключке БАЗОВОГО АДРЕСА на карте (0x220 или 0x240) или значению, хранящемуся в EEPROM карты для карт, у которых есть EEPROM и для их переключки "CONFIG MODE" установлено значение "EEPROM SETTING". Другие значения могут быть свободно выбраны из перечисленных выше опций. Если

указано значение `dma 2`, отличное от значения `dma 1`, карта будет работать в полнодуплексном режиме. Только при `dma 1 = 3`

`dma 2 = 0` допустимый и единственный способ включить захват, поскольку для захвата доступны только каналы 0 и 1.

Общие сведения Настройки являются порт = 0 x 2 2 0 wss_port= 0 x 5 3 0 irq = 1 0 dma 1 = 1 dma 2 = 0 mpu_port= 0 x 3 3 0
mpu_irq= 9 fm_port= 0 x 3 8 8 .

Какие бы каналы IRQ и DMA вы ни выбрали, обязательно зарезервируйте их для устаревшего ISA в вашем BIOS.

4.2.19 Модуль snd-aw2

Модуль для звуковой карты Audiowerk2
Этот модуль поддерживает несколько карт.

4.2.20 Модуль snd-azt2320

Модуль для звуковых карт на базе чипа Aztech System AZT2320
ISA (только PnP). Этот модуль поддерживает несколько карт,
PnP и автозондирование. Поддерживается управление питанием.

4.2.21 Модуль snd-azt3328

Модуль для звуковых карт на базе чипа Aztech AZF3328
PCI. **джойстик** Включить джойстик (по умолчанию
выключен) Этот модуль поддерживает несколько карт.

4.2.22 Модуль snd-bt87x

Модуль для видеокарт на базе чипов Bt87x.

digital_rate Переопределите частоту дискретизации по
умолчанию (Гц) **load_all** Загрузите драйвер, даже если модель
карты неизвестна Этот модуль поддерживает несколько карт.

Примечание: Значение индекса по умолчанию для этого модуля равно -2, т.е. Первый слот исключен.

4.2.23 Модуль snd-ca0106

Модуль для Creative Audigy LS и SB Live 24bit
Этот модуль поддерживает несколько карт.

4.2.24 Модуль snd-cmi8330

Модуль для звуковых карт на базе микросхем C-Media CMI8330
ISA. **isapnp** Обнаружение ISA PnP - 0 = отключить, 1 = включить (по
умолчанию) с помощью `isapnp= 0` , доступны следующие опции:
wssport номер порта для микросхемы CMI8330 (WSS)
wssirq IRQ # для чипа CMI8330 (WSS)
wssdma первый DMA # для чипа CMI8330 (WSS)
sbport номер порта для микросхемы CMI8330 (SB16)
sbirq Номер IRQ для микросхемы CMI8330 (SB16)

sbdma8 8-битный DMA # для чипа CMI8330 (SB16)

sbdma16 16-битный DMA # для чипа CMI8330

(SB16) **fmport** (опционально) порт ввода-вывода OPL3

mpuport (опционально) порт ввода-вывода MPU401

mpuirq (необязательно) MPU401 irq # Этот модуль

поддерживает несколько карт и автоматический

поиск. Поддерживается управление питанием.

Модуль 4.2.25 **snd-cmipci**

Модуль для C-Media CMI8338/8738/8768/8770 Звуковые карты PCI.

mpu_port адрес порта MIDI-интерфейса (только 8338): 0x300, 0x310, 0x320, 0x330 = устаревший порт, 0

=

отключить (по умолчанию)

fm_port адрес порта FM-синтезатора OPL-3 (только 8x38): 0x388 = устаревший порт, 1 = встроенный порт PCI

(по умолчанию 8738), 0 =

отключить **soft_ac3** Программное обеспечение для преобразования необработанных пакетов SPDIF (только модель 033) (по умолчанию = 1)

joystick_port Адрес

порта джойстика (0 = отключить, 1 = автоматически определять) Этот модуль поддерживает автоматический поиск и несколько карт памяти. Поддерживается управление питанием.

4.2.26 Модуль **snd-cs4231**

Модуль для звуковых карт на базе микросхем CS4231 ISA.

порт номер порта для микросхемы CS4231

mpu_port порт # для MPU-401 UART (опционально),

-1 = отключить **irq** IRQ # для чипа

CS4231 **mpu_irq** IRQ # для MPU-401 UART

dma1 первый DMA # для чипа CS4231

dma2 второй DMA # для чипа CS4231

Этот модуль поддерживает несколько карт. Этот модуль не поддерживает

автоматический поиск, поэтому должен быть указан основной порт!!!

Другие порты необязательны. Поддерживается управление питанием.

4.2.27 Модуль **snd-cs4236**

Модуль для звуковых карт на базе CS4232/CS4232A,

CS4235/CS4236/CS4236B/CS4237B/CS4238B/CS4239 микросхем ISA.

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию)

при **isapnp**=0 доступны следующие опции:

порт номер порта для микросхемы CS4236 (настройка PnP - 0x534)

спорт порт управления # для микросхемы CS4236 (настройка PnP - 0x120, 0x210,

0xf00) **mpu_port** номер порта для MPU-401 UART (настройка PnP - 0x300), -1 = отключить

fm_port Номер FM-порта для чипа CS4236 (настройка PnP - 0x388),

-1 = отключить **irq** IRQ # для чипа CS4236 (5,7,9,11,12,15)

mpu_irq IRQ # для MPU-401 UART (9,11,12,15)

dma1 первый DMA # для чипа CS4236 (0,1,3)

dma2 второй номер DMA для чипа CS4236 (0,1,3), -1 = отключить

Этот модуль поддерживает несколько карт. Этот модуль не поддерживает автоматический поиск (если не используется ISA PnP), поэтому необходимо указать основной порт и порт управления!!! Другие порты необязательны. Поддерживается управление питанием. Этот модуль имеет псевдоним snd-cs4232, поскольку он также обеспечивает функциональность старого snd-cs4232.

4.2.28 Модуль snd-cs4281

Модуль для звукового чипа Cirrus Logic CS4281.

dual_codec Идентификатор дополнительного кодека (0 = отключен, по умолчанию) Этот модуль поддерживает несколько карт памяти. Поддерживается управление питанием.

4.2.29 Модуль snd-cs46xx

Модуль для звуковых карт PCI на базе чипов PCI

CS4610/CS4612/CS4614/CS4615/CS4622/CS4624/CS4630/CS4280 .

external_amp Принудительно включить внешний усилитель.

thinkpad Принудительно включить управление CLKRUN на Thinkpad.

mmap_valid Поддержка режима OSS mmap (по умолчанию = 0). Этот модуль поддерживает несколько карт и автоматический поиск. Обычно внешние элементы управления amp и CLKRUN определяются автоматически из идентификаторов субпорядчика PCI / устройства. Если они не работают, явно укажите указанные выше параметры. Поддерживается управление питанием.

4.2.30 Модуль snd-cs5530

Модуль для чипа Cyrix/NatSemi Geode 5530.

4.2.31 Модуль snd-cs5535audio

Модуль для многофункционального сопутствующего PCI-устройства CS5535 Поддерживается управление питанием.

4.2.32 Модуль snd-ctxfi

Модуль для плат Creative Sound Blaster X-Fi (чипы 20k1 / 20k2)

- Creative Sound Blaster X-Fi Titanium Fatal1ty
- Champion Series • Creative Sound Blaster X-Fi Titanium Fatal1ty
- Professional Series • Creative Sound Blaster X-Fi Titanium
- Professional Audio • Creative Sound Blaster X-Fi Titanium

- Creative Sound Blaster X-Fi Elite Pro
- Creative Sound Blaster X-Fi Platinum
- Creative Sound Blaster X-Fi Fatal1ty •
- Creative Sound Blaster X-Fi XtremeGamer •
- Creative Sound Blaster X-Fi XtremeMusic

reference_rate базовая частота дискретизации, 44100 или 48000 (по умолчанию)

кратнократно для ссылки на частоту дискретизации, 1 или 2 (по умолчанию)

подсистема переопределите SSID PCI для проверки; значение состоит из SSVID << 16 | SSDID. Значение по умолчанию равно

нулю, что означает отсутствие переопределения.

Этот модуль поддерживает несколько карт.

4.2.33 Модуль snd-darla20

Модуль для Echoaudio Darla20

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного программного обеспечения в ядре.

4.2.34 Модуль snd-darla24

Модуль для Echoaudio Darla24

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного программного обеспечения в ядре.

4.2.35 Модуль snd-dt019x

Модуль для Diamond Technologies DT-019X / Avance Logic ALS-007 (только PnP)

Этот модуль поддерживает несколько карт. Этот модуль включен только с поддержкой ISA PnP. Поддерживается управление питанием.

4.2.36 Модуль snd-фиктивный

Модуль для фиктивной звуковой карты. Эта "карта" не выполняет никакого вывода или ввода, но вы можете использовать этот модуль для любого приложения, которому требуется звуковая карта (например,

RealPlayer). **pcm_devs** Количество устройств PCM, назначенных каждой карте (по умолчанию = 1, максимум 4)

pcm_substreams Количество подпотоков PCM, назначенных каждому PCM (по умолчанию = 8, до 128)

hrtimer Используйте hrtimer (=1, по умолчанию) или системный таймер (=0)

fake_buffer Поддельные выделения буфера (по умолчанию = 1)

При создании нескольких устройств PCM snd-dummy обеспечивает различное поведение каждого устройства

PCM: * 0 = чередование с поддержкой mmap * 1 = без чередования с поддержкой mmap * 2 = чередование

без mmap * 3 = без чередования без mmap По

умолчанию драйверы snd-dummy не выделяют реальные буферы, а либо игнорируют чтение / запись, либо

mmap одну фиктивную страницу для всех страниц буфера в целях экономии ресурсов. Если вашим

приложениям требуется согласованность данных для чтения / записи в буфере, передайте параметр

fake_buffer=0. Поддерживается управление питанием.

4.2.37 Модуль snd-echo3g

Модуль для карт Echoaudio 3G (Gina3G / Layla3G)

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика прошивки в ядре.

4.2.38 Модуль snd-emu10k1

Модуль для звуковых карт PCI на базе EMU10K1 / EMU10k2.

- Sound Blaster в действии!
- Sound Blaster PCI 512
- Точки доступа Emu (частично поддерживаются)
- Sound Blaster Audigy

extin растровая карта доступных внешних входов для FX8010 (см. Ниже)

расширение растровая карта доступных внешних выходов для FX8010 (см. Ниже)

seq_ports выделенные порты секвенсора (по умолчанию 4)

max_synth_voices ограничение на количество голосов, используемых для wavetable (по умолчанию 64)

max_buffer_size задает максимальный размер буферов wavetable / рсм в единицах измерения МБ. Значение по умолчанию равно 128.

enable_ir включить IR

Этот модуль поддерживает несколько карт и автозондирование.

Ввод & Конфигурации вывода [extin / extout] * Творческая карта с цифровым выходом [0x0003 / 0x1f03] * Творческая карта с цифровым выходом [0x0003 / 0x1f0f] * Творческая карта с цифровым CD входом [0x000f / 0x1f0f] * Творческая карта с цифровым выходом + LiveDrive [0x3fc3 / 0x1fc3] * Творческая карта с цифровым выходом + LiveDrive [0x3fc3 / 0x1fcf] * Творческая карта с цифровым CD входом + LiveDrive [0x3fcf/0x1fcf] * Творческая карта с цифровым выходом + Цифровой ввод-вывод 2 [0x0fc3/0x1f0f] * Творческая карта с цифровым выходом + цифровой ввод-вывод 2 [0x0fc3/0x1f0f] * Творческая карта с цифровым CD входом + цифровой ввод-вывод 2 [0x0fcf/0x1f0f] * Творческая карта 5.1 / с цифровым выходом + LiveDrive [0x3fc3/0x1fff] * Творческая карта 5.1 (с) 2003 [0x3fc3 / 0x7cff]

* Креативная карта со всеми входами и выходами [0x3fff/0x7fff] Управление питанием поддерживается.

4.2.39 Модуль snd-emu10k1x

Модуль для Creative Emu10k1X (ОЕМ-версия SB

Live Dell) Этот модуль поддерживает несколько карт.

4.2.40 Модуль snd-ens1370

Модуль для звуковых карт Ensoniq AudioPCI ES1370 PCI.

- SoundBlaster PCI 64
- SoundBlaster PCI 128

джойстик Включить джойстик (по умолчанию выключен)

Этот модуль поддерживает несколько карт и

автоматический поиск. Поддерживается управление питанием.

Модуль 4.2.41 snd-ens1371

Модуль для звуковых карт Ensoniq AudioPCI ES1371 PCI.

- SoundBlaster PCI 64
- SoundBlaster PCI 128
- SoundBlaster Vibra PCI

джойстик_порт номер порта для джойстика (0x200, 0x208, 0x210, 0x218), 0 = отключить (по умолчанию), 1 = автоматическое определение Этот модуль поддерживает несколько карт и автоматический поиск. Поддерживается управление питанием.

Модуль 4.2.42 snd-es1688

Модуль для звуковых карт ESS AudioDrive ES-1688 и ES-688.

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию)

mpu_port номер порта для порта MPU-401 (0x300, 0x310, 0x320, 0x330),

-1 = отключить (по умолчанию) **mpu_irq** IRQ # для порта MPU-401 (5,7,9,10)

fm_port порт # для OPL3 (опция; использовать тот же порт, что и по

умолчанию) с `isapnp=0`, доступны следующие дополнительные опции:

порт номер порта для микросхемы ES-1688 (0x220, 0x240, 0x260)

irq IRQ # для микросхемы ES-1688 (5,7,9,10)

dma8 DMA # для микросхемы ES-1688 (0,1,3)

Этот модуль поддерживает несколько карт и автоматический поиск (без порта MPU-401) и PnP с чипом ES968.

4.2.43 Модуль snd-es18xx

Модуль для звуковых карт ESS AudioDrive ES-18xx.

isapnp Обнаружение ISA PnP - 0 = отключить, 1 = включить (по умолчанию)

с помощью `isapnp=0`, доступны следующие опции:

порт номер порта для микросхемы ES-18xx (0x220, 0x240, 0x260)

mpu_port номер порта для порта MPU-401 (0x300, 0x310, 0x320,

0x330), -1 = отключить (по умолчанию) **fm_port** порт # для FM

(опционально, не используется) **irq** IRQ # для микросхемы ES-18xx

(5,7,9,10) **dma1** первый номер DMA для микросхемы ES-18xx

(0,1,3) **dma2** первый номер DMA для микросхемы ES-18xx (0,1,3)

Этот модуль поддерживает несколько карт, ISA PnP и автоматический поиск (без порта MPU-401, если не используются собственные процедуры ISA PnP). Когда `dma 2` равно `dma 1` драйвер работает в режиме полудуплекса. Поддерживается управление питанием.

Модуль 4.2.44 snd-es1938

Модуль для звуковых карт на базе микросхем ESS Solo-1 (ES1938, ES1946). Этот модуль поддерживает несколько карт и автозондирование. Поддерживается управление питанием.

Модуль 4.2.45 snd-es1968

Модуль для звуковых карт на базе микросхем ESS Maestro-1/2/2E (ES1968/ES1978). **total_bufsize** общий размер буфера в кБ (1-4096 КБ)
pcm_substreams_p каналы воспроизведения (1-8, по умолчанию =2)
pcm_substreams_c каналы захвата (1-8, по умолчанию = 0)
clock clock (0 = автоматическое определение)
use_pm поддержка управления питанием (0 = выкл., 1 = включено, 2 = авто (по умолчанию)) **enable_mpu** включить MPU401 (0 = выкл., 1 = включено, 2 = авто (по умолчанию)) **джойстик** включить джойстик (по умолчанию выключен) Этот модуль поддерживает несколько карт и автоматический поиск. Поддерживается управление питанием.

Модуль 4.2.46 snd-fm801

Модуль для звуковых карт PCI на базе ForteMedia FM801.
tea575x_tuner Включить тюнер TEA575x; 1 = MediaForte 256-PCS, 2 = MediaForte 256-PCPR, 3 = Medi-
Старшие 16 бит 64-PCR - это номер видео (радио)
устройства + 1; пример: 0x10002 (MediaForte 256-PCPR,
устройство 1) Этот модуль поддерживает несколько карт и автоматический поиск. Поддерживается управление питанием.

4.2.47 Модуль snd-gina20

Модуль для Echoaudio Gina20
Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного программного обеспечения в ядре.

4.2.48 Модуль snd-gina24

Модуль для Echoaudio Gina24
Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного программного обеспечения в ядре.

4.2.49 Модуль snd-gusclassic

Модуль для звуковой карты Gravis UltraSound Classic.
порт номер порта для микросхемы GF1 (0x220, 0x230, 0x240, 0x250, 0x260) **irq** IRQ # для чипа GF1 (3,5,9,11,12,15)
dma1 DMA # для чипа GF1 (1,3,5,6,7)

dma2 DMA # для микросхемы GF1 (1,3,5,6,7,-1 = отключить)

joystick_dac от 0 до 31, (0,59 В-4,52 В или 0,389 В-2,98 В)

количество голосов ограничение по количеству голосов GF1

(14-32) **pcm_voices** зарезервированные голоса PCM Этот модуль поддерживает несколько карт и автоматический поиск.

4.2.50 Модуль snd-gusextreme

Модуль для звуковой карты Gravis UltraSound Extreme (Synergy ViperMax).

порт номер порта для микросхемы ES-1688 (0x220, 0x230, 0x240, 0x250, 0x260)

gf1_port номер порта для микросхемы GF1 (0x210, 0x220, 0x230, 0x240, 0x250, 0x260, 0x270) **mpu_port** номер порта для порта MPU-401 (0x300, 0x310, 0x320, 0x330), -1 = отключить **irq** IRQ # для чипа ES-1688 (5,7,9,10)

gf1_irq IRQ # для чипа GF1 (3,5,9,11,12,15)

mpu_irq IRQ # для порта MPU-401 (5,7,9,10)

dma8 DMA # для микросхемы ES-1688 (0,1,3)

dma1 DMA # для микросхемы GF1 (1,3,5,6,7)

joystick_dac от 0 до 31, (0,59 В-4,52 В или 0,389 В-2,98 В)

количество голосов ограничение по количеству голосов GF1 (14-32)

pcm_voices зарезервированные голоса PCM Этот модуль поддерживает несколько карт и автоматический поиск (без порта MPU-401).

4.2.51 Модуль snd-gusmax

Модуль для звуковой карты Gravis UltraSound MAX.

порт номер порта для микросхемы GF1 (0x220, 0x230, 0x240, 0x250, 0x260) **irq** IRQ # для чипа GF1 (3,5,9,11,12,15)

dma1 DMA # для микросхемы GF1 (1,3,5,6,7)

dma2 DMA # для микросхемы GF1 (1,3,5,6,7,-1 = отключить)

joystick_dac от 0 до 31, (0,59 В-4,52 В или 0,389 В-2,98 В)

количество голосов ограничение по количеству голосов GF1

(14-32) **pcm_voices** зарезервированные голоса PCM Этот модуль поддерживает несколько карт и автоматический поиск.

4.2.52 Модуль snd-hda-intel

Модуль для Intel HD Audio (ICH6, ICH6M, ESB2, ICH7, ICH8, ICH9, ICH10, PCH, SCH), ATI SB450, SB600, R600, RS600, RS690, RS780, RV610, RV620, RV630, RV635, RV670, RV770, VIA VT8251 / VT8237A, SIS966, ULI M5461 [Несколько вариантов для каждого экземпляра карты.] **Модель** принудительно введите название модели

position_fix Исправить указатель DMA; -1 = системное значение по умолчанию: выберите подходящий для каждого оборудования контроллера, 0 = auto: возвращается к LPIB, когда POSBUF не работает, 1 = использовать LPIB, 2 = POSBUF: использовать позиционный буфер, 3 = VIACOMBO: специфичный для VIA обходной путь для захвата, 4 = COMBO: использовать LPIB для воспроизведения, auto для захвата потока

пробная маска Битовая маска для проверки кодеков (по умолчанию = -1, что означает все слоты); Когда установлен бит 8 (0x100),

младшие 8 бит используются в качестве "фиксированных" слотов кодека; т. е. Драйвер проверяет слоты независимо от того, что сообщает аппаратное обеспечение

probe_only Только проверка и никакой инициализации кодека (по умолчанию = выключено); Полезно для проверки начального состояния кодека для отладки

bdl_pos_adj Определяет временную задержку DMA IRQ в выборках. Передача значения -1 заставит драйвер выбрать соответствующее значение в зависимости от микросхемы контроллера.

исправление Указывает файлы ранних "исправлений" для изменения настроек HD-audio перед инициализацией кодеков. Этот параметр доступен только в том случае, если установлен параметр `CONFIG_SND_HDA_PATCH_LOADER=y`. Сначала смотрите hd-audio/примечания. подробности.

beep_mode Выбирает режим регистрации звукового сигнала (0 = выкл., 1 = вкл.); значение по умолчанию устанавливается с помощью `CONFIG_SND_HDA_INPUT_BEEP_MODE` в `kconfig`.

[Одиночные (глобальные) параметры]

single_cmd Используйте отдельные команды `immediate` для взаимодействия с кодеками (только для отладки)

enable_msi Включить прерывание с сигналом сообщения (MSI) (по умолчанию = выкл.)

power_save Автоматический тайм-аут энергосбережения (в секунду, 0 = отключить)

power_save_controller Переведите HD-аудиоконтроллер в режим энергосбережения (по умолчанию = включено)

align_buffer_size Принудительно округляйте размеры буфера / периода до значений, кратных 128 байтам. Это более эффективно с точки зрения доступа к памяти, но не требуется спецификацией HDA и не позволяет пользователям указывать точные размеры периода / буфера. (по умолчанию = включено)

отслеживание Включить / отключить отслеживание (по умолчанию = включено)

Этот модуль поддерживает несколько карт и автоматический поиск.

Смотрите hd-audio/примечания.rst для получения дополнительной информации о драйвере HD-audio.

У каждого кодека может быть таблица моделей для разных конфигураций. Если вашей машины там нет в списке, то устанавливается конфигурация по умолчанию (обычно минимальная). Вы можете передать `модель=<название>` возможность указать определенную модель в таком случае. Существуют разные модели в зависимости от чипа кодека. Список доступных моделей находится в разделе hd-audio/models.rst.

Название модели `generic` рассматривается как особый случай. Когда задана эта модель, драйвер использует универсальный анализатор кодеков без "codecs-patch". Иногда это полезно для тестирования и отладки.

Если конфигурация по умолчанию не работает и одно из вышеперечисленных настроек соответствует вашему устройству, сообщите об этом вместе с помощью `alsa-info.sh output` (с помощью `--no-upload` опция) для ядра bugzilla или alsa-devel ML (см. раздел [Ссылки и адреса](#)).

`power_save` и `power_save_controller` опции предназначены для режима энергосбережения. Смотрите powersave.txt для подробностей.

Примечание 2: Если при выводе появляются звуки щелчка, попробуйте опцию модуля `position_fix = 1` или `2.position_fix=1` будет использовать значение регистра SD_LPIB без коррекции размера FIFO в качестве текущего указателя DMA. `position_fix = 2` заставит драйвер использовать позиционный буфер вместо чтения регистра SD_LPIB. (Обычно регистр SD_LPIB более точен, чем позиционный буфер.)

`position_fix=3` зависит от устройств VIA. Положение потока захвата проверяется как по LPIB, так и по значениям POSBUF. `position_fix=4` представляет собой комбинированный режим, использующий LPIB для воспроизведения и POSBUF для захвата.

ПРИМЕЧАНИЕ: если вы получаете много сообщений `тайм-аут azx_get_response` сообщения при загрузке, вероятно, проблема в прерываниях (например, маршрутизация irq ACPI). Попробуйте загрузиться с такими параметрами, как `pci = noacpi`. Также вы можете попробовать `single_cmd= 1` опция модуля. Это переключит метод связи между контроллером HDA и кодеками на единый

немедленные команды вместо CORB / RIRB. По сути, режим единой команды предусмотрен только для BIOS, и вы также не получите нежелательных событий. Но, по крайней мере, это работает независимо от irq.

Помните, что это последнее средство, и его следует избегать, насколько это возможно ... ДОПОЛНИТЕЛЬНЫЕ ПРИМЕЧАНИЯ

К `тайм-аут azx_get_response` ПРОБЛЕМЫ: На некоторых аппаратных средствах вам может потребоваться добавить соответствующую опцию `probe_mask`, чтобы избежать `тайм-аута azx_get_response` вместо этого описанная выше проблема. Эта операция- выполняется, когда доступ к несуществующему или нерабочему слоту кодека (вероятно, модемному) вызывает остановку связи по шине HD-audio. Вы можете увидеть, какие слоты кодека проверяются, включив `CON-FIG_SND_DEBUG_VERBOSE`, или просто по имени файла в файлах `codec proc`. Затем ограничьте интервалы зондированием с помощью опции `probe_mask`. Например, `probe_mask= 1` означает зондирование только первого интервала, а `probe_mask= 4` означает только третий слот.

Поддерживается управление питанием.

4.2.53 Модуль snd-hdsp

Модуль для аудиоинтерфейса (ов) RME

Hammerfall DSP Этот модуль поддерживает несколько карт.

Примечание: Данные встроенного ПО могут быть автоматически загружены через hotplug, когда `CONFIG_FW_LOADER` установлен.

В остальном- вам необходимо загрузить прошивку с помощью утилиты `hdsploader`, входящей в пакет `alsa-tools`. Прошивка

данные находятся в `alsa-firmware package`. Примечание:

модуль `snd-page-alloc` выполняет работу, которую ранее выполнял модуль `snd-hammerfall-mem`. Он выделит буферы заранее, когда будут найдены какие-либо карты HDSP. Чтобы обеспечить надежное распределение буфера, загрузите модуль `snd-page-alloc` на ранней стадии последовательности загрузки. Смотрите раздел *Раннее выделение буфера* Раздел.

4.2.54 Модуль snd-hdspm

Модуль для платы RME HDSP MADI.

precise_ptr Включить точный указатель или отключить.

line_outs_monitor По умолчанию отправлять потоки воспроизведения

на аналоговые выходы. **enable_monitor** По умолчанию включить

аналоговый выход на канале 63/64. Подробнее см. `hdspm.txt`.

4.2.55 Модуль snd-ice1712

Модуль для звуковых карт PCI на базе Envy24 (ICE1712).

- MidiMan M Audio Delta
- 1010 • MidiMan M Audio
- Delta 1010LT • MidiMan M
- Audio Delta DiO 2496 •
- MidiMan M Audio Delta 66 •
- MidiMan M Audio Delta 44 •
- MidiMan M Audio Delta 410 •
- MidiMan M Audio Audiophile
- 2496 • TerraTec EWS 88MT •
- TerraTec EWS 88D • TerraTec EWX
- 24/96 • TerraTec DMX 6Fire

- Фаза 88 TerraTec •
- Саундтрек Hoontech DSP 24
- Значение DSP 24
- саундтрека Hoontech • Саундтрек
- Hoontech DSP 24 Media
- 7.1 • Event Electronics, EZ8 •
- Digigram VX442 • Lionstracs,
- Mediastaton • Terrasoniq TS 88

Модель Используйте данную модель платы, одну из следующих: delta1010, dio2496, delta66, delta44, audio-
файл, delta410, delta1010lt, vx442, ewx2496, ews88mt, ews88mt_new, ews88d, dmx6fire, dsp24,
dsp24_value, dsp24_71, ez8, phase88, mediastation
omni Поддержка ввода-вывода Omni для MidiMan M-Audio Delta44/66
cs8427_timeout время ожидания сброса для чипа CS8427 (приемопередатчика S / PDIF) с разрешением MC, значение по умолчанию
равно 500 (0,5 секунды)

Этот модуль поддерживает несколько карт и автоматический поиск. Примечание: Потребительская часть
используется не со всеми картами на базе Envy24 (например, в MidiMan Delta siree). Примечание:
Поддерживаемая плата определяется путем считывания EEPROM или SSID PCI (если EEPROM недоступна). Вы
можете переопределить модель, передав `Модель` опция модуля на случай, если драйвер настроен
неправильно или вы хотите попробовать другой тип для тестирования.

4.2.56 Модуль snd-ice1724

Модуль для звуковых карт PCI на базе Envy24HT (VT/ICE1724), Envy24PT (VT1720).

- MidiMan M Audio Revolution
- 5.1 • MidiMan M Audio
- Revolution 7.1 • MidiMan
- M Audio Audiophile 192 •
- AMP Ltd AUDIO2000 • TerraTec
- Aureon 5.1 Sky • TerraTec
- Aureon 7.1 Space • TerraTec
- Aureon 7.1 Universe • TerraTec
- Phase 22 • TerraTec Phase
- 28 • AudioTrak Prodigy
- 7.1 • AudioTrak Prodigy
- 7.1 LT • AudioTrak Prodigy
- 7.1 XT • AudioTrak Prodigy
- 7.1 HIFI • AudioTrak Prodigy
- 7.1 HD2 • AudioTrak Prodigy
- 192 • Pontis MS300
- Albatron K8X800 Pro II
- Chaintech ZNF3-150
- Chaintech ZNF3-250

- Chaintech 9CJS
- Chaintech AV-710
- Шаттл SN25P
- Onkyo SE-90PCI
- Onkyo SE-200PCI
- ESI Juli@ •
- ESI Maya44 • Hercules Fortissimo
- IV • EGO-SYS
- WaveTerminal 192 M

revo51, revo71, amp2000, prodigy71, **Модель** Используйте

данную модель платы, одну из следующих: prodigy71lt, prodigy71xt, prodigy71hifi, prodigyhd2, prodigy192, juli, aureon51, aureon71, universe, ap192, k8x800, phase22, phase28, ms300, av710, se200pci, se90pci, fortissimo4, sn25p, WT192M, maya44

Этот модуль поддерживает несколько карт и автоматический поиск. Примечание:

Поддерживаемая плата определяется путем считывания EEPROM или SSID PCI (если EEPROM недоступна). Вы можете переопределить модель, передав `Модель` опция модуля на случай, если драйвер настроен неправильно или вы хотите попробовать другой тип для тестирования.

4.2.57 Модуль snd-indigo

Модуль для Echoaudio Indigo

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика прошивки в ядре.

4.2.58 Модуль snd-indigodj

Модуль для Echoaudio Indigo DJ

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика прошивки в ядре.

4.2.59 Модуль snd-indigoio

Модуль для Echoaudio Indigo IO

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного программного обеспечения в ядре.

Модуль snd 4.2.60-intel8x0

Модуль для материнских плат AC'97 от Intel и совместимых с ними.

- Intel i810/810E, i815, i820, i830, i84x,
- MX440 ICH5, ICH6, ICH7, 6300ESB, ESB2 • SiS 7012
- (SiS 735) • NVidia nForce, nForce2,
- nForce3, MCP04, CK804, CK8, CK8S, MCP501
- AMD AMD768, AMD8111 • ALi m5455

ac97_clock База синхронизации кода AC'97 (0 = автоматическое определение)

ac97_quirk Обходной путь AC'97 для странного оборудования; Смотрите *Вариант AC97 Quirk* раздел ниже.

buggy_irq Включить обходной путь для прерываний с ошибками на некоторых материнских платах (по умолчанию да на чипах nForce,

в противном случае отключено)

buggy_semaphore Включить обходной путь для оборудования с глючащими семафорами (например, на некоторых ноутбуках ASUS tops) (по

умолчанию отключено) **spdif_aclink** Используйте S / PDIF по AC-link вместо прямого подключения от микросхемы контроллера (0 = выкл., 1 = включено, -1 = по умолчанию)

Этот модуль поддерживает один чип и автозондирование.

Примечание: последний драйвер поддерживает автоматическое определение тактовой частоты чипа. если вы по-прежнему

сталкиваетесь со слишком быстрым воспроизведением, укажите тактовую частоту явно с помощью опции модуля `ac97_clock = 41194`.

Порты джойстика / MIDI не поддерживаются этим драйвером. Если на вашей материнской плате установлены эти устройства,

используйте модули ns558 или snd-mpu401 соответственно.

Поддерживается управление питанием.

Модуль snd 4.2.61-intel8x0m

Модуль для модемов MC97 с чипсетом Intel ICH (i8x0).

• Intel i810/810E, i815, i820, i830, i84x, MX440 ICH5, ICH6, ICH7

• SiS 7013 (SiS 735)

• NVidia nForce, nForce2, NForce2s, nForce3

• AMD AMD8111

• ALi m5455

ac97_clock База синхронизации кодека AC'97 (0 = автоматическое определение)

Этот модуль поддерживает одну карту и автоматический поиск.

Примечание: Значение индекса по умолчанию для этого модуля равно -2,

т.е. Первый слот исключен. Поддерживается управление питанием.

4.2.62 Модуль snd-interwave

Модуль для ультразвуковых PnP Gravis, Dynasonic 3-D/Pro, STB Sound Rage

32 и других звуковых карт на базе чипа AMD InterWave (tm).

joystick_dac от 0 до 31, (0,59 В-4,52 В или 0,389 В-2,98 В)

midi 1 = включить MIDI UART, 0 = Отключение MIDI UART (по умолчанию)

pcm_voices зарезервированные голоса PCM для синтезатора (по умолчанию

2) **эффект** 1 = Включить межволновые эффекты (по умолчанию 0);

требуется 8 голосов **isapnp** Обнаружение ISA PnP - 0 = отключить, 1 =

включить (по умолчанию) с помощью `isapnp= 0` , доступны следующие опции:

порт номер порта для межволнового чипа (0x210, 0x220, 0x230, 0x240,

0x250, 0x260) **irq** IRQ # для межволнового чипа (3,5,9,11,12,15)

dma1 DMA # для межволнового чипа (0,1,3,5,6,7)

dma2 DMA # для микросхемы InterWave (0,1,3,5,6,7, -1 = отключить)

Этот модуль поддерживает несколько карт, автоматический поиск и ISA PnP.

4.2.63 Модуль snd-interwave-stb

Модуль для UltraSound 32-Pro (звуковая карта от STB, используемая Compaq) и других звуковых карт на базе чипа AMD InterWave (tm) со схемой TEA6330T для расширенного управления

басами, высокими частотами и основной громкостью. **joystick_dac** от 0 до 31, (0,59 В-4,52 В или

0,389 В-2,98 В) **midi** 1 = Включить MIDI UART, 0 = отключить MIDI UART (по умолчанию)

pcm_voices зарезервированные голоса PCM для синтезатора (по умолчанию 2)

эффект 1 = Включить межволновые эффекты (по умолчанию 0); требуется 8 голосов

isapnp Обнаружение ISA PnP - 0 = отключить, 1 = включить (по умолчанию)

с помощью `isapnp=0`, доступны следующие опции:

порт номер порта для микросхемы InterWave (0x210, 0x220, 0x230, 0x240, 0x250, 0x260)

порт_tc порт # управления тоном (шина i2c) для микросхемы TEA6330T (0x350, 0x360, 0x370,

0x380) **irq** IRQ # для межволнового чипа (3,5,9,11,12,15)

dma1 DMA # для межволнового чипа (0,1,3,5,6,7)

dma2 DMA # для микросхемы InterWave (0,1,3,5,6,7, -1 = отключить)

Этот модуль поддерживает несколько карт, автоматический поиск и ISA PnP.

4.2.64 Модуль snd-jazz16

Модуль для набора микросхем Media Vision Jazz16. Набор микросхем состоит из 3 микросхем:

MVD1216 + MVA416 + MVA514. **порт** номер порта для микросхемы SB DSP (0x210,

0x220, 0x230, 0x240, 0x250, 0x260) **irq** IRQ # для микросхемы SB DSP (3,5,7,9,10,15)

dma8 DMA # для микросхемы SB DSP (1,3)

dma16 DMA # для микросхемы SB DSP (5,7)

mpu_port Номер порта MPU-401 (0x300, 0x310, 0x320, 0x330)

mpu_irq MPU-401 irq # (2,3,5,7)

Этот модуль поддерживает несколько карт.

4.2.65 Модуль snd-korg1212

Модуль для платы Korg 1212 IO PCI

Этот модуль поддерживает несколько карт.

4.2.66 Модуль snd-layla20

Модуль для Echoaudio Layla20

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика прошивки в ядре.

Модуль snd 4.2.67-layla24

Модуль для Echoaudio Layla24

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного ПО в ядре.

4.2.68 Модуль snd-lola

Модуль для плат Digigram Lola PCI-e

Этот модуль поддерживает несколько плат.

4.2.69 Модуль snd-lx6464es

Модуль для плат Digigram LX6464ES

Этот модуль поддерживает несколько плат.

4.2.70 Модуль snd-maestro3

Модуль для микросхем Allegro/Maestro3

external_amp включить внешний усилитель (включен по умолчанию)

amp_gpio PIN-код GPIO для внешнего усилителя (0-15) или -1 для pin-кода по умолчанию (8

для allegro, 1 для других) Этот модуль поддерживает автоматический поиск и несколько

микросхем. Примечание: привязка усилителя зависит от оборудования. Если звук

отсутствует, хотя все каналы не отключены, попробуйте указать другое подключение gpio через опцию

amp_gpio. Например, ноутбуку Rapasonic может потребоваться опция `amp_gpio= 0 x 0 d` .

Поддерживается управление питанием.

Модуль snd-mia 4.2.71

Модуль для Echoaudio Mia

Этот модуль поддерживает несколько карт памяти. Драйвер требует поддержки загрузчика прошивки в ядре.

Модуль snd-miro 4.2.72

Модуль для звуковых карт Miro: miroSOUND PCM 1 pro,

miroSOUND PCM 12, miroSOUND PCM 20 Radio. **порт** Порт

(0x530, 0x604, 0xe80, 0xf40) **irq** IRQ # (5,7,9,10,11)

dma1 1-й dma # (0,1,3)

dma2 2-й dma # (0,1)

mpu_port Номер порта MPU-401 (0x300, 0x310, 0x320,

0x330) **mpu_irq** MPU-401 irq # (5,7,9,10)

fm_port FM-порт # (0x388)

wss включить режим WSS

ide включить встроенную поддержку ide

4.2.73 Модуль snd-mixart

Модуль для звуковых карт Digigram miXart8.

Этот модуль поддерживает несколько карт. Примечание: Одна плата miXart8 будет представлена в виде 4 карт alsa. Смотрите MIXART.txt для получения подробной информации. Когда драйвер скомпилирован в виде модуля и поддерживается прошивка hotplug, данные прошивки загружаются через hotplug автоматически. Установите необходимые файлы прошивки в alsa-firmware package. Если загрузчик fw с горячей подклкючением недоступен, вам необходимо загрузить прошивку с помощью утилиты mixartloader из пакета alsa-tools.

4.2.74 Модуль snd-mona

Модуль для Echoaudio Mona

Этот модуль поддерживает несколько карт. Драйвер требует поддержки загрузчика встроенного программного обеспечения в ядре.

4.2.75 Модуль snd-mpu401

Модуль для устройств MPU-401 UART.

порт номер порта или -1 (отключить)

irq Номер IRQ или -1 (отключить)

pnp Обнаружение PnP - 0 = отключить, 1 = включить (по умолчанию)

Этот модуль поддерживает несколько устройств и PnP.

4.2.76 Модуль snd-msnd-classic

Модуль для звуковых карт Turtle Beach MultiSound Classic, Tahiti или Monterey.

io Номер порта для карты msnd-classic

irq IRQ # для карты msnd-classic

mem Адрес памяти (0xb0000, 0xc8000, 0xd0000, 0xd8000, 0xe0000 или 0xe8000)

write_ndelay включить запись ndelay (по умолчанию = 1)

calibrate_signal сигнал калибровки (по умолчанию = 0)

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию)

цифровая Присутствует цифровая дочерняя плата (по умолчанию = 0)

cfg Порт конфигурации (0x250, 0x260 или 0x270) по умолчанию = PnP

Сброс Сброс настроек всех устройств

mpu_io Порт ввода-вывода MPU401

mpu_irq MPU401 irq#

ide_io0 IDE порт #0

ide_io1 IDE порт # 1

ide_irq IDE irq#

joystick_io Порт ввода-вывода джойстика

Драйверу требуются файлы прошивки `turtlebeach/msndinit.bin` и `turtlebeach/msndperm.bin` в соответствующем каталоге прошивки.

Смотрите Документацию / sound / oss / MultiSound для получения важной информации об этом драйвере. Обратите внимание, что его выпуск прекращен, но запись базы знаний Voyetra Turtle Beach для него по-прежнему доступна по адресу <http://www.turtlebeach.com>

4.2.77 Модуль snd-msnd-pinnacle

Модуль для мультизвуковых карт Pinnacle/Фиджи Turtle Beach.

ioport Порт # для pinnacle / карты Фиджи

irq IRQ # для pinnacle / карты Фиджи

mem Адрес памяти (0xb0000, 0xc8000, 0xd0000, 0xd8000, 0xe0000 или

0xe8000) **write_ndelay** включить запись ndelay (по умолчанию = 1)

calibrate_signal сигнал калибровки (по умолчанию = 0)

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию)

Драйверу требуются файлы встроенного ПО `turtlebeach/pndspini.bin` и `turtlebeach/pndspem.bin` в соответствующем каталоге прошивки.

4.2.78 Модуль snd-mtpav

Модуль для многопортового MIDI-интерфейса MOTU MidiTimePiece AV (на параллельном порту). **порт** Номер порта ввода-вывода для МТРАВ (0x378, 0x278, по умолчанию = 0x378) **irq** IRQ # для МТРАВ (7,5, по умолчанию = 7) **hwports** количество поддерживаемых аппаратных портов по умолчанию = 8. Модуль поддерживает только 1 карту. У этого модуля нет опции включения.

4.2.79 Модуль snd-mts64

Модуль для Ego Systems (ESI) Miditerminal 4140

Этот модуль поддерживает несколько устройств. Требуется `parport` (конфигурационный параметр).

4.2.80 Модуль snd-nm256

Модуль для микросхем NeoMagic NM256AV/ZX

playback_bufsize максимальный размер кадра воспроизведения в кБ (4-128 КБ)

capture_bufsize максимальный размер кадра захвата в кБ (4-128 КБ)

force_ac97 0 или 1 (отключено по умолчанию)

buffer_top укажите верхний адрес буфера

use_cache 0 или 1 (отключено по умолчанию)

vaio_hack псевдоним `buffer_top` = 0x25a800

reset_workaround включить обходной путь сброса AC97 для некоторых ноутбуков

reset_workaround2 включите расширенное решение для сброса AC97 для некоторых других ноутбуков

Этот модуль поддерживает один чип и автоматический поиск. Поддерживается управление питанием.

Примечание: на некоторых ноутбуках адрес буфера не может быть определен автоматически или вызывает зависание во время инициализации. В таком случае явно укажите верхний адрес буфера с помощью опции `buffer_top`. Для примера, Sony F250: `buffer_top=0x25a800` Sony F270: `buffer_top=0x272800` Драйвер поддерживает только кодек `ac97`. Можно принудительно инициализировать / использовать `ac97`, хотя он не обнаружен. В таком случае используйте `force_ac97=1` опция - но *НЕ* гарантируйте, работает ли это! Примечание: Микросхема NM256 может быть подключена внутри с помощью кодеков, отличных от AC97. Этот драйвер поддерживает только кодек AC97 и не будет работать с машинами с другими (скорее всего, CS423х или OPL3SAх) чипами, даже если устройство обнаружено в `Ispci`. В таком случае попробуйте другие драйверы, например `snd-cs4232` или `snd-opl3sa2`. У некоторых есть ISA-PnP, но у некоторых нет ISA PnP. Вам нужно будет указать `isapnp=0` и соответствующие аппаратные параметры в случае без ISA PnP. Примечание: некоторым ноутбукам требуется обходной путь для СБРОСА AC97. Для известного оборудования, такого как Dell Latitude LS и Sony PCG-F305, этот обходной путь включается автоматически. Для других ноутбуков с жестким зависанием вы можете попробовать `reset_workaround = 1` вариант.

Примечание: У ноутбуков Dell Latitude CSx есть другая проблема, связанная со СБРОСОМ AC97. На этих ноутбуках по умолчанию включена опция `reset_workaround2`. Этот вариант стоит попробовать, если предыдущий параметр `reset_workaround` не помогает. Примечание: Этот драйвер действительно дерьмовый. Это перенос с драйвера OSS, который является результатом черной магии обратного проектирования. Обнаружение кодека завершится ошибкой, если драйвер загружен *после* X-сервер, как описано выше. Возможно, вы сможете принудительно загрузить модуль, но это может привести к зависанию. Следовательно, убедитесь, что вы загрузили этот модуль *перед* X, если вы столкнулись с подобной проблемой.

4.2.81 Модуль `snd-opl3sa2`

Модуль для звуковых карт Yamaha OPL3-SA2 / SA3.

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить

(по умолчанию) при `isapnp=0` доступны следующие опции:

port номер порта управления для микросхемы OPL3-SA (0x370)

sb_port Номер порта SB для микросхемы OPL3-SA (0x220, 0x240)

wss_port Номер порта WSS для микросхемы OPL3-SA (0x530, 0xe80, 0xf40, 0x604) **midi_port** номер порта для MPU-401 UART (0x300, 0x330), -1 =

отключить **fm_port** Номер FM-порта для микросхемы OPL3-SA (0x388), -1 =

отключить **irq** Номер IRQ для микросхемы OPL3-SA (5,7,9,10)

dma1 первый номер DMA для микросхемы Yamaha OPL3-SA (0,1,3)

dma2 второй DMA # для микросхемы Yamaha OPL3-SA (0,1,3), -1 = отключить

Этот модуль поддерживает несколько карт и ISA PnP. Он не поддерживает автоматический поиск (если не используется ISA PnP) поэтому должны быть указаны все порты !!! Поддерживается управление питанием.

4.2.82 Модуль `snd-opti92x-ad1848`

Модуль для звуковых карт на базе чипов OPTi 82c92х и Analog

Devices AD1848. Модуль также работает с картами OAK Mozart.

isapnp Обнаружение ISA PnP - 0 = отключить, 1 = включить (по умолчанию)

с помощью `isapnp=0`, доступны следующие опции:

порт номер порта для микросхемы WSS (0x530, 0xe80, 0xf40, 0x604)

mpu_port номер порта для MPU-401 UART (0x300, 0x310, 0x320, 0x330) **fm_port** номер порта для устройства OPL3 (0x388)

irq Номер IRQ для микросхемы WSS (5,7,9,10,11)

mpu_irq IRQ # для MPU-401 UART (5,7,9,10)

dma1 первый DMA # для микросхемы WSS (0,1,3) Этот модуль поддерживает только одну карту, автозондирование и PnP.

4.2.83 Модуль snd-opti92x-cs4231

Модуль для звуковых карт на базе чипов OPTi 82c92x и Crystal CS4231.

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию) при `isapnp=0` доступны следующие опции:

порт номер порта для микросхемы WSS (0x530, 0xe80, 0xf40, 0x604)

mpu_port номер порта для MPU-401 UART (0x300, 0x310, 0x320, 0x330) **fm_port** номер порта для устройства OPL3 (0x388)

irq Номер IRQ для микросхемы WSS (5,7,9,10,11)

mpu_irq IRQ # для MPU-401 UART (5,7,9,10)

dma1 первый DMA # для микросхемы WSS (0,1,3)

dma2 второй DMA # для микросхемы WSS (0,1,3) Этот модуль поддерживает только одну карту, автоматический поиск и PnP.

4.2.84 Модуль snd-opti93x

Модуль для звуковых карт на базе чипов OPTi 82c93x.

isapnp Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию) при `isapnp=0` доступны следующие опции:

порт номер порта для микросхемы WSS (0x530, 0xe80, 0xf40, 0x604) **mpu_port** номер порта для MPU-401 UART (0x300, 0x310, 0x320, 0x330) **fm_port** номер порта для устройства OPL3 (0x388)

irq Номер IRQ для микросхемы WSS (5,7,9,10,11)

mpu_irq IRQ # для MPU-401 UART (5,7,9,10)

dma1 первый DMA # для микросхемы WSS (0,1,3)

dma2 второй DMA # для микросхемы WSS (0,1,3) Этот модуль поддерживает только одну карту, автозондирование и PnP.

4.2.85 Модуль snd-oxygen

Модуль для звуковых карт на базе чипа C-Media CMI8786/8787/8788:

- Asound A-8788
- Asus Xonar DG/DGX

- AuzenTech X-Meridian
- AuzenTech X-Meridian 2G
- Bgears b-Enspirer
- Club3D Theatron DTS
- HT-Omega Claro (plus)
- HT-Omega Claro halo (XT)
- Kuroutoshikou CMI8787-HG2PCI
- Razer Barracuda AC-1
- Sondigo Inferno
- TempoTec HiFier Fantasia
- TempoTec HiFier Serenade

Этот модуль поддерживает автоматический поиск и несколько карт памяти.

Модуль `snd-pcsp` 4.2.86

Модуль для встроенного динамика ПК.

pcspm Отключите звук PCM из динамика ПК. Остаются только звуковые сигналы.

nforce_wa включите обходной путь набора микросхем nForce. Ожидайте плохого звука.

Этот модуль поддерживает системные звуковые сигналы, своего рода воспроизведение PCM и даже несколько элементов управления микшером.

4.2.87 Модуль `snd-pcxhr`

Модуль для плат Digigram PCXHR Этот модуль поддерживает несколько плат.

Модуль `snd` 4.2.88-`portman2x4`

Модуль для MIDI-интерфейса с параллельным портом MIDI Portman 2x4 Этот модуль поддерживает несколько карт.

4.2.89 Модуль `snd-powertac` (только для `ppc`)

Модуль для встроенных звуковых чипов PowerMac, iMac и iBook

включить звуковой сигнал включить звуковой сигнал с помощью PCM (включен по умолчанию) Модуль поддерживает автоматический поиск чипа. Примечание: у драйвера могут возникнуть проблемы с порядковым номером. Поддерживается управление питанием.

4.2.90 Модуль `snd-pxa2xx-ac97` (только для `arm`)

Модуль для драйвера AC97 для чипа Intel PXA2xx только для архитектуры ARM. Поддерживается управление питанием.

4.2.91 Модуль snd-riptide

Модуль для чипа Conexant Riptide

joystick_port Номер порта джойстика (по умолчанию:

0x200) **mpu_port** Номер порта MPU401 (по умолчанию:

0x330) **opl3_port** Порт OPL3 # (по умолчанию: 0x388)

Этот модуль поддерживает несколько карт. Драйверу требуется поддержка загрузчика прошивки в ядре. Вам необходимо установить файл прошивки `riptide.hex` к стандартному пути прошивки (например, `/lib/firmware`).

4.2.92 Модуль snd-rme32

Модуль для звуковых карт RME Digi32, Digi32 Pro и Digi32/8 (далее - Prodif32, Prodif96 и Prodif Gold). Этот модуль поддерживает несколько карт.

4.2.93 Модуль snd-rme96

Модуль для звуковых карт RME Digi96, Digi96 / 8 и Digi96 / 8 PRO / PAD / PST. Этот модуль поддерживает несколько карт.

4.2.94 Модуль snd-rme9652

Модуль для RME Digi9652 (Hammerfall, Звуковые карты Hammerfall

(Light)). **precise_ptr** Включить точный указатель (работает

ненадежно). (по умолчанию = 0) Этот модуль поддерживает несколько карт.

Примечание: модуль `snd-page-alloc` выполняет работу, которую ранее выполнял модуль `snd-hammerfall-mem`. Он будет распределять буферы заранее, когда будут найдены какие-либо карты RME9652. Чтобы обеспечить надежное выделение буфера, загрузите модуль `snd-page-alloc` на ранней стадии последовательности загрузки. Смотрите раздел [Раннее выделение буфера](#) Раздел.

4.2.95 Модуль snd-sa11xx-uda1341 (только для arm)

Модуль для Philips UDA1341TS на звуковой карте Compaq iPAQ H3600.

Модуль поддерживает только одну карту. Модуль не имеет опций включения и индексирования. Управление питанием поддерживается.

4.2.96 Модуль snd-sb8

Модуль для 8-разрядных карт SoundBlaster: SoundBlaster 1.0, SoundBlaster

2.0, SoundBlaster Pro **порт** номер порта для микросхемы SB DSP

(0x220, 0x240, 0x260) **irq** Номер IRQ для микросхемы SB DSP (5,7,9,10)

dma8 DMA # для микросхемы SB DSP (1,3)

Этот модуль поддерживает несколько карт и автоматический поиск.

Поддерживается управление питанием.

Модули 4.2.97 snd-sb16 и snd-sbawe

Модуль для 16-разрядных карт SoundBlaster: SoundBlaster 16 (PnP), SoundBlaster AWE 32

(PnP), SoundBlaster AWE 64 PnP

mic_agc Автоматическая регулировка усиления микрофона - 0 = отключить, 1 = включить

(по умолчанию) **csp** Поддержка чипов ASP/CSP - 0 = отключить (по умолчанию), 1 =

включить **isapnp** Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию)

при isapnp=0 доступны следующие параметры:

порт номер порта для микросхемы SB DSP 4.x (0x220, 0x240, 0x260)

mpu_port номер порта для MPU-401 UART (0x300, 0x330), -1 = отключить

awe_port базовый порт # для синтезатора EMU8000 (0x620, 0x640, 0x660) (только для

модуля snd-sbawe) **irq** IRQ # для микросхемы SB DSP 4.x (5,7,9,10)

dma8 8-разрядный DMA # для микросхемы SB DSP 4.x (0,1,3)

dma16 16-разрядный DMA # для микросхемы SB DSP 4.x (5,6,7) Этот модуль

поддерживает несколько карт, автоматический поиск и ISA PnP. Примечание: Для использования карт

Vibra16X в 16-разрядном полудуплексном режиме необходимо отключить 16-разрядный

DMA с параметром dma16 = -1. Параметр модуля. Кроме того, все платы типа Sound

Blaster 16 могут работать в 16-битном полудуплексном режиме через 8-битный канал

DMA, отключив их 16-битный канал DMA. Поддерживается управление питанием.

4.2.98 Модуль snd-sc6000

Модуль для звуковой карты Gallant SC-6000 и более поздних

моделей: SC-6600 и SC-7000. **порт** Порт # (0x220 или 0x240)

mss_port MSS Порт # (0x530 или 0xe80)

irq IRQ # (5,7,9,10,11)

mpu_irq MPU-401 IRQ # (5,7,9,10), 0 - нет MPU-401 irq

dma DMA # (1,3,0)

джойстик Включить игровой порт - 0 = отключить (по

умолчанию), 1 = включить Этот модуль поддерживает несколько карт.

Эта карта также известна как Audio Excel DSP 16 или Zoltrix AV302.

4.2.99 Модуль snd-sscape

Модуль для звуковых карт ENSONIQ.

порт Порт # (настройка PnP)

wss_port Порт WSS # (настройка PnP)

irq IRQ # (настройка PnP)

mpu_irq MPU-401 IRQ # (настройка PnP)

dma DMA # (настройка PnP)

dma2 2 - й DMA # (настройка PnP, -1 для отключения)

джойстик Включить игровой порт - 0 = отключить (по умолчанию), 1 = включить

Этот модуль поддерживает несколько карт.

Драйвер требует поддержки загрузчика прошивки в ядре.

Модуль 4.2.100 `snd-sun-amd7930` (только на `sparc`)

Модуль для звуковых чипов AMD7930, найденный на Sparcs. Этот модуль поддерживает несколько карт.

Модуль 4.2.101 `snd-sun-cs4231` (только на `sparc`)

Модуль для звуковых чипов CS4231, найденный в Sparcs. Этот модуль поддерживает несколько карт.

4.2.102 Модуль `snd-sun-dbri` (только в `sparc`)

Модуль для звуковых чипов DBRI, найденный в Sparcs. Этот модуль поддерживает несколько карт.

4.2.103 Модуль `snd-wavefront`

Модуль для звуковых карт Turtle Beach Maui, Tropez и Tropez+.

use_cs4232_midi Используйте интерфейс CS4232 MPU-401 (недоступен внутри вашего компьютера) **isapnp** Обнаружение PnP ISA - 0 = отключить, 1 = включить (по умолчанию) при isapnp=0 доступны следующие параметры: **cs4232_pcm_port** Номер порта для интерфейса CS4232 PCM.

cs4232_pcm_irq IRQ # для интерфейса CS4232 PCM (5,7,9,11,12,15).

cs4232_mpu_port Номер порта для интерфейса CS4232 MPU-401.

cs4232_mpu_irq IRQ # для интерфейса CS4232 MPU-401 (9,11,12,15).

ics2115_port Порт # для ICS2115

ics2115_irq IRQ # для ICS2115

fm_port FM OPL-3 Порт #

dma1 DMA1 # для интерфейса CS4232 PCM.

dma2 DMA2 # для интерфейса CS4232 PCM.

Ниже приведены варианты функций `wavefront_synth`:

wf_raw Предположим, что нам нужно загрузить ОС (по умолчанию: нет); Если да, то во время загрузки драйвера состояние платы игнорируется, и мы все равно перезагружаем плату и загружаем прошивку.

fx_raw Предположим, что процессу FX требуется помощь (по умолчанию: yes); Если false, мы оставим FX-процессор в любом состоянии, в котором он находится при загрузке драйвера. По умолчанию загружается микропрограмма и связанные коэффициенты, чтобы настроить ее на работу по умолчанию, что бы это ни значило. **debug_default** Параметры отладки для инициализации карты

wait_usecs Как долго ждать без перехода в спящий режим, usecs (по умолчанию: 150); Это магическое число, кажется, дает

довольно оптимальную пропускную способность, основанную на моих ограниченных экспериментах. Если вы хотите поиграть с этим и найти лучшее соотношение цены и качества, милости просим. Помните, идея состоит в том, чтобы получить число, которое заставляет нас просто напряженно ждать как можно большего количества команд волнового фронта, не получая при этом числа настолько большого, что мы перегружаем весь процессор.

В частности, с этим числом из примерно 134 000 ожиданий состояния только около 250 приводят к переходу в спящий режим. **sleep_interval** Как долго нужно спать в ожидании ответа (по умолчанию: 100)

попытки сна Сколько раз нужно пытаться перейти в режим сна во время ожидания (по умолчанию: 50)

ospath Путь к обработанной прошивке ICS2115 OS (по умолчанию: wavefront.os); Имя пути к прошивке ISC2115 OS

. В последней версии он обрабатывается через платформу загрузки встроенного ПО, поэтому он должен

быть установлен по правильному пути, обычно /lib/firmware .

reset_time Как долго ждать вступления в силу сброса (по умолчанию: 2)

время проверки оперативной памяти Сколько секунд ждать проверки оперативной памяти (по умолчанию: 20)

osrun_time Сколько секунд ждать запуска ICS2115 OS (по умолчанию: 10)

Этот модуль поддерживает несколько карт и ISA PnP. Примечание: файл

прошивки `wavefront.os` в более ранней версии находился в /etc . Теперь он загружается через загрузчик прошивки и должен

находиться в соответствующем пути к прошивке, например /lib/firmware . Скопируйте (или символическую ссылку) файл соответствующим образом, если вы получите сообщение об ошибке, связанной с загрузкой прошивки после обновления ядра.

4.2.104 Модуль snd-sonicvibes

Модуль для звуковых карт S3 SonicVibes PCI. * PINE Schubert 32 PCI

реверберация Включение реверберации - 1 = включить, 0 = отключить (по умолчанию); для этого

звуковая карта должна иметь встроенную SRAM. **mge** Включение усиления микрофона - 1 =

включить, 0 = отключить (по умолчанию) Этот модуль поддерживает несколько карт и автоматический поиск.

4.2.105 Модуль snd-serial-u16550

Модуль для последовательных MIDI-портов UART16550A.

порт номер порта для чипа UART16550A

irq IRQ # для чипа UART16550A, -1 = режим опроса

Скорость скорость в бодах (9600,19200,38400,57600,115200) 38400 = по умолчанию

База база для делителя в бодах (57600,115200,230400,460800) 115200 = по

умолчанию **выходы** количество MIDI-портов в последовательном порту (1-4) 1 = по

умолчанию **адаптер**

Тип адаптера. 0 = Soundcanvas, 1 = MS-124T, 2 = MS-124W S / A, 3 = MS-124W M / B, 4 =

универсальный Этот модуль поддерживает несколько карт. Этот модуль не поддерживает автоматический поиск, поэтому необходимо указать основной порт!!! Другие опции необязательны.

4.2.106 Модуль snd-trident

Модуль для звуковых карт Trident 4DWave DX / NX. * Best Union Miss Melody 4DWave

PCI * HIS 4DWave PCI * Warpspeed ONSpeed 4DWave PCI * AzTech PCI 64-Q3D *

Addonics SV 750 * ШИКАРНЫЙ True Sound 4Dwave * Shark Predator4D-PCI * Jatон

SonicWave 4D * SIS SI7018 PCI Audio * Цифровой саундтрек Hoontech 4DWave NX

pcm_channels максимальное количество каналов (тембров), зарезервированных для PCM **wavetable_size** максимальный размер таблицы сигналов в кБ (4-?кБ) Этот модуль поддерживает несколько карт и автозондирование. Поддерживается управление питанием.

4.2.107 Модуль snd-ua101

Модуль для аудио / MIDI интерфейсов Edirol UA-101 / UA-1000.
Этот модуль поддерживает несколько устройств, автоматический поиск и горячее подключение.

4.2.108 Модуль snd-usb-audio

Модуль для устройств USB audio и USB MIDI.

vid Идентификатор поставщика для устройства (необязательно)

pid Идентификатор продукта для устройства (необязательно)

nrpacks Максимальное количество пакетов в URB (по умолчанию: 8)

device_setup Магический номер для конкретного устройства (необязательно); Влияние зависит от устройства По умолчанию: 0x0000

ignore_ctl_error Игнорировать любой USB-контроллер, касающийся интерфейса микшера (по умолчанию: нет)

автоблокировка Включить автоматический выбор часов для устройств UAC2 (по умолчанию: да)

quirk_alias Список псевдонимов Quirk, передавать строки типа `0 1 2 3 abcd: 5 6 7 8 beef`, который применяет существующую особенность для устройства 5678:beef к новому устройству 0123: abcd.

Этот модуль поддерживает несколько устройств, автоматический поиск и горячее подключение.

ПРИМЕЧАНИЕ.: **nrpacks** параметр может быть динамически изменен через sysfs. Не указывайте значение больше 20. Изменение через sysfs не требует проверки работоспособности. ПРИМЕЧАНИЕ: **ignore_ctl_error= 1** может помочь, когда вы получаете ошибку при доступе к элементу микшера, такую как URB ошибка -22. Это происходит на каком-нибудь глючном USB-устройстве или контроллере. ПРИМЕЧАНИЕ: опция **quirk_alias** предоставляется только для тестирования / разработки. Если вы хотите получить надлежащую поддержку, обратитесь в upstream для статического добавления соответствующей функции в код драйвера.

4.2.109 Модуль snd-usb-caiaq

Модуль для аудиоинтерфейсов caiaq UB,

- Native Instruments RigKontrol2 •
- Native Instruments Kore Controller •
- Native Instruments Audio Kontrol 1
- Native Instruments Audio 8 DJ

Этот модуль поддерживает несколько устройств, автоматический поиск и горячее подключение.

4.2.110 Модуль snd-usb-ux2y

Модуль для устройств Tascam USB US-122, US-224 и US-428.

Этот модуль поддерживает несколько устройств, автоматический поиск и горячее подключение.

Примечание: вам необходимо загрузить прошивку через `ux2yloader` утилита, включенная в пакеты alsa-tools и alsa-firmware.

4.2.111 Модуль snd-via82xx

Модуль для материнских плат AC'97 на базе мостов VIA 82C686A/686B, 8233, 8233A, 8233C, 8235, 8237 (южный)

.

mpu_port 0x300, 0x310, 0x320, 0x330, в противном случае выполните настройку BIOS [только через VIA686A / 686B]

джойстик Включить джойстик (по умолчанию выключен) [только через VIA686A / 686B]

ac97_clock Базовая частота кодека AC'97 (по умолчанию 48000 Гц)

dxs_support поддержка каналов DXS, 0 = авто (по умолчанию), 1 = включить, 2 = отключить, 3 = только 48k, 4 = нет

VRA, 5 = включить любую частоту дискретизации и разные частоты дискретизации на разных каналах [только VIA8233 / C, 8235, 8237]

ac97_quirk Обходной путь AC'97 для странного оборудования; Смотрите [Вариант AC97 Quirk](#) раздел ниже.

Этот модуль поддерживает один чип и автозондирование.

Примечание: на некоторых материнских платах SMP, таких как MSI 694D, прерывания могут генерироваться неправильно. В таком случае, пожалуйста, попробуйте установить версию SMP (или MPS) в BIOS равной 1.1 вместо значения по умолчанию 1.4. Тогда номеру прерывания будет присвоено значение меньше 15. Вы также можете обновить свой BIOS.

Примечание: VIA8233 / 5 / 7 (не VIA8233A) может поддерживать каналы DXS (прямой звук) в качестве первого PCM. На этих каналах одновременно может воспроизводиться до 4 потоков, и контроллер может выполнять преобразование частоты дискретизации с отдельными частотами для каждого канала. По умолчанию (`dxs_support = 0`), выбрана фиксированная скорость 48k за исключением известных устройств, поскольку на выходе часто присутствует шум, за исключением 48k на некоторых материнских платах из-за ошибки BIOS. Пожалуйста, попробуйте один раз. `dxs_support = 5` и если это работает с другими частотами дискретизации (например, 44,1 кГц при воспроизведении mp3), пожалуйста, сообщите нам поставщика подсистемы PCI / идентификатор устройства (вывод `lspci -nv`). Если `dxs_support= 5` не работает, попробуйте `dxs_support= 4`; если это тоже не сработает, попробуйте `dxs_support=1`. (`dxs_support=1` обычно используется для старых материнских плат. Правильно реализованная плата должна работать с 4 или 5.) Если она по-прежнему не работает, а настройка по умолчанию в порядке, `dxs_support = 3` правильный выбор. Если настройка по умолчанию вообще не работает, попробуйте `dxs_support = 2` чтобы отключить каналы DXS. В любом случае, пожалуйста, сообщите нам результат и идентификаторы поставщика подсистемы / устройства. Смотрите [Ссылки и адреса](#) ниже.

Примечание: для MPU401 на VIA823x дополнительно используйте драйвер `snd-mpu401`. Опция `mpu_port` предназначена только для чипов VIA686.

Поддерживается управление питанием.

Модуль snd 4.2.112-via82xx-modem

Модуль для модема VIA82xx AC97

ac97_clock Тактовая частота кодека AC'97 (по умолчанию 48000 Гц)

Этот модуль поддерживает одну карту и автоматический поиск.

Примечание: Значение индекса по умолчанию для этого модуля равно -2, т.е. первый слот исключен. Поддерживается управление питанием.

4.2.113 Модуль snd-virmidi

Модуль для виртуальных устройств `rawmidi`. Этот модуль создает виртуальные устройства `rawmidi`, которые подключаются к соответствующим портам

ALSA sequencer. **midi_devs** MIDI-устройства # (1-4, по умолчанию = 4)

Этот модуль поддерживает несколько карт.

4.2.114 Модуль snd-virtuoso

Модуль для звуковых карт на базе чипов Asus AV66/AV100/AV200, то есть Xonar D1, DX, D2, D2X, DS, DSX, Essence ST (Deluxe), Essence STX (II), HDAV1.3 (Deluxe) и HDAV1.3 Slim. Этот модуль поддерживает автоматический поиск и несколько карт памяти.

4.2.115 Модуль snd-vx222

Модуль для карт Digigram VX-Pocket VX222, V222 v2 и Mic.

микрофон Включить микрофон на V222 Mic (Нью-Йорк)

ibl Фиксируйте размер IBL. (по умолчанию = 0, минимальный размер) Этот модуль поддерживает несколько карт.

Когда драйвер скомпилирован в виде модуля и поддерживается прошивка hotplug, данные прошивки загружаются через hotplug автоматически. Установите необходимые файлы прошивки в alsa-firmware package. Если загрузчик fw hotplug недоступен, вам необходимо загрузить прошивку с помощью утилиты vxloader из пакета alsa-tools. Чтобы автоматически вызвать vxloader, добавьте следующее в /etc/modprobe.d/alsa.conf

```
установить snd-vx 2 2 2 /sbin/modprobe --snd-vx 2 2 2 для первого использования\
&& /usr/bin/vxloader
```

(для ядер 2.2 / 2.4 добавьте после установки /usr/bin /vxloader вместо этого в /etc/modules.conf.) Размер IBL уменьшает период прерываний для PCM. Меньший размер дает меньшую задержку, но также приводит к большему потреблению процессора. . Размер обычно равен 126. По умолчанию (= 0) выбирается наименьший размер. Возможные значения IBL можно найти в файле /proc/asound/cardX/vx-status proc. Поддерживается управление питанием.

4.2.116 Модуль snd-vxpocket

Модуль для карт Digigram VX-Pocket VX2 и 440 PCMCIA.

ibl Захват размера IBL. (по умолчанию = 0, минимальный размер)

Этот модуль поддерживает несколько карт. Модуль компилируется только тогда, когда в ядре поддерживается PCMCIA. В более старом ядре 2.6.x для активации драйвера через диспетчер карт вам необходимо настроить /etc/pcmcia/vxpocket.conf. Смотрите файл sound/pcmcia/vx/vxpocket.c. 2.6.13 или более поздней версии для ядра больше не требуется файл конфигурации.

Когда драйвер скомпилирован в виде модуля и поддерживается прошивка hotplug, данные прошивки загружаются через hotplug автоматически. Установите необходимые файлы прошивки в alsa-firmware package. Если загрузчик fw hotplug недоступен, вам необходимо загрузить прошивку с помощью утилиты vxloader из пакета alsa-tools. О модуле capture IBL смотрите описание модуля snd-vx222. Примечание: Драйвер snd-vx440 объединен с драйвером snd-vxpocket начиная с версии ALSA 1.0.10. Поддерживается управление питанием.

4.2.117 Модуль snd-ymfpci

Модуль для микросхем Yamaha PCI (YMF72x, YMF74x и YMF75x).

mpu_port 0x300, 0x330, 0x332, 0x334, 0 (отключить) по умолчанию, 1 (автоматическое определение только для YMF744 /754) **fm_port** 0x388, 0x398, 0x3a0, 0x3a8, 0 (отключить) по умолчанию 1 (автоматическое определение только для YMF744/754) **joystick_port** 0x201, 0x202, 0x204, 0x205, 0 (отключить) по умолчанию, 1 (автоматическое определение)

rear_switch включить общий переключатель заднего/ линейного ввода (bool) Этот модуль поддерживает автоматический поиск и несколько микросхем. Поддерживается управление питанием.

4.2.118 Модуль snd-pdaudiocf

Модуль для звуковой карты Sound Core PDAudioCF.
Поддерживается управление питанием.

Опция 4.3 AC97 Quirk

Параметр ac97_quirk используется для включения / переопределения обходного пути для определенных устройств в драйверах для бортовых контроллеров AC'97, таких как snd-intel8x0. На некотором оборудовании поменялись местами выходные контакты между Master и наушниками или Surround (из-за путаницы спецификаций AC'97 от версии к версии :-)) Драйвер обеспечивает автоматическое определение известных проблемных устройств, но некоторые из них могут быть неизвестны или обнаружены ошибочно. В таком случае передайте правильное значение с помощью этой опции. Принимаются следующие строки:

По умолчанию Не переопределяйте настройку по умолчанию

Нет Отключите функцию quirk

hp_only Свяжите основные элементы управления и элементы управления наушниками как единый элемент управления **swap_hp** Поменяйте местами наушники и основные элементы управления

swap_surround Поменяйте местами элементы управления master и surround

ad_sharing Для AD1985 включите бит OMS и используйте наушники

alc_jack Для ALC65x включите режим jack sense

inv_eapd Перевернутая реализация EAPD

mute_led Привязать бит EAPD для включения / выключения индикатора отключения звука

Для обратной совместимости также принимаются соответствующие целочисленные значения -1, 0,

Например, если `Master` регулировка громкости не влияет на ваше устройство, а только на наушники `Влияет` передайте опцию модуля ac97_quirk = hp_only.

4.4 Настройка карт, отличных от ISAPnP.

Когда ядро настроено с поддержкой ISA-PnP, модули, поддерживающие платы isapnp, будут иметь опции модуля `isapnp`. Если эта опция установлена, *Только* будут проверены устройства ISA-PnP. Для проверки карт, отличных от ISA-PnP, вы должны пройти проверку `isapnp = 0` опция вместе с надлежащей конфигурацией ввода-вывода и irq. Если ядро настроено без поддержки ISA-PnP, опция isapnp не будет встроена.

4.5 Поддержка автоматической загрузки модулей

Драйверы ALSA могут быть загружены автоматически по запросу путем определения псевдонимов модулей. Строка `snd-card-%i` запрашивается для устройств, встроенных в ALSA, где `%i` номер звуковой карты от нуля до семи.

Чтобы автоматически загрузить драйвер ALSA для служб OSS, определите строку `звуковой слот-%i` где `%i` означает номер слота для служб OSS, который соответствует картотекой ALSA. Обычно определяют это как тот же самый карточный модуль.

Пример конфигурации для одной карты `emu10k1` выглядит следующим образом:

```
----- /etc/modprobe.d/alsa.conf

псевдоним snd-карты- 0 snd-emu 1 0 k 1

псевдоним звукового слота- 0 snd-emu 1 0 k 1

----- /etc/modprobe.d/alsa.conf
```

Доступное количество автоматически загружаемых звуковых карт зависит от опции модуля `cards_limit` модуля `snd`. По умолчанию установлено значение 1. Чтобы включить автоматическую загрузку нескольких карт, укажите количество звуковых карт в этом параметре.

Когда доступно несколько карточек, было бы лучше указать индексный номер для каждой карточки с помощью опции модуля, также, чтобы порядок карточек оставался неизменным.

Пример конфигурации для двух звуковых карт приведен ниже:

```
----- /etc/modprobe.d/alsa.conf

# Часть ALSA

опции snd cards_limit= 2

псевдоним snd-card- 0 snd-interwave

псевдоним snd-card- 1 snd-ens 1 3 7 1

опции snd-interwave index= 0

опции snd-ens 1 3 7 1 index= 1

# OSS/Свободная часть

псевдоним sound-slot- 0 snd-interwave

псевдоним sound-slot- 1 snd-ens 1 3 7 1

----- /etc/modprobe.d/alsa .conf
```

В этом примере карта `interwave` всегда загружается как первая карта (индекс 0), а `ens1371` - как вторая (индекс 1). Альтернативный

(и новый) способ зафиксировать назначение слотов - использовать `слоты` опцию модуля `snd`. В случае выше укажите примерно следующее:

```
опции snd slots=snd-interwave,snd-ens 1 3 7 1
```

Затем первый слот (`#0`) зарезервирован для `snd-interwave` driver, а второй (`#1`) - для `snd-ens1371`. Вы можете опустить опцию `index` в каждом драйвере, если используется опция `slots` (хотя вы все равно можете использовать их одновременно, если они не конфликтуют). Опция "Слоты" особенно полезна для предотвращения возможного горячего подключения и возникающего в результате конфликта слотов. Например, в приведенном выше случае первые два слота уже зарезервированы. Если какой-либо другой драйвер (например, `snd-usb-audio`) загружен до `snd-interwave` или `snd-ens1371`, ему будет назначен третий или более поздний слот. Если имя модуля указано с помощью `!`, слот будет указан для любых модулей, кроме этого имени. Например, `слоты=!snd-pcsp` будет зарезервирован первый слот для любых модулей, кроме `snd-pcsp`.

4.6 Сопоставление устройств ALSA PCM с устройствами OSS.

<code>/dev/snd/pcmC 0 D 0 [c p]</code>	<code>-> / dev/ audio 0 (/</code>	
<code>/dev/snd/pcmC 0 D 0 [c p]</code>	<code>dev/audio) -> младший 4 -> /</code>	
<code>/dev/snd/pcmC 0 D 1 [c p]</code>	<code>dev/ dsp 0 (/ dev/ dsp) -></code>	
<code>/dev/snd/pcmC 1 D 0 [c p]</code>	<code>младший 3 -> / dev/ adsp 0 (/</code>	<code>-> второстепенный 4 + 1 6 = 2 0</code>
<code>/dev/snd/pcmC 1 D 0 [c p]</code>	<code>dev/ adsp) -> младший 1 2 -></code>	<code>-> второстепенный 3 + 1 6 = 1 9</code>
<code>/dev/snd/pcmC 1 D 1 [c p]</code>	<code>/ dev/ audio 1 -> / dev/</code>	<code>-> второстепенный 1 2 + 1 6 = 2 8</code>
<code>/dev/snd/pcmC 2 D 0 [c p]</code>	<code>dsp 1 -> / dev/ adsp 1 -></code>	<code>-> второстепенный 4 + 3 2 = 3 6</code>
<code>/dev/snd/ pcmC 2 D 0 [c p]</code>	<code>/dev/ audio 2 -> / dev/</code>	<code>-> второстепенный 3 + 3 2 = 3 9</code>
<code>/dev/snd/pcmC 2 D 1 [c p]</code>	<code>dsp 2 -> / dev/ adsp 2</code>	<code>-> второстепенный 1 2 + 3 2 = 4 4</code>

Первый номер из `/dev/snd/pcmC{X} D {Y}[c | p]` выражение означает номер звуковой карты, а второе означает номер устройства. Устройства ALSA имеют либо `c` или `p` суффикс, указывающий направление, захват и воспроизведение соответственно. Пожалуйста, обратите внимание, что приведенное выше отображение устройства может быть изменено с помощью опций модуля `snd-pcm-oss` module.

4.7 Интерфейсы Proc (/proc/asound)

4.7.1 /proc/asound/номер карты/rst#[cp]/oss

стереть стереть всю дополнительную информацию о приложениях OSS
<имя_приложения> <фрагменты> <размер фрагмента> [<параметры>]

<имя_приложения> имя приложения с (более высоким приоритетом) или без пути
<фрагменты> количество фрагментов или ноль, если автоматически
<размер фрагмента> размер фрагмента в байтах или ноль, если автоматически
<параметры> необязательные параметры
отключить приложение пытается открыть rst-устройство для этого канала, но не хочет его использовать.
(Причина ошибка или требуется mmap) Это хорошо для Quake и т.д...

direct не используйте плагины
блокируйте режим принудительной блокировки (rvplayer)
неблочный принудительный неблочный режим
цельнофрагментный записывайте только целые фрагменты (оптимизация влияет только на воспроизведение) **без паузы** не заполняйте паузу впереди, чтобы избежать щелчков
ошибка-ptr Возвращает блоки пробелов в GETOPTR ioctl вместо заполненных блоков

Пример:

```
echo "x 1 1 amp 1 2 8 1 6 3 8 4 " > /proc/asound/card0/pcm0p/oss
echo "отключить quake 0 0 " > /proc/asound/card0/pcm0c/oss
echo "заблокировать rvplayer 0 0 " > /proc/asound/card0/pcm0p/oss
```

4.8 Раннее выделение буфера

Некоторым драйверам (например, `hdsp`) требуются большие непрерывные буферы, и иногда слишком поздно находить такие пробелы, когда модуль драйвера фактически загружен из-за фрагментации памяти. Вы можете предварительно выделить буферы PCM, загрузив модуль `snd-page-alloc` и предварительно записав команды в его `proc`-файл, например, на ранней стадии загрузки, например `/etc/init.d/* .local` скрипты. Чтение файла `proc/proc/drivers/snd-page-alloc` показывает текущее использование выделения страниц. В письменном виде вы можете отправить следующие команды драйверу `snd-page-alloc`:

- добавить БУФЕРЫ РАЗМЕРА МАСКИ УСТРОЙСТВА ПОСТАВЩИКА

ПОСТАВЩИК и УСТРОЙСТВО - это идентификаторы поставщика PCI и устройства. Они принимают целые числа (для шестнадцатеричного числа необходим префикс 0x). MASK - это маска PCI DMA. Передайте 0, если не ограничено. SIZE - это размер каждого выделяемого буфера. Вы можете передать суффикс `k` и `m` для KB и MB. Максимальное количество - 16 MB. BUFFERS - количество выделяемых буферов. Оно должно быть больше 0. Максимальное число равно 4.

- стереть

Это приведет к удалению всех предварительно выделенных буферов, которые не используются.

4.9 Ссылки и адреса

Домашняя страница проекта ALSA <http://www.alsa-project.org>

Ядро Bugzilla <http://bugzilla.kernel.org/>

ALSA Developers ML <mailto:alsa-devel@alsa-project.org>

alsa-info.sh скрипт <http://www.alsa-project.org/alsa-info.sh>

HD-AUDIO

5.1 Дополнительные сведения о драйвере HD-Audio

Такаши Иваи <tiwai@suse.de>

5.1.1 Общие положения

HD-audio - это новый стандартный встроенный аудиокomпонент на современных ПК после AC97. Хотя Linux поддерживает HD-аудио с давних пор, с новыми машинами часто возникают проблемы. Частично проблема заключается в неисправном BIOS, а остальное - в реализации драйвера. В этом документе кратко описываются методы устранения неполадок и отладки аппаратного обеспечения HD-audio.

Компонент HD-audio состоит из двух частей: микросхемы контроллера и микросхем кодека на шине HD-audio. Linux предоставляет единый драйвер для всех контроллеров, `snd-hda-intel`. Хотя название драйвера содержит слово известного поставщика оборудования, оно относится не только к нему, но и ко всем чипам контроллеров других компаний. Поскольку контроллеры HD-audio должны быть совместимы, в большинстве случаев должен работать один `snd-hda`-драйвер. Но, что неудивительно, существуют известные ошибки и проблемы, характерные для каждого типа контроллера. В драйвере `snd-hda-intel` для них есть множество обходных путей, как описано ниже. Контроллер может иметь несколько кодеков. Обычно у вас есть один аудиокодек и, возможно, один модем кодек. Теоретически, может существовать несколько аудиокодеков, например, для аналоговых и цифровых выходов, и драйвер может работать неправильно из-за конфликта элементов микшера. Это должно быть исправлено в будущем, если такое оборудование действительно существует. Драйвер `snd-hda-intel` имеет несколько различных анализаторов кодеков в зависимости от кодека. У него есть универсальный синтаксический анализатор в качестве запасного варианта, но до сих пор эта функциональность довольно ограничена. Вместо универсального анализатора обычно для реализаций, зависящих от конкретного кодека, используется анализатор, зависящий от конкретного кодека (закодированный в `patch_*.c`). Подробности о проблемах, связанных с конкретным кодеком, объясняются в последующих разделах. Если вас интересует глубокая отладка HD-audio, сначала ознакомьтесь со спецификацией HD-audio. Спецификацию можно найти, например, на веб-странице Intel.:

· <http://www.intel.com/standards/hdaudio/>

5.1.2 Контроллер HD-Audio.

Проблема с положением DMA.

Наиболее распространенной проблемой контроллера является неточное сообщение указателя DMA. Указатель DMA для воспроизведения и захвата может быть считан двумя способами: либо через регистр LPIB, либо через карту позиционного буфера. По умолчанию драйвер пытается считывать данные из позиционного буфера, отображаемого вводом-выводом, и возвращается к LPIB, если позиционный буфер оказывается неработоспособным. Однако на некоторых устройствах это определение не идеально. В таком случае вы можете изменить метод по умолчанию с помощью опции `position_fix`.

`position_fix= 1` означает явное использование метода LPIB. `position_fix= 2` означает использование позиционного буфера. `position_fix= 3` означает использование комбинации обоих методов, необходимой для некоторых контроллеров VIA. The

положение потока захвата корректируется путем сравнения значений LPIB и `position-buffer.position_fix= 4` это другая комбинация, доступная для всех контроллеров, и использует LPIB для воспроизведения и позиционный буфер для потоков захвата. 0 - это значение по умолчанию для всех других контроллеров, автоматическая проверка и резервный вариант для LPIB, как описано выше. Если у вас возникает проблема с повторяющимися звуками, эта опция может помочь. В дополнение к этому, известно, что каждый контроллер неисправен в отношении времени пробуждения. Он запускает несколько выборок перед фактической обработкой данных в буфере. Это вызвало множество проблем, например, с ALSA dmix или JACK. Начиная с ядра 2.6.27, драйвер устанавливает искусственную задержку времени пробуждения. Эта задержка контролируется с помощью опции `bdl_pos_adj` . Когда `bdl_pos_adj` является отрицательным значением (по умолчанию), ему присваивается соответствующее значение в зависимости от микросхемы контроллера. Для чипов Intel это будет 1, а для других - 32. Обычно это работает. Только в том случае, если это не сработает и вы получите предупреждающие сообщения, вам следует изменить этот параметр на другие значения. **Проблема с кодеком**

Менее частой, но более серьезной проблемой является проверка кодека. Когда BIOS ошибочно сообщает о доступных слотах кодеков , драйвер запутывается и пытается получить доступ к несуществующему слоту кодеков. Это часто приводит к полному сбою и прерывает дальнейшую связь с чипами кодека. Симптом появляется обычно в виде сообщений об ошибках, таких как:

```
hda_intel: тайм-аут azx_get_response, переключение в режим опроса:

    последний cmd = 0 x 1 2 3 4 5 6 7 8

hda_intel: тайм-аут azx_get_response, переключение в режим single_cmd:

    последний cmd = 0 x 1 2 3 4 5 6 7 8
```

Первая строка представляет собой предупреждение, и обычно оно относительно безвредно. Это означает, что ответ кодека не уведомляется через IRQ. Драйвер использует метод явного опроса для считывания ответа. Это дает очень небольшую нагрузку на процессор , но вы вряд ли это заметите. Однако вторая строка является фатальной ошибкой. Если это происходит, обычно это означает, что что-то действительно не так. Скорее всего, вы обращаетесь к несуществующему слоту кодека. Таким образом, если появляется второе сообщение об ошибке, попробуйте сузить исследуемые слоты кодека с помощью опции `probe_mask` . Это битовая маска, и каждый бит соответствует слоту кодека. Например, чтобы проверить только первый слот, передайте `probe_mask= 1` . Для первого и третьего слотов передайте команду `probe_mask= 5` (где 5 = 1/4) и так далее. Начиная с ядра 2.6.29, драйвер имеет более надежный метод проверки, поэтому эта ошибка может возникать редко, хотя. На компьютере со сломанным BIOS иногда требуется заставить драйвер проверить слоты кодеков, которые оборудование не сообщает об использовании. В таком случае поверните бит 8 (0x100) в `probe_mask` опция включена. Затем остальные 8 битов передаются в качестве слотов кодека для безусловного тестирования. Например,, `probe_mask = 0 x 1 0 3` будет принудительно проверять слоты кодека 0 и 1 независимо от того, что сообщает аппаратное обеспечение.

Обработка прерываний

В HD-аудио драйвер для MSI использует по умолчанию (если имеется) с ядром 2.6.33 как MSI работает лучше на каком-нибудь Ма- Китае, и вообще, это лучше для производительности. Однако контроллеры Nvidia показали плохие регрессии с MSI (особенно в сочетании с чипсетом AMD), поэтому мы отключили MSI для них. Похоже, есть и другие устройства, которые не работают с MSI. Если вы видите регресс в отношении качества звука (заикание и т.д.) Или блокировку в последнем ядре, попробуйте передать `enable_msi= 0` опция для отключения MSI. Если это сработает, вы можете добавить известное неисправное устройство в черный список, определенный в `hda_intel.c` . В таком случае, пожалуйста, сообщите и верните исправление вышестоящему разработчику.

5.1.3 HD-аудиокодека

Вариант модели

Наиболее распространенной проблемой, связанной с драйвером HD-audio, являются неподдерживаемые функции кодека или неправильно подобранная конфигурация устройства. Большая часть кода, зависящего от кодека, имеет несколько предустановленных моделей, либо для переопределения настроек BIOS, либо для предоставления более полных функций. Драйвер проверяет SSID PCI и просматривает таблицу статической конфигурации, пока не будет найдена какая-либо соответствующая запись. Если у вас новая машина, вы можете увидеть сообщение, подобное

приведенному ниже: `hda_codec: ALC 8 8 0 : автоматическое тестирование BIOS.`

Между тем, в более ранних версиях вы бы увидели сообщение типа:

`hda_codec: Неизвестная модель ALC 8 8 0 , пытаюсь выполнить автопробу из BIOS...`

Даже если вы увидите такое сообщение, НЕ ПАНИКУЙТЕ. Сделайте глубокий вдох и возьмите полотенце. Прежде всего,

это информационное сообщение, без предупреждения, без ошибки. Это означает, что SSID PCI вашего устройства

отсутствует в списке известных предустановленных моделей (white-). Но это не означает, что драйвер неисправен.

Многие драйверы кодеков обеспечивают механизм автоматической настройки на основе настроек BIOS. Кодеки HD-audio обычно имеют виджеты "pin", и BIOS устанавливает конфигурацию по умолчанию для каждого pin, которая указывает местоположение, тип подключения, цвет разъема и т.д. Драйвер HD-audio может выбрать правильное подключение, исходя из этих значений конфигурации по умолчанию. Однако - некоторые коды с поддержкой кодеков, такие как `patch_analog.c`, не поддерживают автоматическое тестирование (пока по состоянию на 2.6.28). И BIOS часто, да, довольно часто ломается. Он устанавливает неправильные значения и портит драйвер. Предустановленная модель (или, недавно названная, "fix-up") предназначена в основном для преодоления такой ситуации. Когда соответствующая предустановленная модель найдена в белом списке, драйвер принимает статическую конфигурацию этой предустановки с правильной установкой pin-кода и т.д. Таким образом, если у вас более новая машина с немного отличающимся SSID PCI (или SSID кодека) от существующей, у вас может быть хороший шанс повторно использовать ту же модель. Вы можете передать `модель=` возможность указать предустановленную модель вместо поиска SSID по PCI (и кодеку). Что

`модель=` доступные значения параметров зависят от микросхемы кодека. Проверьте микросхему вашего кодека в файле `codec proc` (см. раздел "Файл Codec Proc" ниже). В нем будет указано название поставщика / продукта вашего чипа кодека. Затем обратитесь к документации `/sound/HD-Audio-Models.первый файл`, раздел драйвера HD-audio. Вы можете найти список кодеков и `модель=` опции, относящиеся к каждому кодеку. Например, для чипа кодека Realtek ALC262, `pass модель= ultra` для устройств, совместимых с Samsung Q1 Ultra. Таким образом, первое, что вы можете сделать для любого совершенно нового, неподдерживаемого и нерабочего оборудования HD-audio, - это проверить кодек HD-audio и несколько различных `модель=` значения параметров. Если вам повезет, некоторые из них могут хорошо подойти для вашего устройства.

Существует несколько специальных значений параметров модели:

- когда передается 'nofixup', исправления для конкретного устройства в анализаторе кодеков пропускаются.
- когда `общие` передается, парсер, зависящий от конкретного кодека, пропускается и используется только универсальный парсер.

Динамик и выход для наушников

Одна из наиболее частых (и очевидных) ошибок в HD-audio - это бесшумный вывод звука одним или обоими из встроенного динамика и разъема для наушников. Как правило, сначала вам следует попробовать выход на наушники. Выход на динамик часто требует дополнительных элементов управления, таких как внешний усилитель. Таким образом, выход для наушников имеет немного больше шансов. Прежде чем отправлять сообщение об ошибке, дважды проверьте, правильно ли настроен микшер. Последняя версия `snd-hda`-драйвера intel обеспечивает в основном "Основную" регулировку громкости, а также "Фронтальную" громкость (где Front указывает на фронтальные каналы). Кроме того, могут быть отдельные элементы управления "Наушниками" и "Динамиками". То же самое для выхода динамика. В некоторых кодеках может быть переключатель "Внешний усилитель". Включите эту функцию, если она присутствует. Другой связанной с этим проблемой является автоматическое отключение выхода динамика при подключении наушников. Эта функция реализована в большинстве случаев, но не на каждой предустановленной модели или коде поддержки кодека.

5.1. Дополнительные примечания по драйверу HD-Audio 231

В любом случае, попробуйте другую модель, если у вас возникла такая проблема. Некоторые другие модели могут подходить лучше и предоставлять вам более подходящую функциональность. Если ни одна из доступных моделей не работает, отправьте отчет об ошибке. Подробнее смотрите в разделе "Отчет об ошибке".

Если вы достаточно мазохистичны, чтобы устранять проблему с драйвером, обратите внимание на следующее:

- Для выхода на динамик (и на наушники тоже) часто требуется внешний усилитель. Это можно настроить обычно с помощью EAPD verb или определенного GPIO. Если pin-кодек поддерживает EAPD, у вас больше шансов использовать команду SET_EAPD_BTL (0x70c). В других случаях вывод GPIO (в основном это либо GPIO0, либо GPIO1) может включать / выключать EAPD.
- Для некоторых кодеков Realtek требуются специальные коэффициенты, зависящие от производителя, для включения усилителя. Смотрите patch_realtek.c.

- Кодеки IDT могут иметь дополнительные элементы управления включением / отключением питания на каждом аналоговом выводе.

Смотрите patch_sigmatel.c. • Очень редко, но некоторые устройства не принимают команду определения pin-кода до тех пор, пока она не будет запущена.

Выдача

команды GET_PIN_SENSE (0xf09) может привести к сбою связи между кодеками. Некоторые примеры найдены в patch_realtek.c.

Проблемы с захватом

Проблемы с захватом часто возникают из-за отсутствия настроек микшеров. Таким образом, перед отправкой сообщения об ошибке убедитесь, что вы правильно настроили микшер. Например, в дополнение к правильному выбору "Источника захвата" или "Источника входного сигнала" необходимо правильно настроить "Громкость захвата" и "Переключатель захвата". Некоторые устройства имеют функцию "Увеличения громкости микрофона" или переключатель. Когда устройство PCM открывается через PCM "по умолчанию" (без плагина pulse-audio), у вас, вероятно, также будет регулятор громкости цифрового захвата. Это предусмотрено для дополнительного усиления / ослабления сигнала в программном обеспечении, особенно для входов без аппаратного регулятора громкости, таких как цифровые микрофоны. Если это действительно не необходимо, это значение должно быть установлено ровно на 50%, что соответствует 0 дБ - ни дополнительного усиления, ни ослабления. Однако, когда вы используете PCM "hw", то есть PCM необработанного доступа, этот элемент управления не будет иметь никакого влияния. Известно, что некоторые кодеки / устройства имеют довольно плохие аналоговые схемы, а записанный звук содержит определенное смещение по постоянному току. Это не ошибка драйвера. Большинство современных ноутбуков не имеют аналогового подключения CD-входа. Таким образом, запись с CD-входа во многих случаях не будет работать, хотя драйвер предоставляет его в качестве источника записи. Вместо этого используйте CDDA. Автоматическое переключение встроенного и внешнего микрофона при подключении реализовано на некоторых моделях кодеков, но не на каждой модели. Отчасти из-за моей лени, но в основном из-за отсутствия тестеров. Не стесняйтесь отправить патч для улучшения автору.

Прямая отладка

Если ни один из вариантов модели не дает вам лучшего результата, и вы крепкий орешек в борьбе со злом, попробуйте выполнить отладку с помощью установки на устройство исходных данных кодека HD-audio. Доступны некоторые инструменты: hda-emu и hda-анализатор. Подробное описание находится в разделах ниже. Вам необходимо включить hwdep для использования этих инструментов. Смотрите раздел "Конфигурация ядра".

5.1.4 Другие проблемы

Конфигурация ядра

В общем, я рекомендую вам включить опцию звуковой отладки., CONFIG_SND_DEBUG=y, независимо от того, выполняете ли вы отладку или нет. Это активирует макрос snd_printd() и другие, и вы получите дополнительные сообщения ядра при проверке. Кроме того, вы можете включить CONFIG_SND_DEBUG_VERBOSE=y. Но при этом вы получите гораздо больше сообщений. Таким образом, включайте это только тогда, когда вы уверены, что хотите этого.

Не забудьте включить соответствующий `CONFIG_SND_HDA_CODEC_*` Опции. Обратите внимание, что каждый из них соответствует чипу кодека, а не чипу контроллера. Таким образом, даже если `lspci` показывает контроллер Nvidia, вам может потребоваться выбрать опцию для других производителей. Если вы не уверены, просто выберите все да.

`CONFIG_SND_HDA_HWDEP` это полезная опция для отладки драйвера. Когда это включено, драйвер создает аппаратно-зависимые устройства (по одному для каждого кодека), и у вас есть необработанный доступ к устройству через эти файлы устройства. Например, `hwC 0 D 2` будет создан для слота кодека #2 первой карты (#0). Для инструментов отладки, таких как `hda-verb` и `hda-analyzer`, устройство `hwdep` должно быть включено. Таким образом, было бы лучше включать это всегда.

`CONFIG_SND_HDA_RECONFIG` - это новая опция, и это зависит от опции `hwdep`, указанной выше. Когда эта функция включена, у вас будут некоторые файлы `sysfs` в соответствующем каталоге `hwdep`. Смотрите раздел "Реконфигурация HD-audio" ниже.

`CONFIG_SND_HDA_POWER_SAVE` опция включает функцию энергосбережения. Смотрите раздел "Энергосбережение" ниже- низкое.

Программный файл Codec

Программный файл `codec` - это сокровищница для отладки HD-аудио. Он показывает большую часть полезной информации о виджете каждого кодека. Файл `proc` находится в `/proc/asound/card*/codec#*`, по одному файлу на каждый слот кодека. Вы можете узнать поставщика кодека, идентификатор и названия продуктов, тип каждого виджета, возможности и так далее. Однако в этом файле пока не отображается состояние срабатывания разъема. Это связано с тем, что срабатывание разъема может зависеть от состояния триггера. Этот файл будет обнаружен инструментами отладки, а также он может быть передан в эмулятор в качестве основного кодека информация. Смотрите раздел инструменты отладки ниже. Этот файл `proc` также можно использовать для проверки того, используется ли универсальный анализатор. При использовании универсального анализатора имя поставщика / product ID будет отображаться как "Realtek ID 0262" вместо "Realtek ALC262".

Реконфигурация HD-звука.

Это экспериментальная функция, позволяющая динамически перенастраивать кодек HD-audio без перезагрузки драйвера. .

Следующие файлы `sysfs` доступны в каталоге каждого устройства `codec-hwdep` (например, `/sys/class/sound/hwC0D0`):

vendor_id Показывает шестнадцатеричный номер идентификатора поставщика 32-битного кодека. Вы можете изменить значение идентификатора поставщика, записав в этот файл.

subsystem_id Показывает шестнадцатеричный номер идентификатора подсистемы 32-битного кодека. Вы можете изменить значение идентификатора подсистемы путем записи в этот файл.

revision_id Показывает шестнадцатеричный номер идентификатора ревизии 32-битного кодека. Вы можете изменить значение идентификатора ревизии с помощью запись в этот файл.

afg Показывает идентификатор AFG. Доступен только для чтения.

mfg Показывает идентификатор MFG. Доступен только для чтения.

Имя Показывает строку имени кодека. Может быть изменен путем записи в этот файл.

имямодели Показывает текущее значение `модель` параметр. Может быть изменен путем записи в этот файл.

init_verbs Дополнительные глаголы, которые будут выполняться при инициализации. Вы можете добавить глагол, записав его в этот файл. Передайте три числа: `nid`, `verb` и `parameter` (через пробел).

подсказки. Отображает / сохраняет строки подсказок для анализаторов кодеков для любого использования. Его формат `ключ = значение`. Например,,
`передача jack_detect = нет` полностью отключит обнаружение разъема на компьютере.

init_pin_configs Показывает начальные значения конфигурации `pin` по умолчанию, установленные BIOS.

driver_pin_configs Явно показывает значения pin-кода по умолчанию, установленные анализатором кодеков. Это не показывает все значения pin, но только измененные анализатором. То есть, если анализатор не изменит значения конфигурации pin по умолчанию самостоятельно, это ничего не будет содержать.

user_pin_configs Показывает значения конфигурации pin по умолчанию для переопределения настроек BIOS. При вводе этого параметра (с двумя цифрами, NID и value) добавляется новое значение. Указанное значение будет использоваться вместо начального значения BIOS при следующей перенастройке. Обратите внимание, что эта конфигурация будет переопределять даже настройки pin-кода драйвера тоже.

повторная настройка Запускает повторную конфигурацию кодека. Когда в этот файл записывается какое-либо значение, драйвер повторно инициализирует работу и снова анализирует дерево кодеков. Все изменения, внесенные приведенными выше записями sysfs, принимаются во внимание.

Очистить Сбрасывает кодек, удаляет элементы микшера и содержимое PCM указанного кодека и удаляет все глаголы инициализации и подсказки.

Например, если вы хотите изменить значение конфигурации pin-кода по умолчанию для виджета pin-кода 0x14 на 0x9993013f и разрешить повторную настройку драйвера на

```
основе этого состояния, выполните следующие действия: # echo 0 x 1 4 0 x 9 9 9 3 0 1 3 f > /sys/class/sound/hwC 0 D 0 /user_pin_configs
# echo 1 > /sys/class/sound/hwC 0 D 0 /реконфигурация
```

Строки подсказок

В анализаторе кодеков есть несколько переключателей и ручек настройки для лучшего соответствия реальному кодеку или поведению устройства. Многие из них могут быть динамически скорректированы с помощью строк "подсказок", как упоминалось в разделе выше. Например, передавая jack_detect = нет используя sysfs или файл исправления, вы можете отключить обнаружение разъема, таким образом, анализатор кодеков пропустит такие функции, как автоматическое отключение звука или автоматическое переключение микрофона. В качестве логического значения либо да, нет, истина, ложь, 1 или 0 может быть передан.

Универсальный анализатор поддерживает следующие подсказки:

jack_detect (bool) укажите, доступно ли вообще обнаружение гнезда на этом компьютере; по умолчанию true

inv_jack_detect (bool) указывает, что логика обнаружения гнезда инвертирована

trigger_sense (bool) указывает, что для обнаружения гнезда требуется явный вызов

Глагол AC_VERB_SET_PIN_SENSE

inv_eapd (bool) указывает, что EAPD реализован в инвертированной логике

pcm_format_first (bool) устанавливает формат PCM перед тегом потока и идентификатором канала

sticky_stream (bool) сохраняйте формат PCM, тег потока и идентификатор как можно дольше; по умолчанию true

spdif_status_reset (bool) сбрасывайте биты состояния SPDIF при каждой настройке потока SPDIF

pin_amp_workaround (bool) выходной вывод может иметь несколько значений amp

single_adc_amp (bool) АЦП могут иметь только одиночные входные усилители

auto_mute (bool) включение / выключение функции автоматического отключения звука в наушниках; значение по умолчанию true

auto_mic (bool) включение / выключение функции автоматического переключения микрофона; значение по умолчанию true

line_in_auto_switch (bool) включить / отключить функцию автоматического переключения ввода строки; по умолчанию false

need_dac_fix (bool) ограничивает ЦАП в зависимости от количества каналов

primary_hp (bool) в качестве основных выходов используйте разъемы для наушников; по умолчанию true

multi_io (bool) попробуйте протестировать конфигурацию с несколькими входами-выходами (например, общий линейный вход / объемное звучание, микрофонные / clfe разъемы) **multi_cap_vol (bool)** обеспечивает несколько томов записи

inv_dmic_split (bool) обеспечивает разделение громкости внутреннего микрофона /переключатель для цифровых микрофонов с

фазоинвертированием **indep_hp (bool)** обеспечьте независимый поток PCM для наушников и соответствующее управление микшером,

если доступно

add_stereo_mix_input (bool) добавьте стереомикс (аналогово-циклический микс) во входной мультиплексор, если он доступен

add_jack_modes (bool) добавьте элементы управления перечислением "Режим xxx Jack Mode" к каждому разъему ввода-вывода, чтобы можно было изменять

возможности VREF смещения усилителя для наушников и микрофона

power_save_node (bool) расширенное управление питанием для каждого виджета, контролирующее уровень энергопотребления (D0/D3) каждого узла виджета в зависимости от фактического вывода и состояний потока

power_down_unused (bool) отключите неиспользуемые виджеты, подмножество power_save_node, и они будут удалены в будущем

add_hp_mic (bool) добавьте наушники для записи источника, если это возможно

hp_mic_detect (bool) включите / отключите общий вход hp / mic для одного встроенного микрофонного корпуса; значение по умолчанию true

vmaster (bool) включить / отключить элемент управления virtual Master; по умолчанию true

mixer_nid (int) задает NID виджета микшера с аналоговой обратной связью

Раннее исправление

Когда CONFIG_SND_HDA_PATCH_LOADER=T установлен, вы можете передать "патч" в виде файла прошивки для изменения настроек HD-audio перед инициализацией кодека. Это может работать в основном так же, как реконфигурация через sysfs, описанная выше, но она делает это перед первой настройкой кодека.

Файл исправления - это обычный текстовый файл, который выглядит следующим образом:

```
[кодек]
0 x 1 2 3 4 5 6 7 8 0 xabcd 1 2 3 4 2

[модель]
auto

[pincfg]
0 x 1 2 0 x 4 1 1 1 1 1 f 0

[глагол]
0 x 2 0 0 x 5 0 0 0 x 0 3
0 x 2 0 0 x 4 0 0 0 x ff

[подсказка]
jack_detect = нет
```

В файле должна быть строка [кодек]. Следующая строка должна содержать три цифры, указывающие на кодек-идентификатор

поставщика (0x12345678 в примере), идентификатор подсистемы кодека (0xabcd1234) и адрес (2) кодека. Остальные записи исправлений применяются к этому указанному кодеку до тех пор, пока не будет указана другая запись кодека. При добавлении 0 или отрицательного числа к первому или второму значению проверка соответствующего поля будет пропущена. Это будет полезно для действительно неисправных устройств, которые неправильно инициализируют SSID.

Строка [модель] позволяет изменить название модели каждого кодека. В приведенном выше примере это значение будет изменено на model=auto. Обратите внимание, что это переопределяет параметр module.

После [pincfg] строка, содержимое анализируется как исходные настройки pin-кода по умолчанию, точно так же, как user_pin_configs sysfs выше. Значения также могут быть показаны в файле user_pin_configs sysfs.

Аналогично, строки после [глагол] анализируются как init_verbs записи sysfs и строки после [подсказка] анализируются как подсказки записи sysfs соответственно.

Другой пример переопределения идентификатора поставщика кодека с 0x12345678 на 0xdeadbeef выглядит следующим образом: [кодек]

```
0 x 1 2 3 4 5 6 7 8 0 xabcd 1 2 3 4 2

[vendor_id]
0 xdeadbeef
```


Аналогичным образом вы можете переопределить кодек subsystem_id с помощью [subsystem_id], идентификатор редакции через строка [revision_id] . Также имя чипа кодека может быть перезаписано через [chip_name] линия.

```
[кодек]
0 x 1 2 3 4 5 6 7 8 0 xabcd 1 2 3 4 2

[subsystem_id]
0 xffff 1 1 1 1

[revision_id]
0 x 1 0

[chip_name]

Мои собственные новости- 0 0 0 2
```

Драйвер hd-audio считывает файл через request_firmware(). Таким образом, файл исправления должен быть расположен в соответствующем пути к прошивке, обычно /lib/firmware. Например, когда вы передаете опцию patch=hda-init.fw, должен присутствовать файл /lib/firmware/hda-init.fw.

Опция модуля исправления специфична для каждого экземпляра card, и вам нужно указать одно имя файла для каждого экземпляра, разделенное запятыми. Например, если у вас есть две платы, одна для встроенного аналога и одна для видеоплаты HDMI, вы можете выбрать опцию исправления, как показано ниже:

```
опции snd-hda-intel patch = встроенный патч, hdm1-патч
```

Энергосбережение

Энергосбережение - это своего рода автоматическое приостановление работы устройства. Когда устройство неактивно в течение определенного времени, устройство автоматически выключается для экономии электроэнергии. Время отключения указывается с помощью параметра power_save опция модуля, и эту опцию можно динамически изменять через sysfs.

Энергосбережение не будет работать, если в некоторых кодеках включена аналоговая обратная связь. Убедитесь, что вы отключаете все ненужные маршруты передачи сигналов, когда хотите сэкономить электроэнергию.

Функция энергосбережения может вызывать слышимые щелчки при каждом включении / выключении питания в зависимости от устройства. Некоторые из них, боюсь, решаемы, но некоторые сложны. Некоторые дистрибутивы, такие как openSUSE, автоматически включают функцию энергосбережения при отключении кабеля питания. Таким образом, если вы слышите шумы, сначала заподозрите энергосбережение. Смотрите /sys/module/snd_hda_intel/parameters/power_save, чтобы проверить текущее значение. Если оно отличное от нуля, функция включена.

Последнее ядро также поддерживает runtime PM для чипа контроллера HD-audio. Это означает, что HD-аудио контроллер также включается / выключается динамически. Эта функция включена только для определенных контроллеров таких чипов, как Intel LynxPoint. Вы можете принудительно включить / отключить эту функцию, установив power_save_controller опция, которая также доступна в каталоге /sys/module/snd_hda_intel/parameters.

Точки трассировки

Драйвер hd-audio предоставляет несколько основных точек трассировки.

hda: hda_get_response	отслеживает ответ от RIRB (только при чтении из драйвера кодека).		
hda: hda_send_cmd	отслеживает каждую запись CORB пока hda: hda_bus_reset	отслеживает сброс шины из-за фатальной ошибки и т.Д.	
hda: hda_unsol_event	отслеживает не- запрошенные события, и hda: hda_power_down	и hda: hda_power_up	отслеживание отключения / увеличения мощности с помощью режима энергосбережения .

Включение всех точек трассировки можно выполнить следующим образом

```
# echo 1 > /sys/kernel/debug/трассировка/события/hda/включить
```

затем после нескольких команд вы можете выполнять трассировку из файла /sys/kernel/debug/tracing/trace. Например, когда вы хотите отследить, какая команда кодека отправлена, включите точку трассировки следующим образом:


```
# cat
/sys/kernel/debug/tracing/трассировка # трассировщик:
nop #
# CPU # ВРЕМЯДАЧА-МЕШКА ФУНКЦИЯ
# | | | / |
<...>- 7 8 0 7 [ 0 0 2 ] 1 0 5 1 4 7 . 7 7 4 8 8 9 : hda_send_cmd: [ 0 : 0 ] val=e 3 a 0 1 9
<...>- 7 8 0 7 [ 0 0 2 ] 1 0 5 1 4 7 . 7 7 4 8 9 3 : hda_send_cmd: [ 0 : 0 ] val=e 3 9 0 1 9
<...>- 7 8 0 7 [ 0 0 2 ] 1 0 5 1 4 7 . 9 9 9 5 4 2 : hda_send_cmd: [ 0 : 0 ] val=e 3 a 0 1 a
<...>- 7 8 0 7 [ 0 0 2 ] 1 0 5 1 4 7 . 9 9 9 5 4 3 : hda_send_cmd: [ 0 : 0 ] val=e 3 9 0 1 a
<...>- 2 6 7 6 4 [ 0 0 1 ] 3 4 9 2 2 2 . 8 3 7 1 4 3 : hda_send_cmd: [ 0 : 0 ] значение = e 3 a 0 1 9
<...>- 2 6 7 6 4 [ 0 0 1 ] 3 4 9 2 2 2 . 8 3 7 1 4 8 : hda_send_cmd: [ 0 : 0 ] значение=e 3 9 0 1 9
<...>- 2 6 7 6 4 [ 0 0 1 ] 3 4 9 2 2 3 . 0 5 8 5 3 9 : hda_send_cmd: [ 0 : 0 ] значение = e 3 a 0 1 a
<...>- 2 6 7 6 4 [ 0 0 1 ] 3 4 9 2 2 3 . 0 5 8 5 4 1 : hda_send_cmd: [ 0 : 0 ] значение=e 3 9 0 1 a
```

Здесь [0 : 0] указывает номер карты и адрес кодека, а val показывает значение, отправленное на кодек, соответственно. Значение является упакованным значением, и вы можете декодировать его с помощью программы hda-decode-verb, включенной в пакет hda-emu, приведенный ниже. Например, значение e3a019 предназначено

```
для установки значения левого выходного усилителя равным 25. % hda-декодирование-платон 0 xe 3 a 0 1 9
исходное значение = 0 x 0 0 e 3 a 0 1 9
cid = 0, nid = 0 x 0 e, verb = 0 x 3 a 0, parm = 0 x 1 9
исходное значение: verb = 0 x 3 a 0, parm = 0 x 1 9
verbname = set_amp_gain_mute
amp raw val = 0 xa 0 1 9
вывод, слева, idx = 0, отключение звука = 0, val = 2 5
```

Дерево разработки

Последние коды разработки для HD-audio находятся в sound git tree:

- [git://git.kernel.org/pub/scm/linux/kernel/git/tiwai/sound.git](https://git.kernel.org/pub/scm/linux/kernel/git/tiwai/sound.git)

Основная ветвь или ветви for-next могут использоваться в качестве основных ветвей разработки в целом, в то время как разработка для текущего и следующего ядер находится в ветвях for-linus и for-next соответственно.

Отправка отчета об ошибке

Если какие-либо параметры модели или модуля не работают для вашего устройства, пришло время отправить отчет об ошибке разработчикам. Укажите следующее в своем отчете об ошибке:

- Поставщик оборудования, названия продукта и модели
- Версия ядра (и версия драйвера ALSA, если вы собирали извне) •

alsa-info.sh вывод; запуск с параметром --без загрузки . Смотрите раздел ниже об alsa-info

Если это регрессия, в лучшем случае отправьте выходные данные alsa-info

как рабочего, так и нерабочего ядер. Это действительно полезно, потому

что мы можем напрямую сравнивать регистры кодеков. Отправьте отчет

об ошибке следующим образом: **ядро-bugzilla** <https://bugzilla.kernel.org>

/ **alsa-devel ML** alsa-devel@alsa-project.org

5.1.5 Инструменты отладки

В этом разделе описаны некоторые инструменты, доступные для отладки проблем с HD-audio.

alsa-info

Скрипт `alsa-info.sh` является очень полезным инструментом для сбора информации об аудиоустройстве. Он включен в пакет `alsa-utils`. Последнюю версию можно найти в репозитории `git`:

• `git://git.alsa-project.org/alsa-utils.git`

Скрипт также можно получить напрямую по следующему URL:

• `http://www.alsa-project.org/alsa-info.sh`

Запустите этот скрипт от имени `root`, и он соберет важную информацию, такую как списки модулей, параметры модуля, содержимое файла `proc`, включая файлы `proc` кодека, выходные данные микшера и элементы управления. По умолчанию информация будет храниться на веб-сервере на `alsa-project.org`. Но, если вы отправите отчет об ошибке, было бы лучше запустить с `--no-upload` выберите и прикрепите сгенерированный файл.

Есть несколько других полезных опций. Смотрите `--справка` вывод опции для получения подробной информации. Когда возникает ошибка проверки или когда драйвер явно назначает несоответствующую модель, было бы полезно загрузить драйвер с помощью `probe_only= 1` выберите опцию (в лучшем случае после холодной перезагрузки) и запустите `alsa-info` в этом состоянии. При использовании этой опции драйвер не будет настраивать микшер и РСМ, а просто попытается проверить слот кодека. После проверки доступен файл `proc`, так что вы можете получить исходную информацию о кодеке до внесения изменений в драйвер. Конечно, драйвер нельзя использовать с `probe_only= 1`. Но вы можете продолжить настройку через файл `hwdep sysfs`, если включена опция `hda-reconfig`. Используя `probe_only` маска 2 пропускает сброс кодеков HDA (используйте `probe_only= 3` в качестве опции модуля). Интерфейс `hwdep` может использоваться для определения инициализации кодека BIOS.

hda-глагол

`hda-verb` - это небольшая программа, которая позволяет вам напрямую обращаться к HD-аудиокодеку. С ее помощью вы можете выполнить `raw` команду HD-audio codec verb. Эта программа обращается к устройству `hwdep`, поэтому вам необходимо включить конфигурацию ядра `CONFIG_SND_HDA_HWDEP=y` заранее.

Программа `hda-verb` принимает четыре аргумента: файл устройства `hwdep`, NID виджета, глагол и параметр. Когда вы получаете доступ к кодеку в слоте 2 карты 0, обычно передайте `/dev/snd/hwC0D2` в качестве первого аргумента. (Однако реальное имя пути зависит от системы.)

Второй параметр - это номер виджета-id для доступа. Третьим параметром может быть либо шестнадцатеричное/значное число, либо строка, соответствующая глаголу. Аналогично, последний параметр является значением для записи или может быть строкой для типа параметра.

```
% hda-глагол /dev/snd/hwC 0 D 0 0 x 1 2 0 x 7 0 1 2

nid = 0 x 1 2 , verb = 0 x 7 0 1 , param = 0 x 2

значение = 0 x 0

% hda-глагол /dev/snd/hwC 0 D 0 0 x 0 ПАРАМЕТРЫ VENDOR_ID

nid = 0 x 0 , verb = 0 x f 0 0 , param = 0 x 0

значение = 0 x 1 0 ec 0 2 6 2

% hda-глагол /dev/snd/hwC 0 D 0 2 set_a 0 xb 0 8 0

nid = 0 x 2 , глагол = 0 x 3 0 0 , параметр = 0 xb 0 8 0

значение = 0 x 0
```

Хотя с помощью этой программы вы можете вводить любые команды, состояние драйвера не всегда будет обновляться.

Например, значения громкости обычно кэшируются в драйвере, и, таким образом, изменение значения виджета `amp` напрямую через `hda-verb` не изменит значение микшера.

Программа `hda-verb` теперь включена в `alsa-tools`:

• `git://git.alsa-project.org/alsa-tools.git`

Кроме того, старый автономный пакет находится в каталоге `ftp`:

• `ftp://ftp.suse.com/pub/people/tiwai/misc/`

Также доступен репозиторий git:

• [git://git.kernel.org/pub/scm/linux/kernel/git/tiwai/hda-verb.git](https://git.kernel.org/pub/scm/linux/kernel/git/tiwai/hda-verb.git)

Смотрите файл README в архиве, чтобы узнать больше о программе hda-verb.

hda-анализатор

hda-анализатор предоставляет графический интерфейс для доступа к элементу управления raw HD-audio, основанный на привязке pyGTK2. Это более мощная версия hda-verb. Программа предоставляет вам простой в использовании графический интерфейс для отображения информации о виджете и настройки значений amp, а также вывода, совместимого с proc. HDA-анализатор:

• <http://git.alsa-project.org/?p=alsa.git;a=деревя;f=hda-анализатор>

является частью репозитория alsa.git в alsa-project.org:

• [git://git.alsa-project.org/alsa.git](https://git.alsa-project.org/alsa.git)

Codecgraph

Codecgraph - это служебная программа для генерации графика и визуализации соединения кодек-узел чипа codec . Это особенно полезно, когда вы анализируете или отлаживаете кодек без надлежащей таблицы данных. Программа анализирует данный программный файл codec и преобразует в SVG с помощью программы graphviz. Деревья tarball и GIT находятся на веб-странице по адресу:

• <http://helllabs.org/codecgraph/>

hda-emu

hda-emu - это эмулятор HD-audio. Основная цель этой программы - отладить HD-аудиокодек без использования реального оборудования. Таким образом, он не имитирует поведение при реальном вводе-выводе звука, а просто сбрасывает изменения регистра кодека и внутренние изменения ALSA-драйвера при проверке и эксплуатации HD-audio драйвера. Для имитации программе требуется proc-файл кодека. Заранее подготовьте файл proc для целевого кодека или выберите пример кодека из коллекций codec proc в архиве. Затем запустите программу с файлом proc, и программа hda-emu начнет разбор файла кодека и

```
имитирует драйвер HD-audio: % hda-кодеки emu/stac 9 2 0 0 -dell-d 8 2 0 -ноутбук

# Разбор.

hda_codec:  Неизвестная модель для STAC 9 2 0 0 , использование BIOS по умолчанию

hda_codec:  pin-код  0 8  bios pin config  4 0 c 0 0 3 fa
.....
```

Программа предоставляет вам только очень тупой интерфейс командной строки. Вы можете получить дампы proc-файла в текущем состоянии, получить список элементов управления (микшера), установить / получить значение элемента управления, смоделировать работу PCM , имитацию подключения разъема и т.д. Программа находится в репозитории git ниже:

• [git://git.kernel.org/pub/scm/linux/kernel/git/tiwai/hda-emu.git](https://git.kernel.org/pub/scm/linux/kernel/git/tiwai/hda-emu.git)

Смотрите файл README в репозитории для получения более подробной информации о программе hda-emu.

hda-jack-retask

hda-jack-retask - это удобная программа с графическим интерфейсом для управления выводом HD-audio для переназначения разъема. Если у вас возникли проблемы с назначением гнезда, попробуйте эту программу и проверьте, сможете ли вы получить полезную информацию

Результаты. Как только вы определитесь с правильным назначением pin-кода, его можно исправить либо в коде драйвера, статически, либо путем передачи файла исправления прошивки (см. Раздел "Раннее исправление"). Программа теперь включена в alsa-tools:

- `git://git.alsa-project.org/alsa-tools.git`

5.2 HD-модели, зависящие от аудиокодека

5.2.1 ALC880

3 разъема 3 разъема на задней панели и выход для наушников

3 разъема-выемка 3-разъем сзади, выход HP и SPDIF

5-ступенчатый 5-разъем сзади, 2-разъем спереди

5-ступенчатый-выемка 5-разъем сзади, 2-разъем спереди, выход SPDIF

6 ступеней 6-разъем сзади, 2-разъем спереди

6 ступеней-выемка 6-разъемный с SPDIF выходом

6stack-автоматическое отключение 6-разъемный с определением разъема для наушников

5.2.2 ALC260

gpio1 Включить GPIO1

coef Включить EAPD с помощью таблицы COEF

fujitsu Quirk для FSC S7020

fujitsu-jwse Функция Quirk для FSC S7020 с режимами подключения и поддержкой микрофонов HP

5.2.3 ALC262

inv-dmic Обходной путь с инвертированным внутренним микрофоном

5.2.4 ALC267 / 268

inv-dmic Решение проблемы с инвертированным

внутренним микрофоном **hp-eapd** Отключите HP EAPD при NID 0x15

5.2.5 ALC22x / 23x / 25x / 269 / 27x / 28x / 29x (и модели ALC3xxx от конкретного производителя)

ноутбук-amic Ноутбуки с аналоговым микрофонным входом

ноутбук-dmic Ноутбуки с цифровым микрофонным входом

alc269-dmic Включить обходной путь цифрового микрофона ALC269 (VA)

alc271-dmic Включить обходной путь цифрового микрофона ALC271X

inv-dmic Способ обхода внутреннего микрофона с обратным подключением

гарнитура-микрофон Указывает на комбинированный разъем для гарнитуры (наушники + микрофон) **режим гарнитуры** Более полная поддержка гарнитуры для ALC269 & co

режим гарнитуры-без hp-микрофона Поддержка режима гарнитуры без микрофона для наушников

lenovo-док-станция Включает ввод-вывод док-станции для некоторых Lenovos

hp-gpio-led Поддержка GPIO LED на ноутбуках HP

hp-dock-gpio-mic1-led Док-станция HP с поддержкой микрофона со светодиодной подсветкой

dell-гарнитура-мульти Разъем для гарнитуры, который также можно использовать в качестве

микрофонного входа **dell-док-станция для гарнитуры** Разъем для гарнитуры (без

микрофонного входа), а также стыковочный ввод-вывод **alc283-dac-wcaps** Исправления для Chromebook

с ALC283 **alc283-sense-combo** Определение комбинированного разъема на ALC283

tpt440-док-станция Настройки контактов для поддержки док-станции Lenovo Thinkpad

tpt440 Настройка Lenovo Thinkpad T440s

tpt460 Настройка Lenovo Thinkpad T460 / 560

двойные кодеки Ноутбуки Lenovo с двумя кодеками

5.2.6 ALC66x / 67x / 892

mario Исправление модели Chromebook mario

asus-mode1 ASUS

asus-mode2 ASUS

asus-mode3 ASUS

asus-mode4 ASUS

asus-mode5 ASUS

asus-mode6 ASUS

asus-mode7 ASUS

asus-mode8 ASUS

inv-dmic Решение проблемы инвертирования внутреннего микрофона

dell-гарнитура-multi Разъем для наушников, который также может

использоваться в качестве микрофона **двойные кодеки** Ноутбуки Lenovo с двумя кодеками

5.2.7 ALC680

N/A

5.2.8 ALC88x/898/1150

acer-aspire-4930g Acer Aspire 4930G/5930G/6530G/6930G/7730G

acer-aspire-8930g Acer Aspire 8330G /6935G

acer-aspire Другие Acer Aspire

inv-dmic Обходной путь с инвертированным внутренним микрофоном

отсутствие основного-hp обходной путь VAIO Z / VGC-LN51JGB (для ЦАП со стационарным динамиком)

двойные кодеки Двойные кодеки ALC1220 для игровых мобильных устройств

5.2.9 ALC861/660

N/A

5.2.10 ALC861VD/660VD

N/A

5.2.11 CMI9880

- минимальный размер** 3 разъема сзади
- min_fr** 3 домкрата сзади, 2 домкрата спереди
- полный** 6 домкратов сзади, 2 домкрата спереди
- полный привод** 6-разъем сзади, 2-разъем спереди, ввод/ вывод SPDIF
- все включено** 5-разъем сзади, 2-разъем спереди, вывод SPDIF
- автоматический** автоматическое считывание настроек BIOS (по умолчанию)

5.2.12 AD1882 / AD1882A

- 3stack** 3-стековый режим
- 3stack-автоматическое отключение** 3-стопочный режим с автоматическим отключением передней панели HP (по умолчанию) **6-стопочный режим** 6-стопочный режим

5.2.13 AD1884A / AD1883 / AD1984A / AD1984B

- рабочий стол настольный компьютер с тремя стеками (по умолчанию) ноутбук ноутбук с разъемом HP
 - для мобильных устройств с разъемом HP для мобильных устройств thinkpad Lenovo Thinkpad X300 touchsmart HP Touchsmart
- 5.2.14 AD1884

N/A

5.2.15 AD1981

базовый разъем с 3 разъемами (по умолчанию) hp HP nx6320 thinkpad Lenovo Thinkpad T60/X60/Z60 toshiba Toshiba U205

5.2.16 AD1983

N/A

5.2.17 AD1984

базовая конфигурация по умолчанию thinkpad Lenovo Thinkpad T61/X61 dell_desktop Dell T3400

5.2.18 AD1986A

3stack 3-стековый, с общим окружением

ноутбук только 2-канальный (FSC V2060, Samsung

M50) **ноутбук-іmic** 2-канальный со встроенным

микрофоном **eapd** Постоянно включайте EAPD

5.2.19 AD1988 / AD1988B / AD1989A / AD1989B

6-ступенчатый 6-гнездовой

6-ступенчатый-рыть то же самое с SPDIF

3stack 3-разъемный

3stack-dig то же самое с SPDIF

ноутбук 3-разъемный с автоматическим отключением hp-jack

для ноутбука-dig то же самое с SPDIF

авто чтение BIOS с автоматической настройкой (по умолчанию)

5.2.20 Conexant 5045

cap-mix-amp Исправлен максимальный уровень

входного сигнала в виджете микширования **toshiba-p105** Toshiba

P105 quirk **hp-530** HP 530 quirk

5.2.21 Conexant 5047

cap-mix-amp Исправлен максимальный уровень входного сигнала в виджете mixer

5.2.22 Conexant 5051

lenovo-x200 Причуда Lenovo X200

5.2.23 Conexant 5066

стерео-dmic Обходной путь для инвертированного цифрового

стереомикрофона **gpio1** Включить вывод GPIO1

вывод для наушников-микрофона Включить NID микрофона наушников

0x18 без обнаружения **tp410** Thinkpad T400 & co quirks

thinkpad Индикатор отключения звука Thinkpad / mic quirk

lemote-a1004 Lemote A1004 quirk

lemote-a1205 Lemote A1205 quirk

olpc-xo OLPC XO quirk

отключение звука с помощью светодиода eapd Управление

отключением звука с помощью светодиода EAPD **hp-док-станция** Поддержка HP dock

индикатор отключения звука-gpio Управление светодиодом отключения звука через GPIO

5.2.24 STAC9200

ссылка Справочная доска

oqo OQO Модель 2

dell-d21 Dell (неизвестно)

dell-d22 Dell (неизвестно)

dell-d23 Dell (неизвестно)

dell-m21 Dell Inspiron 630m, Dell Inspiron 640m

dell-m22 Dell Latitude D620, Dell Latitude D820

dell-m23 Dell XPS M1710, Dell Precision M90

dell-m24 Dell Latitude 120L

dell-m25 Dell Inspiron E1505n

dell-m26 Dell Inspiron 1501

dell-m27 Dell Inspiron E1705/9400

шлюз-m4 Ноутбуки Gateway с управлением EAPD

gateway-m4-2 Ноутбуки Gateway с управлением

EAPD **panasonic** Panasonic CF-74

автоматический Настройка BIOS (по умолчанию)

5.2.25 STAC9205/9254

ссылка Справочная плата

dell-m42 Dell (неизвестно)

dell-m43 Dell Precision

dell-m44 Dell Inspiron

eapd Включите EAPD (например, Gateway T1616)

автоматическая настройка BIOS (по умолчанию)

5.2.26 STAC9220/9221

ссылка Справочная плата

3stack D945 3stack

5stack D945 5stack + SPDIF

intel-mac-v1 Intel Mac Type 1

intel-mac-v2 Intel Mac Type 2

intel-mac-v3 Intel Mac Type 3

intel-mac-v4 Intel Mac Type 4

intel-mac-v5 Intel Mac Type 5

intel-mac-автоматически Intel Mac (определяет тип в соответствии с идентификатором подсистемы)

macmini Intel Mac Mini (эквивалент type 3)

macbook Intel Mac Book (эквивалент type 5)

macbook-pro-v1 Intel Mac Book Pro 1-го поколения (эквалайзер

type 3) **macbook-pro** Intel Mac Book Pro 2-го поколения

(эквалайзер type 3) **imac-intel** Intel iMac (эквалайзер типа 2)

imac-intel-20 Intel iMac (более новая версия) (эквалайзер типа

3) **ecs202** Микросхемы ECS/PC

dell-d81 Dell (неизвестно)

dell-d82 Dell (неизвестно)

dell-m81 Dell (неизвестно)

dell-m82 Dell XPS M1210

авто настройка BIOS (по умолчанию)

5.2.27 STAC9202/9250 /9251

ссылка Справочная плата, базовая конфигурация

m1 Некоторые ноутбуки серии Gateway MX (NX560XL)

m1-2 Некоторые ноутбуки серии Gateway MX (MX6453)

m2 Некоторые ноутбуки серии Gateway MX (M255)

m2-2 Некоторые ноутбуки серии Gateway MX

m3 Некоторые ноутбуки серии Gateway MX

m5 Некоторые ноутбуки серии Gateway MX (MP6954)

m6 Некоторые ноутбуки серии Gateway NX

автоматическая настройка BIOS (по умолчанию)

5.2.28 STAC9227/9228/9229/ 927x

ссылка Контрольная плата

ссылка-нет-jd Контрольная плата без определения разъема HP /

Mic **3stack** D965 3stack

5stack D965 5stack + SPDIF

5stack-no-fp D965 5stack без передней панели

dell-3stack Dell Dimension E520

dell-bios Исправления при настройке Dell BIOS

dell-bios-amic Исправления в настройках Dell BIOS, включая

аналоговый микрофон **volknob** Исправления в виджете

регулировки громкости 0x24 **авто** настройка BIOS (по умолчанию)

5.2.29 STAC92HD71B*

ссылка Справочная плата

dell-m4-1 Настольные компьютеры Dell

dell-m4-2 Настольные компьютеры Dell

dell-m4-3 Настольные компьютеры Dell

hp-m4 HP mini 1000

hp-dv5 Серия HP dv

hp-hdx Серия HP HDX

hp-dv4-1222nr HP dv4-1222nr (с поддержкой светодиодов)

автоматическая настройка BIOS (по умолчанию)

5.2.30 STAC92HD73*

ссылка Справочная плата

нет-jd Настройка BIOS, но без определения разъема

intel Intel DG45 * мобильные устройства

dell-m6-amic Настольные компьютеры / ноутбуки Dell с аналоговыми микрофонами

dell-m6-dmic Настольные компьютеры Dell / ноутбуки с цифровыми микрофонами

dell-m6 Настольные компьютеры Dell / ноутбуки с обоими типами микрофонов

dell-eq Настольные компьютеры / ноутбуки Dell

alienware Alienware M17x

asus-mobo Настройки контактов для ASUS mobo с выходом 5.1 / SPDIF

автоматическое Настройка BIOS (по умолчанию)

5.2.31 STAC92HD83*

ссылка Справочная плата

ссылка на микрофон Справочная плата с управлением питанием для портов

dell-s14 Ноутбук Dell

dell-vostro-3500 Ноутбук Dell Vostro 3500

hp-dv7-4000 HP dv-7 4000

hp_cNB11_intquad Модели HP CNB с 4 динамиками

hp-zephyr HP Zephyr

hp-led HP со сломанным BIOS для индикатора отключения звука

hp-inv-led HP со сломанным BIOS для перевернутого индикатора отключения звука

hp-mic-led HP со светодиодом отключения микрофона

разъем для подключения гарнитуры Dell Latitude с 4-контактным разъемом для

подключения гарнитуры **hp-envy-bass** Исправление контактов для басового динамика HP Envy (NID 0x0f)

hp-envy-ts-bass Исправление вывода для низкочастотного динамика HP Envy TS (NID 0x10)

hp-bnb13-eq Настройка аппаратного эквалайзера для ноутбуков HP

hp-envy-ts-bass Поддержка HP Envy TS bass

авто настройка BIOS (по умолчанию)

5.2.32 STAC92HD95

hp-led Поддержка светодиодов для ноутбуков HP

hp-bass Настройка частоты низких частот для HP Spectre 13

5.2.33 STAC9872

vaio Ноутбук VAIO без SPDIF

авто настройка BIOS (по умолчанию)

5.2.34 Cirrus Logic CS4206/4207

mbp53 MacBook Pro 5,3

mbp55 MacBook Pro 5,5

imac27 iMac 27 дюймов

imac27_122 iMac 12,2

Apple Универсальная особенность

Apple **mbp101** MacBookPro 10,1

mbp81 MacBookPro 8,1

mba42 MacBookAir 4,2

авто настройка BIOS (по умолчанию)

5.2.35 Cirrus Logic CS4208

mba6 MacBook Air 6,1 и 6,2

gpio0 Включить GPIO 0 amp

mbp11 MacBookPro 11,2

macmini MacMini 7,1

авто Настройка BIOS (по умолчанию)

5.2.36 ЧЕРЕЗ VT17xx / VT18xx / VT20xx

авто настройка BIOS (по умолчанию)

5.3 Элементы управления микшером для конкретного HD-аудиокодека

В этом файле описываются элементы управления микшером для конкретного кодека.

5.3.1 Кодеки Realtek

Режим канала Это элемент управления перечислением для изменения настроек канала объемного звучания, появляется только при доступны каналы объемного звучания. Здесь указывается количество используемых каналов: "2ch", "4ch", "6ch", и "8ch". В зависимости от конфигурации, это также управляет перенастройкой гнезд для нескольких разъемов ввода-вывода.

Режим автоматического отключения звука Это элемент управления перечислением для изменения режима автоматического отключения звука разъемов для наушников и линейного выхода- . Если на устройстве имеются встроенные динамики, разъемы для наушников и / или линейного выхода, появится это управление. Когда есть только разъемы для наушников или линейного выхода, он выдает состояние "Отключено" и "Включено". При включении громкоговоритель автоматически отключается при подключении к разъему. При наличии разъемов для наушников и линейного выхода отображается "Отключено", "Только для динамиков" и "Линейный выход + динамик". Если выбран режим "Только громкоговоритель", подключение к наушникам или разъему линейного выхода отключает громкоговорители, но не линейные выходы. При выборе линейный выход + динамик подключение к разъему для наушников отключает звук как динамиков, так и линейных выходов.

5.3.2 Кодеки IDT / Sigmatel

Аналоговая обратная связь Этот элемент управления включает / отключает аналоговую схему обратной связи. Это отображается только тогда, когда для параметра "обратная связь" в подсказке по кодеку установлено значение true (см. HD-Audio.txt). Обратите внимание, что в некоторых кодеках аналоговая обратная связь и обычное воспроизведение PCM являются исключительными, т. е. Когда это включено, вы не услышите никакого PCM потока.

Поменять центр/LFE Меняет порядок расположения каналов center и LFE. Обычно левый соответствует центру , а правый - LFE. Когда он включен, слева - LFE, а справа - по центру.

Наушники как линейный выход. Когда этот элемент управления включен, считайте разъемы для наушников разъемами линейного выхода. То есть, наушники не отключают автоматически другие линейные выходы, и HP-амп не настроен на контакты.

Режим микрофонного разъема, режим линейного разъема и т.д. Эти перечисления регулируют направление и смещение входного разъема. булавки. В зависимости от типа разъема он может быть установлен как "Микрофонный вход" и "Линейный вход" для определения смещения входного сигнала, или он может быть установлен в положение "Линейный выход", когда вывод является разъемом с несколькими входами / выводами для каналов объемного звучания.

5.3.3 С ПОМОЩЬЮ кодеков

Smart 5.1 Элемент управления перечислением для переназначения разъемов ввода-вывода для выходов объемного звучания. Когда он включен, кор- отвечающие входные разъемы (обычно линейный вход и микрофонный вход) переключаются как выходные разъемы surround и CLFE .

Независимый HP Когда этот элемент управления перечислением включен, выход на наушники направляется из отдельного потока (третий PCM, такой как hw: 0,2) вместо основного потока. В случае, если ЦАП для наушников используется совместно с боковым или CLFE-канальным ЦАП, ЦАП переключается на наушники автоматически.

Микширование с обратной связью Элемент управления перечислением, определяющий, включен ли аналоговый маршрут с обратной связью или нет. Когда он включен, аналоговый маршрут с обратной связью микшируется с передним каналом. Кроме того, тот же маршрут используется для выходов наушников и динамиков. В качестве побочного эффекта, когда установлен этот режим, индивидуальные регуляторы громкости больше не будут доступны для наушников и динамиков, поскольку к виджету микшера подключен только один ЦАП .

Динамическая регулировка мощности. Этот элемент управления определяет, включено ли динамическое регулирование мощности для каждого разъема . При включении состояние питания виджетов (D0 / D3) изменяется динамически в зависимости от состояния подключения к разъему для экономии потребляемой мощности. Однако, если ваша система не обеспечивает надлежащее обнаружение гнезда, это не сработает; в таком случае отключите этот элемент управления.

Обнаружение гнезда Это управление предусмотрено только для кодека VT1708, который не выдает надлежащего нежелательного события для каждого разъема. разъем jack. Когда он включен, драйвер опрашивает функцию определения разъема, чтобы могло работать автоматическое отключение звука наушников, в то время как выключение этой функции снизило бы энергопотребление.

5.3.4 Смежные кодеки

Режим автоматического отключения звука Смотрите кодеки Reatek.

5.3.5 Аналоговые кодеки

Режим канала Это элемент управления перечислением для изменения настроек канала объемного звучания, появляется только при наличии каналов объемного звучания. В нем указывается количество используемых каналов: "2ch", "4ch" и "6ch".

В зависимости от конфигурации, это также управляет перенастройкой гнезд для нескольких разъемов ввода-вывода.

Независимый HP Когда этот элемент управления перечислением включен, выход для наушников направляется от отдельного устройства. поток (третий PCM, такой как hw: 0,2) вместо основного потока.

Поддержка 5.4 HD-Audio DP-MST

Для поддержки DP MST audio драйвер кодека HD Audio hdmi вводит виртуальный PIN-код и динамическое назначение rcm. Виртуальный rpin-код является расширением per_rpin. Наибольшее отличие DP MST от legacy заключается в том, что DP MST вводит запись устройства. Каждый rpin-код может содержать несколько записей устройства. Каждая запись устройства ведет себя как rpin-код. Поскольку каждый rpin-код может содержать несколько записей устройства и каждый кодек может содержать несколько rpin-кодов, если мы используем один rcm на per_rpin, будет много PCM. Новое решение заключается в создании нескольких PCM и динамической привязке rcm к per_rpin. Драйвер использует флаг spec-> dyn_pcm_assign, чтобы указать, использовать ли новое решение.

5.4.1 PCM

Будет добавлено дополнительно

5.4.2 Инициализация Rpin-кода

Каждый rpin-код может иметь несколько записей устройства (виртуальных rpin-кодов). На платформе Intel количество записей устройства динамически изменяется. Если подключен концентратор DP MST, он находится в режиме DP MST, и номер записи устройства равен 3. В противном случае номер записи устройства равен 1. Для упрощения реализации все записи устройства будут инициализированы при загрузке, независимо от того, находится ли оно в режиме DP MST или нет.

5.4.3 Список подключений

DP MST повторно использует код списка подключений. Код можно использовать повторно, поскольку записи устройств с одним и тем же PIN-кодом имеют один и тот же список подключений. Это означает, что DP MST получает список подключений для ввода устройства без настройки ввода устройства.

Разъем 5.4.4

Предполагается, что:

- MST должен быть dyn_pcm_assign, и это acomp (для сценария Intel);
- HE-MST может быть или не быть dyn_pcm_assign, это может

быть acomp или !accomp; **Итак, возможны следующие сценарии:**

1. MST (&& dyn_pcm_assign && acomp)
2. HE-MST && dyn_pcm_assign && acomp

3. HE MST && !dyn_pcm_assign && !acompr

В приведенном ниже обсуждении будет проигнорирована разница между MST и HE-MST, поскольку это не слишком сильно влияет на управление домкратом. Драйвер использует массив `struct hdmi_pcm pcm[]` в `hdmi_spec`, а `snd_jack` является членом `hdmi_pcm`. Каждый вывод имеет один указатель `struct hdmi_pcm * pcm`. Для `!dyn_pcm_assign`, `per_pin->pcm` будет статически присвоен `spec->pcm[n]`. Для `dyn_pcm_assign` параметр `per_pin->pcm` будет присвоен параметру `spec->pcm[n]` при горячем подключении монитора.

Разъем для сборки

- `dyn_pcm_assign`

Не будет использовать `hda_jack`, но используйте `snd_jack` в спецификации->
`pcm_rec[pcm_idx].jack` напрямую. • `!dyn_pcm_assign`

Используйте `hda_jack` и назначьте `spec->pcm_rec[pcm_idx].jack = jack->jack` статически.

Включение нежелательного события

Включить нежелательное событие `if!acompr`.

Отслеживать обработку событий горячего подключения

- `acompr`

`pin_eld_notify()` -> `check_presence_and_report()` ->
`hdmi_present_sense()` -> `sync_eld_via_acompr()`. Используйте непосредственно
`snd_jack_report()` в `spec->pcm_rec[pcm_idx].jack` как для
`dyn_pcm_assign`, так и для `!dyn_pcm_assign` • `!acompr`

<code>hdmi_unsol_event()</code>	->	<code>hdmi_intrinsic_event()</code>	->	проверка присутствия и репортажа()	->
<code>hdmi_present_sense()</code>	->	<code>hdmi_prepresent_sense_via_verbs()</code>			

Используйте непосредственно `snd_jack_report()` в `spec->pcm_rec[pcm_idx].jack`
 для `dyn_pcm_assign`. Используйте `hda_jack` механизм для обработки событий `jack`.

5.4.5 Другие будут добавлены позже.

ИНФОРМАЦИЯ, КАСАЮЩАЯСЯ КОНКРЕТНОЙ КАРТЫ

6.1 Поддержка аналогового джойстика в драйверах ALSA

14 октября 2003 г.

Такаши Иваи <tiwai@suse.de>

6.1.1 Общие положения

Прежде всего, вам необходимо включить поддержку GAMEPORT в ядре Linux для использования джойстика с драйвером ALSA. Для получения подробной информации о поддержке gameport обратитесь к Documentation/input/joystick.txt. Поддержка джойстиков драйверами ALSA отличается для карт ISA и PCI. В случае карт ISA (PnP), обычно этим занимается независимый модуль (ps558). Между тем, драйверы ALSA PCI имеют встроенную поддержку gameport. Следовательно, когда драйвер ALSA PCI встроен в ядро, CONFIG_GAMEPORT также должен быть 'y'. В противном случае поддержка игрового порта на этой карте будет (автоматически) отключена. Некоторые модули адаптера проверяют физическое подключение устройства во время загрузки. Было бы безопаснее подключить устройство джойстика перед загрузкой модуля.

6.1.2 Карты PCI

Для карт PCI джойстик включается, когда указана соответствующая опция модуля. Некоторым драйверам не нужны опции, и поддержка джойстика всегда включена. В предыдущей версии ALSA для активации джойстика использовался динамический API управления. Однако он был изменен на параметры статического модуля из-за стабильности системы и управления ресурсами. Следующие драйверы PCI поддерживают джойстик изначально.

Драйвер	Опция	Доступные значения
als4000		
au88x0	модуль joystick_port без джойстика	0 = отключить (по умолчанию), 1 = автоматическое определение, вручную: любой
azf3328		
ens1370	joystick	адрес (например, 0x200) N/A 0 = отключить, 1 = включить, -1 = автоматически (по умолчанию) 0 = отключить (по умолчанию), 1 = включить 0 = отключить (по умолчанию), 1 = включить
ens1371	joystick_port	
cmipci		
cs4281	joystick_port Без	= автоматическое определение, вручную:
cs46xx	джойстика Без	0x200, 0x208, 0x210, 0x218 0 = отключить (по умолчанию), 1 = автоматическое
es1938	джойстика Без	определение, вручную: любой адрес
es1968	джойстика Без	(например, 0x200) N / A N/A N/A 0 = отключить (по умолчанию), 1 = включить N/A
son-		
icvibes	джойстика без	
trident	джойстика без	N/A
via82xx		
ymfpci	джойстика	0 = отключить (по умолчанию), 1 = включить 0 = отключить (по умолчанию), 1 = автоматическое определение, вручную: 0x201, 0x202, 0x204, 0x205

Следующие драйверы не поддерживают gameport изначально, но существуют дополнительные модули. Загрузите соответствующий модуль, чтобы добавить поддержку gameport.

Драйвер	Дополнительный
emu10k1	модуль
fm801	emu10k1-gr fm801-gr

Примечание: модули "pcisgame" и "cs461x" предназначены только для драйверов OSS. Эти драйверы ALSA (cs46xx, trident и au88x0) имеют встроенную поддержку gameport. Как упоминалось выше, драйверы ALSA PCI имеют встроенную поддержку gameport, поэтому вам не нужно загружать модуль ns558 . Просто загрузите "joydev" и соответствующий модуль адаптера (например, "analog").

Платы 6.1.3 ISA

Драйверы ALSA ISA не имеют встроенной поддержки gameport. Вместо этого вам нужно загрузить модуль "ns558" в дополнение к "joydev" и модулю адаптера (например, "analog").

6.2 Краткие сведения о драйвере C-Media 8338/8738/8768/8770

Такаши Иваи <tiwai@suse.de>

6.2.1 Многоканальное воспроизведение спереди/сзади

Микросхема CM8x38 может использовать АЦП в качестве второго ЦАП, так что для воспроизведения спереди / сзади можно использовать два разных стереоканала. Поскольку имеется два ЦАП, оба потока обрабатываются независимо, в отличие от 4/6-канального многоканального воспроизведения, описанного в разделе ниже. По умолчанию драйвер ALSA назначает первое PCM-устройство (т. е. hw: 0,0 для карты # 0) для воспроизведения на передней панели и по 4 / 6 каналам, в то время как второе PCM-устройство (hw: 0,1) назначается второму ЦАП для воспроизведения на задней панели. Между двумя ЦАП есть небольшие различия:

- Первый ЦАП поддерживает форматы U8 и S16LE, в то время как второй ЦАП поддерживает только S16LE.
- Второй ЦАП поддерживает только двухканальное стерео.

¹ Только VIA686A/B
² В микросхемах YMF744/ 754 адрес порта может быть выбран произвольно

Пожалуйста, обратите внимание, что ЦАП SM8x38 поддерживает не непрерывную частоту воспроизведения, а только фиксированную: 5512, 8000, 11025, 16000, 22050, 32000, 44100 и 48000

Гц. Задний выход слышен только при отключенном переключении "Четырехканальный режим". В противном случае сигнал не будет передаваться на задние динамики. По умолчанию он включен.

Предупреждение:

Когда переключатель "Четырехканальный режим" выключен, выход из задних динамиков будет на ПОЛНУЮ громкость с учетом меньшей громкости Master и PCM.¹ Это может привести к повреждению вашего аудиооборудования. Пожалуйста, отключите динамики, прежде чем выключать этот переключатель.

¹ Что ж.. Однажды я получил выходной сигнал с правильной громкостью (то есть такой же, как на передней панели) и был так взволнован. Он был даже включен с "Четырьмя каналами" и режимом "двойного ЦАП". На самом деле я мог слышать отдельные 4 канала из передних и задних динамиков! Но .. после перезагрузки все исчезло. Очень жаль, что я не сохранил дамп реестра в то время.. Возможно, есть неизвестный регистр для достижения этого...

Если на вашей карте есть дополнительный выходной разъем для заднего выхода, воспроизведение с заднего хода должно быть направлено туда по умолчанию. Если нет, то в драйвере есть переключатель управления "Line-In As Rear", который вы можете изменить с помощью alsamixer или каким-либо другим способом. Когда этот переключатель включен, разъем линейного входа используется в качестве заднего выхода. Для заднего выхода есть еще два элемента управления. Переключатель "Заменить ЦАП" используется для обмена маршрутами воспроизведения спереди и сзади, т.е. 2-й ЦАП выводится с переднего выхода.

6.2.2 Многоканальное воспроизведение 4/6

Новейшие чипы SM8738 поддерживают функцию многоканального воспроизведения 4/6. Это особенно полезно для декодирования AC3.

Когда поддерживается многоканальность, имя драйвера имеет суффикс "-MC", например, "SM8738-MC6". Вы можете проверить это название в /proc/asound/cards.

Когда включен 4/6-канальный выход, второй ЦАП принимает до 6 (или 4) каналов. В то время как двойной ЦАП поддерживает две разные скорости или форматы, 4/6-канальное воспроизведение поддерживает только одинаковые условия для всех каналов. Поскольку в режиме многоканального воспроизведения используются оба ЦАП, вы не можете работать в полнодуплексном режиме. Режимы 4.0 и 5.1 определены как pcm "surround40" и "surround51" в alsa-lib. Например,

вы можете воспроизвести файл WAV с 6 каналами, например,

```
% aplay -Dsurround 5 1 sixchannels.wav
```

Для программирования воспроизведения по 4/6 каналам вам необходимо указать каналы PCM по своему усмотрению и установить формат S16LE. Например, для воспроизведения по 4 каналам,

```
snd_pcm_hw_params_set_access(pcm, hw, SND_PCM_ACCESS_RW_INTERLEAVED);

// или mmap, если вам нравится

snd_pcm_hw_params_set_format(pcm, hw, SND_PCM_FORMAT_S 1 6_LE);

snd_pcm_hw_params_set_channels(pcm, hw, 4);
```

и используйте чередующиеся 4-канальные данные.

Есть несколько переключателей, влияющих на подключение динамиков:

Режим линейного подключения элемент управления перечислением для изменения поведения разъема линейного входа. Можно выбрать "Линейный вход", "Задний выход" или

"Выход низких частот". Последнее доступно только с моделью 039 или новее. При выборе "Задний выход" каналы объемного звучания 3 и 4 выводятся на разъем линейного входа.

Режим включения микрофона элемент управления перечислением для изменения режима подключения микрофона. Можно выбрать либо "Вход микрофона", либо "Центр / выход LFE-положить". При выборе "Центральный / низкочастотный выход" центральный и низкочастотный каналы (каналы 5 и 6) выводятся на разъем микрофонного входа.

6.2.3 Цифровой ввод-вывод

CM8x38 обеспечивает отличные возможности SPDIF по очень низкой цене (да, именно по этой причине я купил карту :)

Воспроизведение и захват SPDIF осуществляются с помощью третьего устройства PCM (hw: 0,2). Обычно это присваивается устройству PCM "spdif". Доступные частоты составляют 44100 и 48000 Гц. Для воспроизведения с помощью aplay вы можете

выполнить следующие действия:

```
% aplay -DPCM: 0,2 foo.wav
```

или

```
% aplay -Dspdif foo.wav
```

24-битный формат также поддерживается экспериментально.

Для воспроизведения и захвата по SPDIF используются обычные ЦАП и АЦП соответственно, поэтому вы не можете воспроизводить аналоговый и цифровой потоки одновременно.

Чтобы включить вывод SPDIF, вам необходимо включить управление "IEC958 Output Switch" через микшер или alsactl ("IEC958" - официальное название так называемого S/PDIF). Затем вы увидите, что на карточке загорается красный огонек, так что вы знаете, что это работает, очевидно :) Вход SPDIF всегда включен, поэтому вы можете в любое время прослушать входные данные SPDIF с линейного выхода с переключателем "IEC958 В мониторе" (см. Ниже).

Вы можете воспроизводить через SPDIF даже на первом устройстве (hw: 0,0), но SPDIF включен только при использовании надлежащего формата (S16LE), частоты дискретизации (441100 или 48000) и каналов (2). В противном случае он выключен.

(Также не забудьте включить "Выходной переключатель IEC958".)

Дополнительно имеются соответствующие управляющие переключатели:

IEC958 Микширует аналоговый сигнал Микширует аналоговое воспроизведение PCM и FM-OPL / 3 потоков и выводит через SPDIF. Этот переключатель доступен только на старых моделях микросхем (CM8738, 033 и 037).

Примечание: без этого элемента управления вы можете выводить PCM в SPDIF. Это "микширование" потоков, поэтому, например, это не для вывода AC3 (см. Следующий раздел).

IEC958 В Select Выберите вход SPDIF, внутренний CD-вход (false) и внешний вход (true).

Цикл IEC958 Входные данные SPDIF возвращаются обратно на выход SPDIF (он же байпас)

Авторское право IEC958 Установите бит авторского права.

IEC958 5V Выберите 0,5 В (коаксиальный) или 5 В (оптический) интерфейс. На некоторых картах это не работает, и вам нужно изменить конфигурацию с помощью аппаратного dip-переключателя.

IEC958 В Monitor Ввод SPDIF направляется на ЦАП.

IEC958 инвертирован по фазе Установите формат ввода SPDIF как обратный. [FIXME: это работает не на всех микросхемах ...] **IEC958 В действительном состоянии** Установите флажок определения допустимости ввода. Примечание: Когда включен "Переключатель воспроизведения PCM", вы услышите поток цифрового вывода через аналоговый линейный выход.

6.2.4 Выход AC3 (RAW DIGITAL)

Драйвер поддерживает raw-цифровой (обычно AC3) ввод-вывод по SPDIF. Это можно переключить с помощью IEC958 playback control, но обычно вам нужно получить к нему доступ через alsa-lib. Смотрите документы alsa-lib для получения более подробной информации. В цифровом режиме raw "Переключатель воспроизведения PCM" автоматически отключается, так что незвуковые данные прослушиваются с аналогового линейного выхода. Аналогичным образом отключены следующие переключатели: "IEC958 Mix Analog" и "IEC958 Loop". Переключение возобновляется после автоматического перевода устройства SPDIF PCM в предыдущее состояние. В модели 033 AC3 реализован программным преобразованием в alsa-lib. Если вам нужно обойти программное преобразование подкадров IEC958, выберите опцию модуля "soft_ac3 =0". Это не имеет значения для более новых моделей.

6.2.5 ИНТЕРФЕЙС АНАЛОГОВОГО МИКСЕРА

Интерфейс микшера на CM8x38 аналогичен SB16. Есть мастер, PCM, синтезатор, CD, линейный, микрофонный и ПК громкость воспроизведения динамиков. Синтезатор, CD, Линейный и микрофонный микшеры также оснащены переключателями воспроизведения и захвата, как и SB16.

В дополнение к стандартному микшеру SB, CM8x38 предоставляет больше функций. - Переключатель воспроизведения PCM - PCM переключатель захвата (для захвата данных, отправляемых на ЦАП) - Переключатель усиления микрофона - Громкость захвата микрофона - Воспроизведение через Aux переключатель громкости / и переключатель захвата - переключатель 3D control

6.2.6 MIDI-КОНТРОЛЛЕР

С чипами CM18338 интерфейс MPU401-UART по умолчанию отключен. Вам необходимо установить для параметра модуля "midi_port" действительный адрес порта ввода-вывода, чтобы включить поддержку MIDI. Допустимыми портами ввода-вывода являются 0x300, 0x310, 0x320 и 0x330. Выберите значение, которое не конфликтует с другими картами.

В чипах CM18738 и новее MIDI-интерфейс включен по умолчанию, и драйвер автоматически выбирает адрес порта. Имеется *нет* аппаратная функция wavetable на этом чипе (за исключением синтезатора OPL3 ниже). То, что называется MIDI synth в Windows, является эмуляцией программного синтезатора. В Linux используйте TiMidity или другую программу softsynth для воспроизведения музыки MIDI.

Синтезатор 6.2.7 FM OPL / 3

FM OPL /3 также включен по умолчанию только для первой карты. Установите опцию модуля "fm_port" для других карт. Качество вывода FM OPL / 3, однако, очень странное. Я не знаю почему .. CM18768 и более новые чипы не имеют FM-синтезатора.

6.2.8 Джойстик и модем

Поддерживается устаревший джойстик. Чтобы включить поддержку джойстика, укажите параметр модуля joystick_port=1. Значение 1 означает автоматическое определение. Если автоматическое определение завершается неудачей, попробуйте передать точный адрес ввода-вывода. Модем включается динамически с помощью переключателя управления картой "Модем".

6.2.9 Отладочная информация

Регистры показаны в /proc/asound/cardX/cmipci. Если у вас возникли какие-либо проблемы (особенно неожиданное поведение микшера), пожалуйста, приложите выходные данные этого файла процедуры вместе с отчетом об ошибке.

6.3 Sound Blaster Live mixer / код DSP по умолчанию.

Чипы EMU10K1 имеют DSP-компонент, который может быть запрограммирован для поддержки различных способов обработки образцов, которые описаны здесь. (В этой статье не рассматриваются общие функциональные возможности чипов EMU10K1. Дополнительные сведения см. в разделе руководства.) Драйвер ALSA программирует эту часть чипа с помощью кода по умолчанию (может быть изменен позже), который предлагает следующие функциональные возможности:

6.3.1 IEC958 (S/PDIF) raw PCM

Это PCM-устройство (это 4-е PCM-устройство (индекс 3!) и первое дочернее устройство (индекс 0) для данной карты) позволяет пересылать стереофонические 16-битные потоки с частотой 48 кГц без каких-либо изменений цифрового выхода (коаксиального или оптического). Универсальный интерфейс позволяет создавать до 8 необработанных PCM-устройств, работающих на частоте 48 кГц, с 16-битным младшим порядковым номером. Было бы легко добавить поддержку многоканальных устройств в текущий код, но на данный момент процедуры преобразования существуют только для стерео (2-канальных потоков). Ознакомьтесь с подпрограммами `tram_poke` в `lowlevel` / `emu10k1` / `emufx.c` для получения более подробной информации.

6.3.2 Элементы управления цифровым микшером

Эти элементы управления созданы с использованием инструкций DSP. Они предлагают расширенную функциональность. Здесь описан только стандартный встроенный код в драйвере ALSA. Обратите внимание, что элементы управления работают как аттенюаторы: максимальное максимальное значение - это нейтральное положение, при котором сигнал остается неизменным. Обратите внимание, что если один и тот же пункт назначения упоминается в нескольких элементах управления, сигнал накапливается и может быть преобразован (устанавливается в максимальное или минимальное значение без проверки переполнения).

Объяснение используемых сокращений:

ЦАП цифроаналоговый преобразователь

АЦП аналого-цифровой преобразователь

I2S односторонняя трехпроводная последовательная шина для цифрового звука от Philips Semiconductors (этот стандарт используется для подключения автономных преобразователей ЦАП и АЦП) **LFE** низкочастотные эффекты (сигнал сабвуфера)

AC97 микросхема, содержащая аналоговый микшер, преобразователи ЦАП и АЦП **IEC958 S/PDIF**

FX-bus микросхема EMU10K1 имеет шину эффектов, содержащую 16 аккумуляторов. Каждый из голосов синтезатора может подавать свой выходной сигнал на эти аккумуляторы, и микроконтроллер DSP может работать с полученной суммой.

```
name = 'Громкость воспроизведения волны', index = 0
```

Этот элемент управления используется для смягчения образцы для левой и правой ПКМ на FX-шину аккумуляторов. Алса использует Акку- mulators 0 и 1 для левой и правой образцов ПКМ.

Результирующие выборки передаются на PCM переднего ЦАП слоты кодека AC97.

```
name= "Громкость воспроизведения Wave Surround", index= 0
```

Этот элемент управления используется для смягчения образцы для левой и правой ПКМ на FX-шину аккумуляторов. Алса использует Акку- mulators 0 и 1 для левой и правой образцов ПКМ.

Результирующие выборки передаются на задние ЦАП I2S. Эти ЦАП работают отдельно (они не входят в кодек AC97). `name= 'Громкость воспроизведения в волновом центре', index = 0`

Этот регулятор используется для ослабления сэмплов для левого и правого аккумуляторов PCM FX-bus.

Алса использует Акку- mulators 0 и 1 для левой и правой образцов ПКМ. В результате смешиваются в моно сигнал (одиночный канал) и направлены ??сзади?? правый слот PCM ЦАП кодека AC97.

```
name= 'Громкость воспроизведения Wave LFE', index= 0
```

Этот регулятор используется для ослабления сэмплов для левого и правого аккумуляторов PCM FX-bus.

ALSA использует аккумуляторы 0 и 1 для левого и правого PCM. Результат смешивается с моносигналом

(одноканальный) и передается на ??задний?? левый слот PCM ЦАП кодека AC97.

```
name= 'Громкость захвата волны', индекс = 0,name = 'Переключатель захвата волны', индекс = 0
```

Эти регуляторы используются для ослабления отсчетов для левого и правого аккумуляторов PCM

FX-bus. ALSA использует аккумуляторы 0 и 1 для левого и правого PCM. Результат пересылается в FIFO

захвата АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= 'Громкость воспроизведения на синтезаторе', index= 0
```

Этот элемент управления используется для смягчения образцы для левой и правой миди на FX-шину

аккумуляторов. Алса использует Акку- mulators 4 и 5 для левого и правого миди образцы.

Результирующие сэмплы передаются на PCM переднего ЦАП слоты кодека AC97.

```
name= "Громкость записи синтезатора", index= 0,name= 'Переключатель захвата синтезатора',
```

```
index = 0
```

Эти элементы управления используются для ослабления сэмплов для левого и правого

накопителя MIDI FX-шин. ALSA использует аккумуляторы 4 и 5 для левого и правого PCM. Результат

пересылается на устройство захвата FIFO АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= "Громкость объемного воспроизведения", index= 0
```

Этот регулятор используется для ослабления отсчетов для аккумуляторов FX-шины PCM слева и справа

сзади. ALSA использует аккумуляторы 2 и 3 для отсчетов PCM слева и справа сзади. Результирующие

выборки передаются на задние ЦАП I2S. Эти ЦАП работают отдельно (они не входят в кодек AC97).

```
name= "Громкость записи объемного звучания", индекс = 0,name= "Переключатель записи объемного
```

```
звучания", индекс = 0
```

Эти регуляторы используются для ослабления отсчетов для аккумуляторов

FX-шины PCM слева и справа сзади. ALSA использует аккумуляторы 2 и 3 для отсчетов PCM слева и

справа сзади. Результат пересылается в FIFO захвата АЦП (таким образом, на стандартное устройство

захвата PCM). name= "Центральная громкость воспроизведения", index= 0

Этот регулятор используется для ослабления пробы в аккумуляторе центральной шины PCM FX. ALSA

использует аккумулятор 6 для пробы центральной шины PCM. Результирующая проба направляется на

??задний?? правый слот PCM ЦАП кодека AC97. name= "Громкость воспроизведения LFE", index= 0

Этот регулятор используется для ослабления пробы в аккумуляторе центральной шины PCM FX. ALSA

использует аккумулятор 6 для пробы центральной шины PCM. Результирующая проба направляется на

??задний?? левый слот PCM ЦАП кодека AC97. name= "Громкость воспроизведения AC 9 7 ", index= 0

Этот элемент управления используется для ослабления отсчетов для левого и правого передних слотов

PCM АЦП кодека AC97. Результирующие сэмплы передаются в разъемы PCM переднего ЦАП кодека AC97.

Примечание:

Для стандартных операций этот элемент управления должен быть равен нулю, в противном случае активируется цифровая обратная связь.

```
name = 'Громкость записи AC 9 7 ', index = 0
```

Этот элемент управления используется для ослабления отсчетов для левого и правого передних слотов PCM АЦП кодека AC97. Результат пересылается на устройство FIFO захвата АЦП (таким образом, на стандартное устройство PCM захвата).

Примечание:

Это значение должно быть равно 100 (максимальное значение), в противном случае никакие аналоговые входы кодека AC97 не могут быть захвачены (записаны).

```
name= 'Громкость воспроизведения IEC 9 5 8 TTL', index= 0
```

Этот регулятор используется для ослабления отсчетов с левого и правого цифровых входов IEC958 TTL (обычно используемых приводом CDRом). Результирующие выборки пересылаются в разъемы PCM переднего ЦАП кодека AC97.

```
name= 'Громкость захвата IEC 9 5 8 TTL', index= 0
```

Этот регулятор используется для ослабления отсчетов с левого и правого цифровых входов IEC958 TTL (обычно используемых приводом CDRом). Результирующие выборки пересылаются в FIFO захвата АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= 'Увеличить громкость воспроизведения видео', index= 0
```

Этот элемент управления используется для ослабления отсчетов с цифровых входов видеосигнала с левым и правым масштабированием (обычно используется приводом CDRом). Результирующие выборки передаются в разъемы PCM переднего ЦАП кодека AC97. name= "Увеличить громкость видеозахвата", index = 0

Этот элемент управления используется для ослабления отсчетов с цифровых входов видеосигнала с левым и правым масштабированием (обычно используется приводом CDRом). Результирующие выборки пересылаются в FIFO захвата АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= 'Громкость воспроизведения IEC 9 5 8 в LiveDrive',индекс= 0
```

Этот элемент управления используется для ослабления отсчетов с левого и правого оптического цифрового входа IEC958. Результирующие выборки передаются в передние разъемы PCM ЦАП кодека AC97.

```
name= 'Громкость записи IEC 9 5 8 LiveDrive', index = 0
```

Этот элемент управления используется для ослабления дискретизации с левого и правого оптических цифровых входов IEC958. Результирующие выборки пересылаются на устройство захвата FIFO АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= "Громкость воспроизведения в коаксиальном режиме IEC 9 5 8 ", index= 0
```

Этот элемент управления используется для ослабления отсчетов с левого и правого коаксиальных цифровых входов IEC958. Результирующие отсчеты передаются в разъемы PCM переднего ЦАП кодека AC97.

```
name= 'Громкость коаксиального захвата IEC 9 5 8 ', index = 0
```

Этот регулятор используется для ослабления отсчетов с левого и правого коаксиальных цифровых входов IEC958. Полученный результат отсчеты пересылаются на устройство захвата FIFO АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= 'Линейная громкость воспроизведения LiveDrive', index= 0, name = 'Строка воспроизведения в LiveDrive Vol.-
ume', index= 1
```

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S (в LiveDrive).

Результирующие выборки пересылаются в разъемы PCM переднего ЦАП кодека AC97.

```
name= 'Линейный объем записи LiveDrive', index= 1,name='Объем записи Line LiveDrive
ume', индекс= 1
```

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S (в LiveDrive). Полученный результат отсчеты пересылаются на устройство захвата FIFO АЦП (таким образом, на стандартное устройство захвата PCM).

```
name= 'Переключатель управления тоном', index= 0
```

Этот элемент управления включает или выключает регулятор тембра. Это влияет на сэмплы для переднего, заднего и центрального / низкочастотного выходов.

```
name= 'Регулятор тембра - низкие частоты', index = 0
```

Этот регулятор устанавливает интенсивность низких частот. Нейтрального значения нет!! Когда активирован код управления тоном, сэмплы всегда изменяются. Самое близкое значение к чистому сигналу - 20.

```
name= 'Регулятор тона - высокие частоты', index = 0
```

Этот регулятор устанавливает интенсивность высоких частот. Нейтрального значения нет!! Когда активирован код регулировки тона, сэмплы всегда изменяются. Ближайшее значение к чистому сигналу

равно 20. name= 'Оптический переключатель воспроизведения Raw IEC 9 5 8 ', index= 0

Если этот переключатель включен, то выборки для цифрового выхода IEC958 (S/PDIF) берутся только с raw FX8010 PCM, в противном случае берутся стандартные выборки фронтального PCM.

```
name = "Громкость воспроизведения в наушниках", индекс = 1
```

Этот регулятор ослабляет сэмплы на выходе наушников.

```
name = "Центральный переключатель воспроизведения в наушниках", индекс = 1
```

Если этот переключатель включен, то семпл для центрального PCM выводится на левый выход для наушников (полезно для видеокарт SB Live без отдельного центрального / LFE выхода).

name= "Переключатель воспроизведения LFE в наушниках", index= 1

Если этот переключатель включен, то семпл для центрального PCM выводится на правый выход наушников (полезно для видеокарт SB Live без отдельного центрального / LFE выхода).

6.3.3 Элементы управления, связанные с потоком PCM

name= 'EMU 1 0 K 1 PCM Volume', индекс 0 - 3 1

Ослабление громкости канала в диапазоне 0-0xffff. По умолчанию установлено максимальное значение (без ослабления). Отображение каналов для трех значений выглядит следующим образом. :

- 0 - моно, по умолчанию 0xffff (без затухания)
- 1 - слева, по умолчанию 0xffff (без затухания)
- 2 - справа, по умолчанию 0xffff (без затухания)

name= 'EMU 1 0 K 1 PCM Send Routing', индекс 0 - 3 1

Этот элемент управления определяет пункт назначения - аккумуляторы FX-bus. Существует двенадцать значений с таким отображением:

- 0 - моно, пункт назначения A (FX-bus 0-15), по умолчанию 0
- 1 - моно, пункт назначения B (FX-bus 0-15), по умолчанию 1
- 2 - моно, пункт назначения C (FX-bus 0-15), по умолчанию 2
- 3 - моно, пункт назначения D (FX-bus 0-15), по умолчанию 3
- 4 - слева, пункт назначения A (FX-bus 0-15), по умолчанию 0
- 5 - слева, пункт назначения B (FX-bus 0-15), по умолчанию 1
- 6 - слева, пункт назначения C (FX-bus 0-15), по умолчанию 2
- 7 - слева, пункт назначения D (FX-bus 0-15), по умолчанию 3
- 8 - справа, пункт назначения A (FX-bus 0-15), по умолчанию 0
- 9 - справа, пункт назначения B (FX-bus 0-15), по умолчанию 1
- 10 - справа, пункт назначения C (FX-bus 0-15), по умолчанию 2
- 11 - справа, пункт назначения D (FX-bus 0-15), по умолчанию 3

Не забывайте, что запрещено назначать канал одному и тому же аккумулятору FX-bus более одного раза (это означает, что 0 = 0 && 1 = 0 - недопустимая комбинация).

name= 'Громкость отправки EMU 1 0 K 1 PCM', индекс 0 - 3 1

Указывает ослабление (величину) для данного адресата в диапазоне 0-255. Ниже приводится отображение канала. :

- 0 - моно, A attn назначения, по умолчанию 255 (без ослабления)
- 1 - моно, B attn назначения, по умолчанию 255 (без ослабления)
- 2 - моно, C attn назначения, по умолчанию 0 (отключение звука)
- 3 - моно, D attn назначения, по умолчанию 0 (отключение звука)
- 4 - слева, A attn назначения, по умолчанию 255 (без ослабления)
- 5 - слева, B attn назначения, по умолчанию 0 (отключение звука)
- 6 - слева, C attn назначения, по умолчанию 0 (отключение звука)

- 7 - слева, D attn назначения, по умолчанию 0 (отключение звука)
- 8 - справа, A attn назначения, по умолчанию 0 (отключение звука)
- 9 - справа, B attn назначения, по умолчанию 255 (без ослабления)
- 10 - справа, C attn назначения, по умолчанию 0 (отключение звука)
- 11 - справа, D attn назначения, по умолчанию 0 (отключение звука)

6.3.4 РУКОВОДСТВА /ПАТЕНТЫ

<ftp://opensource.creative.com/pub/doc>

LM4545.pdf Кодек AC97

m2049.pdf Цифровой аудиопроцессор EMU10K1

hog63.ps FX8010 - Архитектура микросхемы DSP для звуковых эффектов

Патенты ВОИС

WO 9901813 (A1) Процессор звуковых эффектов с несколькими асинхронными потоками (14 января 1999 г.) **WO 9901814 (A1)** Процессор с набором команд для звуковых эффектов (14 января 1999 г.)

WO 9901953 (A1) Процессор звуковых эффектов, разделяющий выполнение команд и передачу аудиоданных-гашение (14 января 1999 г.)

Патенты США (<http://www.uspto.gov/>)

US 5925841 Цифровой прибор для отбора проб с использованием кэш-памяти (20 июля 1999 г.)

US 5928342 Процессор аудиоэффектов, интегрированный на одном чипе с многопортовой памятью, в которую несколько асинхронных цифровых сэмплов звука могут быть загружены одновременно (27 июля 1999 г.) **US 5930158** Процессор с набором инструкций для звуковых эффектов (27 июля 1999 г.) **US 6032235** Схема инициализации памяти (Tram) (29 февраля, 2000)

US 6138207 Интерполяционный цикл аудиосэмплов в кэше, подключенном к системной шине, с расстановкой приоритетов и модификация автобусных переходов в соответствии с концами контуров и минимальными размерами блоков (24 октября 2000 г.)

US 6151670 Способ сохранения памяти с использованием пула регистров кратковременной памяти (ноябрь, 21, 2000)

US 6195715 Управление прерываниями для нескольких программ, взаимодействующих с общим прерыванием, с помощью ассоциативного-подключение программ к регистрам GP, определение регистра прерываний, опрос регистров GP и вызов процедуры обратного вызова, связанной с определенным регистром прерываний (27 февраля 2001 г.)

6.4 Микшер Sound Blaster Audigy / код DSP по умолчанию

Он основан на sb-live-mixer.rst.

Чипы EMU10K2 имеют DSP-компонент, который может быть запрограммирован для поддержки различных способов обработки образцов, которые описаны здесь. (В этой статье не рассматриваются общие функциональные возможности чипов EMU10K2 . Дополнительные сведения см. в разделе руководства.) Драйвер ALSA программирует эту часть чипа с помощью кода по умолчанию (может быть изменен позже), который предлагает следующие функциональные возможности:

6.4.1 Элементы управления цифровым микшером

Эти элементы управления созданы с использованием инструкций DSP. Они предлагают расширенную функциональность. Здесь описан только встроенный код по умолчанию в драйвере ALSA. Обратите внимание, что элементы управления работают как аттенюаторы: максимальное максимальное значение - это нейтральное положение, при котором сигнал остается неизменным. Обратите внимание, что если один и тот же пункт назначения упоминается в нескольких элементах управления, сигнал накапливается и может быть преобразован (устанавливается в максимальное или минимальное значение без проверки переполнения).

Объяснение используемых сокращений:

ЦАП цифроаналоговый преобразователь

АЦП аналого-цифровой преобразователь

I2S односторонняя трехпроводная последовательная шина для цифрового звука от Philips Semiconductors (этот стандарт используется для

подключения автономных преобразователей ЦАП и АЦП) **LFE** низкочастотные эффекты (сигнал сабвуфера)

AC97 микросхема, содержащая аналоговый микшер, преобразователи ЦАП и АЦП **IEC958 S/PDIF**

FX-bus микросхема EMU10K2 имеет шину эффектов, содержащую 64 аккумулятора. Каждый из голосов синтезатора может подавать свой выходной сигнал на эти аккумуляторы, и микроконтроллер DSP может работать с полученной суммой.

name= 'Громкость воспроизведения PCM спереди', index = 0

Этот регулятор используется для ослабления сэмплов для аккумуляторов FX-шины PCM слева и справа спереди. ALSA использует аккумуляторы 8 и 9 для сэмплов PCM слева и справа спереди для воспроизведения в формате 5.1. Результирующие сэмплы передаются в разъемы PCM переднего ЦАП

Philips DAC. **name = "Громкость объемного воспроизведения PCM", index = 0**

Этот регулятор используется для ослабления сэмплов для аккумуляторов left и right surround PCM FX-bus. ALSA использует аккумуляторы 2 и 3 для сэмплов left и right surround PCM для воспроизведения в формате 5.1. Результирующие сэмплы передаются в разъемы PCM ЦАП объемного звучания ЦАП

Philips. **name= "Громкость воспроизведения в центре PCM", index = 0**

Этот элемент управления используется для ослабления сэмплов для центрального аккумулятора PCM FX-bus. ALSA использует аккумулятор 6 для центрального сэмпла PCM для воспроизведения 5.1. Результирующий сэмпл передается в центральный разъем PCM ЦАП Philips DAC.

name= "Громкость воспроизведения PCM LFE", index = 0

Этот регулятор используется для ослабления сэмпла для аккумулятора LFE PCM FX-bus. ALSA использует аккумулятор 7 для воспроизведения сэмпла LFE PCM в формате 5.1. Результирующий сэмпл пересылается в слот PCM LFE DAC в ЦАП Philips .

name= "Громкость воспроизведения PCM", index = 0

Этот элемент управления используется для смягчения образцы для левой и правой ПКМ на FX-шину аккумуляторов. Алса использует Нагро- латорс 0 и 1 для левой и правой ПКМ образцы для стерео воспроизведения. Результирующие выборки передаются в разъемы PCM переднего ЦАП ЦАП Philips.

name= 'Громкость захвата PCM', index = 0

Этот регулятор используется для ослабления отсчетов для левого и правого накопителей PCM FX-bus. ALSA использует значения 0 и 1 для левого и правого PCM. Результат пересылается в FIFO захвата АЦП (таким образом, на стандартное устройство захвата PCM).

name= 'Громкость воспроизведения музыки', index= 0

Этот элемент управления используется для смягчения образцы для левой и правой миди на FX-шину аккумуляторов.

Алса использует Акку- mulators 4 и 5 для левого и правого миди образцы. Результирующие сэмплы передаются на PCM переднего ЦАП слоты кодека AC97.

name= "Громкость записи музыки", index = 0

Эти элементы управления используются для ослабления сэмплов для левого и правого аккумуляторов MIDI FX-шины.

ALSA использует аккумуляторы 4 и 5 для левого и правого PCM. Результат пересылается на устройство захвата FIFO АЦП (таким образом, на стандартное устройство захвата PCM).

name= 'Громкость воспроизведения микрофона', index= 0

Этот элемент управления используется для ослабления сэмплов для левого и правого микрофонных входов. Для микрофонного входа используется кодек AC97. Результирующие выборки передаются в разъемы PCM переднего ЦАП ЦАП Philips. Сэмплы пересылаются в Mic capture FIFO (устройство 1 - 16bit / 8kHz mono) также без регулировки громкости. **name= 'Громкость захвата микрофона', index = 0**

Этот регулятор используется для ослабления сэмплов для левого и правого микрофонных входов. Результат пересылается в АЦП capture FIFO (таким образом, на стандартное устройство захвата PCM).

name= "Громкость воспроизведения компакт-диска Audigy", index=0

Этот регулятор используется для ослабления отсчетов с левого и правого цифровых входов IEC958 TTL (обычно используемых приводом CDRом). Результирующие сэмплы передаются в разъемы PCM переднего ЦАП Philips DAC.

name = "Громкость записи компакт-дисков Audigy", index = 0

Этот регулятор используется для ослабления отсчетов с левого и правого цифровых входов IEC958 TTL (обычно используемых приводом CDRом). Результирующие выборки пересылаются в FIFO захвата АЦП (таким образом, на стандартное устройство захвата PCM).

name= 'Оптическая громкость воспроизведения IEC958', index = 0

Этот регулятор используется для ослабления дискретизации с левого и правого оптического цифрового входа IEC958. Результирующие выборки передаются в разъемы PCM переднего ЦАП Philips DAC.

name= 'Объем оптического захвата IEC958', index = 0

Этот элемент управления используется для ослабления отсчетов с левого и правого оптических цифровых входов IEC958. Результат отсчеты передаются на устройство захвата FIFO АЦП (таким образом, на стандартное устройство захвата PCM).

name= 'Громкость воспроизведения Line2', index= 0

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S (в AudigyDrive). Результирующие выборки передаются в разъемы PCM переднего ЦАПА ЦАП Philips.

name= "Объем записи Line2", index = 1

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S (в AudigyDrive). Результирующие выборки пересылаются в FIFO захвата АЦП (таким образом, на стандартное устройство захвата PCM).

name= "Громкость воспроизведения аналогового микса", index= 0

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S от Philips ADC.

Результирующие отсчеты передаются в разъемы PCM переднего ЦАП Philips DAC. Он содержит миксы из аналоговых источников, таких как CD, линейный вход, Aux,

name = "Громкость записи аналогового микса", index = 1

Этот регулятор используется для ослабления отсчетов с левого и правого входов АЦП I2S Philips ADC.

Результирующие выборки пересылаются в FIFO захвата АЦП (таким образом, на стандартное устройство захвата

PCM). **name= 'Громкость воспроизведения Aux2', index= 0**

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S (в AudigyDrive).

Результирующие выборки передаются в разъемы PCM переднего ЦАП Philips DAC.

name = "Громкость записи Aux2", индекс = 1

Этот элемент управления используется для ослабления отсчетов с левого и правого входов АЦП I2S (в AudigyDrive).

Результирующие выборки пересылаются на устройство FIFO захвата АЦП (таким образом, на стандартное

устройство PCM захвата). **name= 'Начальная громкость воспроизведения', index= 0**

Все стереосигналы микшируются вместе и зеркально воспроизводятся для получения окружающего, центрального

и низкочастотного звучания. Этот регулятор используется для ослабления сэмплов для левого и правого

фронтальных динамиков этого микширования. **name= "Громкость воспроизведения объемного звучания", index = 0**

Все стереосигналы микшируются вместе и зеркально воспроизводятся для объемного звучания, центрирования и

LFE. Этот регулятор используется для уменьшения громкости сэмплов для левого и правого динамиков объемного

звучания в этом миксе. **name= "Центральная громкость воспроизведения", index= 0**

Все стереосигналы микшируются вместе и зеркально воспроизводятся для объемного звучания, центрирования и

LFE. Этот регулятор используется для ослабления сэмпла для центрального динамика этого микширования.

name= 'Громкость воспроизведения LFE', index = 0

Все стереосигналы микшируются вместе и зеркально воспроизводятся для объемного звучания, центрирования и

LFE. Этот регулятор используется для ослабления сэмпла для низкочастотного динамика этого микса.

name= 'Переключатель управления тоном', index = 0

Этот элемент управления включает или выключает регулятор тембра. Это влияет на сэмплы для переднего, заднего и центрального / LFE выходов.

name= 'Регулятор тона - Bass', index = 0

Этот регулятор устанавливает интенсивность низких частот. Нейтрального значения нет!! Когда активирован код регулировки тона, сэмплы всегда изменяются. Ближайшее значение к чистому сигналу - 20.

name = 'Регулятор тона - высокие частоты', index = 0

Этот регулятор устанавливает интенсивность высоких частот. Нейтрального значения нет!! Когда активирован код управления тоном, сэмплы всегда изменяются. Самое близкое значение к чистому сигналу - 20.

name= 'Основная громкость воспроизведения', индекс=0

Этот регулятор используется для ослабления сэмплов на фронтальных, объемных, центральных и низкочастотных выходах.

name= 'Оптический переключатель воспроизведения Raw IEC958', index= 0

Если этот переключатель включен, то выборки для цифрового выхода IEC958 (S/PDIF) берутся только с raw FX8010 PCM, в противном случае берутся стандартные выборки фронтального PCM.

6.4.2 Элементы управления, связанные с потоком PCM**name= 'EMU10K1 PCM Volume', индекс 0-31**

Ослабление громкости канала в диапазоне 0-0xffff. По умолчанию установлено максимальное значение (без ослабления). Отображение каналов для трех значений выглядит следующим образом. :

- 0 - моно, по умолчанию 0xffff (без затухания)
- 1 - слева, по умолчанию 0xffff (без затухания)
- 2 - справа, по умолчанию 0xffff (без затухания)

name = 'EMU10K1 PCM Send Routing', индекс 0-31

Этот элемент управления определяет назначение - аккумуляторы FX-bus. Существует 24 значения с таким отображением:

- 0 - моно, пункт назначения A (FX-bus 0-63), по умолчанию 0
- 1 - моно, пункт назначения B (FX-bus 0-63), по умолчанию 1
- 2 - моно, пункт назначения C (FX-bus 0-63), по умолчанию 2
- 3 - моно, пункт назначения D (FX-bus 0-63), по умолчанию 3
- 4 - моно, пункт назначения E (FX-bus 0-63), по умолчанию 0
- 5 - моно, пункт назначения F (FX-bus 0-63), по умолчанию 0
- 6 - моно, пункт назначения G (FX-bus 0-63), по умолчанию 0
- 7 - моно, пункт назначения H (FX-bus 0-63), по умолчанию 0
- 8 - слева, пункт назначения A (FX-bus 0-63), по умолчанию 0
- 9 - слева, пункт назначения B (FX-bus 0-63), по умолчанию 1

- 10 - слева, пункт назначения C (FX-bus 0-63), по умолчанию 2
- 11 - слева, пункт назначения D (FX-bus 0-63), по умолчанию 3
- 12 - слева, пункт назначения E (FX-bus 0-63), по умолчанию 0
- 13 - слева, пункт назначения F (FX-bus 0-63), по умолчанию 0
- 14 - слева, пункт назначения G (FX-bus 0-63), по умолчанию 0
- 15 - слева, пункт назначения H (FX-bus 0-63), по умолчанию 0
- 16 - справа, пункт назначения A (FX-bus 0-63), по умолчанию 0
- 17 - справа, пункт назначения B (FX-bus 0-63), по умолчанию 1
- 18 - справа, пункт назначения C (FX-bus 0-63), по умолчанию 2
- 19 - справа, пункт назначения D (FX-bus 0-63), по умолчанию 3
- 20 - справа, пункт назначения E (FX-bus 0-63), по умолчанию 0
- 21 - справа, пункт назначения F (FX-bus 0-63), по умолчанию 0
- 22 - справа, пункт назначения G (FX-bus 0-63), по умолчанию 0
- 23 - справа, пункт назначения H (FX-bus 0-63), по умолчанию 0

Не забывайте, что запрещено назначать канал одному и тому же аккумулятору FX-bus более одного раза (это означает, что $0 = 0 \& \& 1 = 0$ - недопустимая комбинация).

name= 'Громкость отправки EMU10K1 PCM', индекс 0-31

Указывает ослабление (величину) для данного адресата в диапазоне 0-255. Ниже приводится отображение канала. :

- 0 - моно, сигнал A назначения, по умолчанию 255 (без ослабления)
- 1 - моно, сигнал B назначения, по умолчанию 255 (без ослабления)
- 2 - моно, сигнал C назначения, по умолчанию 0 (отключение звука)
- 3 - моно, сигнал D назначения, по умолчанию 0 (отключение звука)
- 4 - моно, сигнал E назначения, по умолчанию 0 (отключение звука)
- 5 - моно, сигнал F назначения, по умолчанию 0 (отключение звука)
- 6 - моно, сигнал G назначения, по умолчанию 0 (отключение звука)
- 7 - моно, сигнал H назначения, по умолчанию 0 (отключение звука)
- 8 - слева, сигнал A назначения, по умолчанию 255 (без ослабления)
- 9 - слева, сигнал B назначения, по умолчанию 0 (отключение звука)
- 10 - слева, C attn назначения, по умолчанию 0 (отключение звука)
- 11 - слева, D attn назначения, по умолчанию 0 (отключение звука)
- 12 - слева, E attn назначения, по умолчанию 0 (отключение звука)
- 13 - слева, F attn назначения, по умолчанию 0 (отключение звука)
- 14 - слева, G attn назначения, по умолчанию 0 (отключение звука)
- 15 - слева, H attn назначения, по умолчанию 0 (отключение звука)
- 16 - справа, A attn назначения, по умолчанию 0 (отключение звука)
- 17 - справа, B attn назначения, по умолчанию 255 (без ослабления)
- 18 - справа, C attn назначения, по умолчанию 0 (отключение звука)

- 19 - справа, D attn назначения, по умолчанию 0 (отключение звука)
- 20 - справа, E attn назначения, по умолчанию 0 (отключение звука)
- 21 - справа, F attn назначения, по умолчанию 0 (отключение звука)
- 22 - справа, G attn назначения, по умолчанию 0 (отключение звука)
- 23 - справа, H attn назначения, по умолчанию 0 (отключение звука)

6.4.3 РУКОВОДСТВА /ПАТЕНТЫ

<ftp://opensource.creative.com/pub/doc>

LM4545.pdf Кодек AC97

m2049.pdf Цифровой аудиопроцессор EMU10K1

hog63.ps FX8010 - Архитектура микросхемы DSP для звуковых эффектов

Патенты ВОИС

WO 9901813 (A1) Процессор звуковых эффектов с несколькими асинхронными потоками (14 января 1999 г.) **WO 9901814 (A1)** Процессор с набором команд для звуковых эффектов (14 января 1999 г.)

WO 9901953 (A1) Процессор звуковых эффектов, разделяющий выполнение команд и передачу аудиоданных-гашение (14 января 1999 г.)

Патенты США (<http://www.uspto.gov/>)

US 5925841 Цифровой прибор для отбора проб с использованием кэш-памяти (20 июля 1999 г.)

US 5928342 Процессор аудиоэффектов, интегрированный на одном чипе с многопортовой памятью, в которую несколько асинхронных цифровых сэмплов звука могут быть загружены одновременно (27 июля 1999 г.) **US 5930158** Процессор с набором инструкций для звуковых эффектов (27 июля 1999 г.) **US 6032235** Схема инициализации памяти (Tram) (29 февраля 2000 г.)

US 6138207 Интерполяционное зацикливание аудиосэмплов в кэше, подключенном к системной шине, с расстановкой приоритетов

и модификация автобусных переходов в соответствии с концами контуров и минимальными размерами блоков (24 октября 2000 г.)

US 6151670 Способ сохранения памяти с использованием пула регистров кратковременной памяти (ноябрь, 21, 2000)

US 6195715 Управление прерываниями для нескольких программ, взаимодействующих с общим прерыванием, с помощью ассоциативного-подключение программ к регистрам GP, определение регистра прерываний, опрос регистров GP и вызов обратного вызова процедура, связанная с определенным регистром прерываний (27 февраля 2001 г.)

6.5 Низкая задержка, многоканальный Аудио с ГНЕЗДОМ и с emu10k1/emu10k2

Этот документ представляет собой руководство по использованию устройств на базе emu10k1 с разъемом для многоканальной записи с низкой задержкой. Вся моя недавняя работа, направленная на то, чтобы позволить пользователям Linux использовать все возможности своего оборудования, была вдохновлена проектом kX. Без их работы я бы никогда не открыл для себя истинную мощь этого оборудования.

<http://www.kxproject.com>

• Ли Ревелл, 2005.03.30

До недавнего времени, пользователи еmu10k1 в Linux не имели доступа к тем же многоканальным функциям с низкой задержкой, которые предлагает функция "kX ASIO" их драйвера Windows. Начиная с версии ALSA 1.0.9, этого больше нет! Для тех, кто не знаком с kX ASIO, он состоит из 16 каналов захвата и 16 каналов воспроизведения. С ядром Linux версии 2.6.9 задержки до 64 (1,33 мс) или даже 32 (0,66 мс) кадров должны работать хорошо. Настройка немного сложнее, чем в Windows, поскольку вам нужно выбрать правильное устройство для использования JACK. На самом деле, для пользователей qjackctl это говорит само за себя - выберите Duplex, затем для захвата и воспроизведения выберите многоканальные устройства, установите входные и выходные каналы на 16, а частоту дискретизации на 48000 Гц. Командная строка выглядит

```
следующим образом: /usr/local/bin /jackd -R -dalsa -r 4 8 0 0 0 -p 6 4 -n 2 -D -Chw: 0 , 2 -Phw: 0 , 3 -S
```

Это даст вам 16 входных и 16 выходных портов.
16 выходных портов подключены к 16 шинам FX (или первым 16 из 64, для Audigy). Преобразование из шины FX в физический вывод описано в sb-live-mixer.rst (или audigy-mixer.rst). 16 входных портов подключены к 16 физическим входам. Вопреки распространенному мнению, все карты еmu10k1 являются многоканальными картами. К какому из этих входных каналов подключены физические входы, зависит от модели карты. Настоятельно рекомендуется метод проб и ошибок; схемы распиновки для карты были спроектированы некоторыми предприимчивыми пользователями kX и доступны в Интернете. Здесь полезен Meterbridge , а форумы kX полны полезной информации. Каждый входной порт будет соответствовать либо цифровому (SPDIF) входу, либо аналоговому входу, либо никакому. Единственным исключением является SBLive! 5.1. На этих устройствах второй и третий входные порты подключены к центральному / LFE выходу. Вы по-прежнему будете видеть 16 каналов захвата, но только 14 доступны для входов записи. Эта диаграмма, заимствованная из kxfxlib/da_asio51.cpp, описывает сопоставление портов JACK с каналами FXBUS2 (многодорожечный вход для записи) и EXTOUT (физический выход).
Сопоставления JACK (и ASIO) на картах 10k1 5.1 SBLive:

ДЖЕК	Эпилог	FXBUS2(nr)
захват_1	asio14	FXBUS2(0xe)
захват_2 захват_3	asio15	FXBUS2(0xf)
~	asio0	FXBUS2 (0x0)
захват_4	Центр	EXTOUT(0x11) //
~ захват_5	LFE	сопоставлен с центром
захват_6	asio3	EXTOUT(0x12) //
захват_7	asio4	сопоставлен с LFE
захват_8	asio5	FXBUS2(0x3) FXBUS2(0x4)
захват_9	asio6	FXBUS2(0x5) FXBUS2(0x6)
захват_10	asio7	FXBUS2 (0x7)
захват_11	asio8	FXBUS2 (0x8)
захват_12	asio9	FXBUS2 (0x9)
захват_13	asio10	FXBUS2 (0xa)
захват_14	asio11	FXBUS2 (0xb)
захват_15	asio12	FXBUS2 (0xb) FXBUS2
захват_16	asio13	(0xc),, FXBUS2 (0xd)

ЗАДАЧА: описать использование ld10k1 / ql010k1 в сочетании с разъемом JACK

Микшер VIA82xx 6.6

На многих платах VIA82xx не работает кнопка Выбор источника входного сигнала управление микшером. Установка значения Input 2 на таких платах приведет к зависанию записи или сбою ЕЮ (ошибка ввода / вывода) через эмуляцию OSS. Этот элемент управления следует оставить в положении Input 1 для таких карточек.

Руководство 6.7 по использованию M-Audio Audiophile USB с ALSA и разъемом Jack

Версия v1.5

Thibault Le Meur <Тибо.ЛеМур@supelec.fr>

Этот документ представляет собой руководство по использованию аудиофильского USB-устройства M-Audio (tm) с ALSA и разъемом JACK.

6.7.1 История

- Версия v1.4 - Тибо Ле Мер (2007-07-11)
 - Добавлен Низкий уровень Порядковый номер Природа из 16-битных режимов найдено Автор: Hakan Леннесталь <Hakan.Lennestall@brfsodrahamn.se>
- Изменение структуры документа • версия 5 - Thibault Le Meur (2007-07-12)
- Добавлена информация о интерфейсе AC3 / DTS passthru

6.7.2 Спецификации аудиофильского USB и правильное использование

Эта часть является напоминанием важных фактов о функциях и ограничениях устройства. Устройство имеет 4 аудиоинтерфейса и 2 MIDI-порта:

- Аналоговый стереовход (Ai)
 - Этот порт поддерживает 2 пары линейных аудиовходов (1/4 "TS и RCA) - При подключении разъемов 1/4"TS (jack) разъемы RCA отключены
- Аналоговый стереовыход (Ao) • Цифровой стереовход (Di) • Цифровой стереовыход (Do) • Midi In (Mi) • Midi Out (Mo) Внутренний

ЦАП/АЦП имеет следующие характеристики:

- глубина дискретизации 16 или 24 бита • частота дискретизации от 8 кГц до 96 кГц • Два интерфейса не могут использовать разную глубину дискретизации одновременно.

Более того, документация Audiophile USB содержит следующее предупреждение: Пожалуйста, выйдите из любого приложения audio, запущенного перед переключением между разрядностью

Из-за ограничения полосы пропускания USB 1.1 одновременно может быть активировано ограниченное количество интерфейсов, в зависимости от выбранного режима воспроизведения звука:

- 16 бит / 48 кГц ==> 4 канала на входе + 4 канала на выходе
 - Ai + Ao + Di+Do • 24 бита/48 кГц ==> 4 входных канала + 2 выходных канала или 2 входных канала + 4 выходных канала
 - Ai + Ao+ Do или Ai + Di + Ao или Ai + Di+ Do или Di + Ao+Do • 24 бита/96 кГц ==> 2 канала на входе _или_ 2 канала на выходе (только полудуплексный режим)
 - Ai, или Ao, или Di, или Do

Важные факты о цифровом интерфейсе:

- Порт Do дополнительно поддерживает сквозную передачу AC-3 в кодировке объемного звучания и DTS, хотя я этого не делал тестировал его под Linux
- Обратите внимание, что при этой настройке может быть включен только интерфейс Do
- Помимо записи цифрового аудиопотока, включение порта Di - это способ синхронизации устройства с внешними часами сэмплирования
- Как следствие, порт Di должен быть включен, только если подключен активный цифровой источник.
- Включение Di, когда ни один цифровой источник не подключен, может привести к ошибке синхронизации (например звук воспроизводится с нечетной частотой дискретизации)

6.7.3 Поддержка аудиофильского USB MIDI в ALSA

Аудиофильские USB MIDI порты будут поддерживаться автоматически после того, как будут загружены следующие модули:

- snd-usb-audio
- snd-seq-midi

Никаких дополнительных настроек не требуется.

6.7.4 Поддержка аудиофильского USB-аудио в ALSA

Аудиофункции аудиофильского USB-устройства выполняются модулем snd-usb-audio. Этот модуль может работать в режиме по умолчанию (без каких-либо параметров для конкретного устройства) или в "расширенном" режиме с параметром для конкретного устройства под названием `device_setup`.

Режим драйвера Alsa по умолчанию

Поведение драйвера snd-usb-audio по умолчанию заключается в перечислении возможностей устройства при запуске и активации требуемого режима, когда того требуют приложения: например, если пользователь записывает в 24-битном режиме глубины и сразу после этого хочет переключиться в 16-битный режим глубины, модуль snd-usb-audio перенастроит устройство на лету. Преимущество такого подхода заключается в том, что драйвер позволяет автоматически переключаться между частотой дискретизации и глубиной в соответствии с потребностями пользователя. Однако те, кто использует устройство под управлением Windows, знают, что устройство должно работать не так: под управлением Windows приложения должны быть закрыты перед использованием панели управления m-audio для переключения режима работы устройства. Таким образом, как мы увидим в следующем разделе, этот режим драйвера Alsa по умолчанию может привести к неправильным настройкам устройства. Давайте пока вернемся к режиму драйвера Alsa по умолчанию. В этом случае аудиофильские интерфейсы сопоставляются с устройствами alsa_pcm следующим образом (я полагаю, что индекс устройства равен 1):

- hw: 1,0 - это Ao при воспроизведении и Di при захвате
- hw: 1,1 - это Do при воспроизведении и Ai при захвате
- hw: 1,2 - это Do в сквозном режиме AC3 / DTS

В этом режиме устройство использует байтовое кодирование с большим порядковым номером, так что поддерживаемые аудиоформаты - S16_BE для 16-битных режимов глубины и S24_3BE для 24-битного режима глубины. Одним исключением является порт hw: 1,2, который, как сообщалось, мало совместим с Endian (предположительно поддерживает S16_LE), но фактически обрабатывает только потоки S16_BE. Это было исправлено в ядре 2.6.23 и выше, и теперь сообщается, что интерфейс hw: 1,2 имеет начальный регистр в этом режиме драйвера по умолчанию. Примеры:

• воспроизведение raw-файла в кодировке S24_3BE на порт Ao:

```
% aplay -D hw: 1, 0 -c 2 -t raw -text r 4 8 0 0 0 -fs 2 4 _3 BE.raw
```

• запись raw-файла в кодировке S24_3BE из порта искусственного интеллекта:

```
% запись -D hw: 1, 1 -c 2 -t raw -text r 4 8 0 0 0 -fs 2 4 _3 BE.raw
```

• воспроизведение raw-файла в кодировке S16_BE на порт Do:

```
% aplay -D hw: текст 1, 1 -c 2 -t raw -r 4 8 0 0 0 -fs 1 6 _BE.raw
```

• воспроизведение файла с образцом ac3 на порт Do:

```
% aplay -D hw: 1, 2 --channels= 6 ac 3 _S 1 6 _BE_encoded_file.raw
```

Если вас устраивает режим драйвера Alsa по умолчанию и у вас нет никаких проблем с этим режимом, то вы можете пропустить следующую главу.

Расширенная настройка модуля

Из-за аппаратных ограничений, описанных выше, инициализация устройства, выполненная драйвером Alsa в режиме по умолчанию, может привести к повреждению устройства. Например, особенно досадной проблемой является то, что звук, получаемый с интерфейса искусственного интеллекта, звучит искаженно (как будто усиленный чрезмерно высоким усилением громкости). Для людей, сталкивающихся с этой проблемой, модуль snd-usb-audio имеет новый параметр модуля с именем device_setup (этот параметр был введен в выпуске ядра 2.6.17)

Инициализация рабочего режима аудиофильского USB-устройства

Что касается аудиофильского USB-устройства, это значение позволяет пользователю указать:

- глубину выборки
- частоту дискретизации
- используется ли порт Di или нет

При инициализации с помощью device_setup = 0 x 0 0 , модуль snd-usb-audio ведет себя так же, как и в случае, когда этот параметр опущен (см. параграф "Режим драйвера Alsa по умолчанию" выше) Другие режимы описаны в следующих подразделах.

16-разрядные режимы

Поддерживаются два режима::

```
•device_setup = 0 x 0 1
```

-режим 16 бит 48 кГц с отключенным

Di - Ai, Ao, Do могут использоваться

одновременно -hw: 1,0 недоступен в

режиме захвата -hw: 1,2 недоступен

```
•device_setup = 0 x 1 1
```

- режим 16 бит 48 кГц с включенным Di

- Одновременно могут использоваться Ai, Ao, Di, Do

- hw: 1,0 доступен в режиме захвата
- hw: 1,2 недоступен

В этих режимах устройство работает только в 16-битных режимах. До ядра 2.6.23 устройства, о которых сообщалось, были с большим порядком, хотя на самом деле они были с маленьким порядком, так что воспроизведение файла было вопросом использования:

```
% aplay -D hw: 1, 1 -c 2 -t raw -r 4 8 0 0 0 -fs 1 6 BE
```

тест_1 6_le.raw где "test_S16_LE.raw" на самом деле был образцом файла с небольшим расширением.

Благодаря Хакану Леннесталу (который обнаружил, что в этих режимах у устройства есть младшие порядковые номера) было зафиксировано исправление (ожидается в ядре 2.6.23), и ALSA теперь сообщает о младших порядковых интерфейсах. Таким образом, воспроизвести файл теперь так же

```
просто, как использовать: % aplay -D hw: 1, 1 -c 2 -t raw -r 4 8 0 0 0 -fs 1 6_LE test_S 1 6_LE.raw
```

24-разрядные режимы

Поддерживаются три режима:

```
•device_setup = 0 x 0 9
```

- режим 24 бит 48 кГц с отключенным Di
- Одновременно могут использоваться Ai, Ao, Do - hw: 1,0 недоступен в режиме захвата - hw: 1,2 недоступен

```
•device_setup= 0 x 1 9
```

- режим 24 бит 48 кГц с включенным Di
- одновременно могут использоваться 3 порта от {Ai, Ao, Di, Do} - hw: 1,0 доступен в режиме захвата, и к Di должен быть подключен активный цифровой источник - hw: 1,2 недоступен

```
•device_setup = 0 x 0 D или 0 x 1 0
```

- режим 24 бит 96 кГц
- Di включен по умолчанию для этого режима, но его не нужно подключать к активному источнику - Одновременно может использоваться только 1 порт из {Ai, Ao, Di, Do} - hw: 1,0 доступен в режиме захвата
- hw: 1,2 недоступен В этих режимах устройство совместимо только с Big-Endian (см. "Режим драйвера ALSA по умолчанию" выше для получения примера команды aplay)

AC3 без режима DTS passthru

Благодаря Хакану Леннесталу, теперь у меня есть отчет, в котором говорится, что этот режим работает.

```
•device_setup = 0 x 0 3
```

- режим 16 бит 48 кГц с включенным только портом Do
- AC3 с DTS passthru
- Внимание: при такой настройке порт Do сопоставляется с устройством pcm hw: 1,0

Командная строка, используемая для воспроизведения WAV-файлов в кодировке AC3 /

DTS в этом режиме: % aplay -D hw: 1, 0 --каналы= 6 файлов ac3_S16_LE_encoded_file.raw

Как использовать параметр device_setup параметр

Параметр можно задать:

- Путем ручного тестирования устройства (от имени root)::

```
# modprobe -r snd-usb-audio
# индекс modprobe snd-usb-audio= 1 device_setup= 0 x 0 9
```

- Или при настройке параметров модулей в файле конфигурации ваших модулей (обычно в файле .conf в каталоге /etc/modprobe.d/::

```
псевдоним snd-карты- 1 snd-usb-audio
параметры snd-usb-audio index= 1 device_setup= 0 x 0 9
```

ВНИМАНИЕ при инициализации устройства

- Для правильной инициализации на устройстве требуется, чтобы перед включением устройства модулю был задан параметр device_setup. устройство включено. Итак, если вы используете метод "ручного зондирования", описанный выше, позаботьтесь о том, чтобы включить устройство ПОСЛЕ этой инициализации. • Несоблюдение этого требования приведет к неправильной настройке устройства. В этом случае выключите устройство, снимите проверку с модуля snd-usb-audio, затем проверьте его еще раз с помощью правильного параметра device_setup и затем (и только после этого) снова включите устройство. • Если вы правильно инициализировали устройство в допустимом режиме, а затем хотите переключиться в другой режим (возможно, с другой глубиной выборки), пожалуйста, воспользуйтесь также следующей процедурой:

- сначала выключите устройство
- де-зарегистрировать СНД-USB-аудио
- модули (команды modprobe -P) - изменения параметр Автор: изменения в device_setup параметр в

```
/etc/modprobe.d/ * .conf
```

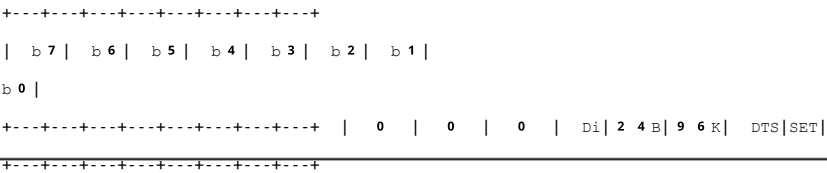
- включите устройство • Решение этой последней проблемы было применено к ядру 2.6.23, но этого может быть недостаточно для обеспечения "стабильности" инициализации устройства.

Технические подробности для хакеров

Этот раздел предназначен для хакеров, желающих узнать подробности о внутреннем устройстве устройства и о том, как ALSa поддерживает это.

Аудиофильский USB-порт device_setup структура

Если вы хотите понять магические числа device_setup для аудиофильского USB, вам нужно очень базовое понимание двоичных вычислений. Однако это не обязательно для использования параметра, и вы можете пропустить этот раздел. device_setup имеет длину в один байт и следующую структуру:



Где:

- b0 - бит SET бит
 - он ДОЛЖЕН быть установлен, если инициализирован device_setup
 - b1 - бит DTS
 - он установлен только для цифрового выхода с DTS / AC3 - эта настройка не тестировалась
 - b2 - флаг выбора частоты
 - При установке в 1 диапазон ставок равен 48.1-96 кГц - В противном случае диапазон частот дискретизации составляет 8-48 кГц
 - b3 - флаг выбора разрядности
 - При установке в 1 длина сэмплов составляет 24 бита
 - В противном случае они имеют длину 16 бит
 - Обратите внимание, что b2 подразумевает b3, поскольку режим 96 кГц поддерживается только для 24-битных выборок
 - b4 - это флаг цифрового ввода
 - При установке в 1 устройство предполагает, что подключен активный цифровой источник
 - Не следует включать Di, если на порту не отображается источник (это приводит к проблемам с синхронизацией)
 - b4 подразумевается под b2 (поскольку одновременно включен только один порт, ошибка синхронизации возникнуть не может)
 - b5 -b7 зарезервированы для будущего использования и должны быть установлены в 0
 - может стать Ao, Do, Ai для

b7, b6, b4 соответственно Внимание:

- значение, которое вы присвоите device_setup, не проверяется
 - например, при выборе 0x05 (16 бит 96 кГц) произойдет сбой возврата к 0x09, поскольку b2 подразумевает b3.
_there_will_be_no_warning_ в /var/log/ сообщениях
- Аппаратные ограничения из-за ограничения шины USB не проверены
 - выбор b2 подготовит все интерфейсы для работы с частотой 24 бит / 96 кГц, но вы сможете использовать только один из них одновременно же время

Но

Сведения о реализации USB для этого устройства

Вы можете смело пропустить этот раздел, если вас не интересует взлом драйверов.

В этом разделе описываются некоторые внутренние аспекты устройства и обобщаются данные, которые я получил с помощью отслеживания USB драйверов Windows и Linux. Аудиофильский USB-накопитель M-Audio имеет 7 USB-интерфейсов: "USB-интерфейс":

- USB-интерфейс №0
- USB-интерфейс №1
 - Функция управления звуком

- USB-интерфейс nb.2

- Аналоговый выход

- Интерфейс USB №3

- Цифровой выход

- Интерфейс USB №4

- Аналоговый вход

- Интерфейс USB №5

- Цифровой вход

- Интерфейс USB №6

- MIDI-интерфейс, совместимый с MIDIMAN quirk Каждый интерфейс

имеет 5 настроек (AltSet 1,2,3,4,5), за исключением:

- Интерфейс 3 (цифровой выход) имеет

дополнительный AltSet nb.6 • Интерфейс 5 (цифровой вход) не имеет AltSet

nb.3 и 5 Вот краткое описание возможностей AltSettings:

- Значения высот 1 соответствуют

- глубина 24 бита, режим дискретизации 48,1-96 кГц

- Адаптивное воспроизведение (Ao и Do), синхронный захват (Ai) или

асинхронный захват (Di) • Настройки 2 соответствуют

- Глубина 24 бита, режим дискретизации 8-48 кГц

- Захват и воспроизведение в асинхронном режиме (Ao, Ai, Do, Di)

- AltSettings 3 соответствует

- Глубина 24 бита, режим дискретизации 8-48 кГц

- Синхронный захват (Ai) и адаптивное воспроизведение (Ao, Do)

- Настройки 4 соответствуют

- 16-битная глубина, режим дискретизации 8-48 кГц

- Захват и воспроизведение в асинхронном режиме (Ao, Ai, Do, Di)

- Высотные настройки 5 соответствуют

- 16-битная глубина, режим дискретизации 8-48 кГц

- Синхронный захват (Ai) и адаптивное воспроизведение (Ao, Do)

- AltSettings 6 соответствует

- 16-битная глубина, режим дискретизации 8-48 кГц

- Воспроизведение с синхронизацией (Do), аудиоформат типа III IEC1937_AC-3

Для обеспечения правильной инициализации устройства, драйвер *должен знать* как будет использоваться устройство:

- если выбран DTS, должен быть зарегистрирован только интерфейс 2 с набором высот nb.6

- при частоте 96 кГц должны быть выбраны только наборы высот nb.1 для каждого интерфейса

- если выборки используют 24 бита / 48 кГц, то набор высот 2 должен использоваться при подключении цифрового входа и только

AltSet примечание.3, если цифровой вход не подключен

- если в выборках используется 16 бит / 48 кГц, то AltSet 4 следует использовать, если подключен цифровой вход, и только

AltSet примечание.5, если цифровой вход не подключен

Когда `device_setup` задается в качестве параметра для модуля `snd-usb-audio`, функция `parse_audio_endpoints` использует функцию `quirk`, вызываемую `audiophile_skip_setting_quirk` для предотвращения регистрации в драйвере настроек, не соответствующих `device_setup`.

6.7.5 Поддержка аудиофильского USB и разъема Jack

В этом разделе рассматривается поддержка аудиофильского USB-устройства в разъеме Jack. При использовании Jackd с устройством возможны 2 основные проблемы.:

- поддержка устройств Big-Endian в 24-разрядных режимах
- поддержка каналов 4 входа / 4 выхода

Прямая поддержка в Jackd

Джек поддерживает устройства big endian только в последних версиях (спасибо Андреасу Штайнмецу за его первый патч big-endian). Я не могу точно вспомнить, когда эта поддержка была выпущена в jackd, давайте просто скажем, что с jackd версии 0.103.0 все почти в порядке (всего лишь небольшая ошибка влияет на 16-битные устройства Big-Endian, но поскольку вы внимательно прочитали вышеприведенные параграфы, теперь вы используете kernel >= 2.6.23, и ваши 16-битные устройства теперь имеют младшие конечные разряды ;-)). Вы можете запустить jackd со следующей командой для воспроизведения с

```
помощью Ao и записи с помощью Ai: % jackd -R -dalsa -Phw: 1,0 -r 4 8 0 0 0 -p 1 2 8 -n 2 -D -Chw: 1,1
```

Используется плунжер Alsa

Если у вас недавно не установлен Jackd, вы можете перейти на использование `Alsa plug converter`.

Например, вот один из способов запустить Jack с 2 каналами воспроизведения на Ao и 2

```
каналами захвата от Ai: % jackd -R -dalsa -dplughw: 1 -r 4 8 0 0 0 -p 2 5 6 -n 2 -D -Cplughw: 1,1
```

Однако вы можете увидеть следующее предупреждающее сообщение: Похоже, вы используете программное обеспечение ALSA

уровень "подключения", вероятно, результат использования устройства ALSA по умолчанию. Это

менее эффективно, чем могло бы быть. Рассмотрите возможность использования аппаратного

устройства вместо уровня подключения. **Получение 2 входных и / или выходных интерфейсов в Jack**

Как вы можете видеть, запуск сервера Jack таким образом позволит включить только 1 стереовход (Di

или Ai) и 1 стереовыход (Ao или Do).

Это связано со следующими ограничениями:

- Разъем может одновременно открывать только одно устройство захвата и одно устройство воспроизведения

- Аудиофильский USB-порт рассматривается как 2 (или три) Устройства Alsa: hw: 1,0, hw:

1,1 (и необязательно hw: 1,2) Если вы хотите получить поддержку Ai + Di и / или Ao + Do с помощью Jack,

вам нужно будет объединить устройства Alsa в одно логическое "сложное" устройство. Если вы

хотите попробовать, я рекомендую ознакомиться с информацией с этой страницы: <http://www.sound-man.co.uk/linuxaudio/ice1712multi.html> Он связан с другим устройством (ice1712), но может быть адаптирован

под аудиофильский USB.

под аудиофильский USB.

Включение нескольких аудиофильских USB-интерфейсов для Jackd, безусловно, потребует:

- Убедитесь, что в вашей версии Jackd есть исправление MMAP_COMPLEX (см. страницу ice1712)
- (возможно) исправление файла `alsa-lib / src / pcm / pcm_multi.c` (см. страницу ice1712)

- определите несколько устройств (комбинация hw: 1,0 и hw: 1,1)
- в вашем файле .asoundrc • запустите jackd с этого устройства

На данный момент у меня не было успеха в тестировании этого, если у вас есть какие-либо успехи с такого рода настройками, пожалуйста, напишите мне электронное письмо.

Драйвер 6.8 Alsa для Digigram miXart8 и miXart8AES / EBU

звуковые карты

Digigram <alsa@digigram.com>

6.8.1 ОБЩИЕ ПОЛОЖЕНИЯ

miXart8 - это многоканальная звуковая карта для обработки и микширования звука, которая имеет 4 стереовхода и 4 стереовыхода. miXart8AES/EBU аналогичен с дополнительной платой, которая предлагает дополнительные 4 цифровых стереовхода и выхода. Кроме того, дополнительная плата обеспечивает синхронизацию внешних часов (AES / EBU, Word Clock, Time Code и Video Synchro) На материнской плате установлен PowerPC, который обеспечивает встроенное кодирование и декодирование трег, преобразования с дискретизацией и различные эффекты. Драйвер вообще не работает должным образом, пока не будут загружены определенные прошивки, т. е. Не появятся ни PCM, ни микшерные устройства. Используйте mixartloader, который можно найти в пакете alsa-tools.

6.8.2 ВЕРСИЯ 0.1.0

Одна плата miXart8 будет представлена в виде 4 плат alsa, каждая с 1 устройством стереоаналогового захвата "pcm0c" и 1 устройством стереоаналогового воспроизведения "pcm0r". В miXart8AES/EBU дополнительно имеется 1 стереофонический цифровой вход "pcm1c" и 1 стереофонический цифровой выход "pcm1r" на карту.

Форматы

U8, S16_LE, S16_BE, S24_3LE, S24_3BE, FLOAT_LE, FLOAT_BE Частоты дискретизации: 8000-48000 Гц

непрерывно Воспроизведение

Например, устройства воспроизведения настроены на максимальное количество подпотоков. Аппаратное микширование выполняют 4 подпотока. При желании это значение можно изменить максимум на 24 подпотока.

Монофонические файлы будут воспроизводиться на левом и правом каналах. Каждый канал можно отключить для каждого потока, чтобы использовать 8 аналоговых / цифровых выходов отдельно. **Захват**

На устройство захвата приходится один подпоток. Например, поддерживаются только стереоформаты.

Микшер

<Master> и <Master Capture> аналоговый регулятор громкости воспроизведения и записи PCM.

<PCM 0-3> и <PCM Capture> цифровой регулятор громкости каждого аналогового подпотока.

<AES 0-3> и <AES Capture> цифровая регулировка громкости каждого подпотока AES/EBU.

<Мониторинг> Переключение с 'pcm0c' на 'pcm0r' с цифровым управлением громкостью и отключением звука.

Примечание: для достижения наилучшего качества звука старайтесь поддерживать затухание 0 на регуляторах громкости PCM и AES, которое установлено на 219 в диапазоне от 0 до 255 (около 86% с alsamixer)

6.8.3 ЕЩЕ НЕ РЕАЛИЗОВАНО

- поддержка внешних часов (AES / EBU, Word Clock, Time Code, Video Sync) •
- Аудиоформаты MPEG • монофоническая запись • встроенные эффекты и преобразования
- сэмплирования • связанные потоки

6.8.4 ПРОШИВКА

[Начиная с версии 2.6.11, прошивка может загружаться автоматически с помощью hotplug при установке FIG_FW_LOADER. Загрузчик mixart необходим только для старых версий или когда вы встроите драйвер в ядро.]

Для автоматической загрузки прошивки после загрузки модуля используйте команду install. Например, добавьте следующую запись в /etc/modprobe.d/mixart.conf для драйвера miXart:

```
установить snd-mixart /sbin/modprobe --snd-mixart для первого использования && \
/usr/bin/mixartloader
```

(для ядер 2.2 / 2.4 вместо этого добавьте "snd-mixart /usr/bin/vxloader после установки" в /etc/modules.conf.)

Прошивка двоичные файлы устанавливаются в каталог /usr/доли/алса/встроенного (или /usr/местные/доли/алса/прошивки, зависит- ских префикс возможность настроить). Будет файл miXart.conf, который определяет файлы изображений dsp. Авторские права на файлы прошивки принадлежат Digigram SA

АВТОРСКИЕ ПРАВА на 6.8.5

Авторское право (с) 2003 Digigram SA <alsa@digigram.com> Распространяется под лицензией GPL.

Драйвер 6.9 ALSA BT87x

Введение 6.9.1

Возможно, вы заметили, что карты захвата bt878 на самом деле имеют две функции

```
PCI: $ lspci
[ ... ]
0 0 : 0 a . 0  Мультимедийный  видеоконтроллер:  Brooktree  Corporation  Bt 8 7 8  (rev.  0 2 )
0 0 : 0 a . 1  Мультимедийный  контроллер:  Brooktree  Corporation  Bt 8 7 8  (rev.  0 2 )
[ ... ]
```

Первый выполняет видео, он обратно совместим с bt848. Второй выполняет аудио. snd-bt87x - это драйвер для второй функции. Это звуковой драйвер, который можно использовать для записи звука (и *Только* запись, без воспроизведения). Поскольку большинство ТВ-карт поставляются с коротким кабелем, который можно подключить к линейному входу вашей звуковой карты, вам, вероятно, не нужен этот драйвер, если все, что вы хотите делать, это просто смотреть телевизор ... Некоторые карты не утруждают себя подключением чего-либо к выводам аудиовхода чипа, а некоторые другие карты используют функцию аудио для передачи видеоданных MPEG, поэтому вполне возможно, что запись звука может не работать с вашей картой.

Состояние драйвера 6.9.2

Драйвер теперь стабилен. Однако он не знает о многих телевизионных картах и отказывается загружаться для карт, о которых он не знает.

Если драйвер выдает жалобу ("Найдена неизвестная ТВ-карта, аудиодрайвер не загружается"), вы можете указать

`load_all= 1` возможность принудительно заставить драйвер попытаться использовать функцию захвата звука вашей карты.

Если частота записываемых данных неправильная, попробуйте указать значение `digital_rate` опция со значениями, отличными от значений по умолчанию 32000 (часто это 44100 или 64000).

Если у вас неизвестная карта, пожалуйста, отправьте идентификатор и название платы по адресу alsa-devel@alsa-project.org, независимо от того, работает захват звука или нет, чтобы будущие версии этого драйвера знали о вашей карте.

6.9.3 Режимы воспроизведения звука.

Микросхема поддерживает два разных режима (цифровой / аналоговый). `snd-bt87x` регистрирует два устройства PCM, по одному для каждого режима. Их нельзя использовать одновременно.

6.9.4 Режим цифрового звука

Первое устройство (`hw: X, 0`) воспроизводит 16-битный стереозвук. Частота дискретизации зависит от внешнего источника, который снабжает Bt87x цифровым звуком через интерфейс I2S.

6.9.5 Режим аналогового аудио (A/D)

Второе устройство (`hw: X, 1`) воспроизводит 8 или 16-битный монофонический звук. в диапазоне от 119466 до 448000 Гц (да, эти цифры настолько высоки).

установили опцию `CONST-FIG_SND_BT87X_OVERCLOCK`, максимальная частота дискретизации составляет 1792000 Гц, но аудиоданные становятся непригодными для использования после 896000 Гц на моей карте. Микросхема имеет три аналоговых входа. Следовательно, вы получите микшерное устройство для управления ими. Получайте удовольствие,

Клеменс

Автор: Клеменс Ладиш clemens@ladisch.de большие части скопированы с `btaudio.txt` Гердом Кноппом kraxel@bytesex.org

Поддерживаемые частоты дискретизации:

Если вы

6.10 Примечания по поддержке звука USB Maya44

Примечание:

Ниже приведен оригинальный документ патча Райнера, на котором основан текущий код `maya44`. Некоторые содержимое может быть устаревшим, но я сохраняю его здесь в качестве ссылки - `tiwai`

14 февраля 2008 г.

Rainer Zimmermann mail@lightshed.de

СОСТОЯНИЕ РАЗРАБОТКИ 6.10.1

Этот драйвер разрабатывается по инициативе Петра Маковски (oponek@gmail.com) и финансируется Ларсом Бергманном. Разработкой занимается Райнер Циммерманн (Rainer.Zimmermann.mail@lightshed.de). ESI предоставила образец карты Maуa44 для разработки. Однако, к сожалению, оказалось трудно получить подробную информацию о программировании, поэтому мне (Райнер Циммерманн) пришлось выяснить некоторую информацию о конкретной карте путем эксперимента и догадок. Некоторая информация (в частности, несколько битов GPIO) по-прежнему отсутствует. Это первая тестовая версия драйвера Maуa44, выпущенная для рассылки alsa-devel (5 февраля 2008 г.). Следующие функции работают, как было протестировано Райнером Циммерманом и Петром Маковски:

- воспроизведение и запись со всеми частотами дискретизации • уровень ввода / вывода • перекрестное микширование
- переключатель линии / микрофона
- переключатель фантомного питания
- аналоговый монитор, он же байпас

Следующие функции *должны* работать, но не протестированы полностью:

- Аналоговый канал 3 + 4 - переключение входов S/PDIF • Выход S/PDIF • все входы / выходы на плате расширения M/IO/DIO
- выбор внутренних / внешних часов

В частности, мы были бы признательны за тестирование этих функций всем, у кого есть доступ к плате расширения M / IO / DIO. Вещи, которые, похоже, не работают:

- Измерители уровня ("многодорожечные") в "alsamixer", похоже, не реагируют на сигналы (если это ошибка, то она вероятно, была бы в существующем коде ICE1724).
- Пыл 2.1, кажется, работает только через Джек, не используем ALSA напрямую или через OCC. Это еще предстоит выяснить.

6.10.2 СВЕДЕНИЯ О ДРАЙВЕРЕ

следующие файлы были добавлены:

- pci/ice1724/maуa44.c - конкретный код Maуa44
- pci/ice1724/maуa44.h • pci/ice1724/ice1724.patch
- pci/ice1724/ice1724.h.patch - ПРЕДЛАГАЕМЫЙ патч для ice1724.h (см. ЧАСТОТЫ ДИСКРЕТИЗАЦИИ) • i2c/other/wm8776.c - процедуры низкоуровневого доступа к кодекам Wolfson WM8776 • включают/wm8776.h

Обратите внимание, что код wm8776.c не зависит от карты и фактически не регистрирует кодек в инфраструктуре ALSA. Это делается в maуa44.c, главным образом потому, что некоторые элементы управления WM8776 используются специфичными для Maуa44 способами и должны иметь соответствующие названия. следующие файлы были созданы в pci/ice1724, просто #включая соответствующий файл из дерева ядра alsa- :

- wtm.h
- vt1720_mobo.h
- revo.ч
- prodigy192.ч
- понтис.ч
- фаза.х
- майя44.х
- джули.х
- ореон.х
- amp.h
- envy24ht.h
- se.h
- prodigy_hifi.h

Я надеюсь, что это правильный способ делать вещи.

6.10.3 ЧАСТОТА ДИСКРЕТИЗАЦИИ

Карта Maуa44 (или, точнее, кодеки Wolfson WM8776) обеспечивает максимальную частоту дискретизации 192 кГц для воспроизведения и 92 кГц для захвата. Поскольку микросхема ICE1724 допускает только одну глобальную частоту дискретизации, это выполняется следующим образом:

- установка частоты дискретизации на любом устройстве с открытым PCM на плате maуa44 всегда будет устанавливать *глобальную* частоту дискретизации для всех каналов воспроизведения и захвата.
- В текущем состоянии драйвера настройка частоты до 192 кГц разрешена даже для устройств захвата.

ИЗБЕГАЙТЕ ЗАПИСИ СО СКОРОСТЬЮ ВЫШЕ 96 кГц, хотя может показаться, что это работает. На самом деле кодек не может осуществлять запись с такой скоростью, что означает низкое качество. Я предлагаю некоторый дополнительный код для ограничения частоты дискретизации при настройке на устройстве capture pcm. Однако из-за глобальной частоты дискретизации эта логика была бы несколько проблематичной. Предлагаемый код (в настоящее время деактивированный) содержится в ice1712.h.patch, ice1724.c и maуa44.c (в pci / ice1712).

6.10.4 ЗВУКОВЫЕ УСТРОЙСТВА.

Устройства PCM соответствуют входам / выходам следующим образом (предполагается, что Maуa44 - это карта № 0):

- hw: вход 0,0 - стерео, аналоговый вход 1 + 2 • hw: выход 0,0 - стерео, аналоговый выход 1 + 2 • hw: вход 0,1 - стерео, аналоговый вход 3 + 4 Или вход S/PDIF • hw: выход 0,1 - стерео, аналоговый выход 3 + 4 (и выход SPDIF)

6.10.5 ПРИСВОЕНИЕ ИМЕН ЭЛЕМЕНТАМ УПРАВЛЕНИЯ МИКШЕРОМ

(для получения дополнительной информации о потоке сигналов, пожалуйста, обратитесь к блок-схеме на стр.24 руководства ESI Maуa44 или к программному обеспечению ESI для Windows). **PCM** (цифровой) выходной уровень для канала 1 + 2 **PCM 1** то же самое для канала 3 + 4

Фантом микрофона + 48 В переключатель фантомного питания + 48 В для электростатических микрофонов на входе 1/2.

Убедитесь, что он не включен, когда к входу 1/2 подключен любой другой источник. Это может привести к повреждению источника и / или платы тауа44.

Микрофонный / линейный вход если переключатель включен, входное гнездо 1/2 служит микрофонным входом (моно), в противном случае линейным входом (стерео). **Байпас** аналоговый байпас от входа АЦП к выходу для канала 1 + 2. То же, что "Монитор" в Windows драйвер.

Обход 1 то же самое для канала 3 + 4.

Перекрестное микширование перекрестный микшер с каналов 1 + 2

на каналы 3 + 4 **Перекрестное микширование 1** переключение с

каналов 3 + 4 на каналы 1 + 2 **Выход IEC958** переключатель для вывода S / PDIF.

Это не поддерживается драйвером ESI для Windows. S / PDIF должен выводить тот же сигнал, что и канал 3 + 4. [непроверено!]

Цифровые выходные селекторы Эти переключатели обеспечивают прямую цифровую маршрутизацию от АЦП к ЦАП. Каждый переключатель определяет, откуда поступают цифровые входные данные на один из ЦАП. Они не

поддерживаются портированным драйвером ESI для Windows. Для нормальной работы все они должны быть установлены в положение "PCM out". **Н / W** Канал выходного источника 1

Н / W 1 Канал выходного источника 2

Н / W 2 Канал выходного источника 3

Н / W 3 Канал выходного источника 4

Н / W 4... Н / W 9 неизвестная функция, оставлена для включения тестирования.

Возможно, некоторые из них управляют выводами S / PDIF. Если они окажутся неиспользуемыми, они исчезнут в более поздних версиях драйверов.

Выбираемые значения для каждого из селекторов цифровых выходных сигналов:

PCM out Выход ЦАП соответствующего канала (настройка по умолчанию)

Вход 1... Вход 4 прямая маршрутизация с выхода АЦП выбранного входного канала

6.11 Программный интерфейс ALSA-драйвер DSP MADI

(переведено с немецкого, поэтому хорошего английского нет;-), 2004 - винфрид Ритч

В драйвер добавлена полная функциональность. Поскольку некоторые элементы управления и опции запуска являются стандартными для ALSA , ниже описаны и обсуждаются только специальные элементы управления.

6.11.1 Аппаратные функции

Передача звука

• количество каналов - зависит от режима передачи.

Количество выбранных каналов составляет от 1..Nmax. Причиной использования меньшего количества каналов является только выделение ресурсов, поскольку неиспользуемые каналы DMA отключены и выделяется меньше памяти . Таким образом, также можно масштабировать пропускную способность системы PCI. (Важно только для плат с низкой производительностью). • Однокоростной - 1 ..64 канала

Примечание:

(Примечание: при выборе 56-канального режима для передачи или в качестве приемника по

MADI передается / принимается только 56 каналов, но для микшера доступны все 64 канала,

поэтому количество каналов зависит от драйвера) • Двойная скорость - 1 .. 32 канала

Примечание:

Примечание: При выборе 56-канального режима для режима передачи / приема по MADI передаются / принимаются только 28 каналов, но для микшера доступны все 32 канала, поэтому количество каналов зависит от драйвера

• Четырехскоростной - 1 .. 16 каналов

Примечание:

При выборе 56-канального режима для режима передачи / приема по MADI передаются / принимаются

только 14 каналов, но для микшера доступны все 16 каналов, поэтому количество каналов зависит

от драйвера • Формат - 32-битный строчный порядковый номер со знаком (SNDRV_PCM_FMTBIT_S32_LE)

• Частоты дискретизации -

Одноростной - 32000, 44100, 48000

Двойная скорость - 64000, 88200, 96000 (непроверено)

Четырехъядерная скорость - 128000, 176400, 192000

(непроверено) • режим доступа - MMIO (отображенный в

память), без чередования (PCM_NON-INTERLEAVED) • размеры

буфера - 64, 128, 256, 512, 1024, 2048, 8192 выборки •

фрагментов - 2 • Аппаратный указатель - 2 модификации

Карта поддерживает считывание фактического указателя буфера, куда DMA выполняет чтение / запись. Поскольку в массовом режиме PCI точность составляет всего 64 байта. ТАКИМ образом, на самом деле это невозможно использовать для функций ALSA- среднего уровня (здесь идентификатор буфера дает лучший результат), но если приложение использует MMIO . Следовательно, его можно настроить во время загрузки с помощью параметра precise-pointer .

Подсказка:

(Подсказка: Поэкспериментировав, я обнаружил, что указатель имеет максимальное значение от 64 до large и никогда не бывает маленьким. Так что, если вы вычтете 64, у вас всегда будет безопасный указатель для записи, который используется в этом режиме внутри ALSA. Теоретически теперь вы можете получить задержку всего в 16 выборок, что составляет четверть от возможностей прерывания.) • Точное отключение указателя прерывание, используемое для вычисления указателя • Точное включение указателя используется аппаратный указатель.

Контроллер

Поскольку DSP-MADI-Mixer имеет 8152 фейдера, нет смысла использовать стандартные элементы управления микшером, поскольку это нарушило бы большинство (особенно графических) графических интерфейсов ALSA-Mixer. Таким образом, управление микшером обеспечивается двумерным контроллером, использующим интерфейс hwdep.

Также ко всем 128 + 256 пиковым и среднеквадратичным измерителям можно получить доступ через hwdep-интерфейс. Поскольку это может быть проблемой производительности, если всегда копировать и преобразовывать пиковые и среднеквадратичные уровни, даже если вам нужен только один, я решил экспортировать аппаратную структуру, чтобы при необходимости какой-нибудь драйвер-гуру мог реализовать отображение в памяти микшера или пикометров через ioctl, или также выполнять только копирование без преобразования. Тестовое приложение показывает использование контроллера.

• Управление задержками - не реализовано !!!

Примечание:

Примечание: В Windows-драйвере задержка доступна из панели управления, но размеры буфера контролируются с помощью ALSA из hwparams-вызовов и не должны изменяться в состоянии выполнения, я не реализовывал это здесь.

• Системные часы - приостановлены!!!!

- Название - "Режим системных часов" - Доступ - чтение- запись
- Значения - "Master", "Slave"

Примечание:

!!!! Это аппаратная функция, но она противоречит контроллеру источника синхронизации, который является своего рода стандартом ALSA. I имеет смысл перевести карту в специальный режим (master на определенной частоте или slave), поскольку даже без использования аудиоприложения студия должна иметь рабочие настройки синхронизации. Так что используйте вместо этого контроллер источника синхронизации!!!!

• Источник синхронизации

- Имя - "Образец источника синхронизации"

- Доступ - чтение- запись

- Значения - "Автосинхронизация", "Внутренний 32,0 кГц", "Внутренний 44,1 кГц", "Внутренний 48,0 кГц", "Внутренний 64,0 кГц", "Внутренний 88,2 кГц", "Внутренний 96,0 кГц"

Выберите ведущий на определенной частоте, а также скоростной режим или ведомый (автосинхронизация). Смотрите также "Предпочтительная ссылка на синхронизацию"

Предупреждение:

!!!! Это не чисто аппаратная функция, но она была реализована ALSA некоторыми ALSA-драйверами ранее, поэтому я тоже ее использую. !!!

• Предпочтительная ссылка на синхронизацию

- Имя - "Предпочтительная ссылка на синхронизацию" - Доступ - Чтение- запись - Значения - "Word", "MADI"

В режиме автоматической синхронизации можно выбрать предпочтительный источник синхронизации. Если он недоступен, по возможности используется другой.

Примечание:

Примечание: Поскольку скорость передачи данных MADI намного выше, чем у word-clock, карта должна лучше синхронизироваться в режиме MADI. Но поскольку RME-PLL очень хорош, то и с word-clock проблем почти нет. Я так и не нашел разницы.

• TX 64 канала

- Название - "Режим TX 64

каналов" - Доступ -

Чтение - запись - Значения - 0 1

Использование 64-канального режима (1) или 56-канального режима для передачи MADI (0).

Примечание:

Примечание: Этот элемент управления предназначен только для вывода. Режим ввода определяется автоматически с помощью аппаратной отправки MADI.

• Очистить TMS

- Название - "Очистить

маркер дорожки" - Доступ - чтение-запись

- Значения - 0 1

Не используйте для уменьшения 5 аудиобитов в AES в качестве дополнительных битов. • Безопасный режим для автоматического ввода

- Название - "Безопасный режим"

- Доступ - чтение-запись

- Значения - 0 1 (по умолчанию включено)

Если включено (1), то в случае сбоя оптического или коаксиального соединения происходит переключение на рабочее соединение без сбоя выборки. Это полезно, только если вы используете второе в качестве резервного подключения.

• Ввод

- Имя - "Выбор ввода"

- Доступ - Чтение - запись

- Значения - оптический коаксиальный

Выбор входа, оптического или коаксиального. Если активен безопасный режим, это предпочтительный вход.

Микшер

• Название микшера

- "Микшер"

- Доступ - чтение, запись

- Значения - <номер канала 0-127> <Значение 0-65535>

Здесь в качестве первого значения берется индекс канала для получения/установки соответствующего канала микширования, где 0-63 - это фейдер ввода-вывода, а 64-127 - фейдер воспроизведения-вывода. Значение 0 означает отключенный канал 0, а 32768 - усиление, равное 1.

• Chn 1-64

быстрый микшер для утилит ALSA-mixer.

Диагональ микшерной матрицы реализована от воспроизведения до вывода. • Линейный вывод

-Название - "Линейный выход"

-Доступ - чтение

- запись -Значения - 0 1

Включение и выключение аналогового выхода, которое не имеет никакого отношения к микшированию или маршрутизации. аналоговые выходы отражают канал 63,64.

Информация (доступ только для чтения)

• Частота дискретизации

-Название - "Системная частота дискретизации"

-Доступ - только для чтения

получаем частоту дискретизации.

• Измерена внешняя частота.

-Название - "Внешняя частота"

-Доступ - только для чтения

Должно быть "Частота автосинхронизации", но используется ALSA-Scheme. Сообщается о частоте внешней выборки, которая использовалась при автосинхронизации. • Состояние синхронизации MADI

-Название - "Статус блокировки синхронизации MADI" -Доступ - Чтение - Значения - 0,1,2

MADI-вход 0 = разблокирован, 1 = заблокирован или 2 = синхронизирован. • Состояние синхронизации часов Word

-Название - "Состояние блокировки часов Word" -Доступ - чтение - Значения - 0,1,2

Для ввода Word Clock 0 = разблокировано, 1 = заблокировано или 2 = синхронизировано. • Автосинхронизация.

-Имя - "Ссылка на автосинхронизацию"

-Доступ - чтение

-Значения - "WordClock", "MADI", "Нет"

Ссылка на синхронизацию - это либо "WordClock", "MADI", либо нет. • RX 64ch - noch nicht implementiert

MADI-ресивер работает в 64-канальном режиме вместо 56-канального. • AB_inр - не тестировался

Используется вход для автоматического ввода.

• фактическое положение буфера - не реализовано

!!! это внутренняя функция ALSA, поэтому управление не используется!!!

6.11.2 Вызывающий параметр

- массив `index` `int` (`min` = 1, `max` = 8)

Значение индекса для интерфейса RME

HDSPPM. картотека в ALSA примечание: Стандарт ALSA

- массив строк идентификаторов (`min` = 1, `max` = 8)

Строка идентификаторов для интерфейса RME HDSPPM.

примечание: Стандарт ALSA

- включить массив `int` (`min` = 1, `max` = 8)

Включить / отключить определенные звуковые карты HDSPPM.

примечание: Стандарт ALSA

- массив `precise_ptr` `int` (`min` = 1, `max` = 8)

Включить точный указатель или отключить.

Примечание:

примечание: Используйте только тогда, когда приложение поддерживает это (что является особым случаем).

- массив `line_outs_monitor` `int` (`min` = 1, `max` = 8)

По умолчанию отправляйте потоки воспроизведения на аналоговые выходы.

Примечание:

примечание: каждый канал воспроизведения микшируется с одним и тем же пронумерованным выходным каналом (маршрутизируется). Это противоречит соглашению ALSA, согласно которому все каналы должны быть отключены после загрузки драйвера, но использовалось раньше на других картах, поэтому исторически я использую его снова)

- массив `int` `enable_monitor` (`min` = 1, `max` = 8)

По умолчанию включить аналоговый выход на канале 63/64.

Примечание:

примечание: здесь аналоговый выход включен (но не маршрутизируется).

6.12 MIDI-драйвер Serial UART 16450/16550

Параметр модуля адаптера позволяет выбрать либо:

- 0 - Поддержка Roland Soundcanvas (по умолчанию)
- 1 - Поддержка Midiator MS-124T (1)
- 2 - Midiator MS-124W в режиме S / A (2)
- 3 - MS-124W в режиме M / B (3)
- 4 - Универсальное устройство с поддержкой нескольких входов (4)

Для Midiator MS-124W необходимо настроить физические переключатели M-S и A-B на Midiator в соответствии с выбранным вами режимом драйвера. В режиме Roland Soundcanvas поддерживаются несколько подпотоков ALSA raw MIDI (midiCnD0-midiCnD15). Всякий раз, когда вы выполняете запись в другой подпоток, драйвер отправляет нестандартную последовательность MIDI-команд F5 NN, где NN - номер подпотока плюс 1. Модули Roland используют эту команду для переключения между различными "частями", поэтому эта функция позволяет обрабатывать каждую часть как отдельный необработанный подпоток MIDI. Драйвер не предоставляет способа отправить F5 00 (без выбора) или вообще не отправлять последовательность

команд F5 NN; возможно, так и должно быть. Пример использования простого последовательного преобразователя: /sbin/setserial /dev/ttyS 0 uart отсутствует

/sbin/modprobe snd-serial-порт u 1 6 5 5 0 = 0 x 3 f 8 irq = 4 скорости = 1 1 5 2 0 0

Пример использования для Roland SoundCanvas с 4 MIDI-портами:

/sbin/setserial /dev/ttyS 0 uart отсутствует

/sbin/modprobe snd-serial-порт u 1 6 5 5 0 = 0 x 3 f 8 irq = 4 выхода = 4

В режиме MS-124T поддерживается один подпоток raw MIDI (midiCnD0); параметр модуля outs автоматически устанавливается равным 1. Драйвер отправляет одни и те же данные на все четыре разъема MIDI Out. Установите переключатель A-B и параметр speed module на соответствие (A=19200, B= 9600). Пример использования для MS-124T с переключателем A-B в

положении A: /sbin/setserial /dev/ttyS 0 uart отсутствует

/sbin/modprobe snd-последовательный порт u 1 6 5 5 0 = 0 x 3 f 8 irq = 4 адаптера = 1 \

скорость= 1 9 2 0 0

В режиме S/A MS-124W поддерживается один подпоток raw MIDI (midiCnD0); параметр модуля outs автоматически устанавливается равным 1. Драйвер отправляет одни и те же данные на все четыре разъема MIDI Out на полной скорости MIDI. Пример использования для S / A режима:

/sbin/setserial /dev/ttyS 0 uart отсутствует

/sbin/modprobe snd-последовательный порт u 1 6 5 5 0 = 0 x 3 f 8 irq = 4 адаптера = 2

В режиме MS-124W M/B драйвер поддерживает 16 подпотоков ALSA raw MIDI; параметр модуля outs автоматически устанавливается равным 16. Номер подпотока дает битовую маску того, на какие разъемы MIDI Out должны быть отправлены данные , при этом midiCnD1 отправляется на выход 1, midiCnD2 - на выход 2, midiCnD4 - на выход 3 и midiCnD8 - на выход 4. Таким образом, midiCnD15 отправляет данные на все 4 порта. В качестве особого случая midiCnD0 также отправляет данные на все порты, поскольку нет смысла отправлять данные ни на какие порты. Режим M / B требует дополнительных затрат на выбор MIDI-выхода для каждого байта, поэтому совокупная скорость передачи данных по всем четырем MIDI-выходам составляет не более одного байта на каждые 520 us, по сравнению с полной скоростью передачи данных MIDI в один байт на каждые 320 us на порт. Пример использования для режима M/B:

/sbin/setserial /dev/ttyS 0 uart отсутствует

/sbin/modprobe snd-serial-порт u 1 6 5 5 0 = 0 x 3 f 8 irq = 4 адаптера = 3

Аппаратный режим ввода-вывода MS-124W в настоящее время не поддерживается. Этот режим позволяет MIDI-выходам действовать независимо с удвоенной совокупной пропускной способностью M / B, но не позволяет отправлять один и тот же байт одновременно нескольким MIDI-выходам. Протокол M / A требует, чтобы драйвер переключал линии управления модемом в условиях временных ограничений, поэтому реализовать его было бы немного сложнее, чем в других режимах. Модели Midiator, отличные от MS-124W и MS-124T, в настоящее время не поддерживаются. Обратите внимание, что буква суффикса имеет значение; MS-124 и MS-124B несовместимы, как и другие известные модели MS-101, MS-101B, MS-103 и MS-114. У меня есть документация (tim.mann@compaq.com), которая частично охватывает эти модели, но нет устройств для экспериментов. Поддержка MS-124W тестируется на реальном устройстве. Поддержка MS-124T не тестировалась, но должна работать.

Универсальный драйвер поддерживает несколько подпотоков ввода и вывода через один последовательный порт.

Аналогично режиму Roland Soundcanvas, F5 NN используется для выбора соответствующего входного или выходного потока (в зависимости от 288

направление передачи данных). Дополнительно сигнал CTS используется для регулирования потока данных. Количество входов задается параметром `ins`.

6.13 Контроллеры ввода SPDIF Imagination Technologies

Контроллер ввода SPDIF Imagination Technologies содержит следующие элементы управления:

- `name= 'Маска захвата IEC958', index=0`

Этот элемент управления возвращает маску, которая показывает, какие из битов состояния IEC958 могут быть считаны с помощью элемента управления 'IEC958 Capture Default'.

- `name='IEC958 Capture Default', index = 0`

Этот элемент управления возвращает биты состояния, содержащиеся в принимаемом потоке SPDIF. 'IEC958 Маска захвата' показывает, какие биты могут быть считаны из этого элемента управления.

- `name='SPDIF При многочастотном получении', индекс = 0`
- `name='SPDIF При многочастотном получении', индекс=1`
- `name='SPDIF При многочастотном получении', индекс = 2`
- `name='SPDIF При многочастотном получении', индекс = 3`

Этот элемент управления используется для попытки определения до четырех различных частот дискретизации. Активную частоту можно получить, считывая значение параметра "SPDIF в фиксированной частоте". Когда значение этого элемента управления установлено в {0,0,0,0}, скорость, указанная в `hw_params`, будет определять единственную скорость, которую будет фиксировать блок. В противном случае частота, указанная в `hw_params`, будет проигнорирована, и блок попытается выполнить захват для каждой из четырех установленных здесь частот дискретизации. Если требуется менее четырех частот, одну и ту же частоту можно указать более одного раза

- `name= 'SPDIF В частоте блокировки', index= 0`

Этот элемент управления возвращает активную частоту захвата или 0, если блокировка не была получена

- `name= 'SPDIF B Lock TRK', index= 0`

Этот элемент управления используется для изменения характеристик блокировки / подавления дрожания блока. Большие значения увеличивают диапазон блокировки, но уменьшают подавление дрожания.

- `name= 'SPDIF В пороге получения блокировки', index = 0`

Этот элемент управления используется для изменения порога получения блокировки.

- `name= 'SPDIF В пороге снятия блокировки', index = 0`

Этот элемент управления используется для изменения порога, при котором снимается блокировка.

Символы

`_snd_rawmidi_transmit_ack` (функция C), 46
`_snd_rawmidi_transmit_peek` (функция C), 45

B

`байт_то_фреймы` (функция C), 21
`bytes_to_samples` (функция C), 20

C

`copy_from_user_toio` (функция C), 6
`copy_to_user_fromio` (функция C), 6

D

`devm_snd_dmaengine_pcm_register` (функция C), `snd_card_new` (функция C), 1
68
`devm_snd_soc_register_card` (функция C), 68
`devm_snd_soc_register_component` (функция C), 67 `snd_component_add` (функция C), 3
`devm_snd_soc_register_platform` (функция C), 67

F

`frame_aligned` (функция C), 21
`frames_to_bytes` (функция C), 21

P

`params_buffer_bytes` (функция C), 24
`params_buffer_size` (функция C), 23
`params_channels` (функция C), 23
`params_period_size` (функция C), 23
`params_periods` (функция C), 23
`params_rate` (функция C), 23
`pcm_format_to_bits` (функция C), 26

S

`samples_to_bytes` (функция C), 21
`snd_ac97_bus` (функция C), 40
`snd_ac97_get_short_name` (функция C), 40
`snd_ac97_mixer` (функция C), 41
`snd_ac97_pcm_assign` (функция C), 42
`snd_ac97_pcm_close` (функция C), 43
`snd_ac97_pcm_double_rate_rules` (функция C), 43
`snd_ac97_pcm_open` (функция C), 42
`snd_ac97_read` (функция C), 39
`snd_ac97_resume` (функция C), 41
`snd_ac97_set_rate` (функция C), 42
`snd_ac97_suspend` (функция C), 41

`snd_ac97_tune_hardware` (функция C), 41
`snd_ac97_update` (функция C), 39
`snd_ac97_update_bits` (функция C), 40
`snd_ac97_update_power` (функция C), 41
`snd_ac97_write` (функция C), 39
`snd_ac97_write_cache` (функция C), 39
`snd_BUG` (функция C), 88
`snd_BUG_ON` (функция C), 89
`snd_card_add_dev_attr` (функция C), 2
`snd_card_disconnect` (функция C), 1
`snd_card_file_add` (функция C), 3
`snd_card_file_remove` (функция C), 3
`snd_card_free` (функция C), 2
`snd_card_free_when_closed` (функция C), 2
`snd_card_register` (функция C), 2
`snd_card_set_id` (функция C), 2
`snd_compr` (тип C), 55
`snd_compr_avail` (тип C), 50
`snd_compr_caps` (тип C), 51
`snd_compr_codec_caps` (тип C), 51
`snd_compr_metadata` (тип C), 51
`snd_compr_ops` (тип C), 54
`snd_compr_params` (тип C), 50
`snd_compr_runtime` (тип C), 53
`snd_compr_stream` (тип C), 54
`snd_compr_tstamp` (тип C), 50
`snd_compress_register` (функция C), 49
`snd_compressed_buffer` (тип C), 49
`snd_ctl_activate_id` (функция C), 37
`snd_ctl_add` (функция C), 35
`snd_ctl_add_slave` (функция C), 44
`snd_ctl_add_slave_uncached` (функция C), 44
`snd_ctl_add_vmaster_hook` (функция C), 43
`snd_ctl_boolean_mono_info` (функция C), 38
`snd_ctl_boolean_stere_info` (функция C), 38
`snd_ctl_enum_info` (функция C), 38
`snd_ctl_find_id` (функция C), 37
`snd_ctl_find_numid` (функция C), 37
`snd_ctl_free_one` (функция C), 35
`snd_ctl_make_virtual_master` (функция C), 43
`snd_ctl_new` (функция C), 35
`snd_ctl_new1` (функция C), 35
`snd_ctl_notify` (функция C), 34
`snd_ctl_register_ioctl` (функция C), 38
`snd_ctl_register_ioctl_compat` (функция C), 38

[snd_ctl_remove \(функция C\)](#), 36
[snd_ctl_remove_id \(функция C\)](#), 36
[snd_ctl_remove_user_ctl \(функция C\)](#), 36
[snd_ctl_rename_id \(функция C\)](#), 37
[snd_ctl_replace \(функция C\)](#), 36
[snd_ctl_sync_vmaster \(функция C\)](#), 44
[snd_ctl_unregister_ioctl \(функция C\)](#), 38

[snd_ctl_unregister_ioctl_compat \(функция C\)](#), 38
[snd_device_disconnect \(функция C\)](#), 4
[snd_device_free \(функция C\)](#), 4
[snd_device_initialize \(функция C\)](#), 1
[snd_device_new \(функция C\)](#), 4
[snd_device_register \(функция C\)](#), 4
[snd_dma_alloc_pages \(функция C\)](#), 7
[snd_dma_alloc_pages_fallback \(функция C\)](#), 7
[snd_dma_disable \(функция C\)](#), 88
[snd_dma_free_pages \(функция C\)](#), 8
[snd_dma_pointer \(функция C\)](#), 88
[snd_dma_program \(функция C\)](#), 88
[snd_dmaengine_dai_dma_data \(тип C\)](#), 33
[snd_dmaengine_pcm_close \(функция C\)](#), 33

[snd_dmaengine_pcm_close_release_chan \(функция C\)](#)- [snd_pcm_drain_done \(функция C\)](#), 18
[тион](#)), 33
[snd_dmaengine_pcm_config \(тип C\)](#), 34
[snd_dmaengine_pcm_open \(функция C\)](#), 32
[snd_dmaengine_pcm_open_request_chan \(функция C\)](#)- [snd_pcm_format_little_endian \(функция C\)](#), 27
[тион](#)), 33
[snd_dmaengine_pcm_pointer \(функция C\)](#), 32
[snd_dmaengine_pcm_pointer_no_residue \(функция C\)](#)- [snd_pcm_format_set_silence \(функция C\)](#), 27
[тион](#)), 32
[snd_dmaengine_pcm_prepare_slave_config \(функция C\)](#)- [snd_pcm_format_silence_64 \(функция C\)](#), 27
[тион](#)), 82
[snd_dmaengine_pcm_register \(функция C\)](#), 82
[snd_dmaengine_pcm_request_channel \(функция C\)](#), [snd_pcm_format_width \(функция C\)](#), 27
32
[snd_dmaengine_pcm_set_config_from_dai_data \(C snd_pcm_group_for_each_entry \(функция C\)](#), 20
[функция](#)), 31
[snd_dmaengine_pcm_trigger \(функция C\)](#), 32
[snd_dmaengine_pcm_unregister \(функция C\)](#), 83
[snd_enc_flac \(тип C\)](#), 53
[snd_enc_real \(тип C\)](#), 52
[snd_enc_vorbis \(тип C\)](#), 52
[snd_free_dev_iram \(функция C\)](#), 7
[snd_free_pages \(функция C\)](#), 7
[snd_hwdep_new \(функция C\)](#), 83
[snd_hwparams_to_dma_slave_config \(функция C\)](#), [snd_pcm_hw_constraint_ratnums \(функция C\)](#), 14
31
[snd_info_create_card_entry \(функция C\)](#), 49
[snd_info_create_module_entry \(функция C\)](#), 48
[snd_info_free_entry \(функция C\)](#), 49
[snd_info_get_line \(функция C\)](#), 48
[snd_info_get_str \(функция C\)](#), 48
[snd_info_register \(функция C\)](#), 49
[snd_interval_div \(функция C\)](#), 10
[snd_interval_list \(функция C\)](#), 11
[snd_interval_muldivk \(функция C\)](#), 10
[snd_interval_mulkdiv \(функция C\)](#), 10
[snd_interval_ranges \(функция C\)](#), 11

[snd_interval_ratden \(функция C\)](#), 11
[snd_interval_ratnum \(функция C\)](#), 10
[snd_interval_refine \(функция C\)](#), 9
[snd_jack_add_new_kctl \(функция C\)](#), 84
[snd_jack_new \(функция C\)](#), 84
[snd_jack_report \(функция C\)](#), 85
[snd_jack_set_key \(функция C\)](#), 84
[snd_jack_set_parent \(функция C\)](#), 84
[snd_jack_types \(тип C\)](#), 83
[snd_lookup_minor_data \(функция C\)](#), 5
[snd_malloc_dev_iram \(функция C\)](#), 7
[snd_malloc_pages \(функция C\)](#), 6

[snd_mpu401_uart_interrupt \(функция C\)](#), 47
[snd_mpu401_uart_interrupt_tx \(функция C\)](#), 47
[snd_mpu401_uart_new \(функция C\)](#), 47
[snd_pcm_add_chmap_ctls \(функция C\)](#), 16
[snd_pcm_capture_avail \(функция C\)](#), 21
[snd_pcm_capture_empty \(функция C\)](#), 23
[snd_pcm_capture_hw_avail \(функция C\)](#), 22
[snd_pcm_capture_ready \(функция C\)](#), 22
[snd_pcm_chmap_substream \(функция C\)](#), 26

[snd_pcm_format_big_endian \(функция C\)](#), 27
[snd_pcm_format_cpu_endian \(функция C\)](#), 24
[snd_pcm_format_linear \(функция C\)](#), 26

[snd_pcm_format_name \(функция C\)](#), 8
[snd_pcm_format_physical_width \(функция C\)](#), 27

[snd_pcm_format_signed \(функция C\)](#), 26

[snd_pcm_format_size \(функция C\)](#), 27
[snd_pcm_format_unsigned \(функция C\)](#), 26

[snd_pcm_gettime \(функция C\)](#), 24

[snd_pcm_hw_constraint_integer \(функция C\)](#), 12
[snd_pcm_hw_constraint_list \(функция C\)](#), 13
[snd_pcm_hw_constraint_mask \(функция C\)](#), 12
[snd_pcm_hw_constraint_mask64 \(функция C\)](#), 12
[snd_pcm_hw_constraint_minmax \(функция C\)](#), 13
[snd_pcm_hw_constraint_msbits \(функция C\)](#), 14
[snd_pcm_hw_constraint_pow2 \(функция C\)](#), 15
[snd_pcm_hw_constraint_ranges \(функция C\)](#), 13

[snd_pcm_hw_constraint_rattdens \(функция C\)](#), 14

[snd_pcm_hw_constraint_single \(функция C\)](#), 24
[snd_pcm_hw_constraint_step \(функция C\)](#), 14
[snd_pcm_hw_param_first \(функция C\)](#), 15
[snd_pcm_hw_param_last \(функция C\)](#), 16
[snd_pcm_hw_param_value \(функция C\)](#), 15
[snd_pcm_hw_params_choose \(функция C\)](#), 18
[snd_pcm_hw_rule_add \(функция C\)](#), 12
[snd_pcm_hw_rule_noresample \(функция C\)](#), 15
[snd_pcm_kernel_ioctl \(функция C\)](#), 19
[snd_pcm_lib_alloc_vmalloc_32_buffer \(функция C\)](#),
25
[snd_pcm_lib_alloc_vmalloc_buffer \(функция C\)](#), 24

- [snd_pcm_lib_buffer_bytes \(функция C\), 21](#)
[snd_pcm_lib_default_mmap \(функция C\), 19](#)
[snd_pcm_lib_free_pages \(функция C\), 30](#)
[snd_pcm_lib_free_vmalloc_buffer \(функция C\), 30](#)
[snd_pcm_lib_get_vmalloc_page \(функция C\), 31](#)
[snd_pcm_lib_ioctl \(функция C\), 16](#)
[snd_pcm_lib_malloc_pages \(функция C\), 30](#)
[snd_pcm_lib_mmap_iomem \(функция C\), 20](#)
[snd_pcm_lib_period_bytes \(функция C\), 21](#)
[snd_pcm_lib_preallocate_free \(функция C\), 29](#)
[snd_pcm_lib_preallocate_free_for_all \(функция C\), snd_rawmidi_transmit_empty \(функция C\), 45](#)
[29](#)
[snd_pcm_lib_preallocate_pages \(функция C\), 29](#)
[snd_pcm_lib_preallocate_pages_for_all \(функция C\), snd_request_card \(функция C\), 5](#)
[29](#)
[snd_pcm_limit_hw_rates \(функция C\), 28](#)
[snd_pcm_limit_isa_dma_size \(функция C\), 25](#)
[snd_pcm_mmap_data_close \(функция C\), 25](#)
[snd_pcm_mmap_data_open \(функция C\), 25](#)
[snd_pcm_new \(функция C\), 8](#)
[snd_pcm_new_internal \(функция C\), 9](#)
[snd_pcm_new_stream \(функция C\), 8](#)
[snd_pcm_notify \(функция C\), 9](#)
[snd_pcm_period_elapsed \(функция C\), 16](#)
[snd_pcm_playback_avail \(функция C\), 21](#)
[snd_pcm_playback_data \(функция C\), 22](#)
[snd_pcm_playback_empty \(функция C\), 22](#)
[snd_pcm_playback_hw_avail \(функция C\), 22](#)
[snd_pcm_playback_ready \(функция C\), 22](#)
[snd_pcm_prepare \(функция C\), 19](#)
[snd_pcm_rate_bit_to_rate \(функция C\), 28](#)
[snd_pcm_rate_mask_intersect \(функция C\), 28](#)
[snd_pcm_rate_range_to_bits \(функция C\), 28](#)
[snd_pcm_rate_to_rate_bit \(функция C\), 28](#)
[snd_pcm_running \(функция C\), 20](#)
[snd_pcm_set_ops \(функция C\), 9](#)
[snd_pcm_set_runtime_buffer \(функция C\), 24](#)
[snd_pcm_set_sync \(функция C\), 9](#)
[snd_pcm_sgbuf_get_addr \(функция C\), 25](#)
[snd_pcm_sgbuf_get_chunk_size \(функция C\), 25](#)
[snd_pcm_sgbuf_get_ptr \(функция C\), 25](#)
[snd_pcm_sgbuf_ops_page \(функция C\), 30](#)
[snd_pcm_start \(функция C\), 18](#)
[snd_pcm_stop \(функция C\), 18](#)
[snd_pcm_stop_xrun \(функция C\), 18](#)
[snd_pcm_stream_linked \(функция C\), 20](#)
[snd_pcm_stream_lock \(функция C\), 17](#)
[snd_pcm_stream_lock_irq \(функция C\), 17](#)
[snd_pcm_stream_lock_irqsave \(функция C\), 20](#)
[snd_pcm_stream_str \(функция C\), 26](#)
[snd_pcm_stream_unlock \(функция C\), 17](#)
[snd_pcm_stream_unlock_irq \(функция C\), 17](#)
[snd_pcm_stream_unlock_irqrestore \(функция C\), 17](#)
[snd_pcm_substream_to_dma_direction \(функция C\), snd_soc_dai_set_tristate \(функция C\), 65](#)
[33](#)
[snd_pcm_suspend \(функция C\), 19](#)
[snd_pcm_suspend_all \(функция C\), 19](#)
[snd_pcm_trigger_done \(функция C\), 23](#)
[snd_power_wait \(функция C\), 3](#)
[snd_printd \(функция C\), 88](#)
[snd_printd_ratelimit \(функция C\), 89](#)
[snd_printdd \(функция C\), 89](#)
[snd_printk \(функция C\), 88](#)
[snd_rawmidi_new \(функция C\), 46](#)
[snd_rawmidi_receive \(функция C\), 45](#)
[snd_rawmidi_set_ops \(функция C\), 47](#)
[snd_rawmidi_transmit \(функция C\), 46](#)
[snd_rawmidi_transmit_ack \(функция C\), 46](#)
[snd_rawmidi_transmit_peek \(функция C\), 45](#)
[snd_register_device \(функция C\), 5](#)
[snd_soc_add_card_controls \(функция C\), 62](#)
[snd_soc_add_codec_controls \(функция C\), 62](#)
[snd_soc_add_component_controls \(функция C\), 62](#)
[snd_soc_add_dai_controls \(функция C\), 62](#)
[snd_soc_add_dai_link \(функция C\), 60](#)
[snd_soc_add_platform \(функция C\), 66](#)
[snd_soc_add_platform_controls \(функция C\), 62](#)
[snd_soc_cache_sync \(функция C\), 58](#)
[snd_soc_card_jack_new \(функция C\), 85](#)
[snd_soc_cnew \(функция C\), 61](#)
[snd_soc_codec_exit_regmap \(функция C\), 59](#)
[snd_soc_codec_force_bias_level \(функция C\), 58](#)
[snd_soc_codec_get_bias_level \(функция C\), 58](#)
[snd_soc_codec_get_dapm \(функция C\), 58](#)
[snd_soc_codec_init_bias_level \(функция C\), 58](#)
[snd_soc_codec_init_regmap \(функция C\), 59](#)
[snd_soc_codec_set_jack \(функция C\), 85](#)
[snd_soc_codec_set_sysclk \(функция C\), 63](#)
[snd_soc_component_async_complete \(функция C\), 69](#)
[snd_soc_component_exit_regmap \(функция C\), 66](#)
[snd_soc_component_get_dapm \(функция C\), 57](#)
[snd_soc_component_init_regmap \(функция C\), 66](#)
[snd_soc_component_read \(функция C\), 68](#)
[snd_soc_component_test_bits \(функция C\), 69](#)
[snd_soc_component_to_codec \(функция C\), 57](#)
[snd_soc_component_to_platform \(функция C\), 57](#)
[snd_soc_component_update_bits \(функция C\), 68](#)
[snd_soc_component_update_bits_async \(функция C\), 69](#)
[snd_soc_component_write \(функция C\), 68](#)
[snd_soc_dai_digital_mute \(функция C\), 65](#)
[snd_soc_dai_set_bclk_ratio \(функция C\), 64](#)
[snd_soc_dai_set_channel_map \(функция C\), 65](#)
[snd_soc_dai_set_clkdiv \(функция C\), 63](#)
[snd_soc_dai_set_fmt \(функция C\), 64](#)
[snd_soc_dai_set_pll \(функция C\), 63](#)
[snd_soc_dai_set_sysclk \(функция C\), 63](#)
[snd_soc_dai_set_tdm_slot \(функция C\), 64](#)
[snd_soc_dapm_add_routes \(функция C\), 77](#)
[snd_soc_dapm_dai_get_connected_widgets \(функция C\), 76](#)
[snd_soc_dapm_del_routes \(функция C\), 77](#)

snd_soc_dapm_disable_pin (функция C), 81
 snd_soc_dapm_disable_pin_unlocked (функция C),

81

snd_soc_dapm_enable_pin (функция C), 80

snd_soc_dapm_enable_pin_unlocked (функция C), snd_soc_kontrol_platform (функция C), 59

79

snd_soc_dapm_force_bias_level (функция C), 75

snd_soc_dapm_force_enable_pin (функция C), 80

snd_soc_dapm_force_enable_pin_unlocked (функция C) - snd_soc_put_strobe (функция C), 75
 тион), 80

snd_soc_dapm_free (функция C), 82

snd_soc_dapm_get_enum_double (функция C), 78

snd_soc_dapm_get_pin_status (функция C), 82

snd_soc_dapm_get_pin_switch (функция C), 79

snd_soc_dapm_get_volsw (функция C), 78

snd_soc_dapm_ignore_suspend (функция C), 82

snd_soc_dapm_info_pin_switch (функция C), 79

snd_soc_dapm_kontrol_codec (функция C), 58

snd_soc_dapm_kontrol_dapm (функция C), 75

snd_soc_dapm_kontrol_widget (функция C), 75

snd_soc_dapm_nc_pin (функция C), 81

snd_soc_dapm_nc_pin_unlocked (функция C), 81

snd_soc_dapm_new_controls (функция C), 79

snd_soc_dapm_new_widgets (функция C), 78

snd_soc_dapm_put_enum_double (функция C), 78

snd_soc_dapm_put_pin_switch (функция C), 79

snd_soc_dapm_put_volsw (функция C), 78

snd_soc_dapm_set_bias_level (функция C), 76

snd_soc_dapm_stream_event (функция C), 79

snd_soc_dapm_sync (функция C), 77

snd_soc_dapm_sync_unlocked (функция C), 76

snd_soc_dapm_to_codec (функция C), 57

snd_soc_dapm_to_component (функция C), 57

snd_soc_dapm_to_platform (функция C), 57

snd_soc_dapm_weak_routes (функция C), 77

snd_soc_find_dai (функция C), 60

snd_soc_find_dai_link (функция C), 60

snd_soc_get_enum_double (функция C), 71

snd_soc_get_strobe (функция C), 75

snd_soc_get_volsw (функция C), 72

snd_soc_get_volsw_range (функция C), 73

snd_soc_get_volsw_sx (функция C), 73

snd_soc_get_xr_sx (функция C), 74

snd_soc_info_enum_double (функция C), 71

snd_soc_info_volsw (функция C), 72

snd_soc_info_volsw_range (функция C), 73

snd_soc_info_volsw_sx (функция C), 72

snd_soc_info_xr_sx (функция C), 74

snd_soc_jack_add_gpiods (функция C), 87

snd_soc_jack_add_gpios (функция C), 87

snd_soc_jack_add_pins (функция C), 86

snd_soc_jack_add_zones (функция C), 86

snd_soc_jack_free_gpios (функция C), 87

snd_soc_jack_get_type (функция C), 86

snd_soc_jack_gpio (тип C), 56

snd_soc_jack_notifier_register (функция C), 86

snd_soc_jack_notifier_unregister (функция C), 87

snd_soc_jack_pin (тип C), 55

snd_soc_jack_report (функция C), 85

snd_soc_jack_zone (тип C), 56

snd_soc_control_codec (функция C), 59

snd_soc_control_component (функция C), 59

snd_soc_limit_volume (функция C), 74

snd_soc_new_compress (функция C), 75

snd_soc_put_enum_double (функция C), 71

snd_soc_put_volsw (функция C), 72

snd_soc_put_volsw_range (функция C), 73

snd_soc_put_volsw_sx (функция C), 73

snd_soc_put_xr_sx (функция C), 74

snd_soc_read_signed (функция C), 71

snd_soc_register_card (функция C), 65

snd_soc_register_codec (функция C), 67

snd_soc_register_dai (функция C), 66

snd_soc_register_dais (функция C), 66

snd_soc_register_platform (функция C), 67

snd_soc_remove_dai_link (функция C), 60

snd_soc_remove_platform (функция C), 67

snd_soc_runtime_activate (функция C), 70

snd_soc_runtime_deactivate (функция C), 70

snd_soc_runtime_ignore_pmdown_time (C

), 70

snd_soc_runtime_set_dai_fmt (функция C), 61

snd_soc_set_dmi_name (функция C), 61

snd_soc_set_runtime_hwparams (функция C), 71

snd_soc_test_bits (функция C), 70

snd_soc_unregister_card (функция C), 65

snd_soc_unregister_codec (функция C), 67

snd_soc_unregister_component (функция C), 66

snd_soc_unregister_dais (функция C), 65

snd_soc_unregister_platform (функция C), 67

snd_soc_update_bits (функция C), 70

snd_soc_xlate_tdm_slot_mask (функция C), 64

snd_unregister_device (функция C), 6

sndrv_compress_encoder (тип C), 51

SNDRV_COMPRESS_IOCTL_VERSION (функция C), 52

функция-